UNIVERSITY OF CAMBRIDGE

RESEARCH COURSE

Discrete Diffusion

A research course on discrete diffusion generative models, with applications to molecular generation and diffusion language modelling

About this course

This course is a self-contained route from the first principles of diffusion on finite state spaces to the research frontier of discrete diffusion generative modelling. It develops the discrete-time formulation (D3PM), the continuous-time Markov chain framework, concrete score matching and score entropy, masked and uniform-state diffusion, discrete flow matching, and the continuous relaxations that preceded them. It then turns to inference — samplers, decoding orders, remasking, guidance and reinforcement learning — and to the two application areas that have driven the field: molecular and biological sequence design, and diffusion language models. Derivations are given in full, with the intuition stated before the algebra; every result that admits a finite check has been verified numerically, and every code listing has been executed. Each chapter closes with a box of open questions and research directions that are, to the best of the available literature, still unresolved.

Date: October 2026
Keywords: discrete diffusion, masked diffusion, score entropy, discrete flow matching, diffusion language models, molecular generation

Contents

How to use this course **1**
What this is 1
A warning about this field 1
Conventions 2

1 The Case for Discrete Diffusion **3**
Sohl-Dickstein et al.; Austin et al.; Lou et al.; Nie et al. (2026)
1.1 The problem 3
1.2 Why you cannot just use a Gaussian 4
1.3 A map of the field 5
1.4 What discrete diffusion is actually good at 6

2 Mathematical Foundations **8**
Norris; Cover & Thomas; Anderson; Ho et al.; Song et al. (2026)
2.1 Categorical variables and the simplex 8
2.2 Divergences on finite spaces 9
2.3 Variational bounds 10
2.4 Discrete-time Markov chains 10
2.5 Continuous-time Markov chains 12
2.6 The concrete score 19
2.7 Simulating a CTMC: exact, Euler and τ-leaping 19
2.8 Continuous-state diffusion, for contrast 19

3 D3PM: Discrete Diffusion in Discrete Time **27**
Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow and Rianne van den Berg; Emiel Hoogeboom et al. (2021)
3.1 The forward process 27
3.2 The variational bound 28
3.3 The $\boldsymbol{x}_0$-parameterisation 28
3.4 Choosing the transition matrix 29
3.5 Noise schedules 32
3.6 What D3PM achieved, and where it fell short 33

4 Continuous-Time Discrete Diffusion **35**
Andrew Campbell et al.; Aaron Lou, Chenlin Meng and Stefano Ermon (2024)
4.1 Why continuous time 35
4.2 The forward process 36
4.3 The reverse process 36
4.4 The continuous-time ELBO 37
4.5 Sampling: τ-leaping and predictor–corrector 37
4.6 Uniformization 38
4.7 The concrete score and score entropy 38

5 The Interpolating Family: Masked and Uniform Diffusion . . . **47**
Subham Sekhar Sahoo et al.; Jiaxin Shi et al.; Yair Schiff et al.; Patrick Pynadath et al. (2026)
5.1 The forward process . . . 47
5.2 The true posterior . . . 47
5.3 The SUBS parameterisation . . . 48
5.4 Rao–Blackwellisation and the simplified bound . . . 49
5.5 Training and sampling in practice . . . 50
5.6 Reported results . . . 53
5.7 Generalisations . . . 54
5.8 Uniform-state and interpolating diffusion . . . 54

6 Hidden Equivalences: Any-Order Autoregression and What Masked Models Really Learn . . . **61**
Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li and Chongxuan Li (RADD); Benigno Uria et al.; Emiel Hoogeboom et al. (2025)
6.1 Any-order autoregressive models . . . 61
6.2 The equivalence . . . 62
6.3 What this does and does not mean . . . 65
6.4 RADD: the time variable is redundant . . . 66
6.5 The same result, found twice . . . 67
6.6 Other equivalences worth knowing . . . 67

7 Flows and Continuous Relaxations . . . **70**
Itai Gat et al.; Andrew Campbell et al.; Hannes Stark et al.; Xiang Lisa Li et al. (2024)
7.1 Probability paths and velocities . . . 70
7.2 Conditional paths and the marginalisation trick . . . 72
7.3 Training . . . 73
7.4 What flow matching actually adds . . . 73
7.5 Kinetic optimality and related developments . . . 76
7.6 Continuous relaxations: embeddings, simplices and latents . . . 77

8 Sampling and Decoding . . . **82**
Guanghan Wang et al. (ReMDM); Jaeyeon Kim et al.; Luhan Tang et al.; Campbell et al.; Liu et al. (DCD) (2026)
8.1 The problem, stated precisely . . . 82
8.2 Integrators . . . 84
8.3 Decoding order: which tokens to commit . . . 85
8.4 Revision: remasking and correctors . . . 86
8.5 Modelling the dependence . . . 87
8.6 Caching . . . 87
8.7 Distillation . . . 88
8.8 A practical recipe . . . 88

9 Guidance, Control and Reinforcement Learning . . . **93**
Hunter Nisonoff, Junhao Xiong, Stephan Allenspach and Jennifer Listgarten; Yair Schiff et al.; Oussama Zekri and Nicolas Boullé; Siyan Zhao et al. (2025)
9.1 Exact classifier guidance for CTMCs . . . 93
9.2 Guidance in the $\boldsymbol{x}_0$-parameterisation . . . 95
9.3 Reinforcement learning and reward fine-tuning . . . 96
9.4 Preference optimisation . . . 97
9.5 Other reward-based methods . . . 99

10 Building and Measuring . . . **103**
Shen Nie et al.; Jiacheng Ye et al.; Antonio Franca and Alexander Tong; Patrick Pynadath et al. (2026)
10.1 The backbone . . . 103
10.2 Initialising from an autoregressive model . . . 103
10.3 Scaling . . . 104
10.4 Block diffusion: interpolating with autoregression . . . 105
10.5 Variance: the recurring practical problem . . . 105
10.6 Practical pitfalls . . . 105
10.7 The open-model landscape . . . 106
10.8 Evaluation: metrics that lie and metrics that don't . . . 106

11 Diffusion Language Models . . . **115**
Shen Nie et al. (LLaDA); Jiacheng Ye et al. (Dream); Jaeyeon Kim et al.; Marianne Arriola et al. (2026)
11.1 The state of play . . . 115
11.2 Capabilities that come for free . . . 116
11.3 What is genuinely hard . . . 116
11.4 Multimodal diffusion language models . . . 117
11.5 Code generation . . . 117
11.6 Reasoning . . . 117
11.7 Reading the recent literature critically . . . 118

12 Molecular, Graph and Biological Design . . . **120**
Clément Vignac et al.; Seul Lee et al.; Sarah Alamdari et al.; Andrew Campbell et al.; Hannes Stark et al. (2025)
12.1 Graphs: DiGress . . . 120
12.2 Jointly 2D and 3D: MiDi . . . 122
12.3 Strings and fragments: GenMol . . . 123
12.4 Property optimisation . . . 123
12.5 Evaluation . . . 124
12.6 A representative map . . . 124
12.7 Biological sequences: proteins, DNA and RNA . . . 124

13 Theory: Convergence, Expressivity and Limits . . . **130**
Yuchen Liang, Renxiang Huang, Lifeng Lai, Ness Shroff and Yingbin Liang; Guhao Feng et al.; Jaeyeon Kim et al. (2025)
13.1 Convergence: how many steps? . . . 130
13.2 Expressivity: the token/sequence separation . . . 131
13.3 Hardness: masked diffusion trains on harder subproblems . . . 131
13.4 Tightness of the bound . . . 132
13.5 Schedule optimality . . . 132
13.6 Bounding the factorisation error . . . 132
13.7 Sampler correctness, formally . . . 133
13.8 Summary table . . . 134

14 A Research Playbook . . . **136**
Synthesis; practical guidance (2026)
14.1 Build the smallest honest thing first . . . 136
14.2 Choosing a problem . . . 136
14.3 Baselines you must include . . . 137
14.4 Experimental hygiene . . . 137
14.5 Fifteen project ideas . . . 137

14.6 How to position a paper . . . 139

A Notation . . . 141
A.1 Spaces and variables . . . 141
A.2 Processes . . . 141
A.3 Learned objects and losses . . . 142
A.4 Operators . . . 142
A.5 Abbreviations . . . 142
A.6 A reading aid: the same object in four notations . . . 142

B Complete Derivations . . . 144
B.1 The diffusion ELBO telescopes . . . 144
B.2 The D3PM posterior . . . 144
B.3 The masked posterior and the masked NELBO . . . 145
B.4 Time reversal of a CTMC . . . 145
B.5 The continuous-time ELBO integrand . . . 145
B.6 Denoising score entropy recovers the marginal score . . . 146
B.7 Masked diffusion equals any-order autoregression . . . 146
B.8 Closed-form kernels for uniform and absorbing noise . . . 146
B.9 ReMDM's validity bound . . . 147
B.10 Exact guidance . . . 147

C A Working Masked Diffusion Model . . . 148
C.1 The model . . . 148
C.2 Decoding experiments . . . 151
C.3 Checking the theory . . . 152
C.4 Other listings in this book . . . 161
C.5 Scaling this up . . . 161

D Resources and a Reading Order . . . 162
D.1 A reading order . . . 162
D.2 Codebases . . . 162
D.3 Datasets and benchmarks . . . 163
D.4 A compute-budget guide . . . 163
D.5 Where to follow the field . . . 163

Bibliography . . . 165

How to use this course

What this is

This is a course, not a survey. A survey tells you what exists; a course tries to put you in a position to do the work yourself. The target reader is someone who knows deep learning and probability at the level of a strong master's student or first-year PhD student, has seen continuous diffusion models at least once, and now wants to start a research project on *discrete* diffusion — most likely on molecular or biological sequence generation, or on diffusion language models.

The organising principle is that there are really only a handful of ideas in this field, and they recur in different notation. Those ideas are:

1. A **forward process** that destroys structure in a way you can write down in closed form. On finite state spaces this is a Markov chain, and the design space is the set of transition (or rate) matrices.
2. A **reverse process** that is determined, exactly, by the forward process plus one learnable object. In continuous space that object is the score $\nabla_{\boldsymbol{x}} \log p_t$. On discrete spaces it is a *ratio* of probabilities of neighbouring states — the *concrete score* — or equivalently a posterior over clean data.
3. A **training objective** that is a variational bound on the log-likelihood and, after enough algebra, collapses into a weighted cross-entropy.
4. A **sampler** that integrates the reverse process approximately, and whose approximation error — not the model's — is often what limits quality.

Almost every paper in the field is a specific choice in one of these four slots. Once you can see which slot a paper is pushing on, the literature becomes much smaller than it looks.

A warning about this field

Discrete diffusion is moving quickly and, as of late 2026, is in a phase where several widely repeated claims are being revised. Three in particular are worth flagging at the outset, because they shape how you should read everything that follows.

- **Masked diffusion is not a new objective.** The continuous-time bound for absorbing-state diffusion is *exactly* the any-order autoregressive objective that predates it (Uria et al., 2014; Hoogeboom et al., 2022; Ou et al., 2025). This is proved in Chapter 6 and verified numerically. It does not make masked diffusion uninteresting, but it does mean that "diffusion" is doing less work in the explanation than the name suggests.
- **Most reported speedups are sampler claims, not model claims.** The headline advantage of discrete diffusion is parallel generation. But parallel updates assume conditional independence across the positions being updated simultaneously, and natural data violates that. Recent sampler-centric evaluations show the error persists even with an *exact* denoiser (Tang et al., 2026). Chapter 8 is largely about this.
- **Generative perplexity is a broken metric.** It can be driven below that of human text by zero-parameter samplers that emit periodic garbage (Franca and Tong, 2026). If your paper's main table is generative perplexity, the paper is not yet finished. Section 10.8 gives the alternatives.

I have tried throughout to separate (a) results that are theorems, (b) results that are robust empirical findings, and (c) folklore. Where I am uncertain, I say so.

Conventions

The source card. Each chapter opens with a card listing the primary sources it is built on, the year of the most recent one, and tags. The authors and year also appear under the chapter's entry in the table of contents, so the contents page doubles as a reading list.

Boxes. *Intuition* boxes state in words what an equation is doing before the algebra starts; read these first if you are skimming. *Key takeaways* boxes at the end of each chapter are the minimum you should retain. *Open questions and research directions* boxes close each chapter and are the point of the whole exercise: they are problems that, as far as I can establish from the literature, nobody has settled.

Notation. Chapter A is a notation table; keep a finger in it. Two conventions cause endless confusion in this literature and are worth stating now.

- **Row versus column stochastic.** Discrete-time papers (D3PM and descendants) write $\boldsymbol{x}$ as a *row* one-hot vector and use $\boldsymbol{x}\mathbf{Q}$, so $\mathbf{Q}_{ij} = q(x_t = j \mid x_{t-1} = i)$ and rows sum to one. Continuous-time papers often write $\mathrm{d}p_t/\,\mathrm{d}t = \mathbf{Q}_t p_t$ with p_t a *column* vector, so columns of $\mathbf{Q}$ sum to zero. I follow each literature's own convention within its chapter and flag the switch every time, meaning that the first rmQ (transition kernel) is different from the second matrix $\mathbf{Q}_t$ (generator).
- **Time direction.** $t = 0$ is clean data and $t = 1$ (or $t = T$) is pure noise. Generation runs from $t = 1$ down to $t = 0$. Some papers — notably the flow-matching literature — use the opposite convention, with $t = 1$ the data. I use the diffusion convention everywhere and translate when discussing flows.

Code. Every listing in this book was executed before it was typeset, and the printed numbers are the numbers the code produced. The listings are deliberately dependency-light: `numpy`, `scipy` and `torch`, nothing else. They are teaching code, chosen for transparency over speed; Chapter D points to the production repositories. Chapter C contains one complete, trainable masked diffusion model in about ninety lines.

Reading paths. If you have two weeks: Chapters 2, 3, 5, 6 and 8, then whichever of Chapters 11 and 12 matches your project. If you have a term, read it in order. If you only want to start running experiments today, read Chapter C first and come back.

> **On trusting this document**
>
> Every technical claim about a specific paper here was checked against that paper. Where a paper's own wording matters I quote it. Nevertheless: this document is a secondary source, written at a moment when the field is changing monthly. Before you build on any claim in a paper of your own, read the primary source. Chapter D lists every one of them with a link.

CHAPTER 1

The Case for Discrete Diffusion

CHAPTER

Authors	Sohl-Dickstein et al.; Austin et al.; Lou et al.; Nie et al.
Year	2026
Published	Orientation and motivation
Tags	motivation, autoregressive models, parallel generation, landscape

1.1. The problem

A very large fraction of the data we care about generating is discrete and sequential: text, source code, SMILES and SELFIES strings, protein and nucleotide sequences, molecular graphs with categorical node and edge types, tokenised images and audio. For a decade the default answer has been the autoregressive (AR) model: impose an order on the coordinates, factorise

$$p(\boldsymbol{x}) = \prod_{\ell=1}^{L} p(x^{\ell} \mid x^{1}, \ldots, x^{\ell-1}), \tag{1.1}$$

and fit each conditional with a neural network trained by maximum likelihood. This exactly explains why we train LLMs to maximize the probability of each token in the sequence, and not just on the final token.

1. Generation is inherently serial. Sampling L tokens requires L forward passes. You can make each pass cheaper, you can speculate, you can cache, but the dependency chain has length L. For a 1000-token completion this is 1000 sequential steps regardless of how much parallel hardware you own.

2. The order is a modelling commitment. Equation (1.1) is exact for *any* ordering, but a model with finite capacity trained on one ordering learns that ordering's conditionals and no others. This matters concretely. A left-to-right model cannot be asked "what goes in the middle" without special training. The "reversal curse" — a model that has learned "A is B" failing to answer "what is B?" — is in part an artefact of a fixed factorisation order (Berglund et al., 2024; Nie et al., 2025a). In chemistry the problem is sharper still: the canonical SMILES ordering of a molecule is an artefact of a depth-first traversal and carries no chemical meaning, so a left-to-right model is committed to an arbitrary order over an object (a molecule) that has none (Lee et al., 2025).

3. There is no mechanism for revision. Once a token is emitted it is conditioned on forever. An AR model that makes an error at position 3 must build the rest of the sequence on top of it. Humans writing text, and chemists designing molecules, revise.

4. Planning is awkward. Tasks whose solution is a globally consistent assignment rather than a left-to-right story — Sudoku, constraint satisfaction, the "Countdown" arithmetic puzzle, retrosynthesis — fit the AR factorisation badly. Empirically, diffusion language models of comparable size are markedly better at exactly these tasks: Dream-7B scores 81.0 on Sudoku against 21.0 for

the autoregressive Qwen2.5-7B it was initialised from, and 16.0 against 6.2 on Countdown, while being roughly comparable on standard knowledge benchmarks (Ye et al., 2025).

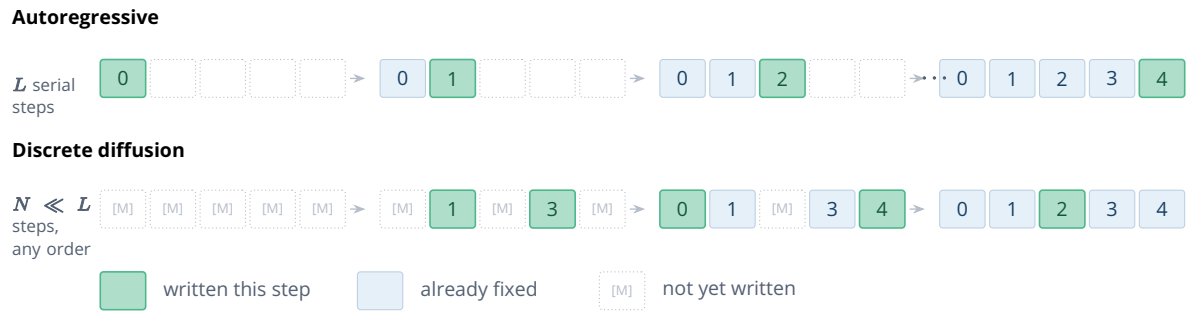


Figure 1.1 The structural difference. An autoregressive model writes one token per forward pass, always left to right, and can never revise: generation costs L sequential steps whatever hardware you have. A discrete diffusion model updates *every* position at every step and commits tokens in an order of its own choosing, so it can stop after $N \ll L$ steps — at the price that tokens committed together are drawn from a product of marginals rather than the joint (Remark 2.4). Which of the two effects dominates is the subject of Chapter 8.

Intuition: what diffusion buys you on discrete data

An autoregressive model answers one question, L times: "given this prefix, what is the next token?" A discrete diffusion model answers a much larger family of questions: "given *this arbitrary subset* of the tokens, what are all the others?" That is strictly more information, it is order-agnostic, and it supports infilling, revision and parallel decoding by construction. The price is that you are asking the network to learn exponentially many conditionals with the same parameter budget — which is the central tension of the field, and the subject of Chapters 6 and 13.

1.2. Why you cannot just use a Gaussian

The obvious move is to take a continuous diffusion model and apply it to discrete data. This fails, and understanding exactly *how* it fails tells you what a discrete formulation has to supply.

A Gaussian diffusion needs a vector space: the forward process $\boldsymbol{z}_t = \sqrt{\alpha_t}\,\boldsymbol{x} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}$ requires that scalar multiples and sums of data points mean something. For a token index, they do not. The three standard repairs each introduce their own problem.

1. **Diffuse in an embedding space** and round back at the end (Li et al., 2022). Now the model must simultaneously learn an embedding geometry and a denoiser in it, and the rounding step is a non-differentiable cliff: almost all of the diffusion trajectory is spent in regions that decode to the same token, so the signal about *which* token is concentrated in a vanishingly small part of the noise schedule.
2. **Diffuse on the probability simplex.** Linear interpolation between a noise point and a vertex of the simplex has pathological behaviour: Stark et al. (2024) prove that for $t > 1/k$ the converged posterior has support on at most $k-1$ vertices, so the marginal vector field becomes increasingly discontinuous as the vocabulary grows, and sampling becomes acutely sensitive to step size.
3. **Diffuse in bits** (Chen et al., 2023). Clean, but it imposes an arbitrary binary code on the vocabulary, so the noise process treats token 7 and token 8 as near-neighbours because `0111` and `1000` differ in four bits while `0111` and `0110` differ in one.

Pynadath et al. (2025) give the cleanest diagnosis. Gaussian noise applied to a one-hot or embedded token corrupts it through two separate mechanisms — *discrete identity corruption* (which token the argmax picks) and *continuous rank degradation* (how the logits are ordered below the argmax) — and these two scale differently with vocabulary size. The result is a tension: a schedule that gives useful signal about the continuous geometry gives almost none about the discrete identity, and vice versa. They call the resulting failure a lack of *token identifiability*.

The alternative is to stop pretending and build the forward process directly out of the only structure a finite set actually has: a Markov chain on it. That is discrete diffusion.

1.3. A map of the field

Table 1.1 is the map. Every row is a choice of forward process and parameterisation; the rest of the book works down it.

Table 1.1 The main families of discrete generative diffusion, by forward process. "Learned object" is what the network actually outputs.

Family	Forward process	Learned object	Representative work
Discrete-time categorical	Markov chain with transition matrix $\mathbf{Q}_t$	$p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_t)$	Sohl-Dickstein et al. (2015); Hoogeboom et al. (2021); Austin et al. (2021)
Masked / absorbing (continuous time)	each token $\to$ `[MASK]` at rate $\beta(t)$	$p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_t)$, i.e. a classifier	Lou et al. (2024); Sahoo et al. (2024); Shi et al. (2024)
Uniform / interpolating	each token $\to$ uniform (or any $\boldsymbol{\pi}$)	concrete score or $\boldsymbol{x}_0$-posterior	Schiff et al. (2025); Sahoo et al. (2025)
General CTMC	arbitrary rate matrix $\mathbf{R}_t$	concrete score $p_t(\boldsymbol{y})/p_t(\boldsymbol{x})$	Campbell et al. (2022); Lou et al. (2024)
Discrete flow matching	prescribed probability path p_t	probability velocity / $\boldsymbol{x}_1$-posterior	Campbell et al. (2024); Gat et al. (2024)
Continuous relaxation	Gaussian in embedding / simplex / bits	$\boldsymbol{\epsilon}$ or $\boldsymbol{x}_0$ in the relaxed space	Li et al. (2022); Dieleman et al. (2022); Stark et al. (2024)

Two things about this table are worth saying immediately, because they are the main structural facts about the field.

First, **the masked/absorbing row has won in practice**, at least for language. Every large open diffusion language model — LLaDA (Nie et al., 2025b), Dream (Ye et al., 2025), LLaDA-MoE, Seed Diffusion — is a masked diffusion model. The reasons are that its objective simplifies to a plain weighted cross-entropy on masked positions (Chapter 5), that it can be initialised from an autoregressive checkpoint, and that it is numerically far better behaved than uniform noise.

Second, **this may be a local optimum**. Sahoo et al. (2026) performed the first scaling-law study across uniform-state, interpolating and masked discrete diffusion, and found that at 1.7B parameters uniform-state diffusion *outperforms* both autoregressive and masked diffusion models on GSM8K despite having worse validation perplexity — which is also a warning about perplexity as a cross-family metric. The field's consolidation on masking is recent and is being actively contested.

1.4. What discrete diffusion is actually good at

It is worth being blunt about where the evidence currently stands, because the marketing and the measurements diverge.

Established. Order-agnostic conditioning and infilling, without task-specific training. Controllable generation via guidance on discrete states, which has a clean exact formulation (Chapter 9) and works well on molecules and regulatory DNA. Planning-style and constraint satisfaction tasks. Likelihoods on small-vocabulary scientific data (molecules, DNA), where uniform-state diffusion is genuinely state of the art (Schiff et al., 2025). Competitive-but-not-better likelihoods on text: masked diffusion models approach autoregressive perplexity to within roughly 15–25% at matched scale (Sahoo et al., 2024).

Contested. The speed advantage. A diffusion model with N denoising steps on an L-token sequence costs N forward passes over the *whole* sequence, so it is only cheaper than AR if $N \ll L$ and if the per-pass cost is comparable. With a fixed small N the parallel updates introduce distributional error that does not go away with a better model (Tang et al., 2026), and theory says that in the worst case the number of steps needed for low *sequence*-level error must scale linearly with L (Feng et al., 2025). The honest statement is that discrete diffusion offers a *tunable* quality/compute trade-off that AR does not, and that the useful operating points are task-dependent.

Not yet established. That diffusion language models will match autoregressive ones at frontier scale. The largest publicly documented open models are in the 7–8B range; commercial systems (Mercury, Gemini Diffusion) report strong latency numbers but limited public detail.

Key takeaways

- Autoregressive modelling imposes serial generation, a fixed factorisation order, no revision mechanism, and a poor fit to globally-constrained tasks. Discrete diffusion attacks all four.
- Gaussian diffusion cannot be transplanted to discrete data without introducing either a rounding cliff, a degenerate vector field on the simplex, or an arbitrary metric on the vocabulary. The clean fix is to build the forward process as a Markov chain on the finite state space itself.
- Every discrete diffusion method is a choice of (i) forward Markov process, (ii) learned object, (iii) variational objective, (iv) sampler. Table 1.1 is the taxonomy; the book works down it.
- Masked (absorbing-state) diffusion dominates practice, for reasons of objective simplicity and numerical stability rather than any proven superiority. Recent scaling studies suggest this consolidation is premature.

Open questions and research directions

1. **Is there a principled way to choose the forward process for a given data type?** We have uniform, absorbing, marginal-matched (Vignac et al., 2023a) and learned schedules, and the choice is made empirically every time. Nothing in the literature predicts, from properties of p_{data} (vocabulary size, sparsity, correlation length), which

corruption process will train best. A theory that does — even a heuristic with one tunable — would be immediately useful.

2. **What is the right notion of "diffusion advantage"?** The field reports perplexity (a likelihood property) and latency (a sampler property) and treats them as a single axis. A principled complexity-theoretic statement of the form "distributions in class $\mathcal{C}$ are learnable in $o(L)$ parallel steps but require $\Omega(L)$ serial steps" does not exist for any non-trivial $\mathcal{C}$. Feng et al. (2025) give one side of this for hidden Markov models; the positive side is open.
3. **Hybrid factorisations.** Block diffusion (Arriola et al., 2025) and Esoteric LMs interpolate between AR and diffusion by hand, choosing a block size. Whether the optimal factorisation can be *learned* per-input — a model that decides how much to parallelise based on the content — is barely explored.
4. **Non-sequence discrete data.** Almost all of the methodology assumes a sequence of exchangeable-position categorical variables. Objects with non-trivial symmetry (graphs up to isomorphism, molecules up to automorphism, sets) are handled by bolting equivariant architectures onto a sequence-shaped diffusion. Whether the *forward process* should itself respect the symmetry is an open design question with very little work on it.

CHAPTER 2

Mathematical Foundations

CHAPTER

Authors	Norris; Cover & Thomas; Anderson; Ho et al.; Song et al.
Year	2026
Published	Toolkit: categorical variables, Markov chains, generators, time reversal
Tags	categorical distributions, Markov chains, generators, time reversal, Fokker–Planck, ELBO

This chapter assembles the machinery. If you are comfortable with continuous-time Markov chains, generator matrices and variational bounds, skim it for notation and move on. Everything here is used repeatedly.

2.1. Categorical variables and the simplex

Fix a finite *vocabulary* $\mathcal{V} = \{1, \dots, K\}$. A sequence of length L lives in $\mathcal{X} = \mathcal{V}^L$, so $|\mathcal{X}| = K^L$: for a 50,000-token vocabulary and a 1024-token context this is 50000^{1024}, which is the only number you need to remember to understand why everything in this field factorises across positions.

It is convenient to identify a token $x \in \mathcal{V}$ with its one-hot vector $\mathbf{e}_x \in \{0,1\}^K$. Then a distribution over $\mathcal{V}$ is a point in the probability simplex

$$\Delta^K = \Big\{ p \in \mathbb{R}^K : p_i \geq 0, \textstyle\sum_i p_i = 1 \Big\}, \tag{2.1}$$

whose vertices are exactly the one-hot vectors. We write $\mathrm{Cat}(x; p)$ for the categorical distribution with probability vector p, so $\mathrm{Cat}(x; \mathbf{e}_y) = \delta_{xy}$.

The key algebraic fact we exploit constantly is that **expectations of one-hot vectors are probability vectors**, and therefore that an inner product with a one-hot vector picks out a coordinate:

$$\langle p, \mathbf{e}_x \rangle = p_x, \qquad \text{so} \qquad \log \langle p, \mathbf{e}_x \rangle = \log p_x \tag{2.2}$$

is the log-likelihood of observing x. Nearly every loss in this book is some weighted average of Equation (2.2).

2.1.1. The algebra of Cat

Two structural facts about Cat are used on almost every page, and both are worth isolating because they are what let the rest of the book proceed by linear algebra rather than by case analysis.

Proposition 2.1 (Cat is affine in its parameter). For $p_1, \dots, p_n \in \Delta^K$ and weights $\lambda_i \geq 0$ with $\sum_i \lambda_i = 1$,

$$\mathrm{Cat}\Big(x; \textstyle\sum_i \lambda_i p_i\Big) \;=\; \sum_i \lambda_i \, \mathrm{Cat}(x; p_i). \tag{2.3}$$

That is, *mixing the parameters and mixing the distributions are the same operation*.

Proof. $\mathrm{Cat}(x; \boldsymbol{p}) = \langle \boldsymbol{p}, \mathbf{e}_x \rangle$ by Equation (2.2), and the inner product is linear in $\boldsymbol{p}$. Non-negativity and normalisation of $\sum_i \lambda_i \boldsymbol{p}_i$ follow from convexity of Δ^K. ∎

This is special. It fails for almost every other parametric family — a mixture of Gaussians is not a Gaussian — and it is the reason every forward process in this book can be written as a single Cat with an interpolated parameter rather than as an explicit mixture. Read in reverse, Equation (2.3) tells you how to *interpret* any expression of the form $\mathrm{Cat}(x;\, \alpha \boldsymbol{u} + (1-\alpha)\boldsymbol{v})$: with probability α draw from $\boldsymbol{u}$, otherwise from $\boldsymbol{v}$. Every forward process in Chapters 3 and 5 is an instance.

Proposition 2.2 (One-hot selection). If $\mathbf{e}_y$ is a one-hot row vector and $\mathbf{M}$ is any $K \times K$ matrix, then $\mathbf{e}_y \mathbf{M}$ is the yth row of $\mathbf{M}$. Consequently $\mathrm{Cat}(x; \mathbf{e}_y \mathbf{Q})$ is "the categorical distribution given by row y of $\mathbf{Q}$".

The second proposition is why transition matrices appear at all: a lookup written as a matrix product composes under multiplication, so t steps of a chain become $\mathbf{Q}_1 \cdots \mathbf{Q}_t$ rather than t nested lookups. It is bookkeeping, but it is the bookkeeping that makes closed-form marginals possible.

Remark 2.3 (Two conventions). Some papers write $\mathrm{Cat}(x; \boldsymbol{p})$ with x an integer index rather than a one-hot vector. The distribution is the same, but only the one-hot convention makes $\mathbf{e}_x \mathbf{Q}$ meaningful, and mixing the two is a reliable source of transposed-index errors. We use one-hot vectors throughout. Note also that Cat is the $n = 1$ case of the multinomial distribution; the "multinomial diffusion" of Hoogeboom et al. (2021) is categorical in this sense, one draw per position.

2.1.2. Factorised distributions

A distribution over $\mathcal{X} = \mathcal{V}^L$ is *factorised* (or product-form, or mean-field) if

$$p(\boldsymbol{x}) = \prod_{\ell=1}^{L} p^{\ell}(x^{\ell}). \tag{2.4}$$

Factorised distributions have $L(K-1)$ parameters instead of $K^L - 1$, so every tractable forward process in this book is factorised, and every practical sampler makes a factorised approximation *somewhere*.

Remark 2.4 (Where the error lives). It cannot be stressed too early: the forward process being factorised is harmless (it is a modelling choice and we have its exact marginals), but the *reverse* process is not factorised, and pretending it is, is the single largest source of error in discrete diffusion sampling. A two-token distribution putting mass $\tfrac{1}{2}$ on $(0,0)$ and $\tfrac{1}{2}$ on $(1,1)$ has marginals $(\tfrac{1}{2}, \tfrac{1}{2})$ at both positions; sampling both positions independently from the correct marginals puts half the mass off-support. Chapter 8 is largely a catalogue of ways to mitigate this.

2.2. Divergences on finite spaces

For distributions p, q on a finite set $\mathcal{S}$,

$$D_{\mathrm{KL}}(q \,\|\, p) = \sum_{s \in \mathcal{S}} q(s) \log \frac{q(s)}{p(s)} \;\geq 0, \tag{2.5}$$

$$\mathrm{TV}(q, p) = \tfrac{1}{2} \sum_{s \in \mathcal{S}} |q(s) - p(s)| \;\leq\; \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(q \,\|\, p)}, \tag{2.6}$$

the inequality in Equation (2.6) being Pinsker's. The KL divergence between two categorical distributions is available in closed form, which is exactly why the discrete-time ELBO of Chapter 3 is computable: each of its terms is a KL between two explicit probability vectors.

The function

$$K(a) = a(\log a - 1) \tag{2.7}$$

appears in Section 4.7; it is the convex generator whose Bregman divergence gives the score entropy loss, and the only property we need is that $a \mapsto b - a\log b + K(a)$, viewed as a function of $b > 0$ for fixed $a > 0$, is minimised at $b = a$ with minimum value 0.

2.3. Variational bounds

The common skeleton of every objective in this book is the following. Suppose we have a latent-variable model $p_\theta(\boldsymbol{x}_0) = \sum_{\boldsymbol{z}} p_\theta(\boldsymbol{x}_0, \boldsymbol{z})$ and an inference distribution $q(\boldsymbol{z} \mid \boldsymbol{x}_0)$. Then

$$-\log p_\theta(\boldsymbol{x}_0) \;\le\; \mathbb{E}_{q(\boldsymbol{z}\mid\boldsymbol{x}_0)}\left[-\log \frac{p_\theta(\boldsymbol{x}_0, \boldsymbol{z})}{q(\boldsymbol{z} \mid \boldsymbol{x}_0)}\right] \;=:\; \mathcal{L}(\boldsymbol{x}_0), \tag{2.8}$$

with gap exactly $D_{\mathrm{KL}}(q(\boldsymbol{z} \mid \boldsymbol{x}_0) \,\|\, p_\theta(\boldsymbol{z} \mid \boldsymbol{x}_0))$. In diffusion models $\boldsymbol{z} = (\boldsymbol{x}_1, \dots, \boldsymbol{x}_T)$ is the whole noising trajectory and q is the *fixed* forward process, so there is no inference network to learn — which is what makes diffusion training stable.

> **Intuition: why the bound telescopes**
>
> Both p_θ and q are Markov chains over the same trajectory. Writing each as a product of one-step kernels and grouping terms with the same $\boldsymbol{x}_t$ turns the single high-dimensional expectation in Equation (2.8) into a *sum of T small, independent problems*, each of which is a KL between two categorical distributions over a single step. This is the only reason diffusion training is practical, and it is worth doing the manipulation once by hand: Chapter B does it in full.

2.4. Discrete-time Markov chains

Let $\boldsymbol{x}_0, \boldsymbol{x}_1, \dots, \boldsymbol{x}_T$ be a Markov chain on $\mathcal{V}$ with one-hot row vectors. A transition matrix $\mathbf{Q}_t \in [0,1]^{K\times K}$ with $[\mathbf{Q}_t]_{ij} = q(x_t = j \mid x_{t-1} = i)$ and unit row sums gives

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\ \mathbf{e}_{x_{t-1}}\mathbf{Q}_t\big)\,, \qquad q(x_t \mid x_0) = \mathrm{Cat}\big(x_t;\ \mathbf{e}_{x_0}\overline{\mathbf{Q}}_t\big)\,, \tag{2.9}$$

where $\overline{\mathbf{Q}}_t = \mathbf{Q}_1\mathbf{Q}_2\cdots\mathbf{Q}_t$. The second equality is the *whole point* of a Markov forward process: we can jump to any noise level in one matrix multiply, and so train on a random t without simulating.

A distribution $\boldsymbol{\pi}$ is *stationary* if $\boldsymbol{\pi}\mathbf{Q} = \boldsymbol{\pi}$. Two cases dominate:

Example 2.5 (Uniform noise). $\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \frac{\beta_t}{K}\mathbf{1}\mathbf{1}^\top$. Each token is resampled uniformly with probability β_t. The matrix is doubly stochastic, so the stationary distribution is uniform and the chain is *ergodic*: every state remains reachable from every other at all times.

Example 2.6 (Absorbing (masked) noise). Extend the vocabulary with one extra symbol `[MASK]` and set $\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{1}\mathbf{e}_{\texttt{[MASK]}}^\top$. Each token jumps to `[MASK]` with probability β_t and is then stuck: `[MASK]` is absorbing. The stationary distribution is $\delta_{\texttt{[MASK]}}$, the chain is *not* ergodic, and the reverse process is correspondingly simpler — there is only one kind of transition to model, namely "reveal a masked token". This is the structure that makes Chapter 5 so clean.

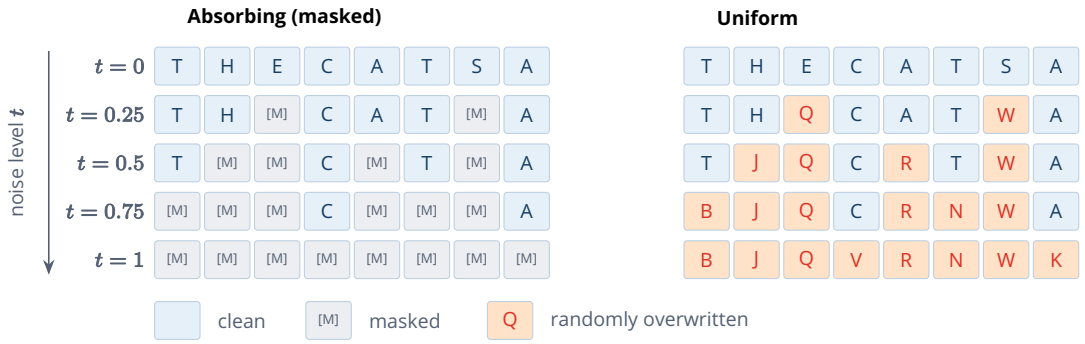


Figure 2.1 The two forward processes of Examples 2.5 and 2.6, acting independently on each position of one sequence. **Left:** absorbing noise replaces a token with `[MASK]` and never revisits it, so the reverse process is told exactly *where* the missing information is and only has to fill blanks. **Right:** uniform noise overwrites tokens with random ones, so the reverse process must infer both which positions are wrong and what belongs there — strictly more work for the same parameter budget, but it can also revise its own output (Section 5.8).

Listing 2.1 builds both and checks the identities in Equation (2.9) numerically, along with the posterior we will derive in Chapter 3.

Listing 2.1 Transition matrices, marginals and the exact posterior, checked against brute-force Bayes.

```python
"""Verify D3PM transition matrices, marginals, posterior, and ELBO identities."""
import torch
torch.manual_seed(0)

K = 6  # vocabulary size (last index is [MASK] in the absorbing case)


def uniform_Q(beta):
    """Q = (1-beta) I + (beta/K) 11^T  -- doubly stochastic, uniform stationary."""
    return (1.0 - beta) * torch.eye(K) + beta / K * torch.ones(K, K)


def absorbing_Q(beta, mask_id=K - 1):
    """Q = (1-beta) I + beta * 1 e_m^T  -- all mass flows to the mask state."""
    Q = (1.0 - beta) * torch.eye(K)
    Q[:, mask_id] += beta
    Q[mask_id, :] = 0.0
    Q[mask_id, mask_id] = 1.0           # [MASK] is absorbing
    return Q


def cumprod_Q(Qs):
    """Qbar_t = Q_1 Q_2 ... Q_t for every t."""
    out, acc = [], torch.eye(K)
    for Q in Qs:
        acc = acc @ Q
        out.append(acc.clone())
    return out


T = 50
betas = torch.linspace(1e-3, 0.4, T)

for name, maker in [("uniform", uniform_Q), ("absorbing", absorbing_Q)]:
    Qs = [maker(float(b)) for b in betas]
```

```
    Qbars = cumprod_Q(Qs)
    # rows are probability distributions
    for Q in Qs + Qbars:
        assert torch.allclose(Q.sum(-1), torch.ones(K), atol=1e-6)
    # q(x_t | x_0) from Qbar must equal repeated single-step application
    p = torch.zeros(K); p[1] = 1.0
    for t in range(T):
        p = p @ Qs[t]
    assert torch.allclose(p, torch.zeros(K).index_put_(
        (torch.tensor(1),), torch.tensor(1.0)) @ Qbars[-1], atol=1e-6)
    print(f"{name}: marginal consistency OK; q(x_T|x_0=1) = {p.numpy().round(4)}")

# ---- posterior q(x_{t-1} | x_t, x_0) ----------------------------------------
Qs = [uniform_Q(float(b)) for b in betas]
Qbars = cumprod_Q(Qs)


def posterior(x0_onehot, xt_onehot, t):
    """Cat(p = (x_t Q_t^T) * (x_0 Qbar_{t-1}) / (x_0 Qbar_t x_t^T))."""
    left = xt_onehot @ Qs[t].T
    right = x0_onehot @ (Qbars[t - 1] if t > 0 else torch.eye(K))
    num = left * right
    return num / num.sum(-1, keepdim=True)


x0 = torch.zeros(K); x0[2] = 1.0
xt = torch.zeros(K); xt[4] = 1.0
t = 20
post = posterior(x0, xt, t)
# brute-force check: enumerate x_{t-1}
brute = torch.zeros(K)
for k in range(K):
    e = torch.zeros(K); e[k] = 1.0
    brute[k] = (x0 @ (Qbars[t - 1] if t > 0 else torch.eye(K)))[k] * (e @ Qs[t])[4]
brute = brute / brute.sum()
assert torch.allclose(post, brute, atol=1e-6)
print("posterior matches brute-force Bayes:", post.numpy().round(4))
```

Running it prints

```
     uniform: marginal consistency OK; q(x_T|x_0=1) = [0.1667 ... 0.1667]
    absorbing: marginal consistency OK; q(x_T|x_0=1) = [0. 0. 0. 0. 0. 1.]
posterior matches brute-force Bayes: [0.0263 0.0263 0.0634 0.0263 0.8314 0.0263]
```

confirming the two stationary distributions of Examples 2.5 and 2.6.

2.5. Continuous-time Markov chains

Continuous time is where the modern formulations live, because it removes the discretisation of the forward process as a hyperparameter and makes the reverse process an object you can integrate as accurately as you like.

> **Definition 2.7 (Generator / rate matrix).** A continuous-time Markov chain (CTMC) on $\mathcal{V}$ is specified by a family of *rate matrices* $\mathbf{Q}_t$. We use the **column convention**: $[\mathbf{Q}_t]_{yx} \geq 0$ for $y \neq x$ is the instantaneous rate of jumping from x to y, and the diagonal is fixed by

$[\mathbf{Q}_t]_{xx} = -\sum_{y \neq x} [\mathbf{Q}_t]_{yx}$, so every column sums to zero. The marginals obey the *Kolmogorov forward equation*

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t = \mathbf{Q}_t \, p_t, \qquad p_t \in \Delta^K \text{ a column vector.} \tag{2.10}$$

Remark 2.8 (The convention trap). Campbell et al. (2022) use the row convention, writing $\partial_t q_t(x) = \sum_y q_t(y) R_t(y, x)$ with $R_t(y, x)$ the rate $y \to x$ and *rows* summing to zero. Lou et al. (2024) use the column convention as above. The two differ by a transpose. Every sign error in this literature — including, in an earlier draft, in this book — traces back to this. When reading a paper, find one equation with an explicit sum and check which index is summed.

If $\mathbf{Q}_t = \overline{\sigma}'(t)\, \mathbf{Q}$ for a fixed matrix $\mathbf{Q}$ and a scalar *total-noise* schedule $\overline{\sigma}(t)$ with $\overline{\sigma}(0) = 0$, then Equation (2.10) integrates in closed form via the matrix exponential:

$$\boxed{\; p_{t|0} \;=\; \exp\!\big(\overline{\sigma}(t)\, \mathbf{Q}\big)\, p_0. \;} \tag{2.11}$$

This is the continuous-time analogue of $\overline{\mathbf{Q}}_t$, and it is the reason the uniform and absorbing cases are tractable: for both, the exponential has a two-line closed form.

Proposition 2.9 (Closed-form kernels). Let $\mathbf{Q}^{\text{unif}} = \mathbf{1}\mathbf{1}^\top - K\mathbf{I}$ and $\mathbf{Q}^{\text{abs}} = \mathbf{e}_{\texttt{[MASK]}}\mathbf{1}^\top - \mathbf{I}$ (whose `[MASK]` column is automatically zero, as it must be for an absorbing state). Writing $\alpha_t = e^{-\overline{\sigma}(t)}$,

$$\exp\!\big(\overline{\sigma}(t)\mathbf{Q}^{\text{unif}}\big) = e^{-K\overline{\sigma}(t)}\mathbf{I} + \tfrac{1}{K}\big(1 - e^{-K\overline{\sigma}(t)}\big)\mathbf{1}\mathbf{1}^\top, \tag{2.12}$$

$$\exp\!\big(\overline{\sigma}(t)\mathbf{Q}^{\text{abs}}\big) = \alpha_t \mathbf{I} + (1 - \alpha_t)\, \mathbf{e}_{\texttt{[MASK]}}\mathbf{1}^\top. \tag{2.13}$$

In words: under uniform noise a token survives with probability $e^{-K\overline{\sigma}(t)}$ and is otherwise uniform; under absorbing noise it survives with probability α_t and is otherwise `[MASK]`.

Proof. Both matrices satisfy a quadratic. For the uniform case put $\mathbf{P} = \frac{1}{K}\mathbf{1}\mathbf{1}^\top$, a projector ($\mathbf{P}^2 = \mathbf{P}$); then $\mathbf{Q}^{\text{unif}} = K(\mathbf{P} - \mathbf{I})$ and $\exp(sK(\mathbf{P}-\mathbf{I})) = e^{-sK}\exp(sK\mathbf{P}) = e^{-sK}\big(\mathbf{I} + (e^{sK}-1)\mathbf{P}\big)$, using $\mathbf{P}^n = \mathbf{P}$. Substituting $\mathbf{P}$ gives Equation (2.12). For the absorbing case put $\mathbf{N} = \mathbf{e}_{\texttt{[MASK]}}\mathbf{1}^\top$; then $\mathbf{N}^2 = \mathbf{e}_{\texttt{[MASK]}}(\mathbf{1}^\top\mathbf{e}_{\texttt{[MASK]}})\mathbf{1}^\top = \mathbf{N}$ since $\mathbf{1}^\top\mathbf{e}_{\texttt{[MASK]}} = 1$, so $\mathbf{N}$ is idempotent and $\exp(s(\mathbf{N}-\mathbf{I})) = e^{-s}\big(\mathbf{I} + (e^s - 1)\mathbf{N}\big) = e^{-s}\mathbf{I} + (1 - e^{-s})\mathbf{N}$, which is Equation (2.13). Both identities are verified numerically to machine precision in the code accompanying this chapter. ∎

Equation (2.13) deserves a name because it is used on nearly every page after Chapter 5: with $\alpha_t = e^{-\overline{\sigma}(t)}$, the masked forward marginal is

$$\boxed{\; q(z_t \mid x) = \mathrm{Cat}\big(z_t;\; \alpha_t x + (1 - \alpha_t)\mathbf{m}\big), \;} \tag{2.14}$$

with $\mathbf{m} = \mathbf{e}_{\texttt{[MASK]}}$. A linear schedule $\alpha_t = 1 - t$ is the default; a cosine schedule $\alpha_t = \cos(\frac{\pi t}{2})$ is also common, and Zhang and Syed (2025) show that the cosine schedule is in a precise sense Fisher–Rao optimal for masked diffusion (Chapter 13).

2.5.1. How fast does the forward process forget?

The matrix exponential in Equation (2.11) is governed entirely by the spectrum of $\mathbf{Q}$, and for the two kernels above the spectrum is as simple as a spectrum can be.

Proposition 2.10 (Spectra and exact contraction rates). With $\mathbf{Q}^{\text{unif}}$ and $\mathbf{Q}^{\text{abs}}$ as in Proposition 2.9:

1. $\operatorname{spec}(\mathbf{Q}^{\text{unif}}) = \{0\} \cup \{-K\}$ with $-K$ of multiplicity $K-1$; and for every $p_0 \in \Delta^K$, writing $\pi^{\text{unif}} = \frac{1}{K}\mathbf{1}$,

$$p_s - \pi^{\text{unif}} \;=\; e^{-Ks}\left(p_0 - \pi^{\text{unif}}\right), \quad \text{so} \quad \|p_s - \pi^{\text{unif}}\|_1 = e^{-Ks}\|p_0 - \pi^{\text{unif}}\|_1. \tag{2.15}$$

2. $\operatorname{spec}(\mathbf{Q}^{\text{abs}}) = \{0\} \cup \{-1\}$ with -1 of multiplicity $K-1$; and the *unabsorbed mass* decays by exactly the survival factor,

$$1 - p_s(\texttt{[MASK]}) \;=\; e^{-s}\left(1 - p_0(\texttt{[MASK]})\right). \tag{2.16}$$

Both are exact identities, not bounds.

Proof. $\mathbf{1}\mathbf{1}^\top$ is rank one with trace K, hence has spectrum $\{K\} \cup \{0\}^{K-1}$; subtracting $K\mathbf{I}$ shifts every eigenvalue by $-K$. Likewise $\mathbf{e}_{\texttt{[MASK]}}\mathbf{1}^\top$ is rank one with trace $\mathbf{1}^\top\mathbf{e}_{\texttt{[MASK]}} = 1$, hence spectrum $\{1\} \cup \{0\}^{K-1}$, and subtracting $\mathbf{I}$ gives $\{0\} \cup \{-1\}^{K-1}$. The eigenvalue 0 is in both cases simple with left eigenvector $\mathbf{1}^\top$ (zero column sums) and right eigenvector the stationary distribution. The identities then follow by reading off the closed forms: substituting p_0 into Equation (2.12) gives $p_s = e^{-Ks}p_0 + (1 - e^{-Ks})\pi^{\text{unif}}$, and rearranging gives Equation (2.15); substituting into Equation (2.13) gives $p_s = e^{-s}p_0 + (1 - e^{-s})\mathbf{e}_{\texttt{[MASK]}}$, whose [MASK] coordinate is $e^{-s}p_0(\texttt{[MASK]}) + 1 - e^{-s}$, which rearranges to Equation (2.16). The ℓ_1 statement holds because Equation (2.15) is coordinatewise. ∎

Intuition: why K and not 1

The factor K in Equation (2.15) is not cosmetic. A uniform jump lands on the state you were already in with probability $1/K$, so a fraction $1/K$ of all jumps does nothing; normalising $\mathbf{Q}^{\text{unif}}$ so that the *total* jump rate out of a state is $K-1$ rather than 1 is what makes the useful rate $O(1)$ and the mixing rate K. The absorbing kernel wastes nothing — every jump is informative, because it is a jump to [MASK] — and so mixes at rate exactly 1. Comparing the two at equal $\bar{\sigma}(t)$ therefore compares different amounts of actual corruption, which is one reason published uniform-versus-masked comparisons are hard to read (Section 10.8).

A second structural fact holds for *every* forward process, not just these two, and it is the reason the forward direction is always the easy one.

Proposition 2.11 (The forward process is a contraction in KL). Let p_t, p'_t be two solutions of Equation (2.10) driven by the same rates $\mathbf{Q}_t$, from different initial conditions p_0, p'_0. Then $t \mapsto D_{\mathrm{KL}}(p_t \,\|\, p'_t)$ is non-increasing. The same holds for total variation and for every f-divergence.

Proof. For $s < t$ the map $p_s \mapsto p_t$ is application of the stochastic transition kernel $\mathbf{P}_{t|s}$, which is the same linear map for both solutions. The data-processing inequality for f-divergences gives $D_f(\mathbf{P}_{t|s}p_s \,\|\, \mathbf{P}_{t|s}p'_s) \le D_f(p_s\|p'_s)$, i.e. $D_f(p_t\|p'_t) \le D_f(p_s\|p'_s)$. KL and TV are f-divergences. ∎

Taking $p'_0 = \pi$ stationary gives $D_{\mathrm{KL}}(p_t \,\|\, \pi)$ monotonically decreasing to zero: the forward process destroys information and never creates it. The generative problem is to invert a strict contraction, which is exactly why it requires a learned object — the reverse map is not a fixed kernel but depends on p_t itself, through Equation (2.17) below.

2.5.2. Time reversal

This is the theorem the whole field rests on.

Theorem 2.12 (Time reversal of a CTMC). Let x_t be a CTMC with rates $[\mathbf{Q}_t]_{yx}$ and marginals p_t, run on $[0, T]$. Then the reversed process $\tilde{x}_u := x_{T-u}$ is a CTMC whose rate of jumping from x to y at reverse-time u (forward-time $t = T - u$) is

$$\big[\widehat{\mathbf{R}}_t\big]_{yx} \;=\; \underbrace{\frac{p_t(y)}{p_t(x)}}_{\text{concrete score}} \cdot \big[\mathbf{Q}_t\big]_{xy}, \qquad y \neq x, \tag{2.17}$$

with the diagonal fixed so that columns sum to zero.

Proof sketch. Write the reverse generator as $[\widehat{\mathbf{R}}_t]_{yx} = [\mathbf{Q}_t]_{xy}\, p_t(y)/p_t(x)$ and verify directly that it reproduces the correct marginal evolution. Acting on p_t,

$$\begin{aligned}
\big(\widehat{\mathbf{R}}_t p_t\big)(y) &= \sum_{x \neq y} [\mathbf{Q}_t]_{xy} \frac{p_t(y)}{p_t(x)} p_t(x) \;-\; p_t(y) \sum_{x \neq y} [\widehat{\mathbf{R}}_t]_{xy} \\
&= p_t(y) \sum_{x \neq y} [\mathbf{Q}_t]_{xy} \;-\; p_t(y) \sum_{x \neq y} [\mathbf{Q}_t]_{yx} \frac{p_t(x)}{p_t(y)} \\
&= -[\mathbf{Q}_t]_{yy}\, p_t(y) \;-\; \sum_{x \neq y} [\mathbf{Q}_t]_{yx} p_t(x) \;=\; -\big(\mathbf{Q}_t p_t\big)(y),
\end{aligned} \tag{2.18}$$

where the third line uses the zero-column-sum property of $\mathbf{Q}_t$. Since $\frac{\mathrm{d}}{\mathrm{d}u} p_{T-u} = -\frac{\mathrm{d}}{\mathrm{d}t} p_t = -\mathbf{Q}_t p_t$, the reverse process with generator $\widehat{\mathbf{R}}_t$ has exactly the right marginals. Markovianity of the reversal follows from the standard reversal argument for Markov jump processes. ∎

Equation (2.17) can look like an algebraic coincidence. It is not: it is the statement that *reversing time means taking an adjoint*, and saying so makes the discrete and continuous cases literally the same theorem. Let the generator act on *functions* (observables) rather than on distributions,

$$(\mathcal{L}_t f)(x) \;=\; \sum_{y \neq x} [\mathbf{Q}_t]_{yx} \big(f(y) - f(x)\big) \;=\; \sum_{y} [\mathbf{Q}_t]_{yx} f(y), \tag{2.19}$$

the second equality because $\mathbf{Q}_t$ has zero column sums, and equip functions with the p_t-weighted inner product $\langle f, g \rangle_{p_t} = \sum_x f(x) g(x) p_t(x) = \mathbb{E}_{p_t}[fg]$.

Proposition 2.13 (Reversal reverses fluxes, and is an adjoint up to $\partial_t \log p_t$). Let $\widehat{\mathcal{L}}_t$ be the generator built from the reverse rates $\widehat{\mathbf{R}}_t$ of Theorem 2.12, and suppose $p_t > 0$.

1. **(Flux reversal.)** For all $y \neq x$,

$$\big[\widehat{\mathbf{R}}_t\big]_{yx}\, p_t(x) \;=\; \big[\mathbf{Q}_t\big]_{xy}\, p_t(y). \tag{2.20}$$

   In words: the probability flux $x \to y$ in the reverse process equals the flux $y \to x$ in the forward process, at the same instant. This determines $\widehat{\mathbf{R}}_t$ uniquely off the diagonal, so Equation (2.17) is not merely *a* valid reversal — it is the only rate matrix that balances fluxes.

2. **(Adjoint identity.)** For all functions f, g on $\mathcal{V}$,

$$\langle \mathcal{L}_t f,\, g \rangle_{p_t} \;=\; \big\langle f,\, \big(\widehat{\mathcal{L}}_t + \partial_t \log p_t\big) g \big\rangle_{p_t}, \tag{2.21}$$

   where $\partial_t \log p_t$ acts by pointwise multiplication. Equivalently $\widehat{\mathcal{L}}_t = \mathcal{L}_t^{*} - \partial_t \log p_t$, with $\mathcal{L}_t^{*}$ the adjoint in $L^2(p_t)$.

3. **(Stationary case.)** If $p_t = \pi$ with $\mathbf{Q}\pi = 0$ then $\partial_t \log p_t \equiv 0$ and $\widehat{\mathcal{L}} = \mathcal{L}^*$ exactly; if in addition detailed balance holds, $[\mathbf{Q}]_{yx}\pi(x) = [\mathbf{Q}]_{xy}\pi(y)$, then $\widehat{\mathcal{L}} = \mathcal{L}$ and the process is its own reversal.

Proof. (i) is Equation (2.17) multiplied through by $p_t(x)$. Uniqueness is immediate: Equation (2.20) is one linear equation per off-diagonal entry with a nonzero coefficient $p_t(x)$.

For (ii), use Equation (2.19) on the left and then the difference form on both sides. Writing $q_{yx} := [\mathbf{Q}_t]_{yx}$,

$$\begin{aligned}\langle \mathcal{L}_t f,\, g\rangle_{p_t} &= \sum_x p_t(x) g(x) \sum_{y\neq x} q_{yx}\big(f(y) - f(x)\big) \\ &= \sum_{x\neq y} q_{yx}\, p_t(x)\, g(x)\, f(y) \;-\; \sum_{x\neq y} q_{yx}\, p_t(x)\, g(x) f(x), \end{aligned} \tag{2.22}$$

while, using (i) in the form $[\widehat{\mathbf{R}}_t]_{xy} = q_{yx}p_t(x)/p_t(y)$,

$$\begin{aligned}\langle f,\, \widehat{\mathcal{L}}_t g\rangle_{p_t} &= \sum_y p_t(y) f(y) \sum_{x\neq y} \frac{q_{yx}p_t(x)}{p_t(y)}\big(g(x) - g(y)\big) \\ &= \sum_{x\neq y} q_{yx}\, p_t(x)\, g(x)\, f(y) \;-\; \sum_{x\neq y} q_{yx}\, p_t(x)\, f(y) g(y). \end{aligned} \tag{2.23}$$

The first terms agree, so with $h := fg$ the difference is

$$\langle \mathcal{L}_t f,\, g\rangle_{p_t} - \langle f,\, \widehat{\mathcal{L}}_t g\rangle_{p_t} = \sum_{x\neq y} q_{yx}\, p_t(x)\, h(y) \;-\; \sum_{x\neq y} q_{yx}\, p_t(x)\, h(x). \tag{2.24}$$

In the first sum, $\sum_{x\neq y} q_{yx}p_t(x) = (\mathbf{Q}_t p_t)(y) - q_{yy}p_t(y)$; in the second, $\sum_{y\neq x} q_{yx} = -q_{xx}$. The two q_{xx} terms cancel and what survives is $\sum_y h(y)\,(\mathbf{Q}_t p_t)(y) = \sum_y f(y)g(y)\,\partial_t p_t(y)$ by Equation (2.10), which is $\langle f,\, g\,\partial_t \log p_t\rangle_{p_t}$. This is Equation (2.21). Part (iii) follows by setting $\partial_t p_t = 0$ and, for the last claim, substituting detailed balance into Equation (2.17). ∎

Remark 2.14 (Why the correction term is the interesting one). It is tempting to state (ii) without the $\partial_t \log p_t$ term — the reversal of a *stationary* chain really is the plain adjoint, and that is the version in most textbooks. For a diffusion model the marginal is moving by construction, and the correction is not optional: dropping it changes the numbers (Listing C.3 checks both the identity and the failure of the uncorrected version). The term is the exact analogue of the schedule derivative $\bar\sigma'(t)$ that appears when you verify $-\bar\sigma'(t)\mathbf{Q}p_t = \widehat{\mathbf{R}}_t p_t$ numerically, and forgetting it is the single most common way to get a plausible-looking but wrong reverse sampler.

The deeper point is that Equation (2.20) is the real content, and it has an exact continuous counterpart. Remark 2.18 observed that the two *forward* equations are one equation, $\partial_t p_t = \mathcal{L}_t^* p_t$. The reversal statements line up the same way: on $\mathbb{R}^d$ the reverse-SDE drift of Equation (2.29) acquires the term $g^2\nabla \log p_t$ exactly where Equation (2.17) acquires the ratio $p_t(y)/p_t(x)$, and both arise from moving p_t across an inner product — an integration by parts on the manifold, a re-indexed double sum on the finite set. Gradient of a log, or ratio of two values: the same object again.

Listing 2.2 verifies Theorem 2.12 as an exact identity at several times, then integrates the reverse ODE numerically back from noise to data.

Listing 2.2 The forward CTMC, the exact time-reversal identity, and the score-entropy minimiser of Section 4.7.

```python
"""CTMC forward process, exact time reversal, and the score-entropy minimiser."""
import numpy as np
from scipy.linalg import expm

rng = np.random.default_rng(0)
S = 5                                          # five states

# ---------------------------------------------------------------------------- #
# 1. Uniform generator.  Convention:  dp/dt = sigma'(t) Q p,  with
#    Q[b, a] = rate(a -> b) for b != a, so columns of Q sum to zero.
# ---------------------------------------------------------------------------- #
Q = np.ones((S, S)) - S * np.eye(S)
assert np.allclose(Q.sum(0), 0)

sig = lambda t: 8.0 * t                        # total noise  sigma_bar(t)
dsig = lambda t: 8.0                           # its derivative
P = lambda t: expm(sig(t) * Q)                 # p_{t|0} = exp(sigma_bar(t) Q)

p0 = rng.dirichlet(np.ones(S) * 2.0)
pt = lambda t: P(t) @ p0
print("p_0 =", p0.round(4), "   p_1 =", pt(1.0).round(4), "(-> uniform)")


# ---------------------------------------------------------------------------- #
# 2. Exact time reversal.  The reverse process has rates
#       rate(b -> a)  =  sigma'(t) Q[b, a] * p_t(a) / p_t(b),
#    i.e. the forward rate reweighted by the ratio  p_t(a)/p_t(b)  -- the
#    *concrete score*.  This is the only thing the network has to learn.
# ---------------------------------------------------------------------------- #
def reverse_generator(t):
    p = pt(t)
    R = dsig(t) * Q.T * (p[:, None] / p[None, :])    # R[a,b] = s'Q[b,a] p_a/p_b
    np.fill_diagonal(R, 0.0)
    np.fill_diagonal(R, -R.sum(0))
    return R


# the reverse generator must satisfy  d/du p_{T-u} = R_{T-u} p_{T-u}, i.e.
#   -sigma'(t) Q p_t  ==  R_t p_t     for every t
for t in (0.05, 0.3, 0.7, 1.0):
    lhs = -dsig(t) * Q @ pt(t)
    rhs = reverse_generator(t) @ pt(t)
    assert np.abs(lhs - rhs).max() < 1e-12, t
print("time-reversal identity  -sigma'(t) Q p_t = R_t p_t  holds exactly")

# integrate the reverse ODE from t=T back to t=delta (early stopping)
T, delta, N = 1.0, 1e-3, 40000
dt = (T - delta) / N
q = pt(T).copy()
for k in range(N):
    t = T - k * dt
    q = q + dt * reverse_generator(t) @ q
    q = np.clip(q, 0, None); q /= q.sum()
print("reverse-simulated p_delta =", q.round(4),
      f"  max abs error vs p_0: {np.abs(q - p0).max():.2e}")

# ---------------------------------------------------------------------------- #
# 3. Denoising score entropy.  Its minimiser over s(x)_y is the true ratio
#    p_t(y)/p_t(x) -- the key fact that makes DSE a valid training objective.
# ---------------------------------------------------------------------------- #
```

```python
t = 0.25
Pt, p_t = P(t), pt(t)                          # Pt[x, x0] = p_t(x | x0)
post = (Pt * p0[None, :]) / p_t[:, None]       # post[x, x0] = p(x0 | x_t = x)

s_dse = np.zeros((S, S))                       # E_{x0 | x}[ p(y|x0)/p(x|x0) ]
for x in range(S):
    for y in range(S):
        if x != y:
            s_dse[x, y] = np.sum(post[x] * Pt[y] / Pt[x])

true_ratio = np.where(np.eye(S) == 0, p_t[None, :] / p_t[:, None], 0.0)
print(f"max |DSE minimiser - true ratio| = {np.abs(s_dse - true_ratio).max():.2e}")
assert np.abs(s_dse - true_ratio).max() < 1e-10
```

Its output:

```
time-reversal identity -sigma'(t) Q p_t = R_t p_t holds exactly
reverse-simulated p_delta = [0.1768 0.2508 0.1061 0.3744 0.092 ]
                  max abs error vs p_0: 7.16e-03
          max |DSE minimiser - true ratio| = 3.33e-16
```

The residual 7×10^{-3} is forward-Euler discretisation error, not model error; the identity itself holds to machine precision.

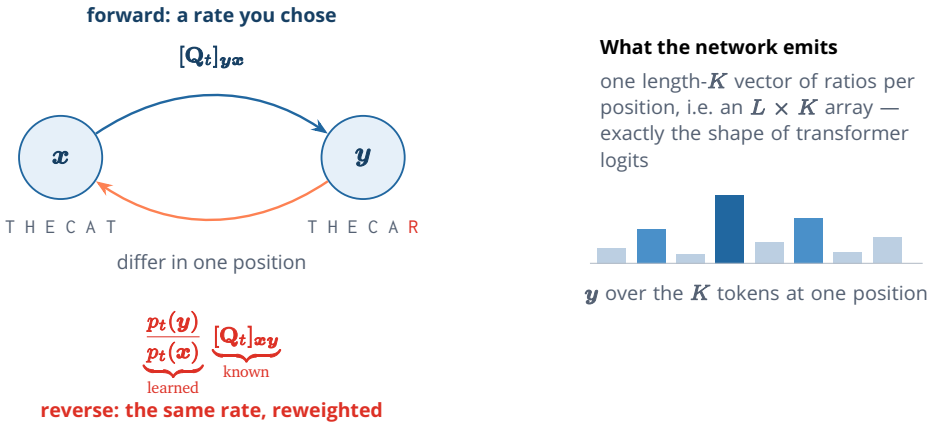


Figure 2.2 Theorem 2.12 in one picture. The forward process jumps between states differing in a single position, at a rate you chose. The exact reverse process uses the *same* rate for the opposite jump, reweighted by how much more or less likely the destination is — the concrete score $p_t(y)/p_t(x)$. Because the forward rate matrix is zero at Hamming distance ≥ 2, only single-site ratios are ever needed, so the learned object is an $L \times K$ array. Everything else in the reverse process is known in closed form.

Intuition: what the reverse rate says

Equation (2.17) is startlingly simple. To reverse the noising process you take the *forward* rate for the opposite jump and multiply it by how much more or less likely the destination is than where you are. If y is twice as likely as x under p_t, you jump $x \to y$ twice as eagerly as the forward process jumped $y \to x$. Everything the network needs to learn is contained in those ratios — and because $\mathbf{Q}_t$ is sparse, only ratios between *neighbouring* states (differing in one position) are ever needed.

2.6. The concrete score

Equation (2.17) motivates the central learned object.

Definition 2.15 (Concrete score). For a distribution p on $\mathcal{X}$ and a neighbourhood structure on $\mathcal{X}$, the *concrete score* at $\boldsymbol{x}$ is the vector of ratios

$$s(\boldsymbol{x})_{\boldsymbol{y}} = \frac{p(\boldsymbol{y})}{p(\boldsymbol{x})}, \qquad \boldsymbol{y} \in \mathcal{N}(\boldsymbol{x}). \tag{2.25}$$

For sequences the natural neighbourhood is the Hamming ball of radius one: $\mathcal{N}(\boldsymbol{x})$ is the set of sequences differing from $\boldsymbol{x}$ in exactly one position, so $|\mathcal{N}(\boldsymbol{x})| = L(K-1)$ and the score is an $L \times K$ array — exactly the shape of a transformer's output logits.

The name is from Meng et al. (2022), who introduced it as the discrete analogue of $\nabla_{\boldsymbol{x}} \log p$: in the continuous limit, $p(\boldsymbol{x} + h\boldsymbol{e})/p(\boldsymbol{x}) \approx 1 + h\,\partial_{\boldsymbol{e}} \log p(\boldsymbol{x})$, so a ratio is a finite-difference score. The crucial property is *normalisation-freedom*: the ratio is unchanged if p is known only up to a constant, exactly as $\nabla \log p$ is.

2.7. Simulating a CTMC: exact, Euler and τ-leaping

Three ways to sample from Equation (2.17), in increasing order of practicality and decreasing order of exactness.

Exact (Gillespie / uniformization). Draw an exponential waiting time, then a jump target; repeat. Exact but the number of jumps is random and each jump touches one coordinate, so an L-token sequence needs $\Omega(L)$ network evaluations.

Euler. Fix a step h and update $p(x_{t-h} = \boldsymbol{y} \mid x_t) = \delta_{\boldsymbol{x}\boldsymbol{y}} + h\,[\widehat{\mathbf{R}}_t]_{\boldsymbol{y}\boldsymbol{x}}$, truncated to the simplex. One network evaluation per step, all coordinates updated in parallel, error $O(h)$ per step *plus* the factorisation error of Remark 2.4.

τ-leaping. Due to Gillespie (2001) and imported to this setting by Campbell et al. (2022): over a window τ, treat the rates as frozen and draw the *number* of jumps along each channel as independent Poissons,

$$n_{x \to y} \sim \mathrm{Poisson}\big(\tau\,[\widehat{\mathbf{R}}_t]_{yx}\big), \tag{2.26}$$

then apply them all at once. For a factorised chain this updates every coordinate in parallel from a single network evaluation. The approximation is that rates do not change within the window; the characteristic failure is that a coordinate draws two or more jumps, which is ambiguous, and implementations either keep the state or apply one jump.

Remark 2.16. It is worth internalising that *all three* of these samplers are exact for the true reverse process in the limit of small steps, and all three suffer from Remark 2.4 at large steps. The interesting samplers of Chapter 8 are the ones that attack the factorisation error rather than the discretisation error.

2.8. Continuous-state diffusion, for contrast

This section is the minimum continuous-diffusion background needed to read the rest of the book, organised so that every object has a discrete counterpart. If you know DDPM and score SDEs well, read Section 2.8.2 — whose Fokker–Planck argument is the same computation as Theorem 2.12, in a different coordinate system — and Section 2.8.5, then move on.

2.8.1. The forward process and its reverse

Gaussian diffusion (Sohl-Dickstein et al., 2015; Ho et al., 2020) corrupts $\boldsymbol{x}_0 \sim p_{\text{data}}$ by

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathcal{N}\big(\boldsymbol{x}_t;\ \sqrt{\alpha_t}\,\boldsymbol{x}_0,\ (1-\alpha_t)\mathbf{I}\big), \qquad \alpha_t : 1 \to 0. \tag{2.27}$$

The two features that make this useful both reappear in the discrete case: the marginal at any t is available in closed form (so you can train on a random t without simulating), and the limiting distribution is a fixed, easy-to-sample reference.

In continuous time (Song et al., 2021) the same process is an SDE

$$\mathrm{d}\boldsymbol{x}_t = f(t)\,\boldsymbol{x}_t\ \mathrm{d}t + g(t)\ \mathrm{d}\boldsymbol{w}_t, \tag{2.28}$$

whose exact time reversal is another SDE,

$$\boxed{\mathrm{d}\boldsymbol{x}_t = \Big[f(t)\boldsymbol{x}_t - g(t)^2 \nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x}_t)\Big]\,\mathrm{d}t + g(t)\ \mathrm{d}\bar{\boldsymbol{w}}_t.} \tag{2.29}$$

Equation (2.29) should be read side by side with Equation (2.17): in both cases the reverse dynamics are the forward dynamics plus *one* unknown function of the marginal, and in both cases that function is a logarithmic derivative of p_t. The continuous case needs a gradient because the state space is a manifold; the discrete case needs a ratio because it is not.

2.8.2. The Fokker–Planck equation, and where Equation (2.29) comes from

Equation (2.29) is usually quoted and almost never derived, which is a pity, because the derivation is short, it explains *why* the score appears, and it is the exact continuous twin of the discrete time-reversal theorem of Chapter 2. The tool is the Fokker–Planck equation.

The equation

An SDE describes a single trajectory. The Fokker–Planck equation describes what the *density* does.

> **Definition 2.17 (Fokker–Planck equation).** Let $\boldsymbol{x}_t$ solve the Itô SDE $\mathrm{d}\boldsymbol{x}_t = f(\boldsymbol{x}_t, t)\,\mathrm{d}t + g(t)\,\mathrm{d}\boldsymbol{w}_t$ on $\mathbb{R}^d$. Its marginal density p_t satisfies
>
> $$\boxed{\partial_t\, p_t(\boldsymbol{x}) = -\nabla\cdot\big[f(\boldsymbol{x},t)\,p_t(\boldsymbol{x})\big] \;+\; \tfrac{1}{2}g(t)^2\,\Delta\, p_t(\boldsymbol{x}),} \tag{2.30}$$
>
> where $\nabla\cdot$ is the divergence and $\Delta = \nabla\cdot\nabla$ the Laplacian.

The two terms do exactly what you would guess. The first is *transport*: the drift f carries probability mass around, and $-\nabla\cdot(fp)$ is the net outflow at each point. The second is *spreading*: the Brownian term smooths the density, and the Laplacian is the canonical smoothing operator. The whole equation is a statement of conservation of probability: mass is moved and smeared, never created or destroyed.

Remark 2.18 (This is the Kolmogorov forward equation). Equation (2.30) *is* the Kolmogorov forward equation of Equation (2.10), for a different state space. Both say $\partial_t p_t = \mathcal{L}_t^*\, p_t$, where $\mathcal{L}_t^*$ is the adjoint of the process's infinitesimal generator. On a finite set the generator is a matrix and $\mathcal{L}_t^*\, p_t = \mathbf{Q}_t p_t$; on $\mathbb{R}^d$ it is the differential operator $\mathcal{L}_t^* = -\nabla\cdot(f\,\cdot) + \tfrac{1}{2}g^2\Delta$. Keeping this correspondence in view is the single most useful thing you can do when reading across the two literatures: Table 2.1 is really just a list of what $\mathcal{L}^*$ looks like in each case.

Step 1: rewrite diffusion as transport

The key algebraic move — and the place the score is born — is the identity

$$\nabla p_t(\boldsymbol{x}) = p_t(\boldsymbol{x})\, \nabla \log p_t(\boldsymbol{x}), \tag{2.31}$$

valid wherever $\boldsymbol{p_t > 0}$. Using it on the Laplacian term,

$$\tfrac{1}{2} g^2 \Delta p_t = \tfrac{1}{2} g^2\, \nabla \cdot \big(\nabla p_t\big) = \tfrac{1}{2} g^2\, \nabla \cdot \big(p_t \nabla \log p_t\big), \tag{2.32}$$

so Equation (2.30) collapses into a pure transport equation:

$$\boxed{\partial_t p_t = -\nabla \cdot \Big[\underbrace{\big(f - \tfrac{1}{2} g^2 \nabla \log p_t\big)}_{=: \boldsymbol{v_t}}\, p_t \Big].} \tag{2.33}$$

Intuition: why the score has to appear

Equation (2.33) says something striking: *the diffusion term is not really doing anything the drift could not do*. Random spreading and a deterministic flow down the density gradient have the same effect on the marginals, and Equation (2.31) is the exchange rate between them. The price of removing the noise is that the equivalent drift depends on $\boldsymbol{\nabla \log p_t}$ — a quantity that depends on the *whole* density, not just on where you are.

That is the entire reason diffusion models need a learned score. The forward SDE spreads mass with noise, which is cheap and local; to undo that spreading you must flow mass back up the density, which requires knowing the density's shape. And this is *exactly* the discrete story: in Equation (2.17) the forward process jumps blindly at a known rate, and the reverse process must reweight each jump by the ratio $\boldsymbol{p_t(y)/p_t(x)}$. Gradient of a log, or ratio of two values: the same object, read on a manifold or on a finite set.

Taking $\boldsymbol{\mathrm{d}x_t = v_t(x_t)\, \mathrm{d}t}$ with no noise at all gives a deterministic ODE with the same marginals as the SDE. This is the *probability-flow ODE*, and Table 2.1 records that it has no general discrete analogue — a fact we revisit at length in Section 5.8.

Step 2: a family of equivalent dynamics

Nothing forced us to push *all* of the diffusion into the drift. For any $\beta \in \mathbb{R}$, split the score term and put part of it back:

$$-\nabla \cdot \big(\boldsymbol{v}_t p_t\big) = -\nabla \cdot \Big[\big(\boldsymbol{v}_t + \tfrac{\beta}{2} \nabla \log p_t\big) p_t\Big] + \tfrac{\beta}{2}\, \nabla \cdot \big(p_t \nabla \log p_t\big), \tag{2.34}$$

and use Equation (2.31) once more on the last term. The result is a one-parameter family of Fokker–Planck equations, all with the same solution $\boldsymbol{p_t}$:

$$\partial_t p_t = -\nabla \cdot \Big[\Big(f + \tfrac{\beta - g^2}{2} \nabla \log p_t\Big) p_t\Big] + \tfrac{\beta}{2}\, \Delta p_t. \tag{2.35}$$

Sanity checks: $\boldsymbol{\beta = g^2}$ returns Equation (2.30), and $\boldsymbol{\beta = 0}$ returns Equation (2.33).

Step 3: reverse the clock and match coefficients

Now run time backwards. Put $\boldsymbol{\tau = T - t}$ and $\boldsymbol{q_\tau := p_{T-\tau}}$, so that $\boldsymbol{\partial_\tau q_\tau = -\,\partial_t p_t\big|_{t=T-\tau}}$. Flipping the sign of Equation (2.35),

$$\partial_\tau q_\tau = +\nabla \cdot \Big[\Big(f + \tfrac{\beta - g^2}{2} \nabla \log q_\tau\Big) q_\tau\Big] - \tfrac{\beta}{2}\, \Delta q_\tau. \tag{2.36}$$

We now *ask* for an SDE $\mathrm{d}\boldsymbol{x} = \bar{\boldsymbol{f}}(\boldsymbol{x},\tau)\,\mathrm{d}\tau + \bar{g}(\tau)\,\mathrm{d}\boldsymbol{w}$ whose own Fokker–Planck equation, $\partial_\tau q = -\nabla\cdot(\bar{\boldsymbol{f}} q) + \tfrac{1}{2}\bar{g}^2\Delta q$, coincides with Equation (2.36). Matching the two terms gives

$$\bar{f} = -\Big(f + \tfrac{\beta - g^2}{2}\nabla \log q_\tau\Big), \qquad \tfrac{1}{2}\bar{g}^2 = -\tfrac{\beta}{2} \implies \bar{g}^2 = -\beta. \tag{2.37}$$

So a reverse *SDE* exists precisely when $\beta \le 0$, and the free parameter β sets how much noise it uses. Choosing $\beta = -g^2$ — the same noise magnitude as the forward process — gives $\bar{g} = g$, and since then $\frac{\beta - g^2}{2} = \frac{-2g^2}{2} = -g^2$,

$$\bar{f} = -\big(f - g^2\,\nabla \log q_\tau\big) = -f + g^2\,\nabla \log q_\tau. \tag{2.38}$$

Written back in the original time variable $t = T - \tau$, where $\mathrm{d}\tau = -\,\mathrm{d}t$ and $\boldsymbol{w}$ becomes a reverse-time Wiener process $\bar{\boldsymbol{w}}$, this is

$$\mathrm{d}\boldsymbol{x}_t = \big[f(\boldsymbol{x}_t, t) - g(t)^2 \nabla \log p_t(\boldsymbol{x}_t)\big]\,\mathrm{d}t + g(t)\;\mathrm{d}\bar{\boldsymbol{w}}_t,$$

which is Equation (2.29) for the linear drift $\boldsymbol{f}(\boldsymbol{x}, t) = \boldsymbol{f}(t)\boldsymbol{x}$. The result is due to Anderson (1982); Song et al. (2021) brought it into generative modelling.

Remark 2.19 (Caveats). The derivation assumes $\boldsymbol{p_t} > 0$ with enough regularity for Equation (2.31) and for the integrations by parts implicit in matching Equation (2.37); it also only matches *marginals*, which is all a generative model needs, not the full path measure. Both caveats have discrete counterparts: Theorem 2.12 divides by $\boldsymbol{p_t}(\boldsymbol{x})$, and the convergence analyses of Chapter 13 need early stopping for exactly this reason.

What the parameter β buys you

The family in Equation (2.37) is worth more than the single equation it was used to produce. Three members are in common use:

- $\beta = 0$, so $\bar{g} = 0$: the **probability-flow ODE**. Deterministic, invertible, and the basis of DDIM, of latent interpolation, and of nearly all distillation in continuous diffusion.
- $\beta = -g^2$, so $\bar{g} = g$: the **standard reverse SDE**, Equation (2.29).
- $\beta < -g^2$, so $\bar{g} > g$: **higher-noise** reverse processes. Injecting extra noise and letting the score push back against it is the "churn" used by some practical samplers, and it is the same idea as a corrector step.

Intuition: one model, many samplers

Notice what just happened. A *single* trained score function supports an entire continuum of samplers with identical marginals, differing only in how much stochasticity they use. You do not retrain to move along it; you choose β at inference time.
This is not a quirk of the Gaussian case. It is the same structure as Theorem 8.7, where a single trained masked diffusion model supports a family of remasking samplers indexed by σ_t, all with the forward process's marginals, with $\sigma_t = 0$ recovering the standard ancestral sampler. In both settings the family exists because the Fokker–Planck (respectively Kolmogorov) equation constrains only the marginals, leaving the dynamics that realise them underdetermined. When you read a paper proposing a new sampler, the first question to ask is which member of this family it has picked, and the second is whether it had to retrain to get there.

2.8.3. Score matching and its discrete obstruction

The score $\nabla_{\boldsymbol{x}} \log p_t$ is learned by *denoising score matching* (Vincent, 2011): minimise

$$\mathbb{E}_t \, \mathbb{E}_{\boldsymbol{x}_0} \, \mathbb{E}_{\boldsymbol{x}_t \mid \boldsymbol{x}_0} \Big[w(t) \, \big\| \boldsymbol{s}_\theta(\boldsymbol{x}_t, t) - \nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) \big\|^2 \Big], \tag{2.39}$$

which works because the conditional score is known analytically ($-(\boldsymbol{x}_t - \sqrt{\alpha_t}\boldsymbol{x}_0)/(1-\alpha_t)$) and because the minimiser of Equation (2.39) is the *marginal* score — the conditional expectation of the conditional score. That last step is a pure L^2-projection argument, and it is the thing that breaks on discrete spaces.

Remark 2.20 (Why L^2 fails on ratios). The naive discrete analogue of Equation (2.39) is to regress onto the ratio $q(\boldsymbol{y} \mid \boldsymbol{x}_0)/q(\boldsymbol{x} \mid \boldsymbol{x}_0)$ in squared error. This is what Meng et al. (2022) first proposed, and it has a real defect: the targets are *positive and unbounded*, so an L^2 loss is scale-mismatched — a ratio of 10^3 and a ratio of 10^6 are both "large" but the loss treats the second as 10^6 times more important. The fix, due to Lou et al. (2024), is to replace the squared error with a Bregman divergence generated by Equation (2.7), which is the *score entropy* of Section 4.7. The projection argument then goes through, and the loss is bounded below by zero with the true ratio as minimiser.

2.8.4. The discrete-time ELBO

The DDPM objective is the variational bound of Equation (2.8) specialised to the chain of Equation (2.27):

$$\mathcal{L} = \mathbb{E}_q \Big[\underbrace{D_{\mathrm{KL}}(q(\boldsymbol{x}_T \mid \boldsymbol{x}_0) \,\|\, p(\boldsymbol{x}_T))}_{\text{prior}} + \sum_{t=2}^{T} \underbrace{D_{\mathrm{KL}}(q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t))}_{\text{diffusion}} \underbrace{- \log p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_1)}_{\text{reconstruction}} \Big]. \tag{2.40}$$

Memorise the shape of Equation (2.40): it is *identical* in the discrete case (Equation (3.3)), with Gaussian KLs replaced by categorical ones. Everything that follows in Chapters 3 and 5 is the work of computing the middle term for a particular choice of $\mathbf{Q}_t$.

Three parameterisations of $p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ are used in the Gaussian case and all three have discrete analogues:

- **$\boldsymbol{\epsilon}$-prediction**: predict the noise. Has no clean discrete analogue (there is no additive noise to subtract), which is why the discrete literature does not use it.
- **$\boldsymbol{x}_0$-prediction**: predict the clean data. This is *the* discrete parameterisation — it becomes a classifier over the vocabulary, and it is what D3PM, MDLM, MD4, LLaDA and Dream all do.
- **score prediction**: predict $\nabla \log p_t$. Becomes concrete score prediction (SEDD).

Intuition: Tweedie's formula and why $\boldsymbol{x}_0$-prediction wins

Tweedie's formula says the posterior mean of the clean signal is an affine function of the score: $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t] = (\boldsymbol{x}_t + (1-\alpha_t)\boldsymbol{s}(\boldsymbol{x}_t))/\sqrt{\alpha_t}$. So score prediction and $\boldsymbol{x}_0$-prediction are the *same model* up to a reparameterisation, and you pick whichever is better conditioned. On discrete data the $\boldsymbol{x}_0$ posterior is a probability vector — bounded, normalised, and exactly what a softmax head produces — whereas the concrete score is an unbounded positive number that diverges as $t \to 0$. That conditioning argument, more than anything else, is why the field converged on $\boldsymbol{x}_0$-prediction. Lou et al. (2024) prove the discrete Tweedie relation (their Theorem 4.1–4.2) and use it to build a better sampler; Ou et al. (2025) push it further and remove the time conditioning altogether (Chapter 6).

2.8.5. A dictionary

Table 2.1 is the translation table. Keep it to hand; much of the discrete literature is most quickly understood as "the obvious analogue", and the places where the analogy *breaks* are where the interesting research is.

Table 2.1 Continuous-to-discrete dictionary.

Continuous (Gaussian)	Discrete (CTMC)	Where it breaks
State space $\mathbb{R}^d$	$\mathcal{V}^L$, finite	no gradients, no interpolation
Score $\nabla_{\boldsymbol{x}} \log p_t$	concrete score $p_t(\boldsymbol{y})/p_t(\boldsymbol{x})$	unbounded; needs Bregman not L^2 (Remark 2.20)
Fokker–Planck Equation (2.30)	Kolmogorov forward Equation (2.10)	same equation, $\partial_t p_t = \mathcal{L}_t^* p_t$ (Remark 2.18)
Forward SDE Equation (2.28)	rate matrix $\mathbf{Q}_t$	design space is much larger and less canonical
Reverse SDE Equation (2.29)	reverse rates Equation (2.17)	—
$q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ Gaussian	$\exp(\bar{\sigma}(t)\mathbf{Q})$ categorical	—
Euler–Maruyama	Euler / τ-leaping	parallel updates factorise the joint (Remark 2.4)
Probability-flow ODE, DDIM	no deterministic analogue in general	the state space is not connected by a flow
Tweedie: score $\leftrightarrow \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t]$	concrete score $\leftrightarrow p(\boldsymbol{x}_0 \mid \boldsymbol{x}_t)$	holds, and is exploited by SEDD and RADD
Classifier guidance $\nabla \log p(c \mid \boldsymbol{x}_t)$	rate reweighting $[p(c \mid \boldsymbol{y})/p(c \mid \boldsymbol{x})]^\gamma$	discrete version is *exact*, not an approximation (Chapter 9)

Two entries deserve comment.

There is no DDIM. The deterministic probability-flow ODE, which underpins almost all fast sampling and distillation in continuous diffusion, has no general discrete counterpart: you cannot move continuously between two points of a finite set. This is the deepest structural difference, and it is why fast sampling for discrete diffusion is hard and why the successful approaches (Chapter 8) either distil in a learned continuous surrogate (Sahoo et al., 2025) or attack dimensional correlations directly (Hayakawa et al., 2025).

Guidance is better, not worse. In the Gaussian case, classifier guidance $\nabla_{\boldsymbol{x}} \log p(c \mid \boldsymbol{x}_t)$ requires a classifier trained on noisy inputs and the guided process is only approximately correct. In the discrete case, reweighting the rates by the likelihood ratio is *exactly* the conditional reverse process (Nisonoff et al., 2025). This is one of the genuine advantages of discrete state spaces and is underexploited.

Key takeaways

- A forward process is useful only if $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ is closed-form. In discrete time this is $\overline{\mathbf{Q}}_t = \prod_{s \le t} \mathbf{Q}_s$; in continuous time it is $\exp(\bar{\sigma}(t)\mathbf{Q})$, which has two-line closed forms for uniform (Equation (2.12)) and absorbing (Equation (2.13)) noise.
- Theorem 2.12 is the foundation: the exact reverse rate is the forward rate for the opposite jump times the ratio $p_t(\boldsymbol{y})/p_t(\boldsymbol{x})$. That ratio — the *concrete score* — is the only thing a network must learn.

- Because the forward rate matrix is sparse (one position at a time), only Hamming-distance-one ratios are needed. The score is therefore an $L \times K$ array: the natural output shape of a transformer.
- Parallel samplers are the reason to care about discrete diffusion and the reason its samples are wrong. Hold Remark 2.4 in mind for the rest of the book.
- The reverse of a diffusion is the forward process plus one unknown function of the marginal: a gradient of $\log p_t$ in continuous space, a ratio of p_t values in discrete space.
- The Fokker–Planck equation Equation (2.30) *is* the Kolmogorov forward equation Equation (2.10) on a different state space: both read $\partial_t p_t = \mathcal{L}_t^* p_t$ for the adjoint of the generator.
- The reverse SDE Equation (2.29) is derived, not postulated. The identity $\nabla p = p \nabla \log p$ converts the diffusion term into transport, which is why the score appears at all; reversing the clock and matching Fokker–Planck coefficients then fixes the reverse drift. The identical strategy proves the discrete Theorem 2.12.
- Matching marginals leaves the dynamics underdetermined, giving a one-parameter family Equation (2.37) of reverse processes from the probability-flow ODE ($\bar{g} = 0$) through the standard reverse SDE ($\bar{g} = g$) to higher-noise samplers. One trained model, many samplers — the same structure as Theorem 8.7 in the discrete case.
- The ELBO has the same three-term shape (Equation (2.40)) in both settings; the work is in computing the middle KL for a given forward process.
- $\boldsymbol{x}_0$-prediction is the parameterisation of choice on discrete data because the target is a normalised probability vector, whereas the concrete score is unbounded and ill-conditioned near $t = 0$.
- The analogy breaks in two important places: L^2 score matching must be replaced by a Bregman divergence, and there is no deterministic probability-flow ODE, hence no DDIM.

Open questions and research directions

1. **Neighbourhood structures beyond Hamming-1.** Definition 2.15 is stated for an arbitrary neighbourhood, but essentially all work uses single-site Hamming balls because that is what makes $\mathbf{Q}_t$ sparse. A forward process with structured multi-site jumps (swap two tokens; move a whole span; edit a substructure in a molecule) would give a concrete score over a richer neighbourhood and could in principle capture correlations that single-site processes cannot. The ELBO machinery goes through; nobody has made it work at scale. Fragment-level flow matching (Lee et al., 2026) is the closest attempt.
2. **Non-factorised tractable forward processes.** We restrict to factorised forward processes purely for tractability of $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$. Are there *non*-factorised chains with closed-form marginals — e.g. built from low-rank or determinantal structure — whose reverse processes are easier to approximate in few steps?
3. **Vocabulary-size scaling of the score.** The concrete score has $L(K-1)$ entries and the softmax over K is the dominant cost at large vocabularies. Hierarchical or latent factorisations of the concrete score (analogous to adaptive softmax) have barely been explored, despite being an obvious lever for frontier-scale diffusion LMs.

4. **τ-leaping with dependence.** Standard τ-leaping draws independent Poissons per channel. A leaping scheme that draws a *coupled* jump count — using a copula, or a learned low-dimensional dependence structure as in Liu et al. (2025) — is the natural attack on Remark 2.4 and remains underexplored relative to how central the problem is.
5. **Is there a useful surrogate for the probability-flow ODE?** Sahoo et al. (2025) show that applying $\arg\max$ to a Gaussian diffusion yields exactly a uniform-state discrete diffusion, which gives a continuous latent to distil in. That construction is specific to uniform noise. Whether an analogous "lift" exists for absorbing-state diffusion — the family everyone actually uses — is open and would immediately unlock consistency distillation for masked diffusion LMs.
6. **Transporting the continuous toolbox.** Many continuous-diffusion advances have no discrete counterpart yet: v-prediction and reparameterisation-invariant losses, EDM-style preconditioning and schedule design, rectified flow and reflow, stochastic interpolants, higher-order solvers (DPM-Solver), and consistency models. Each is a concrete, tractable research project. Reweighted-loss analyses have begun to arrive (Shi and Titsias, 2025), but the preconditioning question — what is the discrete analogue of EDM's input/output scaling? — is essentially untouched.
7. **When is a continuous relaxation actually better?** Pynadath et al. (2025) argue the two regimes (continuous geometry, discrete identity) are in tension and should be decoupled. A clean theory predicting, as a function of K and L, whether a relaxed or native-discrete process is preferable does not exist.
8. **Schedule design.** In continuous diffusion, schedule design is well understood via SNR and is nearly solved in practice. In the discrete case the only optimality result I am aware of is the Fisher–Rao optimality of the cosine schedule for masked diffusion (Zhang and Syed, 2025). Uniform-state, interpolating and general CTMC schedules have no corresponding theory.

CHAPTER 3

D3PM: Discrete Diffusion in Discrete Time

CHAPTER

Authors	Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow and Rianne van den Berg; Emiel Hoogeboom et al.
Year	2021
Published	Advances in Neural Information Processing Systems (NeurIPS)
Link	https://arxiv.org/abs/2107.03006
Tags	D3PM, multinomial diffusion, transition matrices, ELBO, auxiliary loss

Everything in this field descends from one paper. Austin et al. (2021) introduced *Discrete Denoising Diffusion Probabilistic Models* (D3PM), which take the DDPM skeleton of Equation (2.40) and replace the Gaussian kernel by an arbitrary categorical one. Simultaneously Hoogeboom et al. (2021) introduced multinomial diffusion, which is the uniform-noise special case, and both build on the binomial diffusion of Sohl-Dickstein et al. (2015). This chapter derives D3PM completely. The continuous-time formulations of Chapters 4 and 5 and Section 4.7 are limits of it, so it pays to have the discrete-time algebra at your fingertips.

Convention for this chapter

Following the D3PM literature we use **row** one-hot vectors and row-stochastic matrices: $\boldsymbol{x}$ is a row vector, $[\mathbf{Q}_t]_{ij} = q(x_t = j \mid x_{t-1} = i)$, rows sum to one, and marginals propagate as $\boldsymbol{x}\mathbf{Q}$. Chapter 4 switches to the column convention.

3.1. The forward process

Fix a per-step transition matrix $\mathbf{Q}_t$. Acting independently on each position of the sequence,

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathrm{Cat}\big(\boldsymbol{x}_t;\ \boldsymbol{p} = \boldsymbol{x}_{t-1}\mathbf{Q}_t\big), \qquad q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathrm{Cat}\big(\boldsymbol{x}_t;\ \boldsymbol{p} = \boldsymbol{x}_0\overline{\mathbf{Q}}_t\big), \tag{3.1}$$

with $\overline{\mathbf{Q}}_t = \mathbf{Q}_1\mathbf{Q}_2\cdots\mathbf{Q}_t$. This is exactly Equation (2.9) and was verified in Listing 2.1.

3.1.1. The posterior

Training needs $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)$, the *true* one-step denoising distribution given the clean data. By Bayes,

$$\begin{aligned} q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) &= \frac{q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1})\, q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_0)}{q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)} \\ &= \operatorname{Cat}\left(\boldsymbol{x}_{t-1};\ \boldsymbol{p} = \frac{\boldsymbol{x}_t \mathbf{Q}_t^\top \odot \boldsymbol{x}_0 \overline{\mathbf{Q}}_{t-1}}{\boldsymbol{x}_0 \overline{\mathbf{Q}}_t \boldsymbol{x}_t^\top}\right), \end{aligned} \tag{3.2}$$

where $\odot$ is elementwise. Read the three factors: $\boldsymbol{x}_t \mathbf{Q}_t^\top$ is the likelihood of landing on $\boldsymbol{x}_t$ from each candidate $\boldsymbol{x}_{t-1}$, $\boldsymbol{x}_0 \overline{\mathbf{Q}}_{t-1}$ is the prior over $\boldsymbol{x}_{t-1}$ given the data, and the denominator is the scalar normaliser $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$. Listing 2.1 checks Equation (3.2) against brute-force enumeration.

Remark 3.1 (Everything is cheap). Both Equation (3.1) and Equation (3.2) are K-vectors obtained by one or two $K \times K$ matrix multiplies. For the structured $\mathbf{Q}_t$ of Section 3.4 these reduce to $O(K)$ operations with no matrix stored at all — which is what makes the method usable at $K = 50{,}000$.

3.2. The variational bound

With p_θ the learned reverse chain, the bound of Equation (2.8) reads

$$\begin{aligned} \mathcal{L}_{\text{vb}} = \mathbb{E}_{q(\boldsymbol{x}_0)}\Big[&\underbrace{D_{\mathrm{KL}}(q(\boldsymbol{x}_T \mid \boldsymbol{x}_0) \,\|\, p(\boldsymbol{x}_T))}_{\mathcal{L}_T} \underbrace{-\mathbb{E}_{q(\boldsymbol{x}_1 \mid \boldsymbol{x}_0)} \log p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_1)}_{\mathcal{L}_0} \\ &+ \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)} D_{\mathrm{KL}}(q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t))}_{\mathcal{L}_{t-1}} \Big]. \end{aligned} \tag{3.3}$$

The derivation is the telescoping argument of Chapter B; it is word-for-word the DDPM derivation because it uses nothing about the kernel being Gaussian. Note what is now computable: $\mathcal{L}_{t-1}$ is a KL between two *explicit* categorical distributions over K values, so it is an exact sum of K terms, not a Monte Carlo estimate. If $\boldsymbol{\pi}$ is chosen as the stationary distribution of $\mathbf{Q}$ and T is large enough, $\mathcal{L}_T \approx 0$ by construction.

3.3. The x_0-parameterisation

We could let the network output $p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ directly, but that wastes the structure we just derived. Instead, have the network predict the *clean* data and then plug its prediction into Equation (3.2):

$$p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t) \ \propto \sum_{\tilde{\boldsymbol{x}}_0} q(\boldsymbol{x}_{t-1}, \boldsymbol{x}_t \mid \tilde{\boldsymbol{x}}_0)\, \tilde{p}_\theta(\tilde{\boldsymbol{x}}_0 \mid \boldsymbol{x}_t), \tag{3.4}$$

where $\tilde{p}_\theta(\cdot \mid \boldsymbol{x}_t) = \operatorname{softmax}(\mathrm{nn}_\theta(\boldsymbol{x}_t))$ is a per-position softmax over the vocabulary. Three reasons this is the right move:

1. **It is exact at the optimum.** If $\tilde{p}_\theta$ puts all its mass on the true $\boldsymbol{x}_0$, the KL term in Equation (3.3) is exactly zero (Austin et al., 2021).

2. **The network's job is time-invariant.** "Infer the clean token" is the same task at every noise level, so a single network shares statistical strength across t. Predicting $p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ directly makes the target depend on t in a way the network must learn separately.
3. **It enables k-step jumps.** Nothing in Equation (3.4) requires $t-1$; substituting $\overline{\mathbf{Q}}_s$ for any $s < t$ gives a sampler that leaps from t to s. This is where few-step sampling comes from.

3.3.1. The auxiliary loss

Austin et al. (2021) found that the pure bound Equation (3.3) trains poorly and added a cross-entropy term on the $\boldsymbol{x}_0$-prediction:

$$\mathcal{L}_\lambda = \mathcal{L}_{\mathrm{vb}} + \lambda\, \mathbb{E}_{q(\boldsymbol{x}_0)} \mathbb{E}_{q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)} \big[-\log \tilde{p}_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_t) \big]. \tag{3.5}$$

This is no longer a bound on the likelihood, but it is a much better training signal.

Intuition: why the bound alone is a weak signal

At large t, $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)$ is nearly independent of $\boldsymbol{x}_0$: when almost everything has been destroyed, the one-step denoising distribution barely depends on what the data was. So the KL term carries almost no gradient about $\boldsymbol{x}_0$ at high noise, and the network can minimise the bound while learning very little about the data. The auxiliary term asks the question we actually want answered — "what was the clean token?" — at every noise level with equal weight.

This observation turns out to be the hinge of the whole field. In Chapter 5 we will see that for *absorbing* noise the bound *itself* collapses to a weighted version of exactly this cross-entropy, so no auxiliary term is needed and no bound is sacrificed. That is a large part of why masked diffusion won.

3.4. Choosing the transition matrix

This is D3PM's substantive contribution: the recognition that $\mathbf{Q}_t$ is a design space in which you can *encode domain knowledge*, and that the choice matters a great deal.

Uniform. $\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \frac{\beta_t}{K}\mathbf{1}\mathbf{1}^\top$, as in Example 2.5. This is multinomial diffusion (Hoogeboom et al., 2021). Doubly stochastic, uniform stationary distribution, ergodic. The reverse process must learn *both* which tokens are wrong and what to replace them with.

Absorbing. $\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t \mathbf{1}\mathbf{e}_{[\mathtt{MASK}]}^\top$, as in Example 2.6. The reverse process only ever fills in masks, so the problem is strictly easier: the model is told *where* the errors are. Austin et al. (2021) note that the mutual-information schedule for the absorbing case reduces to the $(T-t+1)^{-1}$ schedule of Sohl-Dickstein et al. (2015). This row of the design space is the one that became dominant, and Chapter 5 is devoted to it.

Discretised Gaussian. For *ordinal* data — image pixel intensities, quantised audio — the vocabulary carries a metric, and a transition matrix built by discretising a Gaussian with linearly increasing variance respects it, so that a pixel of value 130 is corrupted to 131 before it is corrupted to 7. Normalised to be doubly stochastic, so the stationary distribution is uniform.

Token-embedding distance. For text there is no ordinal structure, but there is semantic structure. Austin et al. (2021) construct a doubly stochastic $\mathbf{Q}_t$ that transitions preferentially between tokens with similar embeddings, so that corruption is semantic rather than arbitrary.

Marginal-matched. Not in the original paper but important: Vignac et al. (2023a) propose $\mathbf{Q}_t = \alpha_t \mathbf{I} + (1-\alpha_t)\mathbf{1}\boldsymbol{m}^\top$ where $\boldsymbol{m}$ is the *empirical marginal* over token types. The stationary distribution is then $\boldsymbol{m}$ rather than uniform. For sparse graphs this is transformative — the limiting distribution produces graphs with realistic edge density instead of complete graphs — and it is now standard in graph and molecule diffusion (Chapter 12).

Proposition 3.2 (Marginal matching). $\mathbf{Q}(\alpha) = \alpha\mathbf{I} + (1-\alpha)\mathbf{1}\boldsymbol{m}^\top$ is row-stochastic for any $\alpha \in [0,1]$ and $\boldsymbol{m} \in \Delta^K$, and $\boldsymbol{m}$ is stationary: $\boldsymbol{m}^\top \mathbf{Q}(\alpha) = \boldsymbol{m}^\top$.

Proof. Row sums: $\alpha + (1-\alpha)\mathbf{1}^\top\boldsymbol{m} = \alpha + (1-\alpha) = 1$. Stationarity: $\boldsymbol{m}^\top\mathbf{Q} = \alpha\boldsymbol{m}^\top + (1-\alpha)(\boldsymbol{m}^\top\mathbf{1})\boldsymbol{m}^\top = \alpha\boldsymbol{m}^\top + (1-\alpha)\boldsymbol{m}^\top = \boldsymbol{m}^\top$. ∎

3.4.1. One theorem covering three of the five choices

Uniform, absorbing and marginal-matched noise are usually presented as three separate designs, each with its own closed-form $\overline{\mathbf{Q}}_t$ derived by hand. They are in fact one design, and the reason their cumulative products are tractable is a single algebraic property.

Theorem 3.3 (Idempotent noise families are closed under composition). Let $\mathbf{M} \in \mathbb{R}^{K\times K}$ be row-stochastic and *idempotent*, $\mathbf{M}^2 = \mathbf{M}$, and define

$$\mathbf{Q}(\beta) \;:=\; (1-\beta)\mathbf{I} + \beta\mathbf{M}, \qquad \beta \in [0,1]. \tag{3.6}$$

Then

1. each $\mathbf{Q}(\beta)$ is row-stochastic;
2. the family is closed under multiplication, with the *survival probabilities* multiplying:

$$\boxed{\mathbf{Q}(\beta_1)\,\mathbf{Q}(\beta_2) \;=\; \mathbf{Q}\big(1-(1-\beta_1)(1-\beta_2)\big);} \tag{3.7}$$

3. consequently, for any schedule $\{\beta_s\}$,

$$\overline{\mathbf{Q}}_t \;=\; \prod_{s=1}^{t} \mathbf{Q}(\beta_s) \;=\; \bar\alpha_t\,\mathbf{I} + (1-\bar\alpha_t)\,\mathbf{M}, \qquad \bar\alpha_t = \prod_{s=1}^{t}(1-\beta_s). \tag{3.8}$$

Moreover $\mathbf{M} = \mathbf{1}\boldsymbol{v}^\top$ is row-stochastic and idempotent for *every* $\boldsymbol{v} \in \Delta^K$, and then $\boldsymbol{v}$ is the unique stationary distribution and $\operatorname{spec}\mathbf{Q}(\beta) = \{1\} \cup \{1-\beta\}^{K-1}$.

Proof. (i) Row sums: $(1-\beta)\cdot 1 + \beta\cdot 1 = 1$.
(ii) Expand and use $\mathbf{M}^2 = \mathbf{M}$:

$$\begin{aligned}\mathbf{Q}(\beta_1)\mathbf{Q}(\beta_2) &= (1-\beta_1)(1-\beta_2)\mathbf{I} + \big[(1-\beta_1)\beta_2 + \beta_1(1-\beta_2)\big]\mathbf{M} + \beta_1\beta_2\mathbf{M}^2 \\ &= (1-\beta_1)(1-\beta_2)\mathbf{I} + \big[\beta_1+\beta_2-\beta_1\beta_2\big]\mathbf{M},\end{aligned} \tag{3.9}$$

and $\beta_1+\beta_2-\beta_1\beta_2 = 1-(1-\beta_1)(1-\beta_2)$, so the two coefficients are $(1-\beta)$ and β for $\beta = 1-(1-\beta_1)(1-\beta_2)$. (iii) is induction on t with $\bar\alpha_t = \prod_s(1-\beta_s)$.

For the last claim, $(\mathbf{1}\boldsymbol{v}^\top)(\mathbf{1}\boldsymbol{v}^\top) = \mathbf{1}(\boldsymbol{v}^\top\mathbf{1})\boldsymbol{v}^\top = \mathbf{1}\boldsymbol{v}^\top$ because $\boldsymbol{v}^\top\mathbf{1} = 1$; row sums are $\boldsymbol{v}^\top\mathbf{1} = 1$; $\boldsymbol{v}^\top\mathbf{Q}(\beta) = (1-\beta)\boldsymbol{v}^\top + \beta\boldsymbol{v}^\top = \boldsymbol{v}^\top$; and $\mathbf{1}\boldsymbol{v}^\top$ is rank one with trace 1, hence has spectrum $\{1\}\cup\{0\}^{K-1}$, which Equation (3.6) maps to $\{1\}\cup\{1-\beta\}^{K-1}$. ∎

Intuition: survive or resample

Equation (3.8) has a one-line probabilistic reading, and it is the reading to carry forward. Because $\mathbf{M} = \mathbf{1}\boldsymbol{v}^\top$ sends *every* state to the same distribution $\boldsymbol{v}$, applying $\mathbf{Q}(\beta)$ means: *with probability* $1-\beta$ *do nothing, otherwise forget the current state entirely and resample from* $\boldsymbol{v}$. Idempotency is exactly the statement that forgetting twice is the same as forgetting once, and that is why the composition only has to track the probability of never having forgotten, $\bar{\alpha}_t$. So

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathrm{Cat}\big(\boldsymbol{x}_t;\ \bar{\alpha}_t \boldsymbol{x}_0 + (1-\bar{\alpha}_t)\boldsymbol{v}\big), \tag{3.10}$$

by Proposition 2.1 — one formula for uniform ($\boldsymbol{v} = \frac{1}{K}\mathbf{1}$), absorbing ($\boldsymbol{v} = \mathbf{m}$) and marginal-matched ($\boldsymbol{v} = \boldsymbol{m}$). It is also the formula the whole of Chapter 5 is built on, and Equation (2.14) is its $\boldsymbol{v} = \mathbf{m}$ case with $\bar{\alpha}_t = e^{-\bar{\sigma}(t)}$.

Remark 3.4 (What the theorem excludes, and why that matters). The two remaining D3PM choices — discretised-Gaussian and token-embedding-distance — are *not* of the form Equation (3.6), because their $\mathbf{M}$ is not idempotent: a nearest-neighbour corruption applied twice spreads further than applied once. That is precisely the point of them (corruption should respect the metric on $\mathcal{V}$), and it is also precisely why they are more expensive: $\overline{\mathbf{Q}}_t$ has no two-term closed form and must be accumulated numerically, $K \times K$ matrix product by matrix product, and stored per timestep. The design tension is therefore structural rather than incidental: *the kernels that carry domain knowledge about* $\mathcal{V}$ *are exactly the kernels that lose the closed-form marginal.* No published discrete-diffusion model at language scale uses a non-idempotent kernel, and the open questions at the end of this chapter return to whether that is a real limitation or a solved-and-forgotten one.

Listing 3.1 implements this and contrasts it with uniform noise.

Listing 3.1 Marginal-matched transition matrices (DiGress), and why uniform noise is wrong for sparse graphs.

```python
"""Marginal-matched transition matrices (DiGress) vs uniform noise.

For sparse graphs the 'no edge' category dominates.  Uniform noise drives the
limiting distribution to 1/K per category, so fully-noised graphs are dense
and nothing like real data.  Matching the empirical marginal instead keeps
the limiting distribution realistic.
"""
import numpy as np

# empirical marginal over edge types in a sparse molecular graph
m = np.array([0.90, 0.05, 0.03, 0.02])      # none, single, double, triple
K = len(m)


def marginal_Q(a_t):
    """Q_t = a_t I + (1 - a_t) 1 m^T   (row-stochastic, stationary dist. m)."""
    return a_t * np.eye(K) + (1.0 - a_t) * np.outer(np.ones(K), m)


def uniform_Q(a_t):
    """Q_t = a_t I + (1 - a_t) 1 1^T / K   (stationary dist. uniform)."""
    return a_t * np.eye(K) + (1.0 - a_t) * np.ones((K, K)) / K


for name, maker, lim in [("marginal", marginal_Q, m),
```

```
                          ("uniform ", uniform_Q, np.ones(K) / K)]:
    for a in (1.0, 0.5, 0.0):
        Q = maker(a)
        assert np.allclose(Q.sum(1), 1.0)                # row-stochastic
        assert np.allclose(lim @ Q, lim)                 # lim is stationary
    print(f"{name}: row-stochastic and stationary for all a_t; "
          f"limiting dist = {maker(0.0)[0].round(3)}")

# how dense is a fully-noised graph on 20 nodes?  (190 candidate edges)
n_pairs = 20 * 19 // 2
for name, lim in [("marginal", m), ("uniform ", np.ones(K) / K)]:
    print(f"{name}: expected edges in a fully-noised 20-node graph = "
          f"{n_pairs * (1 - lim[0]):.0f} / {n_pairs}")
```

The last two lines of its output make the point quantitatively. With the edge-type marginal of a sparse molecular graph, a fully noised 20-node graph has

```
marginal: expected edges in a fully-noised 20-node graph = 19 / 190
uniform : expected edges in a fully-noised 20-node graph = 142 / 190
```

so uniform noise asks the reverse process to carve a 19-edge molecule out of a near-complete graph, while marginal-matched noise starts it from something with the right density. The denoising network also sees sparse graphs throughout training, which is what makes the expensive structural features of Chapter 12 affordable.

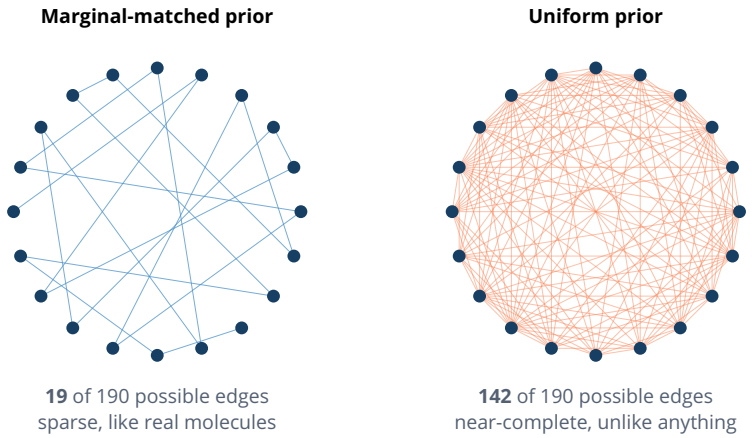


Figure 3.1 The limiting distribution of the forward process is the distribution the reverse process starts from, so it should look like the data. With the edge-type marginal of a sparse molecular graph, a fully noised 20-node graph keeps about 19 of the 190 candidate edges; uniform noise over the four edge types gives about 142 (Listing 3.1). Uniform noise therefore asks the denoiser to carve a molecule out of a near-complete graph, and trains it only on intermediate states that look nothing like chemistry. Matching the marginal (Proposition 3.2) costs nothing but a histogram over the training set, and it is what makes the spectral features of Chapter 12 affordable.

3.5. Noise schedules

Given a family $\mathbf{Q}_t$ parameterised by β_t, the schedule $\{\beta_t\}$ still has to be chosen. D3PM discusses three principles.

- **Linear on β_t.** Simplest; note that linear β_t gives *non*-linear cumulative corruption in $\overline{\mathbf{Q}}_t$.

- **Cosine on cumulative probability.** Adapted from Nichol and Dhariwal (2021) by Hoogeboom et al. (2021): schedule the cumulative stay-probability as a cosine. This is the practical default and there is now a theoretical reason to prefer it (Chapter 13).
- **Mutual-information schedule.** The principled option: choose β_t so that $I(\boldsymbol{x}_t; \boldsymbol{x}_0) \approx (1 - t/T)\, H(\boldsymbol{x}_0)$, i.e. destroy information at a constant rate. This is schedule design by specifying *what the schedule should achieve* rather than its functional form, and it generalises to any $\mathbf{Q}_t$. For the absorbing case it recovers $\beta_t = (T - t + 1)^{-1}$.

Intuition: why $\beta_t = (T - t + 1)^{-1}$ for masking

At step t there are $T - t + 1$ steps remaining and we want every token masked by step T. If a fraction β_t of the still-unmasked tokens is masked at each step, the expected fraction surviving is $\prod_{s \le t}(1 - \beta_s)$. Setting $\beta_t = (T - t + 1)^{-1}$ gives $\prod_{s=1}^{t} \left(1 - \frac{1}{T - s + 1}\right) = \prod_{s=1}^{t} \frac{T - s}{T - s + 1} = \frac{T - t}{T}$, i.e. the survival probability is exactly linear in t — which is the discrete-time version of $\alpha_t = 1 - t$ and masks one token per step in expectation. The whole schedule is just "linear masking".

3.6. What D3PM achieved, and where it fell short

D3PM made discrete diffusion a real method: it reported competitive likelihoods on text8 and on CIFAR-10 at the pixel level, and established that the absorbing kernel was the strongest choice for text. Its limitations drove the next four years:

1. **T is a hyperparameter of the model, not just the sampler.** The bound is defined for a fixed discretisation; changing the number of sampling steps changes the trained object.
2. **The bound needs an auxiliary loss to train well** (Equation (3.5)), so the reported numbers are not clean likelihood bounds for the model you actually trained.
3. **Perplexity was far from autoregressive.** For calibration: D3PM-absorbing reaches 76.90 test perplexity on LM1B at 70M parameters, where a matched autoregressive transformer reaches ≈ 22 and the later MDLM reaches 27.04 (Sahoo et al., 2024). The gap was closed by continuous time plus a better parameterisation, not by bigger models.

Key takeaways

- D3PM is DDPM with a categorical kernel. Equations (3.1) to (3.3) are the whole method, and every term is computable in closed form.
- Predict $\boldsymbol{x}_0$ and substitute into the exact posterior (Equation (3.4)). The target is then time-invariant, the KL is exactly zero at the optimum, and multi-step jumps come for free.
- $\mathbf{Q}_t$ is where domain knowledge goes: uniform, absorbing, ordinal (discretised Gaussian), semantic (embedding distance), or marginal-matched (Proposition 3.2).
- The pure ELBO is a weak learning signal at high noise, hence the auxiliary cross-entropy of Equation (3.5). For absorbing noise this defect disappears — see Chapter 5.
- Schedule by specifying an *objective* (constant information destruction) rather than a functional form.

Open questions and research directions

1. **Learned transition matrices.** $\mathbf{Q}_t$ is chosen by hand in essentially all work. Learning it jointly with the denoiser is attractive — the forward process would adapt corruption to the data's own correlation structure — but it breaks the closed form for $\overline{\mathbf{Q}}_t$ unless the parameterisation is restricted. Low-rank or Kronecker-structured $\mathbf{Q}_t$ with closed-form powers is an obvious, apparently unexplored, target. Recent work on learned forward trajectories for molecules (Seo et al., 2026) is a first step and is restricted to masking schedules rather than the matrix itself.
2. **Position- and token-dependent schedules.** Both the schedule and the matrix are shared across positions. Content-dependent corruption (mask rare tokens later; corrupt a scaffold region less than a decoration region) is used heuristically in a few applied papers but has no clean variational treatment, because $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ then depends on $\boldsymbol{x}_0$ in a way that complicates the bound. Shi et al. (2024) take one step here with state-dependent masking; the general case is open.
3. **Why does the auxiliary loss help so much?** The explanation in the Intuition box above is folklore and, as far as I know, has never been made quantitative. A result of the form "the gradient signal about $\boldsymbol{x}_0$ in $\mathcal{L}_{t-1}$ decays like $f(\alpha_t)$" would both explain Equation (3.5) and tell you how to weight it. The recent reweighted-loss analysis of Shi and Titsias (2025) — showing some reweighted losses are *tighter* variational bounds — is the closest thing and suggests this is tractable.
4. **Ordinal and metric vocabularies are neglected.** The discretised-Gaussian kernel was in the original paper and then almost abandoned, because text has no metric. But quantised audio, pixel values, discretised coordinates in molecule generation, and binned continuous properties all do. Masking throws that structure away. There may be easy wins here.

CHAPTER 4

Continuous-Time Discrete Diffusion

CHAPTER

Authors	Andrew Campbell et al.; Aaron Lou, Chenlin Meng and Stefano Ermon
Year	2024
Published	NeurIPS 2022 / ICML 2024 (best paper)
Link	https://arxiv.org/abs/2310.16834
Tags	CTMC, rate matrices, continuous-time ELBO, concrete score, score entropy, tau-leaping

Campbell et al. (2022) gave the first complete continuous-time framework for discrete denoising models, and it is the cleanest place from which to see the whole field. This chapter sets up the framework, derives the continuous-time ELBO, and explains the samplers it suggests. The two formulations that dominate practice — score entropy (Section 4.7) and masked diffusion (Chapter 5) — are specialisations.

Convention for this chapter and the rest of the book

We now use the **column** convention of Definition 2.7: $[\mathbf{Q}_t]_{yx}$ is the rate $x \to y$, columns sum to zero, and marginals evolve as $\mathrm{d}p_t/\,\mathrm{d}t = \mathbf{Q}_t p_t$. Campbell et al. (2022) themselves use the row convention and write $R_t(x, \tilde{x})$ for the rate $x \to \tilde{x}$; when quoting their results I translate.

4.1. Why continuous time

Going to continuous time is not an aesthetic choice. It buys four concrete things.

1. **T stops being part of the model.** In D3PM the bound is defined for a particular discretisation, so the trained object depends on T. In continuous time the model is the limit, and the number of sampling steps becomes a pure inference-time knob. This is what makes "quality versus compute" a dial rather than a retraining decision.
2. **The bound gets tighter.** The discrete-time ELBO has a discretisation gap; the continuous-time limit removes it. Empirically this is worth several perplexity points: MDLM's continuous-time bound improves on its own discrete-time bound, and the same is reported for MD4 (Sahoo et al., 2024; Shi et al., 2024).
3. **Objectives simplify dramatically.** The T-term sum of KLs in Equation (3.3) becomes a single time-integral of a *pointwise* quantity. For masked diffusion the result is a one-line weighted cross-entropy (Equation (5.8)).
4. **Standard numerical machinery becomes available.** Once the reverse process is an ODE on Δ or a jump process with known rates, you can bring higher-order solvers, adaptive steps, predictor–corrector schemes and uniformization to bear.

4.2. The forward process

Let $\mathbf{Q}_t$ be a rate matrix on $\mathcal{X}$ (Definition 2.7). The forward noising process is the CTMC with these rates started from p_{data}; its marginals obey the Kolmogorov forward equation Equation (2.10).

For this to be usable we need $p_{t|0}$ in closed form, which (as in Equation (2.11)) holds whenever $\mathbf{Q}_t = \overline{\sigma}'(t)\mathbf{Q}$ for a fixed $\mathbf{Q}$. We also need it to factorise across the L positions, so $\mathbf{Q}$ is taken to act on one position at a time:

$$[\mathbf{Q}_t]_{yx} = \begin{cases} \overline{\sigma}'(t)\,[\mathbf{Q}^{\text{tok}}]_{y^\ell x^\ell} & \text{if } \boldsymbol{y}, \boldsymbol{x} \text{ differ only at position } \ell, \\ 0 & \text{if } \mathbf{Ham}(\boldsymbol{y}, \boldsymbol{x}) \geq 2. \end{cases} \tag{4.1}$$

This sparsity — **no simultaneous jumps in the forward process** — is the single most consequential structural assumption in discrete diffusion. It is what makes $p_{t|0}$ a product of per-position kernels, what makes the concrete score an $L \times K$ array, and (as we will see in Chapter 8) what makes parallel sampling approximate.

Remark 4.1 (The forward process has no simultaneous jumps, so neither should the exact reverse). Because $\mathbf{Q}_t$ has zero entries at Hamming distance ≥ 2, Equation (2.17) shows the exact reverse rates are *also* zero at Hamming distance ≥ 2. The exact reverse process therefore changes exactly one token at a time, and it needs $\Theta(L)$ jumps to build an L-token sequence. Every parallel sampler is thus, by construction, simulating something the exact reverse process never does. Hold on to this; it is the cleanest statement of why few-step sampling is lossy, and it is the formal content behind the empirical findings of Tang et al. (2026).

4.3. The reverse process

Apply Theorem 2.12. The reverse rates are

$$[\widehat{\mathbf{R}}_t]_{yx} = \frac{p_t(\boldsymbol{y})}{p_t(\boldsymbol{x})}\,[\mathbf{Q}_t]_{xy}, \qquad \boldsymbol{y} \neq \boldsymbol{x}, \tag{4.2}$$

and everything unknown sits in the ratio. There are two ways to get at it, and they correspond to the two halves of the literature.

Route A: learn the ratio directly. Parameterise $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}, t)_{\boldsymbol{y}} \approx p_t(\boldsymbol{y})/p_t(\boldsymbol{x})$ and train it with a proper loss. This is concrete score matching / score entropy, and leads to SEDD (Section 4.7).

Route B: expand over clean data. Marginalise:

$$\boxed{\frac{p_t(\boldsymbol{y})}{p_t(\boldsymbol{x})} = \sum_{\boldsymbol{x}_0} \frac{p_{t|0}(\boldsymbol{y} \mid \boldsymbol{x}_0)}{p_{t|0}(\boldsymbol{x} \mid \boldsymbol{x}_0)}\, p_{0|t}(\boldsymbol{x}_0 \mid \boldsymbol{x}),} \tag{4.3}$$

which is Campbell et al. (2022)'s key identity. The first factor is known analytically from the forward kernel; the second is the *denoising posterior*, which is intractable and is replaced by a learned $p^{\boldsymbol{\theta}}_{0|t}$. So Route B learns a classifier over clean data — the continuous-time version of the $\boldsymbol{x}_0$-parameterisation of Section 3.3.

Derivation of Equation (4.3). Write $p_t(\boldsymbol{y}) = \sum_{\boldsymbol{x}_0} p_{t|0}(\boldsymbol{y} \mid \boldsymbol{x}_0) p_0(\boldsymbol{x}_0)$ and divide by $p_t(\boldsymbol{x})$:

$$\frac{p_t(\boldsymbol{y})}{p_t(\boldsymbol{x})} = \sum_{\boldsymbol{x}_0} \frac{p_{t|0}(\boldsymbol{y} \mid \boldsymbol{x}_0)}{p_{t|0}(\boldsymbol{x} \mid \boldsymbol{x}_0)} \cdot \frac{p_{t|0}(\boldsymbol{x} \mid \boldsymbol{x}_0) p_0(\boldsymbol{x}_0)}{p_t(\boldsymbol{x})},$$

and the second factor is $p_{0|t}(\boldsymbol{x}_0 \mid \boldsymbol{x})$ by Bayes. ∎

Intuition: the two routes are the same model

Equation (4.3) says the concrete score is a posterior average of *known* ratios. So Route A and Route B differ only in which side of the identity you put the network on. The practical difference is conditioning: the posterior $p_{0|t}$ is a normalised probability vector (bounded, softmax-friendly), while the ratio is unbounded and blows up as $t \to 0$. That is why Route B — and in particular masked diffusion — is numerically better behaved, and why Ou et al. (2025) were able to show that the SEDD score *factorises* into exactly such a posterior times an analytic scalar (Chapter 6).

4.4. The continuous-time ELBO

The discrete-time bound Equation (3.3) has a continuous-time limit. For a CTMC the result is classical (it is a Girsanov-type formula for jump processes) and takes the following form.

Theorem 4.2 (Continuous-time ELBO for CTMCs). Let the forward process have rates $\mathbf{Q}_t$ and marginals p_t on $[0, T]$, and let the model reverse process have rates $\widehat{\mathbf{R}}^\theta_t$. Then

$$-\log p_0^\theta(\boldsymbol{x}_0) \;\le\; D_{\mathrm{KL}}\big(p_{T|0}(\cdot \mid \boldsymbol{x}_0) \,\big\|\, \boldsymbol{\pi}\big) + \int_0^T \mathbb{E}_{p_{t|0}(\boldsymbol{x}|\boldsymbol{x}_0)} \sum_{\boldsymbol{y} \neq \boldsymbol{x}} \Big[\big[\widehat{\mathbf{R}}^\theta_t\big]_{\boldsymbol{y}\boldsymbol{x}} - \big[\widehat{\mathbf{R}}_t\big]_{\boldsymbol{y}\boldsymbol{x}} \log \big[\widehat{\mathbf{R}}^\theta_t\big]_{\boldsymbol{y}\boldsymbol{x}} + C_t(\boldsymbol{x}, \boldsymbol{y}) \Big] \, \mathrm{d}t, \tag{4.4}$$

where C_t does not depend on θ.

The structure to notice is that the θ-dependent part of the integrand is exactly

$$b - a \log b, \qquad a = \big[\widehat{\mathbf{R}}_t\big]_{\boldsymbol{y}\boldsymbol{x}},\; b = \big[\widehat{\mathbf{R}}^\theta_t\big]_{\boldsymbol{y}\boldsymbol{x}}, \tag{4.5}$$

which, with the constant $K(a) = a(\log a - 1)$ of Equation (2.7) added, is a Bregman divergence minimised at $b = a$. Equation (4.5) is the *reason* score entropy has the form it does: it is not a design choice, it falls out of the continuous-time likelihood bound. Section 4.7 makes this precise and Chapter B gives the derivation.

4.5. Sampling: τ-leaping and predictor–corrector

Campbell et al. (2022) introduce two ideas that are still the practical backbone.

τLDR (τ-leaping discrete reverse). As in Chapter 2: over a window τ, freeze the rates and draw the number of jumps along each channel as an independent Poisson with mean $\tau[\widehat{\mathbf{R}}^\theta_t]_{\boldsymbol{y}\boldsymbol{x}}$, then apply them all at once. One network evaluation updates all L positions. The approximation has two parts: the rates are not really constant over the window (an $O(\tau)$ discretisation error, familiar), and the Poissons are drawn independently across positions (the factorisation error of Remark 2.4, which does *not* vanish as the model improves).

Predictor–corrector. This is the more interesting contribution. Observe that the rate matrix

$$\mathbf{R}^c_t = \mathbf{Q}_t + \widehat{\mathbf{R}}_t \tag{4.6}$$

has p_t as its *stationary* distribution: running the forward and reverse dynamics simultaneously leaves the marginal unchanged. So after a τ-leaping *predictor* step takes you (approximately) from t to $t - \tau$, a few *corrector* steps with $\mathbf{R}^c_{t-\tau}$ push the sample's distribution back towards the true $p_{t-\tau}$ without advancing time. In practice $\widehat{\mathbf{R}}_t$ is replaced by the learned $\widehat{\mathbf{R}}^\theta_t$.

Proposition 4.3 (Stationarity of the corrector). With $\widehat{\mathbf{R}}_t$ the exact reverse rates of Equation (4.2), $\mathbf{R}_t^c p_t = 0$.

Proof. From the proof of Theorem 2.12, $\widehat{\mathbf{R}}_t p_t = -\mathbf{Q}_t p_t$. Hence $\mathbf{R}_t^c p_t = \mathbf{Q}_t p_t + \widehat{\mathbf{R}}_t p_t = 0$. ∎

Intuition: correctors buy accuracy with compute, not training

A predictor step moves you forward in time and introduces error. A corrector step spends a network evaluation on *reducing* error at the current time instead of advancing. Proposition 4.3 says the corrector has the right fixed point, so this is not a heuristic. This is the cleanest "inference-time scaling" mechanism in discrete diffusion, and the remasking samplers of Chapter 8 are best understood as a particularly effective family of correctors — indeed Wang et al. (2025b) present ReMDM explicitly as a predictor–corrector method that generalises earlier correctors.

4.6. Uniformization

An alternative to τ-leaping that deserves to be better known. If the total jump rate is bounded by Λ, a CTMC can be simulated *exactly* by drawing jump times from a Poisson process of rate Λ and, at each such time, making a transition according to a discrete-time kernel $\mathbf{I} + \Lambda^{-1}\mathbf{R}$ (which may be a self-transition). There is no discretisation error; the only cost is a random number of steps.

This matters because it is the setting in which the sharpest convergence guarantees are available: Liang et al. (2025) show that for absorbing-state diffusion, uniformization achieves $D_{\mathrm{KL}}(q_\delta \,\|\, p_{T-\delta}) \lesssim d\, e^{-T} + \varepsilon'_{\text{score}}$ with an expected number of steps $\mathbb{E}[N] = O\big(d(\log\log(d/\varepsilon) + \log \delta^{-1})\big)$ — that is, *no* explicit discretisation term at all, unlike τ-leaping. The cost is $\Theta(d)$ steps, so uniformization is the tool for "how accurate can this be in principle" rather than for fast sampling. Chapter 13 collects these results.

4.7. The concrete score and score entropy

Score Entropy Discrete Diffusion (SEDD) (Lou et al., 2024) was the paper that made discrete diffusion competitive with autoregressive language models, and it won the ICML 2024 best paper award for it. The contribution is a *loss function*: the right way to regress onto the concrete score. This chapter derives it, explains why the obvious alternative fails, and covers the samplers SEDD introduced.

4.7.1. The problem with squared error

We want $\boldsymbol{s}_\theta(\boldsymbol{x}, t)_{\boldsymbol{y}} \approx p_t(\boldsymbol{y})/p_t(\boldsymbol{x})$. The natural first attempt, due to Meng et al. (2022), is to mimic Equation (2.39) and regress in squared error onto the conditional ratio. The projection argument works — the minimiser is the marginal ratio — but the loss is badly behaved, for a reason worth stating precisely.

Remark 4.4 (Scale mismatch). The targets $p_t(\boldsymbol{y})/p_t(\boldsymbol{x})$ are positive and unbounded. Under squared error, an error of 0.1 on a target of 1 and an error of 0.1 on a target of 10^6 contribute equally, even though the first is a 10% relative error and the second is 10^{-7}. Worse, as $t \to 0$ the ratios diverge (the data distribution is near-deterministic, so neighbouring states have wildly different probabilities), so the squared loss is dominated entirely by the lowest-noise region and by the largest ratios in it. What we want is a loss that is scale-free in the target: one that penalises *relative* error.

There is a standard construction for exactly this.

4.7.2. Score entropy

Definition 4.5 (Score entropy). With weights $w_{xy} \geq 0$ and $K(a) = a(\log a - 1)$,

$$\mathcal{L}_{\mathrm{SE}} = \mathbb{E}_{x\sim p}\left[\sum_{y\neq x} w_{xy}\left(s_\theta(x)_y - \frac{p(y)}{p(x)}\log s_\theta(x)_y + K\left(\frac{p(y)}{p(x)}\right)\right)\right]. \tag{4.7}$$

Proposition 4.6 (Score entropy is a proper loss). For $a > 0$, the map $b \mapsto b - a\log b + K(a)$ on $(0,\infty)$ is strictly convex, non-negative, and vanishes exactly at $b = a$. Consequently $\mathcal{L}_{\mathrm{SE}} \geq 0$ with equality iff $s_\theta(x)_y = p(y)/p(x)$ for all x in the support of p and all y with $w_{xy} > 0$.

Proof. Let $f(b) = b - a\log b + a\log a - a$. Then $f'(b) = 1 - a/b$, which vanishes only at $b = a$, and $f''(b) = a/b^2 > 0$, so f is strictly convex with a unique minimum at $b = a$. Evaluating, $f(a) = a - a\log a + a\log a - a = 0$. Non-negativity and the equality condition follow. ∎

Proposition 4.6 says score entropy is *a* valid loss. The next proposition says what it *is*, and that identification is what makes its properties free rather than earned one at a time.

Proposition 4.7 (Score entropy is the Bregman divergence of $\varphi(b) = b\log b - b$). For a differentiable strictly convex φ on $(0,\infty)$, the Bregman divergence it generates is $D_\varphi(a\|b) := \varphi(a) - \varphi(b) - \varphi'(b)(a-b)$. Taking $\varphi(b) = b\log b - b$, so that $\varphi'(b) = \log b$,

$$D_\varphi(a\,\|\,b) \;=\; b - a\log b + K(a), \qquad K(a) = a(\log a - 1), \tag{4.8}$$

which is exactly the summand of Equation (4.7). Hence

$$\mathcal{L}_{\mathrm{SE}} = \mathbb{E}_{x\sim p}\Big[\sum_{y\neq x} w_{xy}\, D_\varphi\Big(\tfrac{p(y)}{p(x)}\,\Big\|\, s_\theta(x)_y\Big)\Big]. \tag{4.9}$$

Proof. Substitute: $D_\varphi(a\|b) = (a\log a - a) - (b\log b - b) - \log b\,(a-b) = a\log a - a - b\log b + b - a\log b + b\log b = b - a\log b + a(\log a - 1)$. Equation (4.9) is Equation (4.7) term by term. Numerically checked in Listing C.3. ∎

Three facts now come for free, each of which would otherwise need its own argument.

Corollary 4.8 (Properties inherited from the Bregman structure).

1. *(Properness.)* $D_\varphi(a\|b) \geq 0$ with equality iff $a = b$, by strict convexity of φ — which re-proves Proposition 4.6 with no computation.
2. *(The unique minimiser is the conditional mean.)* For any random $A > 0$, $\arg\min_b \mathbb{E}[D_\varphi(A\|b)] = \mathbb{E}[A]$, and this holds for *every* Bregman divergence. This is precisely why the denoising form Equation (4.11), whose target is a conditional expectation of ratios, recovers the marginal ratio (Theorem 4.9): a Bregman loss regresses onto conditional means, so conditioning on less information gives the marginal.
3. *(Generalised KL.)* If the arguments happen to be normalised, $\sum_i a_i = \sum_i b_i = 1$, then $\sum_i D_\varphi(a_i\|b_i) = \sum_i a_i \log \frac{a_i}{b_i} = D_{\mathrm{KL}}(a\,\|\,b)$. Score entropy is therefore KL extended to unnormalised positive measures, and Equation (4.13) — a likelihood bound expressed through a loss on unnormalised ratios — stops being a surprise.

Proof. (i) is the standard first-order characterisation of strict convexity. For (ii), $\mathbb{E}[D_\varphi(A\|b)] = \mathbb{E}[\varphi(A)] - \varphi(b) - \varphi'(b)(\mathbb{E}[A] - b)$; differentiating in b gives $-\varphi''(b)(\mathbb{E}[A] - b)$, which vanishes only at $b = \mathbb{E}[A]$ since $\varphi'' > 0$. For (iii), the b and $-a$ terms in Equation (4.8) sum to $1 - 1 = 0$ across i, leaving $\sum_i (a_i \log a_i - a_i \log b_i)$. ∎

Intuition: this is just cross-entropy for unnormalised targets

Compare Equation (4.7) with cross-entropy $-\sum_i a_i \log b_i$. If the a_i and b_i were normalised, $\sum_i b_i$ would be the constant 1 and the first term would disappear, leaving exactly cross-entropy. Score entropy is what cross-entropy becomes when the "distributions" are unnormalised positive measures: the $+b$ term is the normalisation penalty that stops the model from simply inflating all its outputs, and $K(a)$ is the constant that makes the minimum zero. Equivalently, it is the Bregman divergence generated by $b \log b$, i.e. a *generalised KL* divergence. That also explains why it is scale-free in the sense Remark 4.4 demands: it penalises b/a, not $b - a$.

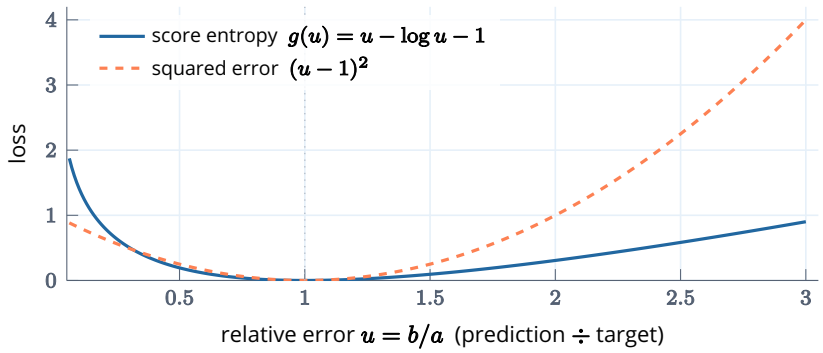


Figure 4.1 Score entropy versus squared error, as functions of the *relative* error. Writing $u = b/a$ for the predicted ratio divided by the true one, the score-entropy integrand of Equation (4.7) is exactly $a\, g(u)$ with $g(u) = u - \log u - 1$, while squared error is $a^2(u-1)^2$. Two consequences. First, the *shape* of the score-entropy penalty depends only on relative error, and its prefactor grows like a; the squared-error prefactor grows like a^2, so a coordinate whose true ratio is 10^6 is weighted 10^{12} times more heavily than one whose ratio is 1 (Remark 4.4). Second, g is strongly asymmetric: badly *under*-predicting a ratio ($u \to 0$) is penalised without bound, which is the behaviour you want when the target is a likelihood ratio.

Making it computable

Equation (4.7) is not usable as written, because it contains the unknown $p(\boldsymbol{y})/p(\boldsymbol{x})$. There are two standard repairs, exactly parallel to implicit and denoising score matching.

Implicit score entropy. Integrate by parts (in the discrete sense of exchanging the roles of $\boldsymbol{x}$ and $\boldsymbol{y}$ in the expectation) to remove the unknown ratio:

$$\mathcal{L}_{\mathrm{ISE}} = \mathbb{E}_{\boldsymbol{x}\sim p}\left[\sum_{\boldsymbol{y}\neq\boldsymbol{x}} \Big(w_{\boldsymbol{x}\boldsymbol{y}}\, \boldsymbol{s}_\theta(\boldsymbol{x})_{\boldsymbol{y}} - w_{\boldsymbol{y}\boldsymbol{x}}\, \log \boldsymbol{s}_\theta(\boldsymbol{y})_{\boldsymbol{x}} \Big)\right]. \tag{4.10}$$

This differs from $\mathcal{L}_{\mathrm{SE}}$ by a θ-independent constant and needs only samples from p. Like implicit score matching it has high variance.

Denoising score entropy. The practical choice. Condition on clean data, where the ratio is known analytically from the forward kernel:

$$\mathcal{L}_{\mathrm{DSE}} = \mathbb{E}_{\boldsymbol{x}_0 \sim p_0}\, \mathbb{E}_{\boldsymbol{x} \sim p_{t|0}(\cdot|\boldsymbol{x}_0)} \left[\sum_{\boldsymbol{y} \neq \boldsymbol{x}} w_{\boldsymbol{x}\boldsymbol{y}} \left(\boldsymbol{s}_\theta(\boldsymbol{x})_{\boldsymbol{y}} - \frac{p_{t|0}(\boldsymbol{y} \mid \boldsymbol{x}_0)}{p_{t|0}(\boldsymbol{x} \mid \boldsymbol{x}_0)} \log \boldsymbol{s}_\theta(\boldsymbol{x})_{\boldsymbol{y}} \right) \right]. \tag{4.11}$$

Only samples from p_0 and from the forward kernel are needed, and the ratio inside is a closed-form expression in α_t (or $\bar{\sigma}(t)$) by Proposition 2.9. The justification is the following projection result, the exact analogue of the one that makes denoising score matching work.

Theorem 4.9 (DSE recovers the marginal score). The minimiser of Equation (4.11) over all functions $\boldsymbol{s}_\theta$ satisfies

$$\boldsymbol{s}_\theta(\boldsymbol{x})_{\boldsymbol{y}} = \mathbb{E}_{\boldsymbol{x}_0 \sim p_{0|t}(\cdot|\boldsymbol{x})} \left[\frac{p_{t|0}(\boldsymbol{y} \mid \boldsymbol{x}_0)}{p_{t|0}(\boldsymbol{x} \mid \boldsymbol{x}_0)} \right] = \frac{p_t(\boldsymbol{y})}{p_t(\boldsymbol{x})}. \tag{4.12}$$

Proof. For fixed $\boldsymbol{x}, \boldsymbol{y}$ the objective is $b - \bar{a} \log b$ up to a constant, where $b = \boldsymbol{s}_\theta(\boldsymbol{x})_{\boldsymbol{y}}$ and $\bar{a} = \mathbb{E}_{\boldsymbol{x}_0|\boldsymbol{x}}[p_{t|0}(\boldsymbol{y} \mid \boldsymbol{x}_0)/p_{t|0}(\boldsymbol{x} \mid \boldsymbol{x}_0)]$ is the posterior mean of the conditional ratio. By Proposition 4.6 the minimiser is $b = \bar{a}$. That $\bar{a}$ equals the marginal ratio is Equation (4.3). ∎

This is the single most important technical fact in this section, and it is verified numerically to machine precision (`3.33e-16`) by Listing 2.2 in Chapter 2.

4.7.3. Diffusion-weighted DSE and the likelihood bound

So far the weights $w_{\boldsymbol{x}\boldsymbol{y}}$ are arbitrary. Theorem 4.2 fixes them: choosing $w_{\boldsymbol{x}\boldsymbol{y}} = [\mathbf{Q}_t]_{\boldsymbol{x}\boldsymbol{y}}$ and integrating over t turns Equation (4.11) into the θ-dependent part of the continuous-time ELBO. SEDD calls the result *diffusion-weighted denoising score entropy*, and the consequence is a genuine likelihood bound (Lou et al., 2024, Thm. 3.6):

$$-\log p_0^\theta(\boldsymbol{x}_0) \;\leq\; \mathcal{L}_{\mathrm{DWDSE}}(\boldsymbol{x}_0) \;+\; D_{\mathrm{KL}}\big(p_{T|0}(\cdot \mid \boldsymbol{x}_0) \,\big\|\, \boldsymbol{\pi}\big)\,. \tag{4.13}$$

This is what makes SEDD a likelihood-based model and lets it report perplexities comparable to autoregressive ones. Note the structure: a trainable term plus a prior-mismatch term that vanishes as T grows.

Intuition: why the diffusion weighting is the right one

The weights decide which of the $L(K-1)$ ratios you spend capacity on. The ELBO's answer is: weight each ratio by the forward rate of the corresponding jump. Transitions the forward process never makes (rate zero) are never needed by the reverse process either (Equation (4.2)), so they get weight zero and cost nothing. For absorbing noise this is drastic — the forward process only ever jumps *to* `[MASK]`, so the reverse process only ever needs ratios for jumps *from* `[MASK]`, and $L(K-1)$ unknowns collapse to the K logits at each masked position. That collapse is the content of Chapter 5.

4.7.4. Sampling

SEDD contributes two samplers beyond Euler and τ-leaping.

Tweedie τ-leaping. Rather than treating the reverse step as a jump process, use Bayes' rule $p_{0|t}(\boldsymbol{x}_0 \mid \boldsymbol{x}_t) = p_{t|0}(\boldsymbol{x}_t \mid \boldsymbol{x}_0) p_0(\boldsymbol{x}_0) / p_t(\boldsymbol{x}_t)$ to obtain the *exact* conditional $p_{t-\Delta t|t}$ in terms of the score, and sample from that. This is the discrete analogue of Tweedie's formula (compare the box in Section 2.8) and corresponds to Lou et al. (2024, Thm. 4.1–4.2).

Analytic (Tweedie) sampling. Because the forward kernel is the matrix exponential Equation (2.11), the transition from t to $t - \Delta t$ can be assembled analytically from the score: the transition density is built as a product of $\exp\big((\overline{\sigma}(t - \Delta t) - \overline{\sigma}(t))\mathbf{Q}\big)$ applied to $\boldsymbol{s}_\theta$ and $\exp\big((\overline{\sigma}(t) - \overline{\sigma}(t - \Delta t))\mathbf{Q}\big)$ evaluated at the current state, with the result clamped to be non-negative and renormalised (Lou et al., 2024, Alg. 2).

Remark 4.10 (Why this is a big deal for step counts). Analytic sampling handles the *known* part of the dynamics ($\mathbf{Q}$) exactly and only approximates the *learned* part. It is an exponential integrator, in the language of ODE solvers. This is where SEDD's headline claim that it matches quality "with about $32\times$ fewer network evaluations" comes from, and it is the clearest existing example of the higher-order-solver research direction flagged at the end of Chapter 4.

Noise schedules. SEDD uses either a geometric schedule $\overline{\sigma}(t) = \sigma_{\min}^{1-t}\sigma_{\max}^{t}$ or a log-linear schedule $\overline{\sigma}(t) = -\log(1-(1-\varepsilon)t)$. The log-linear choice is the one that makes the absorbing case equivalent to $\alpha_t = 1 - (1-\varepsilon)t$, i.e. linear masking.

4.7.5. What SEDD achieved

Taking the abstract's own claims (Lou et al., 2024): SEDD reduces perplexity by roughly 25–75% relative to prior language diffusion models at comparable size, beats GPT-2 on its zero-shot perplexity tasks, and attains roughly 6–8× better *generative* perplexity than un-annealed GPT-2 without temperature scaling, at similar quality with about $32\times$ fewer network evaluations. On OpenWebText at GPT-2 scale, SEDD reaches test perplexity 24.10, which MDLM subsequently improved to 23.21 (Sahoo et al., 2024).

Two caveats, both important and both from later work.

- The generative-perplexity comparison is exactly the kind of claim that Franca and Tong (2026) show can be gamed: they report that MDLM's generative perplexity on OpenWebText (48.3) is less than half SEDD's (125.7) while judging sample quality similar, and that MAUVE actually *favours* SEDD (0.871 vs 0.783). Read Section 10.8 before taking any generative-perplexity number at face value, including SEDD's.
- For the absorbing case, Ou et al. (2025) showed the SEDD score factorises into a time-independent conditional distribution times an analytic scalar, which means much of SEDD's machinery is unnecessary for masked diffusion specifically — and that RADD converges faster for similar final quality. See Chapter 6.

Listing 4.1 trains a concrete-score model on a small problem where the true score is computable, so you can watch Theorem 4.9 happen.

Listing 4.1 Training a concrete-score network with denoising score entropy, and comparing the learned score to the exact one.

```
"""Train a concrete-score network with Denoising Score Entropy (SEDD) on a
problem small enough that the exact concrete score is computable, and check
that the learned score converges to it.

Setting: single categorical variable, K states, uniform-noise CTMC.
    p_{t|0} = exp(sigmabar(t) Q),     Q = 11^T - K I
Target:  s(x)_y = p_t(y) / p_t(x).
```

```
"""
import numpy as np
import torch
import torch.nn as nn
from scipy.linalg import expm

torch.manual_seed(0)
rng = np.random.default_rng(0)

K = 8
Q = np.ones((K, K)) - K * np.eye(K)            # column sums zero
sigbar = lambda t: 6.0 * t                     # total noise
p0 = rng.dirichlet(np.ones(K) * 1.5)           # the data distribution


def kernel(t):
    """p_{t|0}[x, x0] = p(x_t = x | x_0 = x0), in closed form."""
    e = np.exp(-K * sigbar(t))
    return e * np.eye(K) + (1.0 - e) / K * np.ones((K, K))


def exact_score(t):
    """s(x)_y = p_t(y)/p_t(x), zero on the diagonal."""
    p = kernel(t) @ p0
    S = p[None, :] / p[:, None]
    np.fill_diagonal(S, 0.0)
    return S


# ---- the model: a tiny MLP mapping (x, t) -> K positive numbers ------------
class ScoreNet(nn.Module):
    def __init__(self, hidden=128):
        super().__init__()
        self.emb = nn.Embedding(K, hidden)
        self.time = nn.Sequential(nn.Linear(1, hidden), nn.SiLU(),
                                  nn.Linear(hidden, hidden))
        self.net = nn.Sequential(nn.SiLU(), nn.Linear(hidden, hidden),
                                 nn.SiLU(), nn.Linear(hidden, K))

    def forward(self, x, t):
        h = self.emb(x) + self.time(t[:, None])
        return self.net(h).exp()               # positive by construction


def dse_loss(model, batch_size=1024):
    """DSE with the diffusion weighting w_xy = Q[x, y] (constant = 1 here)."""
    t = torch.rand(batch_size) * 0.98 + 0.01
    tn = t.numpy()
    x0 = rng.choice(K, size=batch_size, p=p0)

    # sample x_t ~ p_{t|0}(. | x0) and build the conditional ratio targets
    e = np.exp(-K * sigbar(tn))                                # survive prob
    stay = rng.random(batch_size) < e
    xt = np.where(stay, x0, rng.integers(0, K, batch_size))
    #   p_{t|0}(y|x0) = e*1[y==x0] + (1-e)/K    so the ratio is closed form
    num = (1.0 - e)[:, None] / K + e[:, None] * (
        np.arange(K)[None, :] == x0[:, None])
    den = (1.0 - e) / K + e * (xt == x0)
    ratio = num / den[:, None]                                 # (B, K)

```

```
    xt_t = torch.as_tensor(xt, dtype=torch.long)
    s = model(xt_t, t)                                  # (B, K)
    r = torch.as_tensor(ratio, dtype=torch.float32)
    off = torch.ones(batch_size, K)
    off[torch.arange(batch_size), xt_t] = 0.0           # exclude y == x
    #  score entropy integrand:  s - r log s   (the K(r) term is constant)
    return ((s - r * s.clamp_min(1e-12).log()) * off).sum(-1).mean()


model = ScoreNet()
opt = torch.optim.Adam(model.parameters(), lr=2e-3)
for step in range(2001):
    loss = dse_loss(model)
    opt.zero_grad(); loss.backward(); opt.step()
    if step % 500 == 0:
        # how close is the learned score to the exact one?
        errs = []
        for t in (0.1, 0.3, 0.6):
            with torch.no_grad():
                pred = model(torch.arange(K), torch.full((K,), t)).numpy()
            tgt = exact_score(t)
            mask = ~np.eye(K, dtype=bool)
            errs.append(np.abs(pred[mask] / tgt[mask] - 1).mean())
        print(f"step {step:5d}  DSE {loss.item():8.4f}   "
              f"mean relative score error: "
              + "  ".join(f"t={t}: {e:6.2%}" for t, e in
                          zip((0.1, 0.3, 0.6), errs)), flush=True)
```

Its output, on a single categorical variable with $K = 8$ under uniform noise:

```
step    0   DSE 7.0669   mean relative score error:   t=0.1: 5.34%   t=0.3: 5.38%   t=0.6: 5.51%
step  500   DSE 6.9862                                t=0.1: 1.31%   t=0.3: 0.87%   t=0.6: 0.50%
step 1000   DSE 6.9670                                t=0.1: 0.89%   t=0.3: 0.41%   t=0.6: 0.52%
step 2000   DSE 6.9681                                t=0.1: 1.16%   t=0.3: 0.45%   t=0.6: 0.31%
```

Two things to notice. First, Theorem 4.9 is doing what it promises: optimising a loss that never sees the marginal ratio drives the network towards it, to about 1% relative error. Second, the loss value barely moves ($7.067 \to 6.968$) while the score error falls fivefold — the DSE objective is dominated by a large θ-independent offset, so the loss curve is an almost useless training diagnostic. Track a held-out bound, or a direct score error if you can compute one, instead.

Key takeaways

- Continuous time removes T from the model, tightens the bound, simplifies the objective to a time-integral, and opens up standard numerical machinery.
- The forward rate matrix is token-wise sparse (Equation (4.1)). Consequently the *exact* reverse process also changes one token at a time and needs $\Theta(L)$ jumps (Remark 2.4 and the remark following Equation (4.1)). Parallel samplers are approximations by construction.
- The unknown in the reverse process can be learned directly as a ratio (Route A, score entropy) or expanded over clean data via Equation (4.3) (Route B, x_0-prediction). They are the same model, differently conditioned.

- The continuous-time ELBO's integrand is $b - a \log b$ (Equation (4.5)), which is why score entropy rather than squared error is the right loss — it is forced by the bound, not chosen.
- τ-leaping gives parallel updates; predictor–corrector (Proposition 4.3) spends compute on accuracy at fixed time and has the correct fixed point; uniformization is exact but costs $\Theta(L)$ steps.
- Squared error is the wrong loss for ratios: the targets are positive and unbounded, so L^2 penalises absolute rather than relative error (Remark 4.4).
- Score entropy (Definition 4.5) is the Bregman divergence generated by $b \log b$ — a generalised KL. It is cross-entropy for unnormalised targets, is non-negative, and is minimised exactly at the true ratio (Proposition 4.6).
- Denoising score entropy Equation (4.11) replaces the unknown marginal ratio with the known conditional ratio; its minimiser is still the marginal ratio (Theorem 4.9). This is verified to machine precision in Listing 2.2.
- Weighting by the forward rates makes DSE the θ-dependent part of the continuous-time ELBO and yields the likelihood bound Equation (4.13).
- Analytic (Tweedie) sampling treats the known matrix-exponential part of the dynamics exactly and is the reason SEDD needs far fewer network evaluations than naive Euler.

Open questions and research directions

1. **Higher-order solvers for discrete reverse processes.** Continuous diffusion gained an order of magnitude in sampling efficiency from DPM-Solver and friends, which exploit the semi-linear structure of the probability-flow ODE. The reverse CTMC also has structure — $\widehat{\mathbf{R}}_t$ is a known sparse matrix times a learned ratio — but essentially all samplers in use are first-order Euler or τ-leaping. Exponential integrators that treat the analytic part of $\mathbf{Q}_t$ exactly (which is what SEDD's "analytic" sampler does in a limited way) seem underexplored as a general principle.
2. **Adaptive step sizes.** Nothing in standard practice adapts τ to the local stiffness of the reverse process, even though the rates are known at every step and the error criteria of ODE solving transfer directly. Park et al. (2025) optimise the step schedule offline; online adaptivity is open.
3. **How much can correctors actually buy?** Proposition 4.3 guarantees the right fixed point but says nothing about the *rate* at which corrector steps reduce error, which is what determines whether to spend your nth network evaluation on a predictor or a corrector step. A mixing-time analysis of $\mathbf{R}_t^c$ — its spectral gap as a function of t and of the forward process — would turn corrector scheduling from a hyperparameter into a calculation.
4. **Non-sparse forward processes.** Equation (4.1) is assumed purely for tractability, and the remark after it shows it is precisely what forces $\Theta(L)$ exact reverse steps. A forward process with controlled multi-site jumps would have an exact reverse that *does* move several tokens at once. The ELBO machinery does not care. Nobody has built one at scale, and it is the most direct possible attack on the central limitation of the field.
5. **Variance of the continuous-time estimator.** The time-integral in Equation (4.4) is estimated by sampling t, and for the absorbing case the integrand carries a $1/t$-

type weight whose variance is large (we see this concretely in Chapter 5: a 2×10^5-sample Monte Carlo estimate of a quantity whose exact value is 11.73 returns 11.53). Variance-reduced estimators — importance sampling in t, antithetic masking, Rao–Blackwellisation beyond what MDLM already does — are an active and still-open area (Jia et al., 2025).

6. **Other Bregman divergences.** Score entropy is the Bregman divergence generated by $b \log b$, and that choice is forced if you want the loss to *be* the ELBO. But if you are willing to give up the exact bound — which practitioners do all the time, cf. Equation (3.5) — the whole family of Bregman divergences on $(0, \infty)$ is available, each with different variance and tail behaviour. Shi and Titsias (2025) show some reweighted losses are in fact *tighter* bounds, which suggests the space of bound-preserving losses is larger than assumed and is not well charted.
7. **Score entropy for uniform noise at large K.** The absorbing case collapses to K logits per position. The uniform case genuinely needs all $L(K-1)$ ratios, and the loss sums over $K-1$ terms per position. At $K = 50{,}000$ this is the dominant cost and is why uniform-state diffusion has been under-explored at scale. Sparse, sampled, or hierarchical estimators of the score-entropy sum are an obvious need, especially given that Sahoo et al. (2026) find uniform-state diffusion competitive at 1.7B.
8. **Is the ratio ever the right thing to learn?** Ou et al. (2025) show that for absorbing noise it is not — the time-dependence factors out analytically. Whether a similar factorisation exists for uniform or general CTMCs is open, and a negative result would be just as interesting: it would tell us that absorbing noise is special in a way that explains its dominance.
9. **Target concrete score matching.** Zhang et al. (2025a) propose estimating the concrete score of the *clean* target distribution rather than the noisy intermediates, which lets reward functions and pretrained autoregressive teachers — both of which only operate on clean data — enter the objective directly, and which subsumes many existing methods as special cases. This reframing is recent and its consequences (particularly for distillation from AR models) are barely mapped.
10. **Variance of DSE near $t = 0$.** The conditional ratios diverge as $t \to 0$, so the estimator of Equation (4.11) has heavy tails exactly where the model matters most, and practice handles this with an ad hoc ε cutoff. A principled importance-sampling or control-variate scheme in t — or a reparameterisation that removes the divergence, as RADD effectively does for the masked case — is missing for the general case.

CHAPTER 5

The Interpolating Family: Masked and Uniform Diffusion

CHAPTER

Authors	Subham Sekhar Sahoo et al.; Jiaxin Shi et al.; Yair Schiff et al.; Patrick Pynadath et al.
Year	2026
Published	NeurIPS 2024 / ICLR 2025 / ICML 2025–26
Link	https://arxiv.org/abs/2406.07524
Tags	masked diffusion, SUBS, uniform noise, interpolating family, diffusion duality

This is the most important chapter in the book. Masked (absorbing-state) diffusion is what every large diffusion language model uses, and its objective is simple enough to write on a napkin. Two concurrent papers — MDLM (Sahoo et al., 2024) and MD4 (Shi et al., 2024) — independently derived essentially the same simplification, and a third, RADD (Ou et al., 2025), explained why it had to be so. We derive it here and cover the equivalences in Chapter 6.

5.1. The forward process

Augment the vocabulary with `[MASK]` and let $\mathbf{m} = \mathbf{e}_{\texttt{[MASK]}}$. Each token independently survives with probability α_t and is otherwise replaced by `[MASK]`, where α_t decreases from $\alpha_0 \approx 1$ to $\alpha_1 \approx 0$. From Proposition 2.9, for one position,

$$q(z_t \mid x) = \mathrm{Cat}\big(z_t;\ \alpha_t x + (1-\alpha_t)\mathbf{m}\big), \qquad q(z_t \mid z_s) = \mathrm{Cat}\big(z_t;\ \alpha_{t|s} z_s + (1-\alpha_{t|s})\mathbf{m}\big), \tag{5.1}$$

with $\alpha_{t|s} = \alpha_t / \alpha_s$ for $s < t$. The two structural facts that drive everything:

1. **Masking is absorbing.** Once a token is `[MASK]` it stays `[MASK]` under the forward process. Reversed, this means an unmasked token is never re-masked and is never changed.
2. **The model is told where the errors are.** The reverse process knows exactly which positions need work: the masked ones. This is the entire reason masked diffusion is easier than uniform diffusion.

5.2. The true posterior

For $s < t$ we need $q(z_s \mid z_t, x)$. The absorbing structure makes this almost trivial by cases.

Proposition 5.1 (Masked posterior).

$$q(z_s \mid z_t, x) = \begin{cases} \mathrm{Cat}(z_s;\ z_t), & z_t \neq \mathbf{m}, \\ \mathrm{Cat}\left(z_s;\ \dfrac{(1-\alpha_s)\mathbf{m} + (\alpha_s - \alpha_t)x}{1-\alpha_t}\right), & z_t = \mathbf{m}. \end{cases} \tag{5.2}$$

Proof. If $z_t \neq \mathbf{m}$ then the token survived to time t; since masking is absorbing and $s < t$, it must also have been unmasked at s with the same value. Hence $z_s = z_t$ with probability one.

If $z_t = \mathbf{m}$, apply Bayes within the two possibilities at time s. By Equation (5.1), $\mathbf{Pr}[z_s = x] = \alpha_s$ and $\mathbf{Pr}[z_s = \mathbf{m}] = 1 - \alpha_s$. Given $z_s = \mathbf{m}$ we have $z_t = \mathbf{m}$ with probability 1; given $z_s = x$ we have $z_t = \mathbf{m}$ with probability $1 - \alpha_{t|s} = 1 - \alpha_t/\alpha_s$. So the unnormalised posterior weights are

$$\Pr[z_s = \mathbf{m} \mid z_t = \mathbf{m}] \propto (1 - \alpha_s) \cdot 1, \qquad \Pr[z_s = x \mid z_t = \mathbf{m}] \propto \alpha_s\Big(1 - \tfrac{\alpha_t}{\alpha_s}\Big) = \alpha_s - \alpha_t,$$

whose sum is $1 - \alpha_t$. Dividing gives Equation (5.2). ∎

Intuition: read off the probability of unmasking

Equation (5.2) says: a masked token at time t gets revealed on the way to time s with probability

$$\frac{\alpha_s - \alpha_t}{1 - \alpha_t}, \tag{5.3}$$

and if revealed, it takes its true value x. That is the whole reverse process. With $\alpha_t = 1 - t$ this is $(t - s)/t$: of the t fraction currently masked, you reveal the fraction that "should" be unmasked by time s. The sampler in Listing 5.1 is four lines because of this.

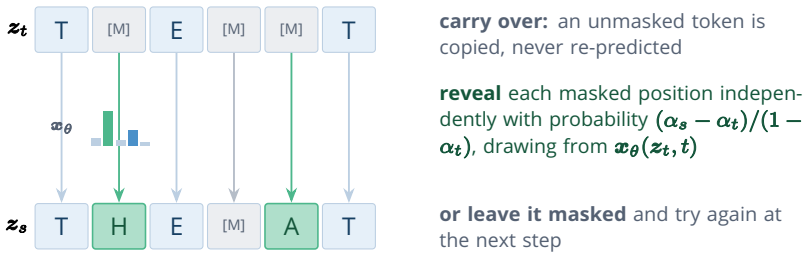


Figure 5.1 One reverse step of a masked diffusion model, from noise level t to $s < t$, which is the whole of Proposition 5.1. Unmasked positions are copied unchanged — the "carry-over unmasking" half of SUBS. Each masked position is independently revealed with probability $(\alpha_s - \alpha_t)/(1 - \alpha_t)$ and, if revealed, takes a sample from the denoiser's distribution over clean tokens at that position; otherwise it stays masked. Note that the positions revealed in the same step are drawn *independently*, which is precisely the approximation of Remark 2.4 and the reason few-step samples go wrong.

5.3. The SUBS parameterisation

The model's reverse kernel $p_\theta(z_s \mid z_t)$ is obtained by replacing the unknown x in Equation (5.2) by a network prediction $x_\theta(z_t, t) \in \Delta^K$. Sahoo et al. (2024) point out that two constraints are implied by Equation (5.2) and should be *built into the parameterisation* rather than learned. They call the pair **SUBS**.

1. **Zero masking probabilities:** $\langle x_\theta(z_t, t), \mathbf{m} \rangle = 0$. The clean data never contains `[MASK]`, so the model should not be able to predict it. Implementation: omit the `[MASK]` logit from the output head entirely, or set it to $-\infty$ before the softmax.
2. **Carry-over unmasking:** $x_\theta(z_t, t) = z_t$ whenever $z_t \neq \mathbf{m}$. Unmasked tokens are copied, not re-predicted. Implementation: overwrite the output at unmasked positions with the input one-hot.

Substituting, the model reverse kernel is

$$p_\theta(z_s \mid z_t) = \begin{cases} \mathrm{Cat}(\boldsymbol{z}_s;\ \boldsymbol{z}_t), & \boldsymbol{z}_t \neq \mathbf{m}, \\ \mathrm{Cat}\left(\boldsymbol{z}_s;\ \dfrac{(1-\alpha_s)\mathbf{m} + (\alpha_s - \alpha_t)\boldsymbol{x}_\theta(\boldsymbol{z}_t, t)}{1-\alpha_t}\right), & \boldsymbol{z}_t = \mathbf{m}. \end{cases} \tag{5.4}$$

Remark 5.2 (Why SUBS matters more than it looks). These two constraints do three things at once.

1. They make the KL term in the ELBO *vanish identically* at unmasked positions, since both posterior and model put all mass on $\boldsymbol{z}_t$. This is what allows the Rao–Blackwellisation of Section 5.4.
2. They remove the possibility of the model "un-deciding": the sampler can never revisit a token. This is simultaneously the reason the objective is simple and the reason masked diffusion cannot correct its own mistakes — the failure that Wang et al. (2025b) attack with remasking (Chapter 8).
3. They make the model's output *time-independent in form*: it is always "a distribution over clean tokens". Ou et al. (2025) take this to its conclusion and show the network need not be conditioned on t at all.

5.4. Rao–Blackwellisation and the simplified bound

Now compute the ELBO. Discretise $[0,1]$ into T steps with $t(i) = i/T$, $s(i) = (i-1)/T$. The generic bound Equation (3.3) has three terms. The prior term vanishes because $\alpha_1 = 0$ makes $q(\boldsymbol{z}_1 \mid \boldsymbol{x}) = \delta_{\mathbf{m}}$ exactly. The reconstruction term vanishes because $\alpha_0 = 1$. What remains is the diffusion term, and by Proposition 5.1 and SUBS it collapses.

Theorem 5.3 (Discrete-time masked NELBO). Under Equation (5.1) and the SUBS parameterisation Equation (5.4),

$$\mathcal{L}_T = \sum_{i=1}^{T} \mathbb{E}_q \left[\frac{\alpha_{t(i)} - \alpha_{s(i)}}{1 - \alpha_{t(i)}} \log \langle \boldsymbol{x}_\theta(\boldsymbol{z}_{t(i)}, t(i)),\, \boldsymbol{x} \rangle \right], \tag{5.5}$$

where the expectation is over the forward process and only masked positions contribute.

Derivation. Fix a step i and a position. If $\boldsymbol{z}_t \neq \mathbf{m}$, both $q(\boldsymbol{z}_s \mid \boldsymbol{z}_t, \boldsymbol{x})$ and $p_\theta(\boldsymbol{z}_s \mid \boldsymbol{z}_t)$ equal $\mathrm{Cat}(\cdot; \boldsymbol{z}_t)$, so the KL is zero: *unmasked positions cost nothing*. If $\boldsymbol{z}_t = \mathbf{m}$, both distributions are supported on $\{\mathbf{m}\} \cup \mathcal{V}$ with the same mass $\frac{1-\alpha_s}{1-\alpha_t}$ on $\mathbf{m}$, and remaining mass $\frac{\alpha_s - \alpha_t}{1-\alpha_t}$ distributed as $\boldsymbol{x}$ (true) or $\boldsymbol{x}_\theta$ (model). A KL between two such distributions is

$$D_{\mathrm{KL}}(q \,\|\, p_\theta) = \frac{\alpha_s - \alpha_t}{1-\alpha_t} \sum_{v \in \mathcal{V}} x_v \log \frac{x_v}{[\boldsymbol{x}_\theta]_v} \;=\; -\frac{\alpha_s - \alpha_t}{1-\alpha_t} \log \langle \boldsymbol{x}_\theta,\, \boldsymbol{x} \rangle, \tag{5.6}$$

using that $\boldsymbol{x}$ is one-hot (so the $x_v \log x_v$ term is zero and the sum picks out one coordinate, Equation (2.2)). Summing over steps and flipping the sign of $\alpha_s - \alpha_t$ gives Equation (5.5). ∎

Intuition: Rao–Blackwellisation here means "skip the unmasked positions"

The textbook ELBO estimator samples $\boldsymbol{z}_s$ and $\boldsymbol{z}_t$ and evaluates a KL. Two exact integrations are available for free: the KL is exactly zero at unmasked positions, and at masked positions the KL over the *whole* categorical distribution reduces to a single log-probability of the correct

token times a scalar. Doing both analytically instead of by sampling is Rao–Blackwellisation, and it cuts the estimator's variance enormously — which Sahoo et al. (2024) identify as a main reason MDLM outperforms its predecessors at the same architecture.

5.4.1. The continuous-time limit

Let $T \to \infty$. Then $\alpha_t - \alpha_s \to -\alpha'_t \, \mathrm{d}t$ and the sum becomes an integral.

Theorem 5.4 (Continuous-time masked NELBO).

$$\mathcal{L}_\infty = \mathbb{E}_q \left[\int_0^1 \frac{\alpha'_t}{1-\alpha_t} \sum_{\ell=1}^{L} \log \langle \boldsymbol{x}_\theta^\ell(\boldsymbol{z}_t, t),\, \boldsymbol{x}^\ell \rangle \; \mathrm{d}t \right]. \tag{5.7}$$

Since $\alpha'_t < 0$ and the logarithm is negative, the integrand is non-negative. Unmasked positions contribute zero by SUBS.

This is Sahoo et al. (2024, Eqs. 10–11); Shi et al. (2024) derive the same expression, written with an explicit $\delta_{\boldsymbol{z}_t,\texttt{[MASK]}}$ indicator in place of the SUBS cancellation.

$$\text{With } \alpha_t = 1 - t: \qquad \mathcal{L}_\infty = \mathbb{E}_q \left[\int_0^1 \frac{1}{t} \sum_{\ell \,:\, z_t^\ell = \texttt{[MASK]}} \big(-\log p_\theta(x^\ell \mid \boldsymbol{z}_t) \big) \, \mathrm{d}t \right]. \tag{5.8}$$

Intuition: this is BERT with a weight

Read Equation (5.8) aloud. Mask each token independently with probability t; ask the network to predict the masked tokens; take the cross-entropy; multiply by $1/t$; average over $t \sim U(0,1)$. That is masked language modelling with a random masking rate and one scalar weight.

The weight is the entire difference between BERT and a generative model. BERT uses a fixed 15% masking rate and no weight, and is therefore not a generative model of anything. Sweeping t over $(0,1)$ and weighting by $-\alpha'_t/(1-\alpha_t)$ makes the objective a *likelihood bound*, and hence makes the model samplable. The $1/t$ factor upweights the nearly-clean regime, where single-token predictions are easy but must be very accurate.

5.5. Training and sampling in practice

Listing 5.1 is a complete masked diffusion model: forward process, loss Equation (5.8), SUBS parameterisation, ancestral sampler from Equation (5.3). It trains in a few minutes on CPU. The full listing with the toy data generator is Chapter C.

Listing 5.1 A complete masked diffusion model: loss and sampler. Full version in Chapter C.

```
"""A complete, minimal masked diffusion language model (MDLM / MD4 style).

Trains on a toy grammar and samples from it.  Importable: `get_trained_model()`
trains once and caches the weights to disk, so the decoding experiments of
Chapter 12 can reuse it.

```

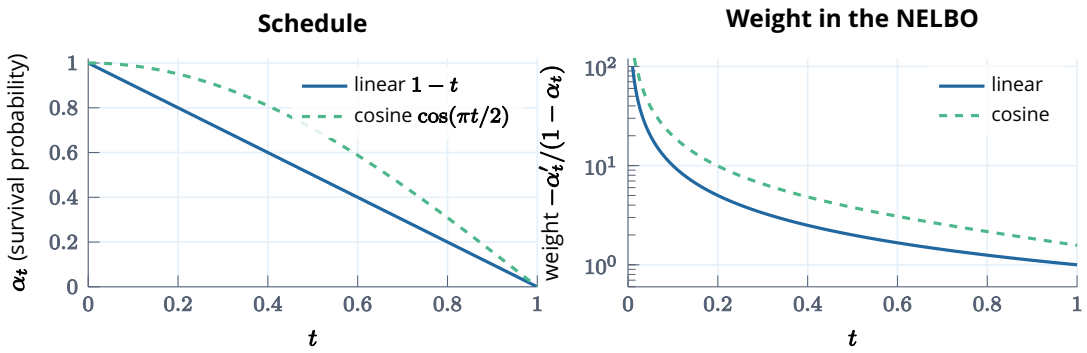


Figure 5.2 **Left:** the two schedules in common use. **Right:** the weight each puts on the cross-entropy in the continuous-time bound Equation (5.7), on a logarithmic scale. Both diverge as $t \to 0$ — like $1/t$ for the linear schedule and $2/t$ for the cosine — so the Monte Carlo estimator of the objective has heavy tails exactly where the model's predictions matter most. In practice the integral is truncated at some $t \geq \varepsilon$, and the value of ε silently changes every reported likelihood (Chapter 10). The cosine schedule is nonetheless Fisher–Rao optimal for masked diffusion (Zhang and Syed, 2025).

```
Run directly to train and evaluate:   python3 appC_masked_lm.py
"""
import os
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
V, L, MASK = 16, 12, 16          # vocab 0..15, plus the mask token with id 16
D, NHEAD, NLAYER = 128, 4, 4
CKPT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "mdlm_toy.pt")


# ---------------------------------------------------------------------------- #
# 1.  Noise schedule.  alpha_t = 1 - t  (linear), so the ELBO weight
#     -alpha'_t / (1 - alpha_t)  is exactly  1/t.
# ---------------------------------------------------------------------------- #
def alpha(t):
    return 1.0 - t                       # alpha_0 = 1, alpha_1 = 0


# ---------------------------------------------------------------------------- #
# 2.  Denoiser: a bidirectional transformer with time conditioning.
#     SUBS is enforced at the output: there is no mask logit at all (zero
#     masking probability) and unmasked positions are copied by the sampler
#     (carry-over unmasking).
# ---------------------------------------------------------------------------- #
class Denoiser(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok = nn.Embedding(V + 1, D)
        self.pos = nn.Embedding(L, D)
        self.time = nn.Sequential(nn.Linear(1, D), nn.SiLU(), nn.Linear(D, D))
        layer = nn.TransformerEncoderLayer(D, NHEAD, 4 * D, batch_first=True,
                                           norm_first=True, dropout=0.0)
        self.body = nn.TransformerEncoder(layer, NLAYER)
        self.head = nn.Linear(D, V)      # no mask logit  ->  SUBS condition 1

    def forward(self, z, t):
```

```
        h = self.tok(z) + self.pos(torch.arange(L, device=z.device))
        h = h + self.time(t[:, None])[:, None, :]
        return self.head(self.body(h))   # (B, L, V) logits over clean tokens


# ---------------------------------------------------------------------------- #
# 3.  Forward process + loss.  Each token is masked independently with
#     probability 1 - alpha_t, and
#       L = E_t E_z  (1/t) * sum_{masked l}  -log p_theta(x^l | z_t).
# ---------------------------------------------------------------------------- #
def nelbo(model, x, eps=1e-3):
    B = x.shape[0]
    t = torch.rand(B, device=x.device) * (1 - eps) + eps
    masked = torch.rand(B, L, device=x.device) < (1.0 - alpha(t))[:, None]
    z = torch.where(masked, torch.full_like(x, MASK), x)
    logits = model(z, t)
    ce = F.cross_entropy(logits.reshape(-1, V), x.reshape(-1),
                         reduction="none").reshape(B, L)
    return ((ce * masked).sum(-1) / t).mean()      # nats per sequence


# ---------------------------------------------------------------------------- #
# 4.  Ancestral sampler.  Going from t to s < t, a masked token is revealed
#     with probability (alpha_s - alpha_t) / (1 - alpha_t) and then drawn from
#     p_theta(. | z_t);  unmasked tokens are carried over unchanged.
# ---------------------------------------------------------------------------- #
@torch.no_grad()
def sample(model, n=8, steps=32, device="cpu"):
    z = torch.full((n, L), MASK, dtype=torch.long, device=device)
    ts = torch.linspace(1.0, 0.0, steps + 1, device=device)
    for i in range(steps):
        t, s = ts[i], ts[i + 1]
        p_unmask = ((alpha(s) - alpha(t)) / (1.0 - alpha(t))).clamp(0, 1)
        probs = model(z, t.expand(n)).softmax(-1)
        draw = torch.multinomial(probs.reshape(-1, V), 1).reshape(n, L)
        do = (torch.rand(n, L, device=device) < p_unmask) & (z == MASK)
        z = torch.where(do, draw, z)
    probs = model(z, torch.zeros(n, device=device)).softmax(-1)
    return torch.where(z == MASK, probs.argmax(-1), z)
```

Trained on a toy grammar in which a 12-token sequence must be an arithmetic progression mod 8 together with a globally consistent flag bit — a distribution with exactly $8 \times 3 \times 2 = 48$ equally likely sequences, hence entropy $\log 48 = 3.87$ nats — the model reaches a NELBO of about 3.1–3.8 nats per sequence, and sampling gives

```
 4 denoising steps   →   6.2% valid
12 denoising steps   →  44.1% valid
48 denoising steps   →  82.0% valid
ground-truth data    → 100.0% valid
```

This is the central empirical fact about masked diffusion, visible in twelve tokens. The model is *the same model* in all three rows; only the number of sampling steps changes. Four steps means unmasking three tokens at a time from the factorised marginals, and the global constraints (common difference, common flag) are destroyed. The error is a property of the sampler, not the network — exactly the point Tang et al. (2026) make at scale with an exact oracle denoiser.

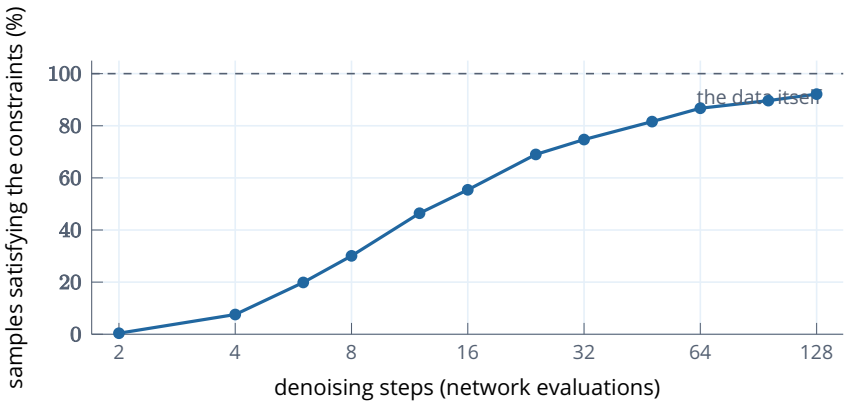


Figure 5.3 The central empirical fact about masked diffusion, visible in twelve tokens. This is *one* trained model (Chapter C) sampled with the ancestral sampler of Equation (5.3); only the number of steps changes. The toy grammar requires an arithmetic progression mod 8 *and* a flag bit that agrees at every position — a global constraint no single position can enforce alone. At two steps the sampler commits several tokens at once from a product of marginals and almost nothing survives; by 128 steps it reaches 92% — but never quite gets there. A decoder that reveals *exactly* one token per step reaches 99% on the same model (Figure 8.1, NFE $= 12$), so the residual gap is not model error: the ancestral sampler reveals each masked position *independently* with probability $(\alpha_s - \alpha_t)/(1 - \alpha_t)$, so however fine the time grid, two tokens can still be revealed together. Nothing about the network changes along this curve, which is why Chapter 8 treats the sampler as a first-class object. (This is a finer sweep, with 2048 samples per point, than the three figures quoted in the text; small differences between them are Monte Carlo noise.)

5.6. Reported results

For calibration, test perplexities on LM1B at ≈ 110M parameters (Sahoo et al., 2024, Table 1):

Table 5.1 Test perplexity on LM1B (lower is better), after Sahoo et al. (2024). "Retrained" rows use the same architecture and token budget as MDLM.

Model	Params	PPL ↓
Autoregressive		
Transformer-XL Base	0.46B	23.5
Transformer (retrained, 33B tokens)	110M	22.32
Transformer (retrained, 327B tokens)	110M	20.86
Diffusion		
BERT-Mouth	110M	142.89
D3PM (absorbing)	70M	76.90
Diffusion-LM	80M	118.62
DiffusionBERT	110M	63.78
SEDD (33B tokens)	110M	32.79
MDLM (33B tokens)	110M	27.04
MDLM (327B tokens)	110M	23.00

On OpenWebText at 262B tokens, SEDD reaches 24.10 and MDLM 23.21. MD4 (Shi et al., 2024) reports 2.75 bits/dim on CIFAR-10 and 3.40 bits/dim on ImageNet 64×64 at the pixel level, which they note beats autoregressive models of similar size — a reminder that masked diffusion is not only for text.

Table 5.1 is worth staring at. The honest summary is: masked diffusion closed most of the gap to autoregressive modelling at matched scale and token budget, and did not close all of it (23.00 vs 20.86, about 10%). The remaining gap is consistent with the theoretical picture in

Chapter 6: the model must spread capacity over all 2^L conditioning patterns rather than the L that an autoregressive model needs.

5.7. Generalisations

State-dependent masking. Shi et al. (2024) generalise to masking schedules that depend on the token's identity — rare tokens can be masked later than common ones — and report improved likelihoods. This is a first step into the content-dependent-schedule territory flagged in Chapter 3.

Any $\boldsymbol{\pi}$. Equation (5.1) with $\mathbf{m}$ replaced by an arbitrary $\boldsymbol{\pi} \in \Delta^K$ gives the *interpolating* family that includes uniform noise ($\boldsymbol{\pi} = \mathbf{1}/K$). The posterior is no longer as simple, since unmasked tokens may now be wrong. That is Section 5.8.

Semi-autoregressive generation. Nothing forces you to denoise the whole sequence at once. MDLM's samplers support generating arbitrary-length text block by block, and Arriola et al. (2025) turn this into a principled interpolation with a per-block KV cache (Chapter 10).

5.8. Uniform-state and interpolating diffusion

Masked diffusion has one structural defect: **it cannot revise**. Once a token is unmasked the SUBS parameterisation freezes it, so an error made at step 3 of a 32-step sample is still there at step 32. Uniform-state diffusion does not have this defect — every token can be changed at every step — at the cost of a harder learning problem. This section covers that trade-off, the recent results suggesting the field moved to masking too quickly, and the hybrid formulations.

5.8.1. The interpolating family

Replace $\mathbf{m}$ in Equation (5.1) by an arbitrary limiting distribution $\boldsymbol{\pi} \in \Delta^K$:

$$q(\boldsymbol{z}_t \mid \boldsymbol{x}) = \mathrm{Cat}\big(\boldsymbol{z}_t;\ \alpha_t \boldsymbol{x} + (1-\alpha_t)\boldsymbol{\pi}\big). \tag{5.9}$$

This single equation covers the whole family:

- $\boldsymbol{\pi} = \mathbf{m}$: masked / absorbing diffusion (Chapter 5).
- $\boldsymbol{\pi} = \mathbf{1}/K$: uniform-state diffusion, i.e. multinomial diffusion (Hoogeboom et al., 2021) in continuous time (Schiff et al., 2025).
- $\boldsymbol{\pi} = \boldsymbol{m}$, the data marginal: marginal-matched diffusion (Vignac et al., 2023a); see Proposition 3.2.
- $\boldsymbol{\pi}$ a mixture of $\mathbf{m}$ and uniform: genuinely *interpolating* models, which can both mask and corrupt.

The posterior, for every member of the family at once

It is worth deriving the posterior once in full generality rather than case by case, because doing so shows exactly *which* step of the masked derivation used the absorbing property — and therefore exactly what breaks for every other $\boldsymbol{\pi}$.

> **Theorem 5.5 (The interpolating posterior).** Let α_t be non-increasing with $\alpha_0 = 1$, let $\alpha_{t|s} := \alpha_t/\alpha_s$ for $s < t$, and let $\boldsymbol{z}_t$ follow Equation (5.9). Then the transition kernel is in the same family,
>
> $$q(\boldsymbol{z}_t \mid \boldsymbol{z}_s) = \mathrm{Cat}\big(\boldsymbol{z}_t;\ \alpha_{t|s}\boldsymbol{z}_s + (1-\alpha_{t|s})\boldsymbol{\pi}\big), \tag{5.10}$$

and for any $\boldsymbol{x}$ and any $\boldsymbol{z}_t$ with $q(\boldsymbol{z}_t \mid \boldsymbol{x}) > 0$, the posterior is the normalised Hadamard product

$$q(\boldsymbol{z}_s \mid \boldsymbol{z}_t, \boldsymbol{x}) = \mathrm{Cat}\left(\boldsymbol{z}_s;\ \frac{\left[\alpha_{t|s}\boldsymbol{z}_t + (1-\alpha_{t|s})\pi_{z_t}\mathbf{1}\right] \odot \left[\alpha_s \boldsymbol{x} + (1-\alpha_s)\boldsymbol{\pi}\right]}{\alpha_t x_{z_t} + (1-\alpha_t)\pi_{z_t}}\right), \tag{5.11}$$

where π_{z_t} and x_{z_t} denote the $\boldsymbol{z}_t$ coordinates of $\boldsymbol{\pi}$ and $\boldsymbol{x}$.

Proof. Equation (5.10) is the continuous-time form of Theorem 3.3: the generator is $\mathbf{Q} = \boldsymbol{\pi}\mathbf{1}^\top - \mathbf{I}$ in the column convention, whose exponential is $\alpha\mathbf{I} + (1-\alpha)\boldsymbol{\pi}\mathbf{1}^\top$ by the same idempotency argument as Proposition 2.9, and the survival factors multiply, so going from s to t uses α_t/α_s. (Equivalently: composing Equation (5.10) with Equation (5.9) must reproduce Equation (5.9) at time t, which forces $\alpha_{t|s}\alpha_s = \alpha_t$.)

For the posterior, Bayes' rule with $\boldsymbol{z}_s \perp \boldsymbol{z}_t \mid$ nothing — the chain is Markov, so $q(\boldsymbol{z}_t \mid \boldsymbol{z}_s, \boldsymbol{x}) = q(\boldsymbol{z}_t \mid \boldsymbol{z}_s)$ — gives

$$q(z_s = k \mid z_t, x) \ \propto\ \underbrace{\left[\alpha_{t|s}\,\delta_{kz_t} + (1-\alpha_{t|s})\pi_{z_t}\right]}_{q(\boldsymbol{z}_t \mid \boldsymbol{z}_s = k)} \cdot \underbrace{\left[\alpha_s x_k + (1-\alpha_s)\pi_k\right]}_{q(\boldsymbol{z}_s = k \mid \boldsymbol{x})}, \tag{5.12}$$

which is the numerator of Equation (5.11) read coordinatewise. Summing over k,

$$\begin{aligned}\sum_k (\cdots) &= \alpha_{t|s}\left[\alpha_s x_{z_t} + (1-\alpha_s)\pi_{z_t}\right] + (1-\alpha_{t|s})\pi_{z_t}\underbrace{\textstyle\sum_k \left[\alpha_s x_k + (1-\alpha_s)\pi_k\right]}_{=1} \\ &= \alpha_t x_{z_t} + \pi_{z_t}\left[\alpha_{t|s} - \alpha_t + 1 - \alpha_{t|s}\right] = \alpha_t x_{z_t} + (1-\alpha_t)\pi_{z_t},\end{aligned} \tag{5.13}$$

using $\alpha_{t|s}\alpha_s = \alpha_t$. The normaliser is therefore $q(\boldsymbol{z}_t \mid \boldsymbol{x})$, as consistency demands. ∎

Both the masked and uniform cases now fall out, and so does the reason the masked one is special.

Corollary 5.6 (Masked and uniform specialisations).

1. $\boldsymbol{\pi} = \mathbf{m}$. If $\boldsymbol{z}_t \neq$ `[MASK]` then $\pi_{z_t} = 0$, the first bracket collapses to $\alpha_{t|s}\boldsymbol{z}_t$, and $q(\boldsymbol{z}_s \mid \boldsymbol{z}_t, \boldsymbol{x}) = \delta_{\boldsymbol{z}_t}$: a revealed token is carried over. If $\boldsymbol{z}_t =$ `[MASK]` then Equation (5.11) becomes $\left[(\alpha_s - \alpha_t)\boldsymbol{x} + (1-\alpha_s)\mathbf{m}\right]/(1-\alpha_t)$, recovering Equation (5.2).
2. $\boldsymbol{\pi} = \mathbf{1}/K$. Then $\pi_{z_t} = 1/K > 0$ for every $\boldsymbol{z}_t$, no bracket collapses, and the posterior has full support on all K categories.

Proposition 5.7 (The posterior collapses exactly on the zero set of $\boldsymbol{\pi}$). Fix $s < t$ with $0 < \alpha_{t|s} < 1$ and $\alpha_s < 1$, and let $\boldsymbol{x}$ be one-hot (clean data). Then $q(\boldsymbol{z}_s \mid \boldsymbol{z}_t, \boldsymbol{x})$ is a point mass **iff** $\pi_{z_t} = 0$. In particular the per-position KL in the ELBO vanishes identically on revealed positions — the Rao–Blackwellisation of Section 5.4 — precisely when the observed state lies outside the support of $\boldsymbol{\pi}$, which for a single absorbing state means $\boldsymbol{\pi}$ is a *vertex* of Δ^K.

Proof. ($\Leftarrow$) If $\pi_{z_t} = 0$ the first bracket of Equation (5.11) is $\alpha_{t|s}\boldsymbol{z}_t$, so the Hadamard product is supported on $\{\boldsymbol{z}_t\}$ alone. ($\Rightarrow$) If $\pi_{z_t} > 0$ then the first bracket is strictly positive in every coordinate (it is $\geq (1-\alpha_{t|s})\pi_{z_t} > 0$), so the support of the posterior equals the support of $\alpha_s\boldsymbol{x} + (1-\alpha_s)\boldsymbol{\pi}$, which contains both $\boldsymbol{x}$ and every k with $\pi_k > 0$. Since $\alpha_s < 1$ and $\boldsymbol{\pi}$ has at least one nonzero coordinate other than $\boldsymbol{x}$ unless $\boldsymbol{\pi} = \boldsymbol{x}$, that support has at least two elements. ∎

Proposition 5.7 is the precise version of the folklore that "masking is easier". The advantage is not a matter of degree: it is a support condition, and it is all-or-nothing. Any $\boldsymbol{\pi}$ with full support — uniform, marginal-matched, or an interpolation — loses the collapse entirely, however close to

a vertex it sits. That discontinuity is why the interpolating family does not come with a smoothly interpolating ELBO, and why Schiff et al. (2025) had to derive the uniform-state bound separately rather than specialising a general one.

5.8.2. Why uniform noise is harder

Three reasons, all consequences of losing the absorbing structure.

1. The model is not told where the errors are. Under masking, a position is either known-correct or known-unknown. Under uniform noise, every position holds a token that may or may not be the true one, and the model must infer *both* which positions are corrupted and what they should be. This is strictly more work for the same parameter budget.

2. The posterior does not collapse. Proposition 5.1 used the absorbing property twice. Without it, the posterior $q(\boldsymbol{z}_s \mid \boldsymbol{z}_t, \boldsymbol{x})$ is a genuine mixture and the ELBO's KL terms do not reduce to a single log-probability: the KL at each position is a sum over all K categories, and the Rao–Blackwellisation of Section 5.4 is unavailable in the same clean form. Schiff et al. (2025) derive a continuous-time ELBO for this case which tightens the variational gap, and this is one of their main contributions.

3. The score has $L(K-1)$ live entries. Under masking, the only reverse transitions with nonzero forward rate are "from `[MASK]`", so the model needs K numbers per masked position. Under uniform noise all $L(K-1)$ ratios are live, and the score-entropy sum in Equation (4.11) runs over $K-1$ terms per position. At $K = 50{,}000$ this is the dominant cost and is the main practical reason uniform-state diffusion was under-studied at scale.

5.8.3. Why uniform noise can still win

And yet. The empirical picture as of late 2026 is considerably more favourable to uniform noise than the folklore suggests.

Small vocabularies. Schiff et al. (2025) report that uniform-state diffusion reaches state-of-the-art performance on small-vocabulary scientific data — molecules and DNA — explicitly challenging the belief that uniform noise lags. The reason is the cost argument above run backwards: at $K = 4$ (DNA) or $K \approx 30$ (SELFIES alphabets, amino acids) the $L(K-1)$ score is cheap, and the ability to revise is worth more than the extra difficulty.

Scaling. Sahoo et al. (2026) conducted the first scaling-law study across uniform-state, interpolating and masked discrete diffusion, and found that at 1.7B parameters uniform-state diffusion remains competitive on likelihood benchmarks and *outperforms both autoregressive and masked diffusion models on GSM8K*, despite having worse validation perplexity. They draw two conclusions worth quoting in substance: that perplexity is a reliable guide *within* a diffusion family but can mislead *across* families, and that a model with worse likelihood scaling may be preferable because it samples faster. They also report masked diffusion being about 12% more FLOPs-efficient when trained with a plain cross-entropy objective.

Error correction at low step counts. This is the structural argument and it is the one to hold on to. Uniform-state diffusion's reverse process is a *corrector by construction*: because every token can change at every step, a sample that is internally inconsistent after an aggressive parallel update can be repaired by subsequent steps. Masked diffusion needs an explicit remasking mechanism bolted on (Chapter 8) to do the same thing.

Intuition: masking trades inference flexibility for training ease

Masking makes training easy by telling the model where to look, and makes inference brittle by forbidding revision. Uniform noise does the reverse. Which is better depends on where your bottleneck is: if you are data-and-compute limited at large K, masking; if you are inference-compute limited or need error correction at small K, uniform. The interpolating family lets you tune between them, and almost nobody has done this systematically — which is itself a research opportunity.

5.8.4. The diffusion duality

Sahoo et al. (2025) found a result that is both elegant and immediately useful.

Theorem 5.8 (The diffusion duality, informally). Applying $\arg\max$ to a Gaussian diffusion yields exactly a uniform-state discrete diffusion. The discrete process's noise parameter is a deterministic transformation $\mathcal{T}$ of the Gaussian one.

In their own words, "the arg max operation transforms Gaussian diffusion into Uniform-state diffusion". The construction is: run a Gaussian diffusion on K logits, and read off the token as the argmax. The induced process on tokens is precisely uniform-state discrete diffusion with a related schedule.

Two consequences, both exploited in their paper's method, Duo.

Curriculum learning. During training, replace the hard discrete input to the denoiser with a tempered softmax $\mathrm{softmax}(\boldsymbol{w}/\tau)$ of the Gaussian latents, keeping the discrete clean token as the target, and anneal $\tau \to 0$. The model starts with a soft, informative input and ends on the true discrete one. The loss is only a valid ELBO in the limit $\tau \to 0$, so this is a curriculum, not a new objective. They report roughly a $2\times$ training speedup, reaching perplexity 35.2 at 510K steps where UDLM needs 1M steps to do about 1.5 points worse.

Discrete Consistency Distillation. This is the important one. Section 2.8 noted that the absence of a deterministic probability-flow ODE is what blocks consistency distillation for discrete diffusion. The duality supplies one: build deterministic trajectories by applying $\arg\max$ to a linear interpolation between the clean sequence and a *fixed* Gaussian noise sample, sample adjacent times s, t separated by δ from these trajectories, and train a student to match the teacher's output distribution under a KL loss, doubling δ between distillation rounds. The reported result is a reduction from 1024 sampling steps to 8 — two orders of magnitude — with minimal quality loss.

Reported perplexities: Duo reaches 33.7 on LM1B with sentence packing (29.9 without) against 36.7 for UDLM, and 25.2 on OpenWebText against 27.4 for UDLM. It outperforms autoregressive models on 3 of 7 zero-shot datasets.

Remark 5.9 (The duality is specific to uniform noise). This cannot be stressed too strongly, because it is the single most actionable open problem in this part of the book. The $\arg\max$ construction produces *uniform*-state diffusion. There is no known analogous lift for absorbing-state diffusion, which is the family every large diffusion language model uses. Finding one — or proving none exists — would directly unlock consistency distillation for LLaDA-class models.

5.8.5. Hybrid discrete–continuous: CANDI

Pynadath et al. (2025) ask why continuous diffusion, which dominates images, underperforms native discrete formulations on discrete data, and answer with *token identifiability*. Gaussian noise applied to a token corrupts it through two mechanisms that scale differently with vocabulary size:

discrete identity corruption (whether the argmax changes) and *continuous rank degradation* (how the ordering of the remaining logits degrades). A schedule that gives useful learning signal about one gives little about the other.

CANDI's answer is to *decouple* the two corruptions so both can be learned simultaneously. The payoffs they report are the two things pure discrete models struggle with:

- **Classifier guidance with off-the-shelf classifiers via simple gradient addition.** Because a continuous channel is maintained, gradient-based guidance works directly, without the discrete machinery of Chapter 9 and without a noise-aware classifier.
- **Better text generation at low numbers of function evaluations** than masked diffusion.

Intuition: why a continuous channel helps at low NFE

At low step counts the limiting factor is that the model must commit to many tokens at once from a factorised distribution (Remark 2.4). A continuous side-channel lets the model carry *partial, graded* information about a token across steps — "probably a verb, leaning towards one of these three" — rather than being forced to either commit fully or know nothing. That is a strictly richer intermediate state than $\{\text{token}\} \cup \{\texttt{[MASK]}\}$, and it is why hybrids help exactly in the regime where masking hurts.

Related: soft-masked diffusion (Hersche et al., 2025) pursues the same idea from the discrete side, interpolating the mask embedding rather than adding a separate continuous channel.

5.8.6. Practical guidance

A rough decision procedure, as of late 2026:

Table 5.2 Choosing a forward process. These are empirical regularities, not theorems.

Situation	Choose	Why
Text, $K \gtrsim 10^4$, large scale	masked	cheap objective, AR initialisation available, everything is tuned for it
DNA ($K = 4$), SELFIES, amino acids	uniform	cheap at small K; revision is worth more than training ease (Schiff et al., 2025)
Sparse graphs / molecular graphs	marginal-matched	limiting distribution has realistic density (Proposition 3.2)
Few-step sampling is the priority	uniform, or masked + remasking, or hybrid	error correction matters more than likelihood
Ordinal vocabulary (quantised values)	discretised Gaussian	respects the metric; neglected in current practice
You want to distil to ≤ 8 steps	uniform	the diffusion duality gives you a consistency objective (Sahoo et al., 2025)

Remark 5.10 (Why the distillation question dominates). Worth making explicit, since it recurs in Chapters 8 and 11. The commercial case for diffusion language models rests almost entirely on latency, and latency is step count times per-step cost. Masked diffusion is the family with the trained checkpoints and the scaling results; distillation is the technique that reduces step count by

orders of magnitude elsewhere in generative modelling; and the bridge between them is missing. Almost every other open problem in this book is a scientific question. This one is on the critical path of whether the field has a product.

Key takeaways

- Absorbing noise makes the posterior trivial (Proposition 5.1): unmasked tokens are frozen, and a masked token is revealed with probability $(\alpha_s - \alpha_t)/(1 - \alpha_t)$.
- SUBS — no mask logit, and copy unmasked tokens — should be hard-coded. It makes the KL vanish at unmasked positions and enables the variance reduction that makes MDLM work.
- The continuous-time bound is a single weighted cross-entropy, Equation (5.7). With $\alpha_t = 1 - t$ it is masked language modelling with a random mask rate and a $1/t$ weight (Equation (5.8)).
- The weight is what separates a generative model from BERT.
- Quality depends strongly on the number of sampling steps even for a fixed, well-trained model (6.2% $\to$ 82.0% valid in the toy experiment). This is factorisation error in the sampler, not model error.
- At matched scale masked diffusion reaches within roughly 10–25% of autoregressive perplexity on text, and beats comparable autoregressive models on pixel-level image likelihood.
- Equation (5.9) with a general $\boldsymbol{\pi}$ covers masked, uniform, marginal-matched and interpolating diffusion in one line.
- Uniform noise is harder to train (the model must localise errors, the posterior does not collapse, the score has $L(K-1)$ live entries) but can revise its own outputs, which masked diffusion structurally cannot.
- At small vocabularies uniform-state diffusion is state of the art, and at 1.7B parameters it beats both AR and masked models on GSM8K despite worse perplexity — so perplexity is not a valid cross-family metric (Schiff et al., 2025; Sahoo et al., 2026).
- The diffusion duality (Theorem 5.8) says uniform-state discrete diffusion is the $\arg\max$ of a Gaussian diffusion. This gives a continuous latent to distil in, and hence 1024 $\to$ 8 sampling steps. No analogue is known for masking.
- Hybrids (CANDI, soft masking) decouple discrete identity from continuous ranking and win specifically at low step counts and for gradient-based guidance.

Open questions and research directions

1. **Is the $1/t$ weight optimal?** It is forced if you want an exact ELBO, but exactness is not obviously what you want: the weight puts enormous emphasis on $t \approx 0$, where the task is nearly trivial, and the resulting estimator has heavy tails (the Monte Carlo estimate in Listing 6.1 returns 11.53 for an exact value of 11.73 with 2×10^5 samples). Shi and Titsias (2025) show some reweightings are *tighter* bounds, and Sahoo et al. (2026) report that masked diffusion is about 12% more FLOPs-efficient when trained with a plain cross-entropy objective rather than the weighted one. Characterising the set of weights that preserve a bound, and finding the variance-optimal member, is a well-posed and open problem.

2. **Capacity allocation across conditioning patterns.** The model must represent $p(\boldsymbol{x}^M \mid \boldsymbol{x}^{\bar{M}})$ for all 2^L subsets M. Nothing in the objective says these should be weighted by the uniform measure over M induced by independent masking, which is simply what the forward process happens to give. Deliberately reweighting the distribution over masking patterns — towards contiguous spans, or towards patterns the sampler actually visits — is barely explored, and Kim et al. (2025) give strong reason to think it matters, since the sampler visits a very different distribution of patterns than training does.
3. **Train/test mismatch in masking patterns.** Training masks independently and uniformly; a confidence-ordered sampler (Chapter 8) produces highly *structured* masking patterns. The model is thus evaluated off its training distribution at inference. Quantifying and closing this gap — "sampler-aware training" — looks like one of the more promising unexploited directions in the field.
4. **Length.** Masked diffusion needs the sequence length fixed in advance, because the number of positions is the dimensionality of the state space. Current practice pads to a maximum length and lets the model emit end-of-sequence tokens, which wastes compute and biases length distributions (the "length bias" noted by Yu et al. (2026)). A clean generative treatment of variable length — a diffusion over sequences *and* their length — does not exist and is a real gap.
5. **Does the absorbing state have to be a single symbol?** A hierarchy of partial masks ("this is a noun", "this is a carbon-containing fragment") would let the forward process destroy information gradually rather than all at once per token, giving the reverse process a coarse-to-fine curriculum. Hierarchical and scale-wise formulations are beginning to appear (Zhou et al., 2025) but the design space is wide open.
6. **A duality for absorbing-state diffusion.** The highest-value problem in this chapter (see Remark 5.10 for why). Theorem 5.8 is uniform-specific. If some continuous process's deterministic trajectories project onto absorbing-state discrete diffusion, consistency distillation becomes available for the entire LLaDA/Dream family. If not, a proof of impossibility would explain the asymmetry.
7. **Optimising $\boldsymbol{\pi}$.** The interpolating family is parameterised by a whole simplex and essentially nobody has searched it. $\boldsymbol{\pi}$ could be learned, annealed over training, made position-dependent, or chosen to minimise a bound on sampler error. A systematic study of even a one-parameter path from $\mathbf{m}$ to uniform would be a useful, cheap paper.
8. **Scaling uniform-state diffusion past 1.7B.** The $L(K-1)$ score-entropy sum is the blocker. Sampled, hierarchical or low-rank estimators of that sum are an engineering problem with a clear payoff, given that Sahoo et al. (2026) find uniform-state competitive exactly where it has been least studied.
9. **Why does uniform diffusion win on GSM8K with worse perplexity?** Sahoo et al. (2026) report the phenomenon but, as far as I can tell, the mechanism is not established. The error-correction hypothesis is plausible and testable: if correct, the advantage should shrink as the step count rises towards L, and should be reproducible by giving a masked model a remasking sampler. Somebody should run that experiment.
10. **Principled hybrids.** CANDI decouples two corruption channels; soft masking interpolates the mask embedding. There is no framework that says how much continuous channel you should keep as a function of K, L and the step budget. The "token identifiability" analysis is the beginning of one.

CHAPTER 6

Hidden Equivalences: Any-Order Autoregression and What Masked Models Really Learn

CHAPTER

Authors	Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li and Chongxuan Li (RADD); Benigno Uria et al.; Emiel Hoogeboom et al.
Year	2025
Published	International Conference on Learning Representations (ICLR)
Link	https://arxiv.org/abs/2406.03736
Tags	any-order autoregression, RADD, time-independence, ELBO tightness, equivalences

This is the chapter that changes how you read the rest of the literature. Masked diffusion, presented in Chapter 5 as a diffusion model, is *exactly* an object that predates diffusion models: an any-order autoregressive model. We prove this, verify it numerically, and work through what it does and does not imply.

6.1. Any-order autoregressive models

An any-order autoregressive model (AO-ARM) (Uria et al., 2014; Hoogeboom et al., 2022) drops the fixed ordering of Equation (1.1). For each permutation σ of $\{1, \dots, L\}$ the chain rule gives a valid factorisation

$$p_\sigma(\boldsymbol{x}) = \prod_{i=1}^{L} p\big(x^{\sigma(i)} \mid x^{\sigma(1)}, \dots, x^{\sigma(i-1)}\big), \tag{6.1}$$

and the AO-ARM objective trains a *single* network to serve all of them:

$$\mathcal{L}_{\mathrm{AO}}(\boldsymbol{x}) = \mathbb{E}_{\sigma \sim \mathrm{Unif}(S_L)} \left[\sum_{i=1}^{L} -\log p_\theta\big(x^{\sigma(i)} \mid x^{\sigma(<i)}\big) \right]. \tag{6.2}$$

By Jensen's inequality this upper-bounds the negative log-likelihood of the uniform mixture $\frac{1}{L!}\sum_\sigma p_\sigma$.

Note what Equation (6.2) asks for: the conditional distribution of one token given an *arbitrary subset* of the others. That is exactly what the masked denoiser of Chapter 5 computes.

6.2. The equivalence

Theorem 6.1 (Masked diffusion is any-order autoregression). Let $\alpha_t = 1 - t$ and let the denoiser output, at every masked position ℓ, the conditional distribution of x^ℓ given the currently unmasked tokens. Then the continuous-time masked NELBO of Theorem 5.4 satisfies

$$\mathcal{L}_\infty(x) \;=\; \mathcal{L}_{\mathrm{AO}}(x) \;=\; \sum_{M \subseteq [L],\, M \neq \emptyset} \frac{(|M|-1)!\,(L-|M|)!}{L!}\,\mathrm{CE}(M), \tag{6.3}$$

where $\mathrm{CE}(M) = -\sum_{\ell \in M} \log p_\theta\big(x^\ell \mid x^{[L]\setminus M}\big)$ is the total cross-entropy at the masked set M. In particular, if the denoiser is *exact* (it outputs the true conditionals of p_{data}) then $\mathcal{L}_\infty(x) = -\log p_{\mathrm{data}}(x)$ exactly: the variational bound is tight.

Proof. Step 1: the NELBO as a sum over masked sets. With $\alpha_t = 1 - t$ we have $\alpha_t' = -1$ and $1 - \alpha_t = t$, so Equation (5.8) reads

$$\mathcal{L}_\infty = \int_0^1 \frac{1}{t}\, \mathbb{E}_{z_t}\big[\mathrm{CE}(M(z_t))\big]\; \mathrm{d}t,$$

where $M(z_t)$ is the set of masked positions. Under the forward process each position is masked independently with probability t, so $\Pr[M(z_t) = M] = t^{|M|}(1-t)^{L-|M|}$ and

$$\mathcal{L}_\infty = \sum_M \mathrm{CE}(M) \int_0^1 t^{|M|-1}(1-t)^{L-|M|}\; \mathrm{d}t = \sum_M \mathrm{CE}(M)\, B\big(|M|,\, L-|M|+1\big),$$

using the Beta integral. Since $B(m, L-m+1) = \frac{\Gamma(m)\Gamma(L-m+1)}{\Gamma(L+1)} = \frac{(m-1)!\,(L-m)!}{L!}$, this is the right-hand side of Equation (6.3). (The $M = \emptyset$ term has $\mathrm{CE} = 0$ and the integral $\int_0^1 t^{-1}(1-t)^L\,\mathrm{d}t$ diverges, but the product is zero; formally one treats $t \in [\varepsilon, 1]$ and lets $\varepsilon \to 0$, exactly as implementations do.)

Step 2: the AO-ARM objective as the same sum. Expand Equation (6.2). For a permutation σ, at step i the conditioning set is $\{\sigma(1), \dots, \sigma(i-1)\}$, so the masked set is $M = \{\sigma(i), \dots, \sigma(L)\}$ with $m := |M| = L - i + 1$, and the term contributed is the single-position cross-entropy at $\ell = \sigma(i) \in M$. Count the permutations producing a given (M, ℓ) pair with $\ell \in M$: the $L - m$ positions outside M fill the first $L - m$ slots in any order ($(L-m)!$ ways), $\sigma(i) = \ell$ is forced, and the remaining $m - 1$ elements of $M \setminus \{\ell\}$ fill the last $m - 1$ slots in any order ($(m-1)!$ ways). Hence

$$\begin{aligned}
\mathcal{L}_{\mathrm{AO}} &= \frac{1}{L!} \sum_M \sum_{\ell \in M} (L-|M|)!\,(|M|-1)!\,\Big(-\log p_\theta\big(x^\ell \mid x^{[L]\setminus M}\big)\Big) \\
&= \sum_M \frac{(|M|-1)!\,(L-|M|)!}{L!}\,\mathrm{CE}(M),
\end{aligned}$$

which is the same expression.

Step 3: tightness. If p_θ gives the true conditionals of p_{data}, then for *every* fixed σ the chain rule gives $\sum_i -\log p_{\mathrm{data}}(x^{\sigma(i)} \mid x^{\sigma(<i)}) = -\log p_{\mathrm{data}}(x)$ exactly. Averaging over σ leaves $-\log p_{\mathrm{data}}(x)$. ∎

Listing 6.1 verifies all three quantities numerically on a random correlated joint distribution over 3^4 sequences.

Listing 6.1 Numerical verification of Theorem 6.1: the exact NLL, the expected any-order autoregressive NLL, and the continuous-time masked NELBO all agree.

```python
"""Numerically verify: continuous-time masked-diffusion NELBO
   == expected any-order autoregressive NLL == exact NLL, when the
   denoiser outputs the true conditional marginals."""
import itertools, math
import numpy as np

rng = np.random.default_rng(0)
L, V = 4, 3                                  # 4 positions, vocab of 3

# a random (correlated) joint distribution over V^L sequences
joint = rng.dirichlet(np.ones(V ** L) * 0.3)
joint = joint.reshape((V,) * L)
seqs = list(itertools.product(range(V), repeat=L))


def cond_marginal(pos, observed):
    """True p(x^pos = . | x^obs) as a length-V vector. `observed` = {pos: val}."""
    out = np.zeros(V)
    for s in seqs:
        if all(s[p] == v for p, v in observed.items()):
            out[s[pos]] += joint[s]
    return out / out.sum()


x = seqs[rng.integers(len(seqs))]            # the datapoint we score
exact_nll = -math.log(joint[x])

# ---- (a) expected any-order autoregressive NLL ------------------------------
ao_nll = 0.0
for sigma in itertools.permutations(range(L)):
    acc, observed = 0.0, {}
    for i in sigma:
        acc -= math.log(cond_marginal(i, observed)[x[i]])
        observed[i] = x[i]
    ao_nll += acc / math.factorial(L)

# ---- (b) continuous-time masked NELBO, in closed form -----------------------
# With alpha_t = 1 - t,   L = int_0^1 (1/t) E_{M ~ Bern(t)^L}[CE(M)] dt
#                           = sum_M  (|M|-1)! (L-|M|)! / L!  *  CE(M)
nelbo = 0.0
for r in range(1, L + 1):
    for M in itertools.combinations(range(L), r):
        observed = {p: x[p] for p in range(L) if p not in M}
        ce = -sum(math.log(cond_marginal(p, observed)[x[p]]) for p in M)
        nelbo += math.factorial(r - 1) * math.factorial(L - r) / math.factorial(L) * ce

# ---- (c) Monte-Carlo estimate of the same integral --------------------------
N, mc = 200_000, 0.0
for t in rng.uniform(1e-4, 1.0, N):          # t ~ U(0,1), weight 1/t
    mask = rng.random(L) < t
    if not mask.any():
        continue
    observed = {p: x[p] for p in range(L) if not mask[p]}
    mc -= sum(math.log(cond_marginal(p, observed)[x[p]]) for p in np.where(mask)[0]) / t
mc /= N

print(f"exact NLL                      : {exact_nll:.6f}")
print(f"expected any-order AR NLL      : {ao_nll:.6f}")
print(f"masked NELBO (closed form)     : {nelbo:.6f}")
```

```
print(f"masked NELBO (Monte Carlo, 2e5)  : {mc:.4f}")
assert abs(ao_nll - exact_nll) < 1e-9 and abs(nelbo - exact_nll) < 1e-9
print("\nAll three agree: with the exact denoiser the masked-diffusion ELBO is TIGHT.")
```

Its output:

```
exact NLL                    : 11.733854
expected any-order AR NLL    : 11.733854
masked NELBO (closed form)   : 11.733854
masked NELBO (Monte Carlo, 2e5) : 11.5305
```

The first three agree to nine decimal places. The fourth is the estimator practitioners actually use, and it is off by 0.2 nats after 2×10^5 samples — which is the variance problem of Chapter 5 made visible.

Intuition: the Beta integral is doing all the work

Why should a time-integral over a continuous noise schedule equal an average over discrete permutations? Because the time variable *is* a reparameterisation of the ordering. Under linear masking, the forward process assigns each position an independent uniform "unmasking time", and the induced ordering of those times is a uniformly random permutation. The $1/t$ weight in Equation (5.8) is exactly the Jacobian of this reparameterisation. So masked diffusion is not an approximation to any-order autoregression; it is the same object in continuous-time coordinates.

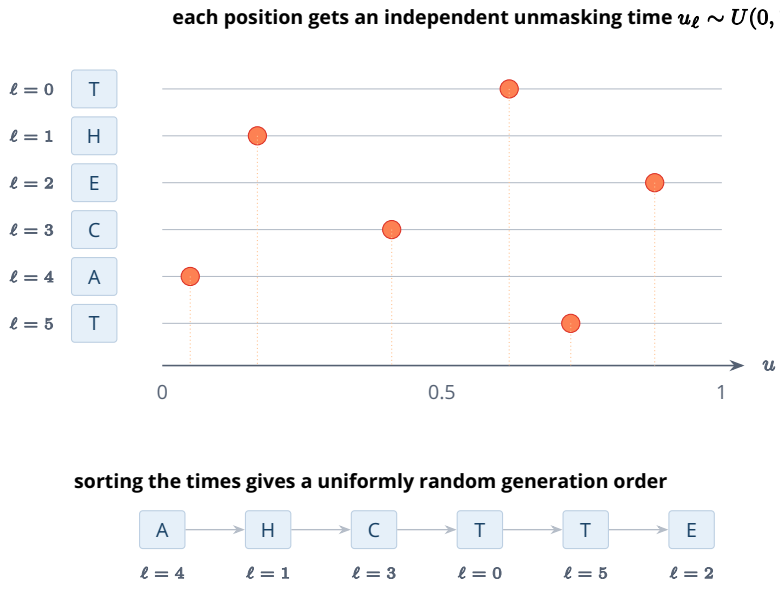


Figure 6.1 Why Theorem 6.1 is true. Under the linear schedule the forward process assigns each position an independent uniform time at which it is unmasked; the *order* of those times is then a uniformly random permutation. Running the reverse process is therefore the same thing as revealing the tokens in a random order, one at a time — that is, sampling from an any-order autoregressive model. The $1/t$ weight in Equation (5.8) is exactly the Jacobian of this change of variables, and integrating it out produces the combinatorial coefficient $(|M|-1)!\,(L-|M|)!/L!$ in Equation (6.3).

6.3. What this does and does not mean

This result is frequently over-read in both directions. Being careful here is worth a page.

It does mean the bound is tight. The "variational gap" of masked diffusion is zero at the optimum (Theorem 6.1, Step 3). So a reported ELBO gap relative to an autoregressive model is *entirely* attributable to the denoiser being imperfect, not to the variational construction. This is a genuinely useful diagnostic: if your masked diffusion model's bound is 3 nats worse than an AR baseline, the model is 3 nats worse, full stop.

But there is a second gap, and it is not variational. The statement above is about the *variational* gap. A finite-capacity model has another one, which the equivalence theorem does not touch and which is worth isolating because it is often conflated with the first. Write

$$p_\sigma(\boldsymbol{x}) := \prod_{i=1}^{L} p_\theta\big(x^{\sigma(i)} \mid \boldsymbol{x}^{\sigma(<i)}\big) \qquad \text{and} \qquad p_{\text{mix}}(\boldsymbol{x}) := \frac{1}{L!}\sum_{\sigma} p_\sigma(\boldsymbol{x}), \tag{6.4}$$

so that p_{mix} is the distribution you actually sample from when you draw a uniformly random order at inference time (Hoogeboom et al., 2022), while the training objective is the *average log-likelihood over orders*, $\mathcal{L}_{\text{AO}}(\boldsymbol{x}) = -\frac{1}{L!}\sum_\sigma \log p_\sigma(\boldsymbol{x})$.

Proposition 6.2 (The any-order objective overshoots by a Jensen gap). For every $\boldsymbol{x}$,

$$\mathcal{L}_{\text{AO}}(\boldsymbol{x}) - \big(-\log p_{\text{mix}}(\boldsymbol{x})\big) \;=\; \log \frac{1}{L!}\sum_{\sigma} p_\sigma(\boldsymbol{x}) \;-\; \frac{1}{L!}\sum_{\sigma} \log p_\sigma(\boldsymbol{x}) \;\geq\; 0, \tag{6.5}$$

with equality **iff** $p_\sigma(\boldsymbol{x})$ takes the same value for every ordering σ. The right-hand side is the Jensen gap of $\log$ on the multiset $\{p_\sigma(\boldsymbol{x})\}_\sigma$, so it grows with the *spread* of the model's per-ordering likelihoods.

Proof. Rearranging the left-hand side gives the right-hand side identically. Non-negativity and the equality condition are Jensen's inequality for the strictly concave $\log$ applied to the uniform average over the $L!$ values $p_\sigma(\boldsymbol{x})$; strict concavity makes the inequality strict unless the values coincide. Verified numerically on a deliberately order-inconsistent model in Listing C.3. ∎

So the training loss is an upper bound on the negative log-likelihood of the model you sample from, and the slack is exactly the model's *order-inconsistency*. Three consequences:

- A perfect model has all $p_\sigma(\boldsymbol{x}) = p_{\text{data}}(\boldsymbol{x})$ and both gaps vanish — consistent with Theorem 6.1, and the reason the two gaps are easy to confuse.
- For a real model the reported bound is *pessimistic* about p_{mix}: a masked diffusion model's true sampling log-likelihood is at least as good as its reported ELBO, by exactly Equation (6.5). Published perplexities therefore understate these models, and the amount is measurable — evaluate a handful of fixed orderings and look at the spread.
- Order-inconsistency is itself the quantity that remasking samplers (Section 8.4) and order-selection heuristics (Section 8.3) exploit: if all orderings agreed, the choice of order could not matter.

It does mean the hard part is capacity allocation. An AR model must represent L conditionals; a masked model must represent $p(x^\ell \mid \boldsymbol{x}^{\bar{M}})$ for all 2^L subsets $\bar{M}$. With the same parameters, each is learned less well. This is the clean explanation for the residual perplexity gap in Table 5.1 and it predicts that the gap should shrink with scale (more capacity) and with data (more coverage of patterns) — which is what Nie et al. (2025a) observe.

It does *not* mean masked diffusion is useless. The extra conditionals are the product. Infilling, arbitrary-order generation, revision, planning and parallel decoding are all exactly the capabilities that the 2^L conditionals buy and the L conditionals do not. Theorem 6.1 relocates the contribution from "a new generative modelling paradigm" to "a scalable training recipe and inference framework for any-order autoregression", which is still a real contribution: AO-ARMs existed for a decade and did not scale.

It does *not* apply to uniform or interpolating diffusion. The proof uses the absorbing structure twice. Section 5.8's models are genuinely not AO-ARMs, which is one reason they are interesting.

It does *not* mean diffusion-specific machinery is wasted. The samplers of Chapter 8 — τ-leaping, remasking, predictor–corrector — have no AO-ARM counterpart and are where most recent gains come from.

6.4. RADD: the time variable is redundant

Ou et al. (2025) push the equivalence into an architectural simplification. Their result, stated in their abstract's own terms, is that "the concrete score in absorbing diffusion can be expressed as conditional probabilities of clean data" multiplied by a time-dependent scalar in analytic form. In symbols, schematically: for absorbing noise,

$$\underbrace{\frac{p_t(\boldsymbol{y})}{p_t(\boldsymbol{x})}}_{\text{concrete score}} = \underbrace{c(t)}_{\text{analytic scalar}} \cdot \underbrace{p_{\text{data}}\left(y^\ell \mid \boldsymbol{x}^{\text{UM}}\right)}_{\text{time-independent}}, \tag{6.6}$$

where $\boldsymbol{x}^{\text{UM}}$ denotes the unmasked tokens of $\boldsymbol{x}$ and $\boldsymbol{y}$ differs from $\boldsymbol{x}$ by revealing position ℓ.

The consequences are immediate and all favourable.

1. **Drop the time conditioning.** The network need not take t as input at all. RADD is "a dedicated diffusion model without time-condition". One fewer thing to learn, and one fewer source of mismatch between training and inference.
2. **Cache across steps.** Because the output depends only on $\boldsymbol{x}^{\text{UM}}$, the network's output is unchanged for any step in which no token is unmasked. This makes caching sound rather than approximate, and underlies the dKV-cache and dLLM-cache line of work (Chapter 8).
3. **Faster convergence.** They report RADD converging faster than SEDD for similar final performance, and state-of-the-art results among diffusion models on five zero-shot language-modelling benchmarks at GPT-2 scale.
4. **The training objectives are equivalent.** Their Theorem 2 establishes that the absorbing-diffusion objective and the AO-ARM objective are equivalent, and that the diffusion upper bound on the negative log-likelihood can be interpreted as an expected AO-ARM negative log-likelihood — i.e. Theorem 6.1.

Intuition: why time has to be redundant

Under absorbing noise, the state $\boldsymbol{z}_t$ *already tells you* the noise level: count the masks. More precisely, everything the model needs is "what are the unmasked tokens and where are they", and t adds only a noisy estimate of a quantity already visible in the input. The analytic scalar $c(t)$ absorbs the rest. Compare uniform noise, where the state does *not* reveal the noise level — you cannot tell a lightly corrupted sequence from a heavily corrupted one

by inspection — and time conditioning is genuinely informative. This is a good diagnostic for the masked-versus-uniform distinction generally.

6.5. The same result, found twice

Zheng et al. (2025) arrived at the time-independence conclusion independently and from a different direction, and their formulation is worth knowing because it is in some ways cleaner.

They reformulate the masked diffusion negative ELBO as a sum over the *number of masked tokens* rather than over continuous time (their Proposition 3.1), and then show that a time-independent network is theoretically sufficient, because the optimal network depends only on the observed tokens (their Proposition 3.2). Training then looks like standard masked modelling, differing mainly in a $1/n$ likelihood weighting where n is the number of masked tokens — which is the discrete counterpart of the $1/t$ weight in Equation (5.8), and makes the Beta-integral combinatorics of Theorem 6.1 almost self-evident.

Their sampling-side contribution is the **first-hitting sampler** (FHS): analytically sample *when* the next token unmasks, then unmask one uniformly chosen position. This is theoretically equivalent to the original reverse process — it is exact, not an approximation — but it proceeds token by token like a masked model rather than stepping through time, and they report it being up to $20\times$ faster in wall-clock terms.

Remark 6.3 (Two lessons). First, the independent rediscovery is reassuring: the time-independence of masked diffusion is a real structural fact, not an artefact of one paper's parameterisation.

Second, note what FHS says about the field's framing. The *exact* sampler for masked diffusion is the one that unmasks one token at a time — which is the remark after Equation (4.1) restated as an algorithm. Time discretisation was never the point; it was an artefact of thinking in diffusion terms about an object that is really an any-order autoregressive model. The same paper also found the float32 Gumbel bug of Section 10.8, which is not a coincidence: taking the sampler seriously as an exact object is what exposes approximations that had been mistaken for properties of the model.

6.6. Other equivalences worth knowing

Masked diffusion and MaskGIT. The MaskGIT family of image generators, developed independently in the vision community, is masked discrete diffusion with a confidence-based decoding order and no likelihood-based justification. Recognising this is useful in both directions: MaskGIT's decoding heuristics transfer to language, and the diffusion ELBO supplies MaskGIT with a principled objective.

Scale-wise visual autoregression. Kumar et al. (2025) argue that scale-wise visual autoregressive modelling (VAR) is "secretly discrete diffusion". Several such identifications have appeared; they are worth tracking because they import tooling in both directions.

Block diffusion. Arriola et al. (2025) make the AR–diffusion interpolation explicit: autoregressive over blocks, diffusion within blocks. Block size 1 is autoregressive; block size L is masked diffusion. Given Theorem 6.1 this is best read as choosing how much of the ordering to fix and how much to average over.

Key takeaways

- Theorem 6.1: the continuous-time masked diffusion NELBO *equals* the any-order autoregressive objective, and both equal the exact NLL when the denoiser is exact. Verified numerically to nine decimals.
- The combinatorial weight is $(|M|-1)!(L-|M|)!/L!$, which arises as a Beta integral: the noise time is a reparameterisation of a random unmasking order, and the $1/t$ weight is its Jacobian.
- Therefore: any ELBO gap is model error, not variational slack; the hard part is allocating capacity across 2^L conditionals; and masked diffusion's real contribution is a scalable training and inference framework for AO-ARMs, not a new probabilistic object.
- For absorbing noise the concrete score factorises into an analytic scalar times a time-independent conditional (Equation (6.6)), so time conditioning is unnecessary and caching across non-unmasking steps is exact (RADD).
- None of this transfers to uniform or interpolating noise, where the state does not reveal the noise level.

Open questions and research directions

1. **Reweight the measure over orderings.** Equation (6.3) exposes the weight on each masked set explicitly. The uniform measure over permutations is an accident of independent masking, not a choice anyone defended. Training with a non-uniform measure — concentrated on contiguous spans, on left-to-right-ish orders, or on the orders a confidence-based sampler actually visits — is a direct, cheap intervention with a clear hypothesis attached, and the result in Kim et al. (2025) that adaptive inference lifts Sudoku accuracy from $< 7\%$ to $\approx 90\%$ suggests the training measure is leaving a great deal on the table. Hong et al. (2026) have begun on this.
2. **Quantify the capacity-allocation cost.** The claim that representing 2^L conditionals rather than L costs you some measurable amount of likelihood is folklore supported by scaling curves. A statement of the form "a model with N parameters achieves AO-ARM excess loss $\geq f(N, L)$" for some non-trivial function class would explain the residual gap in Table 5.1 and predict where it closes.
3. **Does the equivalence survive imperfect denoisers in a useful way?** Theorem 6.1 is an identity between objectives, which holds for any p_θ. But the *mixture* interpretation — that you are bounding the likelihood of $\frac{1}{L!}\sum_\sigma p_\sigma$ — depends on p_θ's conditionals being mutually inconsistent, and the size of the Jensen gap is exactly a measure of that inconsistency. Measuring it empirically for a trained LLaDA-class model would tell us something real about what these models have learned, and nobody appears to have done it.
4. **A RADD-style factorisation for other noise processes.** Equation (6.6) is specific to absorbing noise. Whether a similar separation of an analytic time-dependent scalar from a time-independent learned object exists for uniform or marginal-matched noise is open. A negative result would be informative: it would say the absorbing case is special in exactly the way that explains its dominance.
5. **Exploit the equivalence for evaluation.** Since masked diffusion is AO-ARM, its likelihood can be estimated by averaging over sampled orderings, which is an *unbiased*

estimator of the same quantity as the diffusion ELBO but with different variance. Which estimator is better, and whether a control-variate combination beats both, is a concrete question with immediate practical value given how central likelihood comparisons are to this literature. Ivanov et al. (2026) propose an alternative bound in this spirit.

CHAPTER 7

Flows and Continuous Relaxations

CHAPTER

Authors	Itai Gat et al.; Andrew Campbell et al.; Hannes Stark et al.; Xiang Lisa Li et al.
Year	2024
Published	NeurIPS 2024 / ICML 2024 / NeurIPS 2022
Link	https://arxiv.org/abs/2407.15595
Tags	flow matching, probability velocity, multimodal flows, simplex, Dirichlet, embeddings

Flow matching reframes generative modelling as: *prescribe* a path of distributions from noise to data, then learn the vector field that transports along it. On continuous spaces this gave simpler training and straighter trajectories than diffusion. This chapter develops the discrete analogue, in the two formulations that appeared in 2024: Campbell et al. (2024), which stays in CTMC language and emphasises multimodal (discrete + continuous) generation, and Gat et al. (2024), which builds the flow-matching machinery — paths, velocities, a continuity equation — natively on discrete spaces.

Convention warning

The flow-matching literature uses $t = 0$ for *noise* and $t = 1$ for *data*, the opposite of the diffusion convention used everywhere else in this book. In this chapter I follow the flow-matching convention, because the formulas are unreadable otherwise. So here $p_1 = p_{\text{data}}$ and generation runs $t : 0 \to 1$.

7.1. Probability paths and velocities

Prescribe a path p_t on $\mathcal{X}$ with p_0 a simple source and $p_1 = p_{\text{data}}$. The discrete analogue of a vector field is a *probability velocity* $u_t(y, x)$: the instantaneous rate of flow from state x to state y. This is a rate matrix by another name, and the formal definition is in terms of what it generates.

Definition 7.1 (Generating velocity, Gat et al., 2024). u_t generates p_t if the per-token Euler update

$$X^i_{t+h} \sim \delta_{X^i_t}(\cdot) + h\, u^i_t(\cdot, X_t) \tag{7.1}$$

produces $X_{t+h} \sim p_{t+h} + o(h)$.

The corresponding conservation law is a discrete continuity equation,

$$\dot{p}_t(x) + \operatorname{div}_x(p_t u_t) = 0, \qquad \operatorname{div}_x(v) = \sum_z \big[v(z, x) - v(x, z)\big], \tag{7.2}$$

which is Gat et al. (2024, Eqs. 16, 18). The divergence is the discrete analogue of $\nabla\cdot$: net outflow minus inflow at x. Equating it to $-\dot{p}_t$ is exactly the statement that probability is conserved. Readers who have seen Equation (2.10) will notice that Equation (7.2) *is* the Kolmogorov forward equation with u_t in the role of the generator; the content of flow matching is not a new dynamical object but a new way of choosing and learning it.

7.1.1. How much freedom is there in a velocity?

Equation (7.2) constrains u_t only through a *divergence*, so it cannot pin u_t down. Making that precise is worth the half page, because it is the exact reason flow matching has design freedom that diffusion's construction does not — and it says how much.

It is cleanest in terms of the *flux* $j_t(z,x) := p_t(x)\,u_t(z,x)$, the probability per unit time actually moving from x to z, and its antisymmetric part — the *net* flow $\jmath_t(z,x) := j_t(z,x) - j_t(x,z)$. Only the net flow enters Equation (7.2), since $\mathrm{div}_x(j_t) = \sum_z \jmath_t(z,x)$.

Proposition 7.2 (Gauge freedom of generating velocities). Fix a path p_t with $p_t > 0$ and a time t. Consider one token with vocabulary size K, so that the state graph is the complete graph on K vertices with $E = \binom{K}{2}$ edges.

1. The set of net flows satisfying Equation (7.2) at time t is a non-empty affine subspace of $\mathbb{R}^E$ whose direction is the kernel of the divergence operator, of dimension

$$\dim\ker \mathrm{div} \;=\; E - (K-1) \;=\; \frac{(K-1)(K-2)}{2}. \tag{7.3}$$

2. Hence the generating velocity is **unique** if and only if $K \le 2$. For $K \ge 3$ there is a $\frac{(K-1)(K-2)}{2}$-dimensional family of distinct velocities generating the *same* probability path, and the family of valid rate matrices among them is a polytope of the same dimension whenever one solution has all off-diagonal entries strictly positive.
3. The kernel is spanned by cycles: adding any circulation around a closed loop of states changes u_t but not p_t.

Proof. The divergence operator maps a net flow in $\mathbb{R}^E$ to $\mathbb{R}^K$, and is the (signed) incidence matrix of the graph after orienting each edge. For a connected graph on K vertices the incidence matrix has rank $K-1$, with the all-ones vector spanning the cokernel — which is consistent, because $\sum_x \dot{p}_t(x) = 0$, so the right-hand side always lies in the image. Non-emptiness follows, and the solution set is a coset of $\ker\mathrm{div}$, of dimension $E-(K-1) = \frac{K(K-1)}{2} - K + 1 = \frac{(K-1)(K-2)}{2}$. That kernel is by definition the cycle space of the graph, giving (iii). For (ii), uniqueness means dimension zero, i.e. $K \le 2$; the polytope claim holds because strict positivity is an open condition, so a neighbourhood of an interior solution within the affine space consists of valid rate matrices. Listing C.3 verifies Equation (7.3) for $K = 2,\dots,7$ and exhibits two strictly positive rate matrices on $K = 3$ that give identical $\dot{p}_t$. ∎

Intuition: the path does not know about circulation

A probability path records only where mass *is* at each instant. Two dynamics that move mass around a closed loop at different speeds — say a steady cycle $1 \to 2 \to 3 \to 1$ superimposed on everything — leave every marginal unchanged, because as much arrives at each state as leaves it. The continuity equation is blind to that circulation by construction, and Equation (7.3) counts exactly how many independent circulations exist.

Two consequences. First, this is why Theorem 7.3 is a *construction* rather than an inversion: it does not solve Equation (7.2) for u_t (which is underdetermined) but exhibits one particular solution, obtained by averaging conditional velocities. Second, it is why kinetic-optimality criteria (Section 7.5) are not decoration: there is a genuine $\frac{(K-1)(K-2)}{2}$-dimensional choice to make per token, no criterion is forced on you by the path, and a natural objective is to pick the solution of least "kinetic energy" — the one with no wasted circulation. For a binary vocabulary ($K = 2$) there is nothing to choose, which is why the question never arises in the bit-level literature.

7.2. Conditional paths and the marginalisation trick

Prescribing p_t directly is hopeless. Flow matching's device is to prescribe *conditional* paths, for which the velocity is trivial, and recover the marginal velocity by averaging.

Pick a coupling $\pi(x_0, x_1)$ of source and target (independent is the default) and a conditional path $p_t(\cdot \mid x_0, x_1)$ interpolating δ_{x_0} and δ_{x_1}. The workhorse is the *mixture path*, which per coordinate is

$$p_t(x^i \mid x_0, x_1) = (1 - \kappa_t)\, \delta_{x_0^i}(x^i) + \kappa_t\, \delta_{x_1^i}(x^i), \tag{7.4}$$

with a scheduler $\kappa_t : 0 \to 1$. In words: at time t, each coordinate is the target with probability κ_t and the source otherwise. Note that this is *exactly* the forward process of Equation (5.9) read backwards, with $\kappa_t = \alpha_{1-t}$; masked diffusion is the case $x_0 = \texttt{[MASK]}$ deterministically.

The key structural result is the analogue of the conditional-expectation identity that makes conditional flow matching work.

Theorem 7.3 (Marginal velocity is a posterior average, Gat et al., 2024, Thm. 2).

$$u_t^i(x^i, z) = \sum_{x_0, x_1} u_t^i(x^i, z \mid x_0, x_1)\; p_t(x_0, x_1 \mid z), \qquad p_t(x_0, x_1 \mid z) = \frac{p_t(z \mid x_0, x_1)\pi(x_0, x_1)}{p_t(z)}. \tag{7.5}$$

For the mixture path Equation (7.4) this specialises to something strikingly simple (Gat et al., 2024, Eq. 24):

$$\boxed{u_t^i(x^i, z) = \frac{\dot\kappa_t}{1 - \kappa_t}\Big[p_{1|t}(x^i \mid z) - \delta_z(x^i)\Big].} \tag{7.6}$$

The derivation is two lines and worth doing, because it is where the denoiser appears out of nowhere.

Derivation of Equation (7.6). Write $a_t := \dot\kappa_t/(1 - \kappa_t)$. **Step 1: the conditional velocity.** Claim that

$$u_t^i(x^i, z \mid x_0, x_1) = a_t\big[\delta_{x_1^i}(x^i) - \delta_z(x^i)\big] \tag{7.7}$$

generates Equation (7.4). Substituting into the inflow-minus-outflow form of Equation (7.2), the inflow to x^i is

$$\sum_z p_t(z \mid x_0, x_1)\, a_t\big[\delta_{x_1^i}(x^i) - \delta_z(x^i)\big] = a_t\big[\delta_{x_1^i}(x^i) - p_t(x^i \mid x_0, x_1)\big], \tag{7.8}$$

using $\sum_z p_t(z \mid x_0, x_1) = 1$, while the outflow vanishes identically,

$$p_t(x^i \mid x_0, x_1) \sum_z a_t\big[\delta_{x_1^i}(z) - \delta_{x^i}(z)\big] = p_t(x^i \mid x_0, x_1)\, a_t\,[1 - 1] = 0. \tag{7.9}$$

So the net rate of change is $a_t\big[\delta_{x_1^i} - (1-\kappa_t)\delta_{x_0^i} - \kappa_t\delta_{x_1^i}\big] = a_t(1-\kappa_t)\big[\delta_{x_1^i} - \delta_{x_0^i}\big] = \dot\kappa_t\big[\delta_{x_1^i} - \delta_{x_0^i}\big]$, which is exactly ∂_t of Equation (7.4).

Step 2: marginalise. Equation (7.7) depends on (x_0, x_1) only through x_1^i, and not on x_0 at all. So Equation (7.5) gives

$$u_t^i(x^i, z) = a_t\Big[\sum_{x_0,x_1} \delta_{x_1^i}(x^i)\, p_t(x_0, x_1 \mid z) - \delta_z(x^i)\Big] = a_t\big[p_{1|t}(x^i \mid z) - \delta_z(x^i)\big], \tag{7.10}$$

because marginalising $\delta_{x_1^i}(x^i)$ against the posterior $p_t(x_0, x_1 \mid z)$ is by definition the posterior probability that the clean token at position i equals x^i. ∎

Notice what each step contributed. The *path* choice made the conditional velocity independent of the source x_0, which is why only a *clean-data* posterior survives the averaging; and the fact that the outflow term vanished is what leaves a single scalar schedule coefficient in front. Change the path — to one whose conditional velocity still depends on x_0, as the non-mixture paths of Section 7.5 do — and the learned object is no longer a plain denoiser.

Intuition: it is a denoiser after all

Equation (7.6) says: predict the clean token $p_{1|t}(\cdot \mid z)$, subtract where you currently are, and scale by $\dot\kappa_t/(1-\kappa_t)$. The learned object is a *posterior over clean data* — the same x_0-prediction as in Section 3.3 and Chapter 5, with time reversed. Compare Equation (5.3): with κ_t and α_t matched, the masked diffusion unmasking probability and the DFM Euler step coefficient are the same number. So for the mixture path, *discrete flow matching and discrete diffusion are the same algorithm*, up to time reversal and bookkeeping. The genuine content of DFM lies elsewhere: in the freedom to choose paths and couplings that diffusion's construction does not give you.

7.3. Training

Since the learned object is a posterior, the loss is cross-entropy (Gat et al., 2024, Eq. 28):

$$\mathcal{L}(\theta) = -\sum_i \mathbb{E}_{t,\,(X_0,X_1)\sim\pi,\,X_t\sim p_t(\cdot|X_0,X_1)} \log p^\theta_{1|t}\big(X_1^i \mid X_t\big). \tag{7.11}$$

Compare Equation (5.8): same shape, and the schedule weight has been absorbed into the velocity formula Equation (7.6) rather than appearing in the loss. This is a real difference. Equation (7.11) is *not* in general a likelihood bound — flow matching gives up the ELBO — but it is better conditioned, and its minimiser is the true posterior (Gat et al., 2024, Prop. 5).

Remark 7.4 (The ELBO/conditioning trade-off, again). This is the same trade-off as D3PM's auxiliary loss (Equation (3.5)) and as the reweighted-loss results of Shi and Titsias (2025) and Sahoo et al. (2026): an unweighted cross-entropy trains better than the exactly-weighted bound. Three independent literatures have now rediscovered this. It is probably the single most robust piece of empirical knowledge in the field, and it still lacks a satisfying theory.

7.4. What flow matching actually adds

If the mixture path reproduces discrete diffusion, why bother? Four things, all genuine.

1. Arbitrary couplings. Diffusion's forward process forces $\pi(x_0, x_1) = p_0(x_0)p_{\text{data}}(x_1)$, i.e. independent source and target. Flow matching lets you choose π, and in particular to couple each data point with a *nearby* source point. For discrete data the natural choice is a minimum-Hamming-distance coupling, which shortens trajectories and reduces the number of jumps the reverse process must make. This is the discrete analogue of optimal-transport couplings in continuous flow matching and is comparatively unexploited.

2. Multimodal generation. This is Campbell et al. (2024)'s contribution and the most practically important. Many scientific generation problems are jointly discrete and continuous: a protein is a sequence of amino acids (discrete) *and* a set of backbone frames (continuous); a molecule is atom types and bond types (discrete) and 3D coordinates (continuous). A flow framework handles these in one object by running a discrete flow on the categorical variables and a continuous flow on the Euclidean ones, coupled through a shared network and a shared time. Their application is protein co-design — generating sequence and structure jointly — and this has become the standard formulation for that problem (Section 12.7).

3. Flexible paths. Equation (7.4) is one choice. The general construction (Gat et al., 2024, Thm. 3) allows a convex combination of m components with separate schedulers, for instance a three-term mixture of target, uniform noise and source. This includes processes that mask *and* corrupt, and it is the natural home for the interpolating family of Section 5.8.

4. Remasking and correctors fall out. Because DFM's velocity is defined by the path rather than by a time reversal, you can construct velocities that move *away* from the data — re-masking or re-corrupting — while still generating the prescribed path. Campbell et al. (2024) use this for corrector steps, and the remasking samplers of Chapter 8 can be derived this way.

Listing 7.1 implements the mixture-path Euler sampler.

Listing 7.1 Discrete flow matching with a mixture path: the Euler update Equation (7.1) with the velocity Equation (7.6).

```
"""Discrete flow matching with a mixture path (Gat et al., 2024).

Convention of this file: t = 0 is NOISE, t = 1 is DATA.

Mixture path, per coordinate:
    p_t(x | x_0, x_1) = (1 - kappa_t) delta_{x_0} + kappa_t delta_{x_1}
Marginal velocity for that path (their Eq. 24):
    u_t(y, z) = kappa_t' / (1 - kappa_t) * [ p_{1|t}(y | z) - delta_z(y) ]
Euler step (their Eq. 12):
    X_{t+h} ~ delta_{X_t} + h u_t(., X_t)

We supply the EXACT posterior p_{1|t} by enumeration, so the only error left
is the sampler's.  That isolates integration + factorisation error from model
error -- the same experiment as for masked diffusion in Chapter 7.
"""
import itertools
import numpy as np

rng = np.random.default_rng(0)

L, V = 4, 3
# target: arithmetic progressions mod V  ->  V * (V-1) = 6 valid sequences
support = [tuple((a + i * d) % V for i in range(L))
           for a in range(V) for d in range(1, V)]
seqs = list(itertools.product(range(V), repeat=L))
pdata = np.array([1.0 / len(support) if s in support else 0.0 for s in seqs])
```

```
SEQ = np.array(seqs)                                    # (V^L, L)


def exact_p1_given_t(z, kappa):
    """Exact p_{1|t}(. | z), by enumeration.

    With x_0 uniform and independent of x_1,
        p(z^i | x_1^i) = (1 - kappa)/V + kappa * [z^i == x_1^i],
    so  p(x_1 | z)  is proportional to  pdata(x_1) * prod_i p(z^i | x_1^i).
    """
    n = z.shape[0]
    agree = (SEQ[None, :, :] == z[:, None, :])          # (n, V^L, L)
    lik = ((1.0 - kappa) / V + kappa * agree).prod(-1)  # (n, V^L)
    post = lik * pdata[None, :]
    post /= post.sum(-1, keepdims=True)
    out = np.zeros((n, L, V))
    for v in range(V):                                  # marginalise per site
        out[:, :, v] = (post[:, :, None] * (SEQ[None] == v)).sum(1)
    return out


def dfm_sample(n, steps):
    x = rng.integers(0, V, (n, L))
    h = 1.0 / steps
    for i in range(steps):
        t = i * h
        coef = h / (1.0 - t) if t < 1.0 else 0.0        # h * kappa'/(1-kappa)
        cur = (np.arange(V)[None, None, :] == x[:, :, None]).astype(float)
        probs = cur + coef * (exact_p1_given_t(x, t) - cur)
        probs = np.clip(probs, 0, None)
        probs /= probs.sum(-1, keepdims=True)
        u = rng.random((n, L, 1))
        x = (probs.cumsum(-1) < u).sum(-1).clip(0, V - 1)
    return x


valid = lambda s: np.array([tuple(r) in support for r in s])
for steps in (2, 4, 8, 32, 128):
    s = dfm_sample(4000, steps)
    print(f"{steps:4d} Euler steps -> {100 * valid(s).mean():5.1f}% on-support "
          f"(exact denoiser throughout)")
```

The listing supplies the *exact* posterior $p_{1|t}$ by enumeration, which isolates sampler error from model error completely. The target is the set of arithmetic progressions mod 3 of length 4 — six equally likely sequences — and the result is:

```
   2 Euler steps  →  28.4% on-support
   4 Euler steps  →  65.1% on-support
   8 Euler steps  →  86.1% on-support
  32 Euler steps  →  97.5% on-support
 128 Euler steps  →  99.5% on-support
```

Remark 7.5 (The most important experiment in this book). Read that table again. The denoiser is *exact* — it is the true Bayesian posterior, computed by enumeration, with zero approximation error. And the sampler still puts 72% of its mass off-support at two steps. The quality-versus-steps curve of a discrete diffusion or flow model is therefore **not** primarily a statement about the model. It is a statement about the integrator and about Remark 2.4.

This is the laboratory version of what Tang et al. (2026) establish at scale: few-step discrete diffusion samplers are "not distributionally correct even under an exact oracle denoiser", and the error shrinks only as the step count approaches the sequence length — at which point generation has become autoregressive. If you take one thing from this book into your own research, take the habit of asking, of any claimed improvement, whether it improves the model or the sampler, and benchmarking against an exact denoiser when the problem is small enough to allow it.

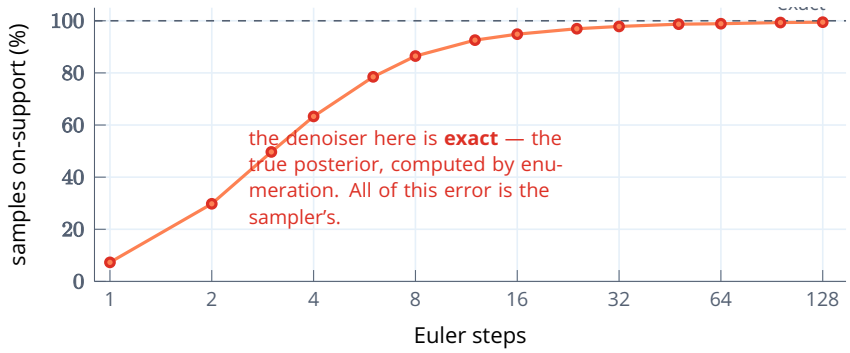


Figure 7.1 The most important experiment in this book (Remark 7.5). A discrete flow-matching sampler on length-4 sequences over a 3-letter alphabet, whose target is the set of arithmetic progressions mod 3. The denoiser is not learned: it is the exact Bayesian posterior, obtained by enumerating all 3^4 sequences, so it has zero approximation error. The sampler nevertheless places most of its mass off-support at two steps, and only approaches correctness as the step count grows well beyond the sequence length. A quality-versus-steps curve is therefore not a statement about a model; it is a statement about the integrator and about Remark 2.4. This is the laboratory version of what Tang et al. (2026) establish at scale with an oracle denoiser. (A finer sweep, with 8000 samples per point, than Listing 7.1; small differences are Monte Carlo noise.)

7.5. Kinetic optimality and related developments

Fisher–Flow and simplex geometry. A parallel line of work puts a Riemannian structure on the simplex and does flow matching there, using the Fisher–Rao metric under which the simplex is (a positive orthant of) a sphere. This gives a principled geometry rather than the pathological linear interpolation that Stark et al. (2024) identified (Section 7.6).

Dirichlet flow matching. Stark et al. (2024) replace the linear path with Dirichlet distributions: for target vertex $\mathbf{e}_i$ the conditional at time t is $\mathrm{Dir}(\mathbf{1} + t\,\mathbf{e}_i)$, which starts uniform on the simplex and concentrates towards the vertex while retaining full support at every time. This fixes the shrinking-support pathology exactly. Because the resulting sampling process is a deterministic map from noise to data, it can be distilled to a single step — they report $O(L)$ speedups over autoregressive models with minimal loss. See Sections 7.6 and 12.7.

Theory. Convergence and error analyses for discrete flow matching have begun to appear (Su et al., 2025), generally finding rates comparable to discrete diffusion. This is an active area.

Remark 7.6 ( ). See Equation (3.5), Equation (7.11), Shi and Titsias (2025) and Sahoo et al. (2026) for four independent instances of "the unweighted cross-entropy trains better than the exact bound".

7.6. Continuous relaxations: embeddings, simplices and latents

Historically this family came first: before anyone had a well-conditioned native discrete formulation, the obvious move was to embed discrete data in a continuous space and run a Gaussian diffusion. The field has largely moved on, but this material is not archaeology. Three reasons it still matters: some of these methods remain the best available for particular tasks (notably Dirichlet flow matching for DNA); the hybrid models of Section 5.8 are a return to these ideas with better diagnosis; and the failure modes are instructive, because they tell you precisely what the native-discrete formulations are buying.

7.6.1. Embedding-space diffusion

Diffusion-LM. Li et al. (2022) learn an embedding $\mathrm{emb} : \mathcal{V} \to \mathbb{R}^d$ jointly with a Gaussian diffusion over embedded sequences, and decode by rounding to the nearest embedding. The motivation was controllable generation: with a continuous latent you can apply gradient-based control (a classifier's gradient) at every denoising step, which is how plug-and-play control works for images.

The method works, and it introduced several ideas that outlasted it (notably the use of gradient-based control for text generation). Its weaknesses are structural:

- **The rounding cliff.** Almost the whole diffusion trajectory lies in regions that round to the same token, so the gradient signal about *which* token is concentrated in a tiny part of the noise schedule. The paper needs an explicit rounding-aware loss term to make training work.
- **Two learning problems at once.** The embedding geometry and the denoiser are learned jointly, and the embedding can collapse or spread in ways that make the diffusion either trivial or impossible.
- **Likelihoods are not comparable.** A continuous density over embeddings is not a probability of a token sequence without careful accounting for the decoder, which is why Diffusion-LM's LM1B perplexity of 118.62 (Table 5.1) is hard to interpret and poor besides.

CDCD. Dieleman et al. (2022) fix the most important of these. Their *Continuous Diffusion for Categorical Data* replaces the rounding loss with a cross-entropy loss computed through a softmax over embedding similarities — so the model is trained to produce the right *token*, not the right vector — and introduces *time warping*, which adapts the noise-level distribution during training to keep the loss informative. Time warping is the idea here with the longest legs: it is self-tuning schedule selection, and the same problem is still being solved by hand in masked diffusion.

Plaid. Gulrajani and Hashimoto (2023) took the likelihood question seriously, producing careful likelihood-based continuous diffusion language models and the first scaling-law comparison with autoregressive models. The conclusion was sobering and, in retrospect, correct: continuous diffusion language models scale worse than autoregressive ones. That result is part of why the field moved to native-discrete formulations.

TESS and simplex-space variants. A related family diffuses over logit or simplex space rather than embedding space, which at least keeps the representation interpretable as a distribution over tokens.

Intuition: what goes wrong, in one sentence

A Gaussian diffusion in $\mathbb{R}^d$ spends its capacity modelling a d-dimensional density, but the information content of the data is $L \log K$ nats, concentrated on K^L isolated points. Almost all of the modelling effort goes into the empty space between the points.

7.6.2. Bit diffusion

Chen et al. (2023) propose something cleaner: encode each token as binary bits, treat the bits as real numbers in $\{-1, +1\}$ ("analog bits"), diffuse with a standard continuous model, and threshold at the end. This has real advantages — it reuses continuous diffusion machinery with no modification, and it scales to large vocabularies with $\log_2 K$ channels instead of K — and real disadvantages.

The disadvantage is the one noted in Chapter 1: the binary code imposes a metric. Tokens 7 (`0111`) and 8 (`1000`) are at Hamming distance 4, while 7 and 6 (`0110`) are at distance 1. Under Gaussian noise on the bits, the corruption process therefore treats semantically unrelated tokens as near-neighbours and vice versa. For *ordinal* data where a Gray code is natural this is fine; for text it is arbitrary. Recent work has returned to bit-level formulations with native discrete machinery instead.

7.6.3. Diffusion on the simplex

The most principled relaxation puts the diffusion on Δ^K, where a point genuinely is a distribution over tokens. The question is which path to take across the simplex, and the answer turns out to matter a great deal.

Why linear interpolation fails

> **Proposition 7.7 (Shrinking support, after Stark et al., 2024, Prop. 1).** Under linear interpolation between a noise point and a vertex of the simplex, for times $t > 1/k$ the converged posterior has support on at most $k - 1$ vertices.

The consequence is that as the vocabulary size K grows, the marginal vector field becomes increasingly discontinuous, and sampling becomes acutely sensitive to the integration step size. This is a crisp, provable statement of why the naive simplex relaxation does not work, and it is worth knowing because the naive construction is the one everybody reinvents first.

Dirichlet flow matching

Stark et al. (2024) replace the linear path with a family that has full support at all times. For target vertex $\mathbf{e}_i$, the conditional distribution at time t is

$$p_t(\cdot \mid \mathbf{e}_i) = \mathrm{Dir}\big(\mathbf{1} + t\,\mathbf{e}_i\big), \tag{7.12}$$

which at $t = 0$ is $\mathrm{Dir}(\mathbf{1})$ — the uniform distribution on the simplex — and concentrates towards $\mathbf{e}_i$ as t grows, while retaining support on the whole simplex at every finite time. The pathology of Proposition 7.7 is removed by construction.

What this buys:

- **Guidance.** They derive the relationship between the score of a mixture of Dirichlets and the resulting flow, which gives both classifier and classifier-free guidance on the simplex. Classifier-free guidance improves unconditional generation as well as hitting design targets.

- **One-step distillation.** Because the sampling process is a *deterministic* map from noise to data — a genuine probability flow, which native discrete formulations do not have (Section 2.8) — it can be distilled to a single step with minimal loss, giving $O(L)$ speedups over autoregressive models. This is the clearest existing demonstration of why a continuous relaxation might be worth the trouble.
- **Strong DNA results.** On distributional similarity for enhancer design (Fréchet biological distance, lower is better) they report 1.9 against 36.0 for an autoregressive language model on Melanoma DNA, and 1.0 against 25.2 on Fly Brain DNA. Guided generation produced cell-type-specific enhancers improving on experimentally validated sequences from the literature.

Fisher–Rao geometry

A further refinement: equip the simplex with the Fisher–Rao metric, under which it is isometric to a positive orthant of a sphere, and do flow matching on that manifold. This gives a canonical geometry rather than an ad hoc path, and connects to the result that the cosine schedule is Fisher–Rao optimal for masked diffusion (Zhang and Syed, 2025) — the appearance of the same metric in both places is not a coincidence and is worth thinking about.

Dirichlet diffusion score models

Before the flow-matching formulation, Avdeyev et al. (2023) built a Dirichlet diffusion score model using Jacobi diffusion processes on the simplex, with applications to DNA sequence design. This is the score-matching counterpart of Equation (7.12) and remains a reasonable baseline in the genomics literature.

7.6.4. Latent and quantised approaches

A different strategy: do not relax the discrete variables, *replace* them. Learn a vector-quantised autoencoder, then run a discrete diffusion over the VQ codes. This is standard practice in image and audio generation (VQ-Diffusion and successors) and has two attractions: the code vocabulary is chosen by you rather than by a tokeniser, and the sequence is much shorter than the raw data.

For text it is less attractive, because text is already discrete and tokenisation is already a learned quantisation — adding another layer usually loses more than it gains. But the tokenisation question itself is live: Yuan et al. (2026) organise their survey of discrete diffusion around exactly this axis, from tokenisation to generation, and several 2025–26 papers revisit what the right discrete units are, including the observation that scale-wise visual autoregression is itself a discrete diffusion (Kumar et al., 2025) and hierarchical semantic-scale formulations (Zhou et al., 2025).

7.6.5. Where this family stands

Key takeaways

- Discrete flow matching prescribes a probability path and learns a probability velocity satisfying the discrete continuity equation Equation (7.2), which is the Kolmogorov equation with the velocity as generator.
- The marginal velocity is a posterior average of conditional velocities (Theorem 7.3); for the mixture path it reduces to $\frac{\dot\kappa_t}{1-\kappa_t}[p_{1|t}(\cdot \mid z) - \delta_z]$, i.e. clean-data prediction again.
- For the mixture path, DFM and discrete diffusion are the same algorithm up to time reversal. The loss is an unweighted cross-entropy rather than the ELBO's weighted one — better conditioned, no longer a bound.

Table 7.1 Continuous relaxations: current status.

Approach	Status	Use it when
Embedding diffusion	superseded for text	you need gradient-based control and nothing else works
Dirichlet FM / simplex	**competitive, state of the art for DNA**	small K; you want one-step distillation or gradient guidance
Analog bits	niche	ordinal vocabularies with a natural (Gray) code; very large K
VQ-latent + discrete diffusion	standard for images/audio	the data is not natively discrete
Hybrid (CANDI, soft-mask)	new, promising	few-step sampling or off-the-shelf classifier guidance (Section 5.8)

- The real additions are: arbitrary source–target couplings, joint discrete+continuous (multimodal) generation, richer multi-component paths, and velocities that can corrupt as well as denoise.
- Multimodal flows (Campbell et al., 2024) are now the standard formulation for protein sequence–structure co-design.
- Embedding-space diffusion (Diffusion-LM) introduced gradient-based control for text but suffers a rounding cliff and poor likelihoods; CDCD's cross-entropy-through-embeddings and *time warping* fixed much of this, and Plaid established that continuous diffusion LMs scale worse than autoregressive ones.
- Analog bits are clean and cheap but impose an arbitrary metric on the vocabulary.
- Linear interpolation on the simplex is provably pathological (Proposition 7.7); Dirichlet paths Equation (7.12) fix it by retaining full support at all times.
- Dirichlet flow matching is genuinely state of the art for DNA sequence design and is one of the very few discrete-data generative models that distils to a single step — because it has a real probability flow.
- The family is not dead: the hybrids of Section 5.8 are these ideas, better diagnosed.

Open questions and research directions

1. **Discrete optimal transport couplings.** The freedom to choose $\pi(x_0, x_1)$ is DFM's clearest advantage over diffusion and is barely used. A minimum-Hamming coupling computed per minibatch (the discrete analogue of minibatch OT in continuous flow matching) is cheap and should shorten trajectories, hence reduce the step count — the field's main bottleneck. I am not aware of a careful study of this, which is surprising.
2. **Is there a "straightness" notion for discrete paths?** Rectified flow works because continuous trajectories can be made straight, and straight trajectories are exactly those an Euler solver integrates without error. The discrete analogue of straightness is unclear — perhaps "each coordinate jumps at most once", which the mixture path already satisfies. If so, the relevant quantity is not straightness but the *correlation* between coordinates' jump times, and a "reflow" procedure would decorrelate them. This seems like a real and tractable question.

3. **Kinetic-optimal paths.** In continuous flow matching one can ask for the path minimising the kinetic energy, which gives optimal transport. The discrete analogue — minimise the expected number of jumps, or the expected total rate — is well posed and, as far as I can tell, mostly unsolved beyond special cases. The answer would *derive* the schedule rather than tuning it.
4. **Multimodal coupling.** In Campbell et al. (2024) the discrete and continuous flows share a network and a time variable but are otherwise independent processes. Genuine coupling — where the discrete jump rate depends on the continuous state and vice versa, as physics would demand (a bond type should depend on an interatomic distance) — is the obvious next step and raises interesting questions about whether the marginalisation trick survives.
5. **Why does the unweighted loss win?** Flagged in Remark 7.6: the same observation has now been made independently by the D3PM, DFM, reweighted-bound and scaling-law literatures. A theory of which reweightings preserve a bound and which minimise estimator variance would unify them.
6. **Why does one-step distillation work here and not for masked diffusion?** The answer — Dirichlet FM has a deterministic noise-to-data map and masked diffusion does not — is clear, but its consequences are not exhausted. Could one *train* a masked diffusion LM and then distil it through a Dirichlet or Fisher–Rao surrogate flow fitted post hoc? This is the same question as Remark 5.10 approached from the relaxation side, and it has the advantage that the relaxation need only be good enough to support distillation, not to be a good generative model itself.
7. **Time warping for discrete diffusion.** CDCD's adaptive noise-level distribution is, as far as I can tell, not used in masked diffusion, where the t-sampling distribution is uniform by default and the estimator variance is a known problem (Chapter 5). Porting time warping is a small, concrete, probably-effective project.
8. **Does the Fisher–Rao metric give a unified story?** The same metric appears in Fisher–Flow's simplex geometry and in the optimality of the cosine schedule for masked diffusion (Zhang and Syed, 2025). A single geometric account of discrete diffusion as a flow in Fisher–Rao geometry, from which both the schedule and the path fall out, looks within reach and would be a nice piece of work.
9. **Relaxations for structured discrete objects.** All of these relax a *categorical* variable. Relaxations of permutations (Sinkhorn/Gumbel), of trees, and of graphs up to isomorphism exist in the broader literature and have hardly been combined with diffusion on the discrete object. Retrosynthesis, parsing and molecular graph-matching are natural targets.
10. **What are the right discrete units?** If the vocabulary is itself a modelling choice (as it is for molecules — SMILES characters, SELFIES tokens, SAFE fragments, motifs), then the tokenisation and the diffusion should be designed together. Lee et al. (2025) get a large win simply by choosing fragment-level units (Chapter 12), which suggests this is underexploited generally.

CHAPTER 8

Sampling and Decoding

CHAPTER

Authors	Guanghan Wang et al. (ReMDM); Jaeyeon Kim et al.; Luhan Tang et al.; Campbell et al.; Liu et al. (DCD)
Year	2026
Published	NeurIPS 2025 / ICML 2025 / preprints
Link	https://arxiv.org/abs/2503.00307
Tags	tau-leaping, decoding order, remasking, predictor-corrector, caching, distillation, parallel decoding

This is where most of the recent progress is, and where most of the recent confusion is. The chapter is organised around one question: given a trained model and a budget of N network evaluations, how do you get the best sample?

8.1. The problem, stated precisely

Recall three facts established earlier:

1. The exact reverse process changes **one token at a time** (Equation (4.1) and the remark after it). An L-token sequence therefore needs $\Theta(L)$ exact reverse jumps.
2. Any sampler that updates several positions in one network evaluation is sampling from a **product of marginals**, not from the joint (Remark 2.4).
3. This error does **not** vanish as the model improves. With an exact, enumerated denoiser, a 4-token flow-matching sampler still places 72% of its mass off-support at two steps (Remark 7.5).

So the whole design space is: how do you spend $N \ll L$ network evaluations so as to minimise the damage from (2)? Before cataloguing the answers it is worth knowing *exactly* how much that damage is, because — unusually for this field — there is a clean closed form.

8.1.1. The cost of a parallel update

Fix a datum and consider a sampler that reveals the L positions in disjoint blocks $S_1, \dots, S_N$ (so $\bigcup_k S_k = [L]$), and that at each step uses the *exact* conditional marginals of the true distribution. Write $S_{<k} = S_1 \cup \cdots \cup S_{k-1}$ for the positions already revealed. The sampler's output distribution is

$$q(\boldsymbol{x}) = \prod_{k=1}^{N} \prod_{i \in S_k} p\big(x^i \,\big|\, \boldsymbol{x}^{S_{<k}}\big), \tag{8.1}$$

whereas the target factorises as $p(\boldsymbol{x}) = \prod_k p(\boldsymbol{x}^{S_k} \mid \boldsymbol{x}^{S_{<k}})$. The two differ only in that Equation (8.1) replaces each block's joint by a product of its marginals. The discrepancy has a name.

Definition 8.1 (Total correlation). For a set of coordinates S, the *total correlation* (or multi-information) is

$$\mathrm{TC}(\boldsymbol{x}^S) \;=\; D_{\mathrm{KL}}\big(p(\boldsymbol{x}^S)\,\big\|\,\textstyle\prod_{i\in S} p(x^i)\big) \;=\; \displaystyle\sum_{i\in S} H(x^i) \;-\; H(\boldsymbol{x}^S) \;\geq\; 0, \tag{8.2}$$

with equality iff the coordinates in S are independent. The conditional version $\mathrm{TC}(\boldsymbol{x}^S \mid \boldsymbol{x}^R)$ replaces every term by its conditional counterpart.

Theorem 8.2 (Parallel decoding costs exactly the correlation it ignores). With q as in Equation (8.1) and an exact denoiser,

$$D_{\mathrm{KL}}(p\,\|\,q) \;=\; \sum_{k=1}^{N} \mathbb{E}_{\boldsymbol{x}^{S_{<k}}\sim p}\Big[\,\mathrm{TC}\left(\boldsymbol{x}^{S_k} \,\middle|\, \boldsymbol{x}^{S_{<k}}\right)\Big]. \tag{8.3}$$

Proof. Take logarithms of the two factorisations and subtract:

$$\begin{aligned} D_{\mathrm{KL}}(p\,\|\,q) &= \mathbb{E}_p\Big[\sum_{k=1}^{N} \log p\big(\boldsymbol{x}^{S_k} \mid \boldsymbol{x}^{S_{<k}}\big) - \sum_{k=1}^{N}\sum_{i\in S_k} \log p\big(x^i \mid \boldsymbol{x}^{S_{<k}}\big)\Big] \\ &= \sum_{k=1}^{N} \mathbb{E}_p\Big[\log \frac{p\big(\boldsymbol{x}^{S_k} \mid \boldsymbol{x}^{S_{<k}}\big)}{\prod_{i\in S_k} p\big(x^i \mid \boldsymbol{x}^{S_{<k}}\big)}\Big]. \end{aligned}$$

For each k, condition on $\boldsymbol{x}^{S_{<k}}$ and take the inner expectation first. The inner quantity is by definition $D_{\mathrm{KL}}\big(p(\boldsymbol{x}^{S_k} \mid \boldsymbol{x}^{S_{<k}}) \,\big\|\, \prod_i p(x^i \mid \boldsymbol{x}^{S_{<k}})\big) = \mathrm{TC}(\boldsymbol{x}^{S_k} \mid \boldsymbol{x}^{S_{<k}})$, and the outer expectation is over $\boldsymbol{x}^{S_{<k}} \sim p$. The telescoping needed no assumption on p. ∎

Equation (8.3) is an identity, not a bound, and it immediately settles several things the book has so far asserted.

Corollary 8.3 (Serial decoding is exact). If $|S_k| = 1$ for every k, then $\mathrm{TC} = 0$ identically and $D_{\mathrm{KL}}(p\,\|\,q) = 0$: a sampler that commits one token per step, from exact conditionals, draws from the target distribution exactly — in any order.

Corollary 8.4 (One-shot decoding costs the whole correlation). If $N = 1$, then $D_{\mathrm{KL}}(p\,\|\,q) = \mathrm{TC}(\boldsymbol{x}) = \sum_i H(x^i) - H(\boldsymbol{x})$.

Example 8.5 (Two perfectly correlated bits). Let p put mass $\frac{1}{2}$ on $(0,0)$ and $\frac{1}{2}$ on $(1,1)$. Then $H(x^1) = H(x^2) = \log 2$ and $H(\boldsymbol{x}) = \log 2$, so $\mathrm{TC} = \log 2$. A one-step sampler therefore incurs exactly $\log 2$ nats, and indeed it produces the uniform distribution on four outcomes, half of which are off-support.

Intuition: parallelism is paid for in nats of correlation

Theorem 8.2 says something sharper than "parallel updates are approximate". It says the *entire* cost of parallelism, measured in KL, is the amount of dependence among the coordinates you chose to commit together, conditioned on what you already know — no more and no less. The model does not enter. Three practical readings follow.

First, the right question when designing a sampler is not "how many tokens can I commit?" but "which *sets* of tokens are nearly conditionally independent?". Second, revealing information reduces the cost of later parallelism, because conditioning typically destroys correlation

— which is why the error per step shrinks as a sample fills in, and why quality-versus-NFE curves are convex. Third, the quantity to estimate at inference time is $\mathrm{TC}(x^S \mid x^{S_{<k}})$; confidence and margin heuristics (Section 8.3) are cheap proxies for it, which is both why they work and why they are not the final word.

Remark 8.6 (Relation to the published bounds). Lavenant and Zanella (2025) bound the factorisation error for *random*-order schedules by $(\mathbb{E}[s_{\max}] - 1)D(\pi)$, with $D(\pi)$ an averaged correlation measure and $s_{\max}$ the largest block size (Section 13.6). Theorem 8.2 is the exact, schedule-specific counterpart: it holds for any fixed partition and replaces the averaged surrogate by the conditional total correlation of the blocks actually used. The two agree in spirit — error is block size times correlation — and the identity is what one would want to estimate in order to choose the blocks adaptively.

Listing C.3 checks Equation (8.3) numerically against brute-force enumeration for several partitions of a five-token sequence; the two sides agree to ten decimal places, and the serial and one-shot corollaries come out at 0 and $\mathrm{TC}(x)$ respectively.

With the cost of parallelism pinned down, the design space has four families of answer, and they compose.

Table 8.1 The four levers available at inference time.

Lever	Mechanism	Representative work
Better integrator	reduce discretisation error at fixed parallelism	analytic/Tweedie sampling (Lou et al., 2024); step-schedule optimisation (Park et al., 2025; Lavenant and Zanella, 2025)
Better *which*	choose *which* positions to commit in parallel	confidence/margin/entropy orders (Kim et al., 2025; Ye et al., 2025); EB-sampler
Revision	allow committed tokens to be changed again	ReMDM (Wang et al., 2025b); correctors (Campbell et al., 2022); DDPD
Model the dependence	stop assuming conditional independence	Discrete Copula Diffusion (Liu et al., 2025); Di4C (Hayakawa et al., 2025)

8.2. Integrators

Covered in Chapter 4 and Section 4.7; summarised here.

Euler and **τ-leaping** are first-order and are what most implementations use. **Analytic (Tweedie) sampling** (Lou et al., 2024) handles the known matrix-exponential part of the dynamics exactly and only approximates the learned part — an exponential integrator — and is the source of SEDD's claim of comparable quality at about $32\times$ fewer network evaluations.

Step schedules. The spacing of the N time points is a free choice and matters. Park et al. (2025) optimise the discretisation schedule offline by minimising a tractable proxy for the compounding decoding error, and report gains across text, image and music tasks at no extra inference cost. This is a cheap win that is often left on the table.

8.3. Decoding order: which tokens to commit

For masked diffusion, the sampler must choose, at each step, which masked positions to unmask. The default from the ELBO is *random*: that is what the forward process prescribes. It is also, empirically, much worse than the alternatives.

8.3.1. Confidence-based orders

Let $p_\theta(\cdot \mid \boldsymbol{z}_t)$ be the per-position posteriors. Common scoring rules for choosing which positions to commit:

$$\text{top-probability:} \quad c^\ell = \max_v p_\theta(v \mid \boldsymbol{z}_t)^\ell, \tag{8.4}$$

$$\text{margin:} \quad c^\ell = p^\ell_{(1)} - p^\ell_{(2)} \quad \text{(gap between top two)}, \tag{8.5}$$

$$\text{negative entropy:} \quad c^\ell = -H\big(p_\theta(\cdot \mid \boldsymbol{z}_t)^\ell\big). \tag{8.6}$$

Unmask the k positions with the highest c^ℓ.

> **Intuition: confidence is a proxy for conditional independence**
>
> Why should committing the most confident positions be right? Because Remark 2.4 says the damage from a parallel update is the *dependence* among the positions you commit. A position whose posterior is nearly deterministic contributes almost no dependence: whatever the others turn out to be, that token was going to be the same. So high confidence ≈ low mutual information with the rest ≈ safe to commit in parallel. This is a heuristic, not a theorem — a set of individually confident positions can still be jointly dependent — but it is a good one, and it explains why margin and entropy work too: all three are proxies for "how much does this position still depend on the others".

Kim et al. (2025) give the sharpest evidence. Their framing is that masked diffusion models are trained on subproblems that are computationally harder than an autoregressive model's — for distributions with a natural left-to-right order, a large fraction of the corresponding masked subproblems is provably intractable at certain noise levels — but that *the model's own logits already contain enough information to find a good unmasking order*, with no retraining. The headline result: on Sudoku, adaptive inference raises the accuracy of pretrained masked diffusion models from under 7% to about 90%, and the adapted models beat autoregressive models with $7\times$ as many parameters trained with teacher forcing on the correct decoding order.

That is one of the largest inference-only gains reported anywhere in generative modelling, and it is a strong argument for treating the sampler as a first-class research object.

8.3.2. Caveats

Confidence orders exploit the model's priors, which may be an artefact. Tang et al. (2026) replace the learned denoiser with an exact HMM posterior and find that LLaDA's confidence-based remasking appears to rely on denoiser priors: with an oracle that lacks them, probability mass concentrates on a few high-frequency transitions as the step count rises. So part of what confidence-based decoding achieves is a useful *bias*, not a better approximation of the target distribution.

Train/test mismatch. Training masks independently and uniformly; a confidence-ordered sampler visits highly structured masking patterns. The model is therefore evaluated off-distribution

at inference. This is flagged in Chapter 5 as an open problem, and it is the main theoretical objection to confidence-based decoding.

Entropy-bounded and adaptive parallelism. Rather than fixing k per step, choose how many tokens to commit based on a bound on the induced error. The EB-sampler (entropy-bounded unmasking) and adaptive parallel decoding do this; Fast-dLLM uses a confidence threshold to decide how many tokens to accept per step.

8.4. Revision: remasking and correctors

The structural defect of masked diffusion is that SUBS freezes committed tokens (Chapter 5). Wang et al. (2025b) call this the "failure to remask property" and give the clean fix.

Theorem 8.7 (ReMDM, after Wang et al., 2025b, Thm. 3.1). Consider a backward process in which a decoded token is kept with probability $1 - \sigma_t$ and remasked with probability σ_t, where

$$0 \;\le\; \sigma_t \;\le\; \min\left\{1,\; \frac{1-\alpha_s}{\alpha_t}\right\}. \tag{8.7}$$

Then the marginals $q_\sigma(z_t \mid x)$ match those of standard masked diffusion, for every such σ_t. Setting $\sigma_t = 0$ recovers the MDLM sampler.

So Equation (8.7) describes a whole *family* of samplers, all valid for the same trained model, differing in how aggressively they revise. The process is non-Markovian and conditions on x — the same device as DDIM in continuous diffusion — and its negative ELBO is a rescaled version of the MDLM objective, which is why the family can be applied to an already-trained MDLM checkpoint with no retraining.

Wang et al. (2025b) present this explicitly as a predictor–corrector method and argue it is more general than the earlier forward–backward and discrete-flow-matching correctors. Their schedule variants (loop, cap, rescale, confidence-based, and a switch time t_{switch}) control when and how much remasking happens.

Reported effects: on OpenWebText, MAUVE approaches that of an autoregressive model with the same architecture as the step count rises, and quality degrades less than baselines at low step counts; on ImageNet, better scaling with inference compute than vanilla masked generation such as MaskGIT; on QM9 molecules, an improved Pareto frontier of novelty against the target property.

Intuition: why remasking is the right primitive

A corrector step (Proposition 4.3) spends a network evaluation reducing error at the current time rather than advancing time. Remasking is what a corrector step *is* for absorbing noise: to change a committed token you must first un-commit it. Theorem 8.7 says the amount of un-committing you are allowed is bounded by how much mass the forward process had on `[MASK]` — you cannot remask more than the marginal permits without changing the distribution you are sampling from. Uniform-state diffusion (Section 5.8) gets this for free, because its reverse process can always change any token; remasking is how you retrofit that capability onto a masked model.

Planner/denoiser decomposition. A related idea: separate the decision of *which* position to update (a planner) from *what* to put there (a denoiser). "Think While You Generate" (DDPD) does this explicitly, training a separate planner that identifies which tokens are wrong. This generalises

both confidence-based ordering and remasking, and is a natural place to apply reinforcement learning.

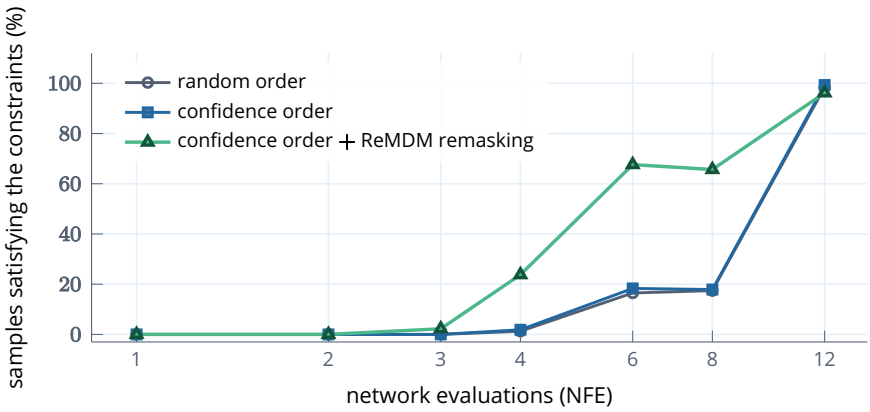


Figure 8.1 Three samplers, one model, matched budget. Every point costs the same number of forward passes; only the inference rule differs. Two lessons. **Revision beats ordering:** allowing the sampler to un-commit tokens (ReMDM, Theorem 8.7) is worth far more at small budgets than choosing a better order — it is a corrector step in the sense of Proposition 4.3, spending compute on repairing an inconsistent partial sample rather than on advancing time. **Remasking costs something at large budgets:** once the sampler is nearly serial it undoes work that was already correct, so σ_t should be annealed to zero. Confidence ordering helps little here because this task's conditional entropy is spread evenly across positions; contrast Kim et al. (2025), who lift Sudoku accuracy from under 7% to about 90% by ordering alone. (A finer sweep, with 2048 samples per point, than Table 8.2; small differences are Monte Carlo noise.)

8.5. Modelling the dependence

The most direct attack on Remark 2.4: stop assuming the positions being updated in parallel are independent.

Discrete Copula Diffusion. Liu et al. (2025) observe that a diffusion model gives good *marginals* but no dependence structure, while a small autoregressive model gives dependence cheaply. Combining a diffusion model's marginals with a copula from an autoregressive model supplies the missing dependence, improving few-step sampling substantially. This is a clean idea and, in my view, under-pursued.

Di4C. Hayakawa et al. (2025) distil a discrete diffusion model specifically so that the student captures *dimensional correlations* that the teacher's factorised one-step prediction does not — the distillation target is the joint, not the marginals. This is the distillation analogue of the copula idea.

Self-speculative and hybrid decoding. Use the diffusion model to propose several tokens in parallel and a cheap verifier (or the model itself, run autoregressively) to accept or reject — the discrete-diffusion analogue of speculative decoding. Set Block Decoding and self-speculative decoding schemes do this. The appeal is that the accept/reject step restores correctness with respect to a serial reference, so the speedup is free of distributional error, which is unusual in this field.

8.6. Caching

A practical but important lever. In an autoregressive transformer, KV caching makes each new token cost $O(L)$ rather than $O(L^2)$. In a diffusion model the whole sequence changes each step,

so naive caching is invalid — which is a large part of why the per-step cost of a diffusion LM is higher than an AR model's per-token cost, and hence why the speed advantage is contested (Chapter 1).

Two observations rescue it. First, Ou et al. (2025)'s result (Equation (6.6)) that for absorbing noise the model output depends only on the unmasked tokens means caching across steps in which nothing was unmasked is *exact*. Second, in practice most positions change slowly, so approximate caching works well: dKV-Cache, dLLM-Cache, Elastic-Cache and Fast-dLLM's block-wise cache all exploit this and report large speedups. These are approximations, and their distributional cost is not usually measured — which, given Section 10.8, it should be.

8.7. Distillation

The lever with the largest headroom and the biggest obstacle (Remark 5.10).

What works. For uniform-state diffusion, the diffusion duality gives deterministic trajectories to distil along, and Discrete Consistency Distillation takes 1024 steps to 8 (Sahoo et al., 2025). For simplex flows, Dirichlet flow matching distils to one step (Stark et al., 2024). For masked diffusion, Di4C distils dimensional correlations (Hayakawa et al., 2025), SDTT applies progressive-distillation-style self-distillation, and certainty-forcing distillation (dParallel) trains the model to become confident earlier so that more tokens can be committed per step.

What does not exist. A consistency-distillation analogue for masked diffusion, for want of a deterministic noise-to-data map.

8.8. A practical recipe

If you are sampling from a masked diffusion language model today, a sensible default stack:

1. Use a **confidence or margin order**, not random. This is the single largest free win (Kim et al., 2025).
2. Use **adaptive parallelism**: commit as many tokens per step as a confidence threshold allows, rather than a fixed k.
3. Add **remasking** with a conservative σ_t schedule (Wang et al., 2025b), especially if you are at low step counts.
4. Use **block/semi-autoregressive** decoding (Arriola et al., 2025) if you need long outputs or KV caching.
5. **Optimise the step schedule** offline (Park et al., 2025).
6. **Measure the distributional cost** of every one of the above, not just generative perplexity. See Section 10.8.

Listing 8.1 implements items 1–3 and compares them on the trained toy model of Chapter 5.

Listing 8.1 Random versus confidence-ordered decoding at matched network-evaluation budgets, on the trained masked diffusion model of Chapter 5.

```
"""Decoding strategies for a masked diffusion model, at matched NFE.

Imports the trained toy model of Appendix C and compares
  (a) random unmasking order            -- what the ELBO prescribes
  (b) confidence-ordered unmasking      -- top-probability first
  (c) confidence order + ReMDM remasking
on the number of samples that satisfy the data's global constraints.
"""
```

```python
import torch

from appC_masked_lm import (Denoiser, L, V, MASK, alpha, batch, valid,
                            get_trained_model)

torch.manual_seed(0)
model = get_trained_model()           # trains once and caches to disk
model.eval()


@torch.no_grad()
def decode(model, n, steps, order="confidence", sigma=0.0):
    """Unmask ceil(L/steps) tokens per step.

    order  : "random" or "confidence"
    sigma  : ReMDM remasking probability applied to already-decoded tokens.
             Theorem 3.1 of Wang et al. (2025) requires
             0 <= sigma <= min{1, (1 - alpha_s) / alpha_t}.
    """
    z = torch.full((n, L), MASK, dtype=torch.long)
    per = max(1, (L + steps - 1) // steps)
    nsteps = (L + per - 1) // per
    for i in range(nsteps):
        t = max(1e-3, 1.0 - i / nsteps)
        s = max(0.0, 1.0 - (i + 1) / nsteps)
        probs = model(z, torch.full((n,), t)).softmax(-1)
        conf, _ = probs.max(-1)
        draw = torch.multinomial(probs.reshape(-1, V), 1).reshape(n, L)

        # --- optional ReMDM corrector step: remask some decoded tokens -----
        if sigma > 0.0 and i > 0:
            cap = min(1.0, (1.0 - alpha(s)) / max(alpha(t), 1e-6))
            sig = min(sigma, cap)
            remask = (torch.rand(n, L) < sig) & (z != MASK)
            z = torch.where(remask, torch.full_like(z, MASK), z)

        # --- choose which masked positions to commit -----------------------
        score = torch.rand_like(conf) if order == "random" else conf.clone()
        score[z != MASK] = -1.0
        k = min(per, int((z == MASK).sum(-1).max().item()))
        if k == 0:
            continue
        idx = score.topk(k, dim=-1).indices
        pick = torch.zeros_like(z, dtype=torch.bool).scatter_(1, idx, True)
        pick &= (z == MASK)
        z = torch.where(pick, draw, z)

    # fill anything still masked
    probs = model(z, torch.full((z.shape[0],), 1e-3)).softmax(-1)
    return torch.where(z == MASK, probs.argmax(-1), z)


print(f"{'NFE':>5} {'random':>10} {'confidence':>12} {'conf+ReMDM':>12}")
for steps in (2, 3, 4, 6, 12):
    r = valid(decode(model, 1024, steps, "random")).float().mean()
    c = valid(decode(model, 1024, steps, "confidence")).float().mean()
    m = valid(decode(model, 1024, steps, "confidence", sigma=0.3)).float().mean()
    print(f"{steps:5d} {100 * r:9.1f}% {100 * c:11.1f}% {100 * m:11.1f}%")
print(f"{'data':>5} {100 * valid(batch(1024)).float().mean():9.1f}%")
```

The result, on the toy grammar of Chapter 5 (percentage of 1024 samples satisfying the global constraints; all three columns use the same number of network evaluations):

Table 8.2 Decoding strategies at matched NFE on the trained toy model. Revision helps far more than ordering here; at high NFE remasking costs budget it cannot recover.

NFE	random order	confidence order	confidence + ReMDM ($\sigma = 0.3$)
2	0.0%	0.0%	0.0%
3	0.0%	0.1%	**2.3%**
4	1.5%	0.8%	**23.6%**
6	16.2%	17.7%	**66.0%**
12	97.8%	**99.3%**	95.9%
ground-truth data		100.0%	

Three things in Table 8.2 are worth dwelling on.

Remasking dominates at low step counts. At four network evaluations, adding a conservative remasking schedule takes validity from 1.5% to 23.6% — a factor of fifteen — and at six evaluations from 16.2% to 66.0%. Nothing about the model changed. This is the corrector effect of Proposition 4.3 in action: spending part of the budget on repairing an inconsistent partial sample beats spending all of it on advancing time.

Remasking has a cost at high step counts. At twelve evaluations — where the sampler is committing one token per step, i.e. effectively autoregressive — remasking *hurts* slightly (95.9% against 97.8%). It undoes work that was already correct. So σ_t should be annealed to zero, which is exactly what ReMDM's cap and rescale schedules do.

Confidence ordering barely helps here, which is informative. The gains are within noise except at NFE $= 12$. Contrast Kim et al. (2025)'s Sudoku result ($< 7\% \to \approx 90\%$). The difference is structural: in Sudoku, conditional determinism is *heterogeneous* — at any point some cells are forced and others are wide open — so confidence carries a strong signal about which positions are safe to commit. In this toy grammar every unrevealed position is about equally uncertain until enough context accrues, so there is nothing for confidence to discriminate. The lesson generalises: **confidence-ordered decoding helps in proportion to how unevenly conditional entropy is distributed across positions**, and on tasks where it is uniform you should not expect much. This is a useful thing to check before attributing a gain to the ordering.

Key takeaways

- The exact reverse process moves one token per jump, so every parallel sampler is approximating something the exact process never does. The resulting error survives a perfect denoiser (Remark 7.5).
- Four composable levers: better integrator, better choice of which positions to commit, revision, and explicitly modelling dependence (Table 8.1).
- Confidence/margin/entropy ordering can be a very large free win — Sudoku accuracy from $< 7\%$ to $\approx 90\%$ with no retraining (Kim et al., 2025) — but only when conditional entropy is unevenly distributed across positions (Table 8.2), and it exploits model priors and puts the model off its training distribution.
- On our toy task *revision* mattered far more than ordering: remasking took validity from 1.5% to 23.6% at four network evaluations, while costing a little at twelve. Anneal σ_t to zero.

- ReMDM (Theorem 8.7) characterises a whole family of marginal-preserving samplers that *can* revise, applicable to already-trained MDLM checkpoints, with $\sigma_t \leq \min\{1, (1-\alpha_s)/\alpha_t\}$.
- Caching is exact across non-unmasking steps for absorbing noise (Equation (6.6)) and approximate otherwise; its distributional cost is rarely measured.
- Consistency distillation exists for uniform-state and simplex-flow models and not for masked diffusion. This is the field's most consequential gap.

Open questions and research directions

1. **A principled criterion for how many tokens to commit.** All current rules are heuristic thresholds on confidence. The right quantity is the *total correlation* (multi-information) among the candidate positions given the current state: commit a set iff its total correlation is below a tolerance. This is not directly computable, but it is estimable — e.g. by comparing the model's factorised prediction against a few autoregressive continuations, or with a small learned estimator. A sampler with an explicit error budget in these terms would be a significant and, I think, achievable advance. Lavenant and Zanella (2025) supply the theoretical half: they bound the factorisation error by (tokens committed per step -1) times a correlation measure $D(\pi)$, and solve for the optimal schedule (Section 13.6). What is missing is an *inference-time estimator* of the relevant correlation, which is an empirical problem rather than a theoretical one.
2. **Sampler-aware training.** The deepest problem in this chapter. Training uses i.i.d. masking; inference uses structured, confidence-chosen masking. Training on the masking distribution the sampler actually induces — by rolling out the sampler during training, or by importance-weighting masking patterns — closes a real train/test gap and nobody has done it properly.
3. **Correctness-preserving acceleration.** Speculative decoding has the beautiful property that it is *exact*: the output distribution is unchanged. Most discrete diffusion acceleration (caching, parallel commits, distillation) changes the distribution and does not measure by how much. Designing accelerations with exactness guarantees — rejection-sampling corrections to parallel updates, say — is wide open and would address Tang et al. (2026)'s critique head-on.
4. **Copulas, properly.** Liu et al. (2025) show that borrowing dependence structure from a small autoregressive model substantially improves few-step sampling. That is a striking result with a cheap implementation and it has not been followed up nearly as much as it deserves. Learning a lightweight dependence model *jointly* with the diffusion model, rather than bolting on an external one, is the obvious next step.
5. **Inference-time scaling with verifiers.** Language models gained a great deal from best-of-n, beam search against a verifier, and tree search. Discrete diffusion's partial states are natural search nodes — a partially unmasked sequence is a much better search state than a prefix, because it constrains the whole output — and this is barely exploited. Test-time-scaling approaches for masked diffusion are just beginning to appear (Misaki and Akiba, 2026).
6. **Report sampler ablations as standard.** A methodological point that is also a research direction: since inference choices can swing results by an order of magnitude more

than architectural ones, papers should report a quality-versus-NFE curve under a *fixed* sampler and separately the gain from their sampler. Almost none do, which makes the literature hard to compare.

CHAPTER 9

Guidance, Control and Reinforcement Learning

CHAPTER

Authors	Hunter Nisonoff, Junhao Xiong, Stephan Allenspach and Jennifer Listgarten; Yair Schiff et al.; Oussama Zekri and Nicolas Boullé; Siyan Zhao et al.
Year	2025
Published	ICLR 2025 / NeurIPS 2025
Link	https://arxiv.org/abs/2406.01572
Tags	classifier guidance, classifier-free guidance, discrete guidance, policy gradient, GRPO, RLHF

Unconditional generation is rarely the goal. You want a molecule with a particular binding affinity, a DNA sequence active in a particular cell type, a protein with a particular stability, a text continuation that is helpful and harmless. This chapter covers the three mechanisms: guidance with a predictor, guidance without one, and reinforcement learning.

The good news, flagged in Table 2.1: discrete guidance is *exact*, where its continuous counterpart is an approximation. This is one of the genuine structural advantages of discrete state spaces and it is still underexploited.

9.1. Exact classifier guidance for CTMCs

Suppose we want to sample from $p_0(\boldsymbol{x} \mid c)$ for some condition c, given an unconditional model and a predictor $p(c \mid \boldsymbol{x}, t)$.

Theorem 9.1 (Exact guided rates, Nisonoff et al., 2025). The reverse process targeting the conditional distribution has rates

$$R_t(\boldsymbol{x}, \tilde{\boldsymbol{x}} \mid c) = \frac{p(c \mid \tilde{\boldsymbol{x}}, t)}{p(c \mid \boldsymbol{x}, t)} R_t(\boldsymbol{x}, \tilde{\boldsymbol{x}}), \qquad \tilde{\boldsymbol{x}} \neq \boldsymbol{x}, \tag{9.1}$$

with the diagonal fixed by conservation, $R_t(\boldsymbol{x}, \boldsymbol{x} \mid c) = -\sum_{\boldsymbol{x}' \neq \boldsymbol{x}} R_t(\boldsymbol{x}, \boldsymbol{x}' \mid c)$. With a guidance strength γ,

$$R_t^{(\gamma)}(\boldsymbol{x}, \tilde{\boldsymbol{x}} \mid c) = \left[\frac{p(c \mid \tilde{\boldsymbol{x}}, t)}{p(c \mid \boldsymbol{x}, t)}\right]^{\gamma} R_t(\boldsymbol{x}, \tilde{\boldsymbol{x}}). \tag{9.2}$$

Why this is exact. Apply Theorem 2.12 to the conditional process. Its marginals are $p_t(\boldsymbol{x} \mid c) \propto p_t(\boldsymbol{x}) p(c \mid \boldsymbol{x}, t)$, so the ratio appearing in the reverse rate factorises:

$$\frac{p_t(\tilde{\boldsymbol{x}} \mid c)}{p_t(\boldsymbol{x} \mid c)} = \frac{p_t(\tilde{\boldsymbol{x}})}{p_t(\boldsymbol{x})} \cdot \frac{p(c \mid \tilde{\boldsymbol{x}}, t)}{p(c \mid \boldsymbol{x}, t)},$$

and the normalising constant cancels. Substituting into Equation (2.17) gives Equation (9.1). ∎

Intuition: why discrete guidance is exact and continuous is not

In continuous space, classifier guidance adds $\gamma \nabla_{\boldsymbol{x}} \log p(c \mid \boldsymbol{x}_t)$ to the score. That is exact *only* at $\gamma = 1$ and only if the classifier is the true noisy posterior; the $\gamma > 1$ version used in practice targets a tilted distribution nobody can write down.

In discrete space the same computation is a finite difference, not a derivative: you simply evaluate the classifier at the candidate state $\tilde{\boldsymbol{x}}$ and at the current state $\boldsymbol{x}$ and take the ratio. No linearisation, no approximation. Equation (9.1) at $\gamma = 1$ is *the* conditional reverse process. ($\gamma \neq 1$ is a different matter, and a more delicate one than it is usually made to sound — Section 9.1.1 takes it apart.)

The price is cost: you need $p(c \mid \cdot)$ at every neighbouring state, i.e. $L(K-1)$ classifier evaluations per step instead of one gradient. That is what the next subsection fixes.

9.1.1. What $\gamma \neq 1$ actually targets

The usual way to describe $\gamma \neq 1$ is that you are "targeting a tilted distribution, by choice" — as though the guided sampler drew from $p_t(\boldsymbol{x})\, p(c \mid \boldsymbol{x})^{\gamma}$ and the only question were whether you wanted that distribution. That is too generous, and it is worth getting right, because $\gamma > 1$ is what everybody actually runs.

Proposition 9.2 (The guided process does not have the tilted marginals). Write $w_t(\boldsymbol{x}) := p(c \mid \boldsymbol{x}, t)$ and define the tilted family

$$\mu_t^{(\gamma)}(\boldsymbol{x}) \;:=\; \frac{p_t(\boldsymbol{x})\, w_t(\boldsymbol{x})^{\gamma}}{Z_t^{(\gamma)}}, \qquad Z_t^{(\gamma)} = \sum_{\boldsymbol{x}} p_t(\boldsymbol{x}) w_t(\boldsymbol{x})^{\gamma}. \tag{9.3}$$

Then $R_t^{(\gamma)}$ of Equation (9.2) has exactly the form $\big[\mu_t^{(\gamma)}(\tilde{\boldsymbol{x}})/\mu_t^{(\gamma)}(\boldsymbol{x})\big]\, \mathbf{Q}_t(\tilde{\boldsymbol{x}}, \boldsymbol{x})$ demanded by Theorem 2.12, and the process it generates has marginals $\mu_t^{(\gamma)}$ **if and only if** $\mu_t^{(\gamma)}$ solves the forward Kolmogorov equation,

$$\partial_t \mu_t^{(\gamma)} \;=\; \mathbf{Q}_t\, \mu_t^{(\gamma)}. \tag{9.4}$$

At $\gamma = 1$ this holds, because $\mu_t^{(1)} = p_t(\cdot \mid c)$ is the time-t marginal of the forward process started from $p_0(\cdot \mid c)$. For $\gamma \neq 1$ it fails in general, and the guided sampler therefore converges to something that is *not* $\mu_0^{(\gamma)}$.

Proof. The form claim is immediate: w_t^{γ} and p_t both appear as ratios, so $Z_t^{(\gamma)}$ cancels. For the "if and only if", recall from the proof of Theorem 2.12 that for *any* strictly positive μ_t the rate matrix $\widehat{\mathbf{R}}_t$ built from μ_t satisfies $\widehat{\mathbf{R}}_t\, \mu_t = -\mathbf{Q}_t\, \mu_t$ — that computation used only the zero-column-sum property of $\mathbf{Q}_t$ and the ratio form, never the fact that μ_t was a marginal. If ν_u denotes the law of the reverse process at reverse-time u, then $\frac{\mathrm{d}}{\mathrm{d}u}\nu_u = \widehat{\mathbf{R}}_{T-u}\nu_u$, so $\nu_u = \mu_{T-u}$ for all u requires $-\partial_t \mu_t = \widehat{\mathbf{R}}_t \mu_t = -\mathbf{Q}_t \mu_t$, which is Equation (9.4). Conversely Equation (9.4) plus the matching initial condition $\nu_0 = \mu_T$ gives $\nu_u = \mu_{T-u}$ by uniqueness of solutions of the linear ODE. ∎

So the error is not a choice of target — it is a genuine inconsistency: the guided rates are, at each instant, the correct reversal of a family that is not being transported by $\mathbf{Q}_t$. Listing C.3 measures it on a five-state chain with an exactly computed noisy classifier, integrating the guided reverse process from $\mu_T^{(\gamma)}$ and comparing the endpoint with $\mu_0^{(\gamma)}$:

	$\mathrm{TV}\left(\text{reverse endpoint},\ \mu_0^{(\gamma)}\right)$	
	2×10^4 steps	8×10^4 steps
$\gamma = 1$	1.8×10^{-5}	4×10^{-6}
$\gamma = 2$	0.01298	0.01298
$\gamma = 4$	0.01228	0.01228

The $\gamma = 1$ row shrinks with the step count, as discretisation error must. The $\gamma \neq 1$ rows do not move at all: the discrepancy survives the continuum limit, which is what makes it a bias rather than an integration artefact.

Remark 9.3 (Which way the bias goes, and why nobody minds). In the experiment above the guided endpoint is *more* concentrated than $\mu_0^{(\gamma)}$ — at $\gamma = 4$ it puts 0.9936 on the top state against the tilted family's 0.9813. That is the generic direction: the inconsistency compounds in the direction the guidance is already pushing, because the reverse rates keep re-tilting a distribution that has already been tilted. For practice this is mostly harmless, and occasionally the point: γ is tuned empirically against a downstream metric, nobody evaluates $\mu_0^{(\gamma)}$, and over-sharpening is often exactly what a practitioner wants from a guidance knob.

It does matter for two things. First, for *theory*: "exact guidance" should be read as exact at $\gamma = 1$ only, and papers that describe $\gamma > 1$ as sampling from $p_t(\boldsymbol{x})p(\boldsymbol{c} \mid \boldsymbol{x})^\gamma$ are describing an approximation. Second, for *evaluation*: if you report a conditional likelihood or a calibration number under a $\gamma > 1$ sampler, it is not the likelihood of the model you think you are measuring, and the discrepancy does not vanish with more sampling steps.

9.1.2. Taylor-approximated guidance

To avoid $L(K-1)$ classifier calls, Nisonoff et al. (2025) approximate the log-ratio by a first-order expansion in the one-hot representation:

$$\log \frac{p^\phi(c \mid \tilde{\boldsymbol{x}}, t)}{p^\phi(c \mid \boldsymbol{x}, t)} \;\approx\; (\tilde{\boldsymbol{x}} - \boldsymbol{x})^\top \nabla_{\boldsymbol{x}} \log p^\phi(c \mid \boldsymbol{x}, t), \tag{9.5}$$

which needs one forward and one backward pass of the predictor for all $L(K-1)$ candidates at once. Substituting into Equation (9.2) gives a guided rate of the form $R_t(\boldsymbol{x}, \tilde{\boldsymbol{x}}) \exp\!\big(\gamma(\tilde{\boldsymbol{x}} - \boldsymbol{x})^\top \nabla_{\boldsymbol{x}} \log p^\phi\big)$. This is Taylor-approximated guidance (TAG), and it is what you should actually implement unless K is tiny.

9.1.3. Predictor-free guidance

The discrete analogue of classifier-free guidance, blending a conditional and an unconditional rate model:

$$R_t^{(\gamma)}(\boldsymbol{x}, \tilde{\boldsymbol{x}} \mid c) = R_t(\boldsymbol{x}, \tilde{\boldsymbol{x}} \mid c)^\gamma \, R_t(\boldsymbol{x}, \tilde{\boldsymbol{x}})^{1-\gamma}. \tag{9.6}$$

Nisonoff et al. (2025) demonstrate the method on class-conditional image generation, SMILES generation conditioned on ring count or lipophilicity (LogP), cell-type-conditioned DNA enhancer design, and stability-guided protein inverse folding conditioned on predicted $\Delta\Delta G$ — a good illustration of how broadly this one mechanism applies.

9.2. Guidance in the $\boldsymbol{x}_0$-parameterisation

If your model predicts clean data rather than rates (which it probably does), Schiff et al. (2025) give the practical forms.

D-CFG (discrete classifier-free guidance). Combine the conditional and unconditional denoiser outputs per token, as a tempered product with guidance strength γ. Train both by randomly dropping the conditioner during training, exactly as in continuous CFG.

D-CBG (discrete classifier-based guidance). Use a separately trained classifier to tilt the denoiser's per-token distribution. The important practical point is that the classifier operates on the model's *clean-data prediction* rather than on the noisy state, so an off-the-shelf classifier trained on clean data can be used — no noise-aware retraining. This is a significant convenience advantage over continuous diffusion.

Hybrid shortcut. As noted in Section 5.8, CANDI (Pynadath et al., 2025) maintains a continuous channel specifically so that guidance becomes "simple gradient addition" with off-the-shelf classifiers. If guidance is your main use case, that is a strong argument for a hybrid model.

Training-free guidance. For molecular generation, Kerby and Moon (2024) explore guidance that requires no additional training at all, using the property predictor only at inference. Worth knowing if labelled data for the property is scarce.

9.3. Reinforcement learning and reward fine-tuning

Guidance tilts the sampling distribution at inference. Reinforcement learning changes the model. For discrete diffusion this is harder than for autoregressive models for one specific reason: **there is no tractable sequence log-probability**. Policy-gradient methods need $\log \pi_\theta(\boldsymbol{y} \mid \boldsymbol{x})$, and a diffusion model gives only a bound on it.

9.3.1. Policy gradients on the CTMC

Zekri and Boullé (2025) attack this directly in the CTMC formulation. Their method, SEPO, derives a discrete REINFORCE-style estimator in which the gradient is a weighted sum over states and their Hamming-distance-one neighbours, with the neighbour probabilities handled by self-normalised importance sampling. The key property, in their words, is that the gradient "does not involve the differentiability of R_t": **non-differentiable rewards are fine**. For molecular and biological design — where the reward is a docking score, a synthesis-accessibility heuristic or a wet-lab assay — this is essential. They add importance sampling from an old policy with PPO-style ratio clipping and advantage estimates.

Reported results: fine-tuning SEDD Medium Absorb with a GPT-2-based reward model trained on HH-RLHF beats the pretrained model under a GPT-3.5-Turbo pairwise judge; and on DNA enhancer design, SEPO leads on most metrics (e.g. 99.5% ATAC-Acc and 7.55 median predicted activity), though not on 3-mer correlation.

9.3.2. GRPO for diffusion language models

For large diffusion language models the practical approach is to adapt GRPO. Zhao et al. (2025) give the recipe in *d1*: masked SFT on curated reasoning traces (1,000 examples from s1K), then RL with *diffu-GRPO*.

The interesting part is the log-probability estimator. They make two approximations:

1. **Mean-field sequence factorisation**: the sequence log-probability is the sum of per-token log-probabilities.
2. **One-step per-token estimate with random prompt masking**: mask each *prompt* token with probability p_{mask} while the completion stays fully masked, and take a single forward pass as

the estimate of $\log \pi_\theta(o^k \mid q)$. This replaces LLaDA's many-pass Monte Carlo estimator with one pass.

Intuition: random prompt masking is a regulariser, not just a cheap estimator

Re-masking the prompt differently at each gradient update creates different "views" of the same rollout. That permits many more gradient updates per batch of online generations (μ up to 12 or 24 in their experiments), which is the expensive part of RL for a diffusion model. So an approximation introduced for tractability turns out to buy sample efficiency — a nice example of the kind of accident that drives applied progress.

Reported gains with LLaDA-8B-Instruct (Zhao et al., 2025):

Table 9.1 d1 results, best generation length per task, after Zhao et al. (2025).

Task	LLaDA-8B-Instruct	d1-LLaDA
GSM8K	78.2	82.1
MATH500	36.2	40.2
Countdown	20.7	42.2
Sudoku	11.7	22.1

Note the pattern: the planning tasks roughly double while the maths tasks improve modestly. The authors attribute the smaller maths gains partly to the base model already being strong there. It is also consistent with the theme of Chapter 1: diffusion's comparative advantage is on globally constrained problems.

9.4. Preference optimisation

Policy gradients need a reward model. Direct preference optimisation (DPO) (Rafailov et al., 2023) removes it, by observing that the KL-regularised optimal policy lets you write the reward as a log-ratio of policy to reference, so pairwise preferences can be fitted directly:

$$\mathcal{L}_{\text{DPO}} = -\,\mathbb{E}_{(\boldsymbol{y}^w,\boldsymbol{y}^l)} \log \sigma\!\left(\beta \log \frac{\pi_\theta(\boldsymbol{y}^w)}{\pi_{\text{ref}}(\boldsymbol{y}^w)} - \beta \log \frac{\pi_\theta(\boldsymbol{y}^l)}{\pi_{\text{ref}}(\boldsymbol{y}^l)}\right). \tag{9.7}$$

This is now the default alignment recipe for autoregressive language models. Porting it to discrete diffusion runs straight into the obstacle of Section 9.3: Equation (9.7) needs four exact sequence log-likelihoods and a diffusion model has none. Three papers resolve this differently, and the differences are instructive.

9.4.1. Substituting the ELBO: VRPO

The direct route is to replace each $\log \pi$ in Equation (9.7) by its ELBO. Zhu et al. (2025) do exactly this, and then take seriously what it costs.

Write $\hat{h}$ for the resulting estimator of the bracketed *preference score* — a linear combination of four ELBOs, each itself estimated by sampling a diffusion time and then a masking pattern. Linearity makes $\hat{h}$ unbiased for the true score h. But $\log \sigma$ is *not* linear, so the loss is biased, and both the bias and the variance of the gradient inherit from $\operatorname{Var}(\hat{h})$. Their Theorem 1 makes this precise by bounding both in terms of that one quantity.

Intuition: alignment becomes a variance-reduction problem

This is the whole contribution in one sentence. Once you accept the ELBO substitution, you are no longer designing an alignment algorithm — you are designing an estimator, and every nat of estimator variance shows up as bias in the thing you are optimising. That reframing is what makes the three fixes below obvious rather than ad hoc, and it is a good template for porting *any* likelihood-based method to diffusion.

VRPO's three fixes follow directly:

1. **Spend more samples.** $n = 8$ per ELBO by default.
2. **Allocate them to timesteps, not masks.** Given a budget n, use n distinct timesteps with one masking pattern each, rather than fewer timesteps sampled more thoroughly. Their Theorem 2 shows this minimises the ELBO estimator's variance — the variance across t dominates.
3. **Antithetic sampling.** Use the *same* timesteps and masks for the policy and the reference ELBO of a given response. Their Theorem 3 shows this reduces the variance of the difference whenever the two are positively correlated, which they are. It costs nothing.

Applied to LLaDA-8B-Instruct for one epoch over 350K preference pairs, this gives LLaDA 1.5: GSM8K $78.6 \to 83.3$, GPQA $33.3 \to 36.9$, HumanEval $49.4 \to 52.4$, IFEval $62.2 \to 66.2$, Arena-Hard $10.0 \to 14.3$. The authors note the gains on MTBench and Arena-Hard are limited.

9.4.2. Deriving DPO natively: D2-DPO

Borso et al. (2025) instead derive the objective inside the CTMC formulation rather than substituting into the autoregressive one. The route: start from the KL-regularised optimal policy; write the path likelihood over a time-discretised chain; replace the intractable reverse path measure by the *forward* process q; expand $\log$ of the transition probability for small Δt; take the continuous-time limit so the sum over steps becomes $t \sim U[0, 1]$; and finish with Jensen, since $-\log \sigma$ is convex.

What drops out for the masking process is pleasingly concrete: the loss is a log-ratio of the denoiser's reconstruction probabilities under the trained and reference models, evaluated *only at masked positions* and weighted by $1/(1-t)$ — the same weight as in Equation (5.7). It needs two forward passes per example.

Remark 9.4 (Read this one with its caveat). The derivation is clean, but the paper is a preliminary study: the only experiment is a toy task on binary strings (fine-tune a model to prefer odd integers), and uniform-noise processes are listed as future work. Treat it as a derivation to build on rather than a validated method. Note also that version 1 was titled *D3PO*, which collides with an unrelated continuous-diffusion paper; search both names.

9.4.3. Decomposing the trajectory: stepwise alignment

Han et al. (2026) avoid trajectory-level likelihoods altogether. They show that if the trajectory reward decomposes as a sum of stepwise rewards, then solving a *separate* alignment problem at each diffusion step — aligning the clean-data posterior $\hat{p}_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_t)$ with a per-step regularisation strength — induces the optimal trajectory distribution. Each subproblem is then trained by distribution matching against a self-normalised reward-weighted target, from offline samples.

Three consequences worth noting. It is *offline*, so no on-policy sampling per step. It accepts arbitrary rewards and N samples per prompt, not just preference pairs — and specialises to a DPO-style log-sigmoid objective when you give it two completions and a Bradley–Terry model. And because everything is evaluated on $\boldsymbol{x}_0$, both likelihoods and rewards are computed on clean

data, which is where reward models actually work — the same observation that motivates target concrete score matching (Zhang et al., 2025a).

Results span both application areas of this book: DNA enhancer design (predicted activity $\mathbf{6.30}$ against $\mathbf{5.61}$ for DRAKES (Wang et al., 2025a), the strongest reward-based baseline), protein inverse folding ($\Delta\Delta G$ $\mathbf{1.400}$ against $\mathbf{1.095}$), and language (LLaDA-8B GSM8K $\mathbf{78.6} \rightarrow \mathbf{81.2}$, IFEval $\mathbf{52.9} \rightarrow \mathbf{55.1}$).

Table 9.2 Three routes from preferences to a fine-tuned discrete diffusion model.

Method	How it handles the intractable likelihood	Status
VRPO (Zhu et al., 2025)	substitute the ELBO, then bound and reduce the estimator variance	validated at 8B; the practical choice today
D2-DPO (Borso et al., 2025)	derive DPO inside the CTMC; continuous-time limit plus Jensen	clean derivation, toy experiments only
Stepwise alignment (Han et al., 2026)	never form a trajectory likelihood; align per-step $\boldsymbol{x}_0$ posteriors	strongest results; offline; handles general rewards

9.5. Other reward-based methods

- **DiffuCoder / coupled-GRPO.** Uses coupled (antithetic) sampling of masking patterns to reduce estimator variance for code generation.
- **TraceRL / TraDo.** Trajectory-aware RL, optimising over the denoising trajectory rather than the final output.
- **MolEditRL.** Discrete diffusion plus RL for structure-preserving molecular editing.
- **Target concrete score matching.** Zhang et al. (2025a) frame reward fine-tuning, preference learning and distillation from autoregressive teachers as instances of one objective, by working with the concrete score of the clean target distribution — where reward functions actually live. This is the most unified treatment currently available and is worth reading before starting an RL project here.

Listing 9.1 implements Equation (9.1) and Equation (9.2).

Listing 9.1 Exact classifier guidance for a CTMC (Theorem 9.1), and the Taylor approximation Equation (9.5).

```
"""Exact classifier guidance for a discrete diffusion model (Nisonoff et al.).

Exact guided rate, for xtilde != x:
    R_t(x, xtilde | c) = [ p(c | xtilde, t) / p(c | x, t) ]^gamma * R_t(x, xtilde)

The exact version needs the predictor at every Hamming-1 neighbour, i.e.
L*(K-1) evaluations.  The Taylor-approximated version (TAG) needs one forward
and one backward pass:
    log p(c|xtilde)/p(c|x)  ~=  (xtilde - x)^T grad_x log p(c | x, t)
"""
import numpy as np
import torch
import torch.nn.functional as F

```

```
torch.manual_seed(0)
L, K = 6, 5                                  # 6 positions, 5 token types


# --- a toy "predictor": p(c=1 | x) is high when the sequence sums to 0 mod 5 -
class Predictor(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.w = torch.nn.Parameter(torch.randn(L, K) * 0.5)

    def forward(self, x_onehot):              # x_onehot: (..., L, K)
        return torch.sigmoid((x_onehot * self.w).sum((-1, -2)))


pred = Predictor()


def exact_guided_rates(x, base_rates, gamma=1.0):
    """x: (L,) ints.  base_rates: (L, K) unguided rates to each neighbour.

    Returns (L, K) guided rates.  Cost: L*(K-1) predictor evaluations.
    """
    x1 = F.one_hot(x, K).float()
    logp_x = torch.log(pred(x1) + 1e-12)
    out = torch.zeros(L, K)
    for ell in range(L):
        for v in range(K):
            if v == x[ell]:
                continue
            y = x.clone(); y[ell] = v
            logp_y = torch.log(pred(F.one_hot(y, K).float()) + 1e-12)
            out[ell, v] = base_rates[ell, v] * torch.exp(
                gamma * (logp_y - logp_x))
    return out


def tag_guided_rates(x, base_rates, gamma=1.0):
    """Taylor-approximated guidance: 1 forward + 1 backward pass."""
    x1 = F.one_hot(x, K).float().requires_grad_(True)
    logp = torch.log(pred(x1) + 1e-12)
    g, = torch.autograd.grad(logp, x1)                    # (L, K)
    # (xtilde - x)^T grad  =  g[ell, v] - g[ell, x_ell]
    delta = g - g.gather(-1, x[:, None])
    out = base_rates * torch.exp(gamma * delta)
    out.scatter_(1, x[:, None], 0.0)                      # no self-transition
    return out


x = torch.randint(0, K, (L,))
base = torch.ones(L, K)                                   # uniform base rates
base.scatter_(1, x[:, None], 0.0)

for gamma in (0.0, 1.0, 2.0):
    e = exact_guided_rates(x, base, gamma)
    a = tag_guided_rates(x, base, gamma)
    rel = ((a - e).abs().sum() / e.abs().sum()).item()
    print(f"gamma={gamma}:  total exact rate {e.sum():7.3f}   "
          f"TAG rate {a.sum():7.3f}   relative discrepancy {rel:6.2%}")

print("\nAt gamma=0 both reduce to the unguided rates:",
```

```
        torch.allclose(exact_guided_rates(x, base, 0.0), base, atol=1e-6))
print("Guidance pushes rate towards neighbours the predictor prefers; the")
print("sum of rates is NOT preserved -- the diagonal absorbs the difference.")
```

Its output quantifies the cost of the approximation:

```
gamma=0.0:   total exact rate 24.000   TAG rate 24.000   relative discrepancy  0.00%
gamma=1.0:   total exact rate 21.660   TAG rate 22.514   relative discrepancy  3.94%
gamma=2.0:   total exact rate 20.159   TAG rate 21.484   relative discrepancy  6.58%
```

The discrepancy grows with γ, as a first-order expansion must: the larger the tilt you ask for, the further the linearisation is being extrapolated. For small K you can and should skip TAG entirely — see the first open question below.

Key takeaways

- Discrete classifier guidance is **exact** (Theorem 9.1): reweight the rate of each jump by the likelihood ratio of the condition at the destination versus the current state. No linearisation, and no noise-aware classifier needed in the $\boldsymbol{x}_0$-parameterisation.
- The cost is $L(K-1)$ predictor evaluations per step; Taylor-approximated guidance Equation (9.5) reduces this to one forward and one backward pass.
- Predictor-free guidance Equation (9.6) and D-CFG are the classifier-free analogues and work as expected.
- Preference optimisation faces the same obstacle: Equation (9.7) needs four exact log-likelihoods. Substitute the ELBO and reduce the resulting estimator variance (VRPO), derive DPO natively in the CTMC (D2-DPO), or never form a trajectory likelihood at all and align the per-step $\boldsymbol{x}_0$ posteriors (stepwise alignment). Table 9.2 compares the three.
- RL for discrete diffusion is hard because the sequence log-probability is intractable. SEPO solves this in the CTMC formulation and crucially does *not* require differentiable rewards; diffu-GRPO solves it for large masked models with a mean-field, one-step estimator plus random prompt masking.
- Variance of the log-probability estimator is the binding constraint on RL here, which is why several methods (VRPO, coupled-GRPO) are explicitly variance-reduction methods.
- The reported RL gains are largest on planning and constraint-satisfaction tasks (Countdown, Sudoku roughly double), consistent with where diffusion has a structural advantage.

Open questions and research directions

1. **Exact guidance is affordable for small K and almost nobody uses it.** For DNA ($K=4$), amino acids ($K=20$) or SELFIES alphabets, $L(K-1)$ classifier evaluations per step is entirely feasible, so Equation (9.1) can be used *exactly* rather than via TAG. Since biological design is precisely where guidance matters most, this is a free improvement that the literature has largely skipped in favour of the approximation developed for large K.
2. **Guidance under parallel sampling.** Theorem 9.1 is a statement about rates, i.e. about single-token jumps. What the correct guided update is when you commit several tokens at once — the regime everybody actually samples in — is not established, and the

obvious thing (guide each position's marginal independently) inherits all of Remark 2.4's problems and then some, because the condition c typically couples positions strongly. This seems to me a clear and important gap.

3. **Credit assignment over the denoising trajectory.** In autoregressive RL, credit assignment follows the token order. In diffusion there is no canonical order, and a reward realised at $t = 0$ must be attributed across a trajectory in which tokens were committed in an order the sampler chose. TraceRL begins on this; the general problem — which is really a question about the right MDP formulation — is open.
4. **RL on the decoding policy itself.** Given Kim et al. (2025)'s result that the *order* is worth more than $7\times$ the parameters, the obvious move is to train the unmasking policy by RL with the task reward, keeping the denoiser frozen. This is a small, well-defined action space (which positions to commit), a clear reward, and a strong prior that it will work. I am not aware of a paper doing it properly.
5. **Reward hacking through the sampler.** Because quality depends so strongly on inference settings (Chapter 8), an RL procedure that optimises a reward at a fixed sampler configuration may be exploiting that configuration rather than improving the model. Nobody reports RL results across a sweep of samplers. They should.
6. **Guidance for multimodal flows.** For joint discrete+continuous generation (Chapter 7 and Section 12.7), guidance must act coherently on both channels — a property predictor that depends on both sequence and structure cannot be decomposed. The correct joint guided process is not, as far as I can tell, written down anywhere.
7. **The rest of the DPO zoo, and IPO in particular.** The variants that followed DPO — IPO, KTO, ORPO, SimPO — have been ported extensively to *continuous* image diffusion, and essentially not at all to the discrete case: Table 9.2 is the whole literature. IPO looks especially worth doing, for a reason specific to the ELBO substitution. IPO replaces $\log \sigma$ with a squared loss, and if $\hat{h}$ is an unbiased estimator of the log-ratio difference then

$$\mathbb{E}\big[(\hat{h} - c)^2\big] = (h - c)^2 + \mathrm{Var}(\hat{h}), \tag{9.8}$$

so the bias introduced by estimating the likelihood is *exactly* the estimator variance — additive, with no unknown constant. Compare VRPO, where the bias depends on the curvature of $\log \sigma$ at a point you do not know, and Theorem 1 of Zhu et al. (2025) has to bound it. Under IPO that analysis is a line of algebra.

Whether Equation (9.8) is a feature or a hazard is the open question. It can be read as free variance regularisation — the objective automatically prefers parameter settings whose ELBO estimator is well behaved — or as a failure mode, since the optimiser may reduce the penalty by collapsing towards degenerate low-variance predictions rather than by fitting preferences. The experiment is cheap: it is VRPO's codebase with one loss swapped, and the diagnostic is whether $\mathrm{Var}(\hat{h})$ falls faster than preference accuracy rises. *(This observation is mine, not a claim from any published paper; treat it as a hypothesis to test.)*

CHAPTER 10

Building and Measuring

CHAPTER

Authors	Shen Nie et al.; Jiacheng Ye et al.; Antonio Franca and Alexander Tong; Patrick Pynadath et al.
Year	2026
Published	ICLR 2025 / NeurIPS 2025 / ICML 2026 workshops
Tags	architectures, AR adaptation, scaling laws, variance, generative perplexity, evaluation

This chapter is the engineering: what network, what conditioning, how to initialise, how it scales, and what breaks.

10.1. The backbone

There is no architectural novelty specific to discrete diffusion. The denoiser is a bidirectional transformer that maps a corrupted sequence to per-position logits over the vocabulary. The differences from a standard decoder-only language model are three:

1. **No causal mask.** Full bidirectional attention: a masked position must attend to tokens on both sides. This is the whole point.
2. **One extra vocabulary entry** for `[MASK]` on the input side, and (with SUBS, Chapter 5) *no* `[MASK]` entry on the output side.
3. **Time conditioning**, usually via a sinusoidal or Fourier embedding of t added to the token embeddings or fed through adaptive layer norm (adaLN), as in diffusion transformers for images.

Everything else — rotary position embeddings, RMSNorm, SwiGLU, flash attention — carries over unchanged.

Remark 10.1 (Time conditioning may be unnecessary). For absorbing noise, Ou et al. (2025) show the model need not be conditioned on t at all (Equation (6.6)), because the state reveals the noise level. Dropping it is free and enables exact caching. For uniform or interpolating noise, keep it.

10.2. Initialising from an autoregressive model

The single most practically important trick in the field. Pretraining a diffusion language model from scratch costs as much as pretraining an autoregressive one; initialising from an existing checkpoint costs a fraction.

Dream-7B. Ye et al. (2025) use the Qwen2.5-7B configuration and start from its pretrained weights. Two details matter:

- **Keep the AR shift.** They retain the autoregressive "shift" operation, so the hidden state at position i predicts the token at $i+1$ during diffusion training. This preserves the geometry the pretrained weights already encode, instead of asking the model to relearn a position-aligned readout.
- **Context-adaptive token-level noise rescheduling (CART).** Instead of one timestep for the whole sequence, each masked token gets a noise level reflecting how much clean context actually informs it, weighted by a mixture of geometric distributions. This is the content-dependent-schedule idea of Chapter 3 realised at scale.

Dream-7B used about 580B tokens from Dolma v1.7, OpenCoder and DCLM-Baseline, with SFT on 1.8M instruction–response pairs from Tulu 3 and SmolLM 2.

DiffuLLaMA / DiffuGPT. Adapting existing autoregressive models (GPT-2, LLaMA) into diffusion models by continued training, with an annealing of the attention mask from causal to bidirectional.

> **Intuition: why AR initialisation works at all**
>
> An autoregressive model has already learned the hard part: the representation of language. What it has not learned is to attend rightwards and to predict from arbitrary conditioning sets. Those are comparatively cheap to acquire — they are a change of interface, not of knowledge. Given Theorem 6.1, you can say this precisely: the AR model has learned one of the $L!$ orderings, and adaptation teaches it the rest, which is a refinement of existing representations rather than new knowledge.

10.3. Scaling

SMDM. Nie et al. (2025a) performed the first careful scaling-law study of masked diffusion models on text. The findings that matter:

- Masked diffusion models follow power-law scaling comparable to autoregressive models, with a worse constant — roughly, a masked diffusion model needs more compute to reach a given loss, but the *exponent* is similar.
- They exhibit genuinely different capability profiles: better at reversal-style tasks (mitigating the reversal curse) and at bidirectional reasoning.

LLaDA. Nie et al. (2025b) scaled a masked diffusion model to 8B parameters and 2.3T training tokens, and showed it competitive with LLaMA3-8B on standard benchmarks — the demonstration that the approach does not fall over at scale.

The 2026 cross-family study. Sahoo et al. (2026) is the most careful comparative scaling work to date and its conclusions are worth stating carefully, because they cut against received wisdom:

1. Masked diffusion is about 12% more FLOPs-efficient when trained with a simple cross-entropy objective rather than the ELBO weighting.
2. Perplexity is a reliable guide *within* a diffusion family but can mislead *across* families.
3. A model with worse likelihood scaling may be preferable because it samples faster — the relevant object is the speed/quality Pareto frontier, not a single number.
4. At 1.7B, uniform-state diffusion is competitive on likelihood benchmarks and *beats* autoregressive and masked diffusion models on GSM8K despite worse validation perplexity.

10.4. Block diffusion: interpolating with autoregression

Arriola et al. (2025) (BD3-LM) make the AR–diffusion trade-off a tunable parameter: autoregressive *across* blocks of B tokens, diffusion *within* each block. $B = 1$ is autoregressive; $B = L$ is masked diffusion. The gains are not merely conceptual:

- **KV caching becomes valid** across blocks, because earlier blocks are fixed. This removes the main per-step cost disadvantage against autoregressive models.
- **Arbitrary-length generation**, because you can keep emitting blocks. Pure masked diffusion needs the length fixed in advance (Chapter 5).
- **Better likelihoods** than full diffusion, as the model fixes part of the factorisation order and so needs fewer conditionals.

They also identify and address a gradient-variance problem in the block objective, which is the recurring practical theme of this chapter.

Esoteric LMs (Eso-LMs) take a related route, fusing AR and masked diffusion to support KV caching within a diffusion framework.

10.5. Variance: the recurring practical problem

The continuous-time objective Equation (5.7) is a time-integral estimated by sampling t, and its integrand carries a $1/(1-\alpha_t)$ weight. The resulting estimator has heavy tails. We saw this concretely in Listing 6.1: a 2×10^5-sample Monte Carlo estimate of a quantity whose exact value is 11.7339 returns 11.5305.

The countermeasures, roughly in order of how standard they are:

1. **Rao–Blackwellisation** (Section 5.4): integrate out everything that can be integrated out. MDLM's main practical advantage over its predecessors.
2. **Low-discrepancy sampling of t**: stratified or antithetic t within a batch rather than i.i.d. uniform.
3. **Coupled / antithetic masking**: for a given t, use complementary masking patterns across a pair of examples. Used by LaViDa (as "complementary masking") and by DiffuCoder's coupled-GRPO.
4. **Importance sampling in t**, including CDCD's *time warping* (Section 7.6), which adapts the t-distribution during training. Underused in masked diffusion.
5. **Reweighting the loss**: give up the exact bound. This is what D3PM's auxiliary loss, DFM's cross-entropy and Sahoo et al. (2026)'s 12% FLOPs win all do.

Explicit variance-focused work has begun to appear (Jia et al., 2025; Shi and Titsias, 2025).

10.6. Practical pitfalls

A checklist of things that commonly go wrong, drawn from the surveys (Yu et al., 2026; Yuan et al., 2026) and from the papers' own appendices.

- **Corpus utilisation.** Each training example contributes gradient signal only at its masked positions, so at a mask rate of t you are using a t fraction of the tokens per pass. Averaged over $t \sim U(0,1)$ this is half the tokens of an autoregressive step, at the same compute. This is a real and often-unstated factor in the efficiency comparison.
- **Time-index sampling.** Sampling one t per *sequence* rather than per *batch* matters; so does whether t is shared within a batch. Get this wrong and the gradient variance jumps.

- **Length bias.** Padding to a maximum length and relying on end-of-sequence tokens biases the length distribution and wastes compute. There is no clean fix (Chapter 5).
- **The ε cutoff.** The $1/t$ weight requires a cutoff $t \geq \varepsilon$. The value of ε silently changes the reported bound. Report it.
- **Evaluation leakage through inference settings.** Reported quality depends more on sampler configuration than on most architectural choices (Chapter 8). Comparisons that do not fix the sampler are not comparisons.
- **SUBS not implemented.** If your model can predict `[MASK]`, or re-predicts unmasked tokens, your loss is not the one in Theorem 5.4 and your bound is invalid. Assert both conditions in your test suite.

10.7. The open-model landscape

For orientation, the publicly documented diffusion language models as of late 2026. Treat parameter counts and claims as reported by the authors.

Table 10.1 Publicly documented diffusion language models. All are masked (absorbing-state) unless noted.

Model	Scale	Notes
LLaDA (Nie et al., 2025b)	8B	from scratch, 2.3T tokens; the reference open model
Dream-7B (Ye et al., 2025)	7B	initialised from Qwen2.5-7B, 580B tokens; CART schedule
LLaDA 1.5	8B	VRPO preference optimisation
BD3-LM (Arriola et al., 2025)	≤1B	block diffusion; KV caching, arbitrary length
Eso-LM	≤1B	AR/diffusion fusion with KV caching
Duo (Sahoo et al., 2025)	≤1B	*uniform-state*; 8-step sampling via distillation
UDLM (Schiff et al., 2025)	small	*uniform-state*; SOTA on molecules/DNA
LLaDA-MoE	7B-A1B	mixture-of-experts diffusion LM
Mercury	undisclosed	commercial (Inception Labs); latency-focused
Gemini Diffusion	undisclosed	commercial (Google DeepMind)
Seed Diffusion	undisclosed	commercial (ByteDance); code-focused

10.8. Evaluation: metrics that lie and metrics that don't

If you read one chapter of this book before writing a paper, read this one. The evaluation practices that the discrete diffusion literature inherited are known to be unsound, the demonstrations are recent and concrete, and a reviewer who knows them will reject a paper built on them.

10.8.1. Four metrics and what each actually measures

1. ELBO / test perplexity. The variational bound on the negative log-likelihood, exponentiated. Measures how well the model assigns probability to held-out data. **Valid use:** comparing models within the same family, with the same noise process, the same ε cutoff, and the same tokeniser. **Invalid use:** comparing across families. Pynadath et al. (2026) put the reason plainly: likelihood bounds from models with different latent structures are unreliable for model selection, because

the intractable KL gap differs across them. Sahoo et al. (2026) reach the same conclusion from the scaling side, and give the sharpest counterexample: at 1.7B, uniform-state diffusion has *worse* validation perplexity and *better* GSM8K accuracy than both masked diffusion and autoregressive baselines.

One exception worth noting. By Theorem 6.1, the masked diffusion ELBO is tight at the optimum, so for masked models a perplexity gap really is model error. That does not license comparing a masked model's bound to a uniform model's.

2. Generative perplexity (gen-PPL). Sample unconditionally from your model, then score the samples under a frozen reference language model (conventionally GPT-2 Large). Lower is supposedly better. This metric is **broken**, and the demonstration is worth going through in detail because it is the most useful single piece of methodological knowledge in the field.

3. MAUVE and other distributional metrics. Compare the *distribution* of generated text to the distribution of human text, rather than scoring samples individually.

4. Downstream task accuracy. GSM8K, HumanEval, MMLU, Sudoku, Countdown. The least gameable and the most expensive.

10.8.2. How generative perplexity is hacked

Franca and Tong (2026) make the argument unanswerable. Their observation is that, by construction, gen-PPL measures only how *predictable* a sample is under the scoring model — not whether it is coherent, diverse, or anything a person would want. So they build **zero-parameter samplers** that are predictable and nothing else.

On LM1B at $L = 128$, where human reference text has gen-PPL 56.9 and unigram entropy 4.33:

Table 10.2 Zero-parameter samplers versus trained models, after Franca and Tong (2026). Gen-PPL lower is "better"; MAUVE higher is better. Note the complete disagreement.

Sampler	gen-PPL ↓	entropy	MAUVE ↑
Human reference (LM1B)	56.9	4.33	—
Periodic ($k = 64$), zero parameters	**29.4**	4.16	**0.004**
Mirror ($k = 5000$), zero parameters	61.3	3.84	0.004
Phrase bank ($m = 1000$), zero parameters	78.1	—	0.004
Top-k i.i.d. ($k = 32$), zero parameters	75.0	2.99	0.004
MDLM (trained)	83.8	—	**0.749**
FMLM (trained, NFE$= 32$), best trained model	78.8	—	—

A sampler that emits a periodic repetition of a 64-token block — containing no model, no training and no information — achieves gen-PPL 29.4, *better than human text and better than every trained model in their table*, while its MAUVE is 0.004 against MDLM's 0.749. On OpenWebText at $L = 1024$, Periodic ($k = 400$) reaches gen-PPL 21.6, beating MDLM (48.3), LangFlow (42.3) and SEDD (125.7).

Intuition: gen-PPL rewards low entropy, and entropy is free

Generative perplexity is (an estimate of) the cross-entropy of your samples under the reference model. The KL divergence you actually care about is

$$D_{\mathrm{KL}}(p_\theta \,\|\, p_{\mathrm{ref}}) = \underbrace{\mathrm{CE}(p_\theta, p_{\mathrm{ref}})}_{\log \text{ gen-PPL}} - \underbrace{H(p_\theta)}_{\text{your model's entropy}}. \tag{10.1}$$

Gen-PPL reports only the *first* term. You can therefore drive it down indefinitely by reducing your own entropy — repeat yourself, lower the temperature, emit templates — without moving closer to the data distribution at all. Equation (10.1) is the whole critique in one line, and it is the organising identity of Pynadath et al. (2026).

Because that identity is the whole argument, it is worth stating it as a proposition and reading off exactly what it does and does not license.

Proposition 10.2 (Generative perplexity is a KL estimate missing a term). For any $p_\theta, p_{\mathrm{ref}}$ with $\operatorname{supp} p_\theta \subseteq \operatorname{supp} p_{\mathrm{ref}}$,

$$D_{\mathrm{KL}}(p_\theta \,\|\, p_{\mathrm{ref}}) \;=\; \mathrm{CE}(p_\theta, p_{\mathrm{ref}}) - H(p_\theta), \tag{10.2}$$

where $\mathrm{CE}(p, r) = -\mathbb{E}_p[\log r]$ and $H(p) = -\mathbb{E}_p[\log p]$. Consequently, for two models A and B,

$$D_{\mathrm{KL}}(p_A \,\|\, p_{\mathrm{ref}}) - D_{\mathrm{KL}}(p_B \,\|\, p_{\mathrm{ref}}) = \underbrace{\left(\mathrm{CE}_A - \mathrm{CE}_B\right)}_{\log \text{ gen-PPL difference}} - \underbrace{\left(H_A - H_B\right)}_{\text{entropy difference}}. \tag{10.3}$$

Proof. $D_{\mathrm{KL}}(p \,\|\, r) = \mathbb{E}_p[\log p - \log r] = -H(p) + \mathrm{CE}(p, r)$. Equation (10.3) is the difference of two copies of Equation (10.2). Checked numerically in Listing C.3. ∎

Corollary 10.3 (What gen-PPL can and cannot rank).

1. *(Matched entropy.)* If $H_A = H_B$ then $D_{\mathrm{KL}}(p_A \,\|\, p_{\mathrm{ref}}) < D_{\mathrm{KL}}(p_B \,\|\, p_{\mathrm{ref}})$ **iff** $\text{gen-PPL}_A < \text{gen-PPL}_B$. At matched entropy, gen-PPL is a faithful ranking.
2. *(Unmatched entropy.)* If $H_A < H_B$, then $\text{gen-PPL}_A < \text{gen-PPL}_B$ implies *nothing* about the KL ordering: the gap $\mathrm{CE}_B - \mathrm{CE}_A$ must exceed $H_B - H_A$ for A to be genuinely closer, and a lower-entropy model gets the first inequality for free.
3. *(Frontier dominance.)* If A's $(H, \log \text{gen-PPL})$ point lies strictly below B's curve — lower cross-entropy at the *same* entropy — then A has strictly smaller KL. This is why an entropy–perplexity frontier (Section 10.8.4) is the right object to plot and a single gen-PPL number is not.

Proof. All three are immediate from Equation (10.3): (i) sets $H_A - H_B = 0$; (ii) observes that the sign of the left-hand side is undetermined when both differences are negative; (iii) is (i) applied at each matched entropy level. ∎

Remark 10.4 (The identity is exact; the measurement is not). Equation (10.2) is an identity about distributions, but neither term is observed. $\mathrm{CE}(p_\theta, p_{\mathrm{ref}})$ is estimated from finitely many samples under a *proxy* reference model (typically GPT-2 or a Llama variant), so it inherits that proxy's own biases — including, awkwardly, a preference for text resembling its own training distribution. $H(p_\theta)$ is in practice estimated by *summing* the denoiser's per-position entropies, and $\sum_\ell H(X^\ell) - H(\boldsymbol{X}) = \mathrm{TC}(\boldsymbol{X}) \geq 0$, so that estimate overstates the true sequence entropy by exactly the total correlation the factorised readout ignores (Definition 8.1). Both error terms

push in the direction of flattering a low-diversity model, which is why the empirical hacks of Section 10.8.2 work as well as they do.

10.8.3. A numerical bug that produced years of results

Before the metric-level critiques, there is a lower-level one that every practitioner in this area should know, because it is a two-line fix with large consequences.

Zheng et al. (2025) found that masked diffusion models' reported generative perplexity was substantially inflated by **32-bit floating-point categorical sampling**. The mechanism: Gumbel-max sampling needs uniform values in $(0, 1)$, but in 32-bit floats the uniforms are capped below 1, so the Gumbel noise is truncated at about 16.6 instead of the 36.7 available in 64-bit. A truncated Gumbel amplifies larger probabilities — it acts like a hidden temperature reduction — and additionally makes unmasking uneven across positions. Both reduce diversity, which (by Equation (10.1)) lowers generative perplexity without improving the distribution.

Their ablation isolates it cleanly (8 samples, 2048 steps):

Table 10.3 The float32 Gumbel-truncation effect, after Zheng et al. (2025). Manually truncating the 64-bit sampler reproduces the 32-bit numbers, confirming the cause.

Sampler	Gen-PPL	Entropy
32-bit (as published)	31.24	5.17
64-bit (correct)	126.11	5.66
64-bit with manual truncation	28.64	5.12

With sampling corrected to 64-bit, they report that entropy returns to about 5.60–5.70 for all models, the true generative perplexity of masked diffusion models is around 100 against below 40 for autoregressive models, and the conclusion is that **masked diffusion models do not beat autoregressive models on text generation under fair evaluation**.

What to do about it

Check the dtype of your Gumbel noise or your `multinomial` call. If you are reproducing a published generative-perplexity number and getting something much worse, this is the first thing to check — and if you are reporting one, state the precision. More generally, this episode is the strongest possible argument for the entropy-paired reporting of the next section: the bug was *visible* in the entropy column (5.17 against 5.66) the whole time. Anybody reporting perplexity alongside entropy would have caught it immediately.

10.8.4. What to report instead

Entropy–perplexity frontiers

Pynadath et al. (2026) turn Equation (10.1) into a protocol. Since a single operating point is ambiguous, sweep the softmax temperature to trace a *frontier* of (entropy, gen-PPL) pairs for each model, and compare models at matched entropy or matched perplexity. Their key formal observation: **frontier dominance** — lower perplexity at *every* entropy — is a sufficient condition for lower KL divergence from the target. Their summary is worth quoting: "Single-point comparisons measure inference settings; generative frontiers measure model quality."

Their concrete recommendations:

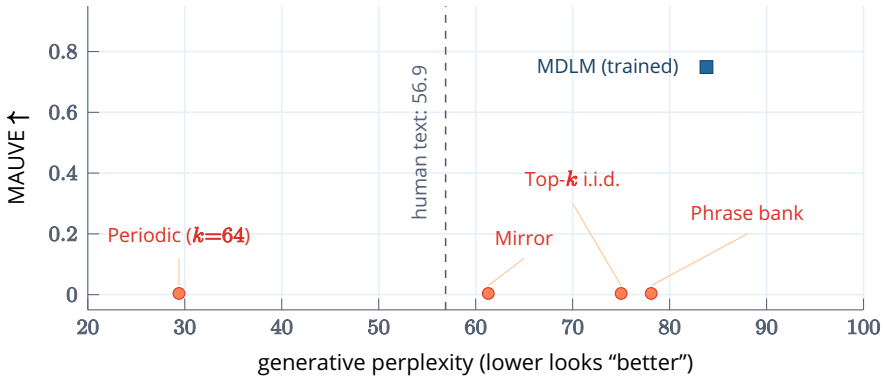


Figure 10.1 Generative perplexity is not a measure of quality. Every orange point is a **zero-parameter** sampler from Franca and Tong (2026) — no model, no training, just repetition or frequent tokens — evaluated on LM1B. "Periodic" emits a repeating 64-token block and scores a generative perplexity of **29.4**, *better than human text* and better than every trained model in their table, while its MAUVE is **0.004** against MDLM's **0.749**. The metric rewards predictability under a frozen scorer, which low-entropy degenerate text supplies in abundance: by Equation (10.1) it reports only the cross-entropy term and ignores your model's own entropy. Report an entropy–perplexity frontier and at least one distributional metric instead (Section 10.8).

- Pretrain on OpenWebText, not LM1B (whose sentence-level shuffling means it cannot assess quality beyond one sentence).
- Use GPT-2 Large as the reference scorer.
- Compare within the empirical OpenWebText unigram-entropy interquartile range, $[5.37, 5.55]$.
- Note that entropy differences of only about **0.3** — well within a reasonable range — can completely reverse method rankings.
- For early method iteration, 50k pretraining steps gave frontiers close to those at 1M steps for MDLM, Duo and CANDI, so you need not train to convergence to compare.

That last point is a genuine gift to anyone on an academic compute budget.

Distributional metrics

Franca and Tong (2026) propose four, in increasing order of cheapness:

- **MAUVE**: compares embedding distributions from GPT-2 Large hidden states. The standard choice, and it correctly assigns **0.004** to all four of their degenerate samplers.
- **Gradient Moment (GM)**: squared distance between the expected log-likelihood gradients of a fixed LM under generated and human text. Periodic scores **435.0** against MDLM's **29.7**.
- **Energy distance (D_E)**: on hand-crafted entity, discourse and surface statistics, with no learned LM at all. Periodic scores **128.144** against MDLM's **0.073**.
- **Surprisal-profile distance (D_{SP})**: compares the mean, variance, skewness and kurtosis of per-token surprisals. It reuses the gen-PPL forward pass, so it costs almost nothing extra — there is no excuse for not reporting it.

10.8.5. Is your sampler correct?

A separate and deeper problem, raised by Tang et al. (2026). Their method is to remove the model from the equation entirely: take a known bigram Markov chain, compute the *exact* HMM posterior by forward–backward smoothing, and use it as an oracle denoiser. Any remaining error is the sampler's.

Their findings:

- Few-step discrete diffusion samplers are "not distributionally correct even under an exact oracle denoiser". Transition-level mismatch persists and shrinks only as the number of steps approaches the sequence length — at which point generation has reduced to autoregressive sampling.
- The cause is the independence approximation in parallel updates: matching marginals implies matching the joint only under strong independence, which natural language violates.
- Sharpening SEDD's score (a temperature-like factor $\beta > 1$) raises transition KL and lowers n-gram diversity — i.e. a trick that improves gen-PPL makes the distribution worse.
- LLaDA's confidence-based remasking appears to rely on *denoiser priors*: with an oracle that lacks them, probability mass concentrates on a few high-frequency transitions as the step count rises.
- NLL and gen-PPL can *improve* under biased sampling, and MAUVE can stay insensitive to transition-level errors. So they recommend transition-level metrics: transition KL, NLL, TV, entropy rate against the oracle kernel, plus support fraction.

We reproduced the core of this in miniature in Remark 7.5: an exact enumerated posterior, and 72% of samples off-support at two steps.

Intuition: why an oracle study is the right experimental design

Any sampler comparison with a learned denoiser confounds two things: whether the sampler integrates the reverse process accurately, and whether it happens to exploit the particular biases of that denoiser. Replacing the denoiser with an exact oracle removes the second, which is the only way to attribute the error. If your problem admits a small exactly-solvable instance — and almost every one does — build that instance and run your sampler on it. It is the cheapest high-value experiment in this field and almost nobody does it.

10.8.6. Non-determinism and reproducibility

Diffusion samplers are stochastic in a way autoregressive greedy decoding is not: the masking pattern, the decoding order and the sampled tokens are all random. Fang et al. (2026) study this and find that dataset-level metrics attenuate non-determinism that is severe at the level of individual generations — so an aggregate number can look stable while individual outputs vary wildly. For any application where single outputs matter (code generation, molecule proposals), report per-sample variance, not just the dataset mean.

10.8.7. Domain-specific metrics

For molecules and biological sequences the metrics are different and, on the whole, better behaved; see Chapter 12 and Section 12.7. The general principle carries over, though: a metric that scores samples individually under a frozen model (a property predictor, a docking score) can be hacked by a generator that finds that model's blind spots, which in the molecular literature is well known as the failure mode of naive property optimisation. Report distributional similarity to the training set and chemical validity alongside any property score.

10.8.8. A reporting checklist

Minimum reporting standard for a discrete diffusion paper

1. **Likelihood:** the ELBO, with the ε cutoff, the number of discretisation steps used to evaluate it, and the tokeniser, all stated. Compare only within-family.
2. **Generation quality:** an entropy–perplexity frontier, not a single gen-PPL (Pynadath et al., 2026). Plus at least one distributional metric (MAUVE, or D_{SP}, which is free).
3. **Sampler:** a quality-versus-NFE curve under a *fixed* sampler, and separately the gain attributable to your sampler. State the decoding order, the parallelism rule, any remasking schedule, any caching, and the temperature.
4. **Oracle check:** if a small exactly-solvable instance of your problem exists, report your sampler's error on it against an exact denoiser (Tang et al., 2026).
5. **Variance:** per-sample, not just dataset-level (Fang et al., 2026).
6. **Compute:** FLOPs for training and for sampling, since the entire case for the approach is a compute trade-off.

Key takeaways

- The architecture is a bidirectional transformer with a mask token and optional time conditioning. Nothing exotic; everything from the AR stack transfers.
- Initialise from an autoregressive checkpoint. Keep the AR shift (Dream), and consider context-adaptive per-token noise schedules.
- Masked diffusion scales with a similar exponent and worse constant than autoregressive modelling, but with a different capability profile (reversal, planning).
- Perplexity does not transfer across diffusion families; use speed/quality Pareto frontiers (Sahoo et al., 2026).
- Block diffusion recovers KV caching and arbitrary-length generation, and improves likelihood, at the cost of fixing part of the ordering.
- Gradient variance is the recurring practical problem. Rao–Blackwellise, use low-discrepancy or coupled sampling, and consider giving up the exact bound — which is worth about 12% of your FLOPs.
- Half your tokens produce no gradient signal per pass. Factor that into any efficiency claim.
- $D_{\mathrm{KL}}(p_\theta \,\|\, p_{\mathrm{ref}}) = \log(\text{gen-PPL}) - H(p_\theta)$. Gen-PPL reports only the first term, so it can be driven arbitrarily low by reducing entropy. Zero-parameter periodic samplers beat every trained model on it (Franca and Tong, 2026).
- Report entropy–perplexity frontiers and compare at matched entropy; frontier dominance is sufficient for lower KL (Pynadath et al., 2026).
- ELBO/perplexity does not transfer across diffusion families. The masked-diffusion bound is tight at the optimum (Theorem 6.1), which makes within-family comparison meaningful and cross-family comparison no more valid.
- Few-step samplers are distributionally incorrect even with an exact oracle denoiser, and the error vanishes only as steps $\to L$ (Tang et al., 2026). Build the oracle instance; it is cheap.

- Some tricks that improve gen-PPL (score sharpening, low temperature) provably make the distribution worse.
- Aggregate metrics hide severe per-generation non-determinism.
- Check your floating-point precision. 32-bit Gumbel-max sampling truncates the noise and silently lowers the effective temperature, which inflated published generative perplexities substantially (Table 10.3). Under corrected sampling, masked diffusion models do *not* beat autoregressive models on text generation (Zheng et al., 2025).

Open questions and research directions

1. **An architecture designed for the task.** Every diffusion LM uses a transformer designed for autoregressive language modelling. But the task is different: the input contains a *known* partition into clean and masked positions, and the model must route information from the former to the latter. An architecture that exploits this explicitly — asymmetric attention between clean and masked tokens, separate parameter sets, iterative refinement with explicit state — has not been seriously attempted. This strikes me as the most obvious unexplored direction in the whole field.
2. **Corpus utilisation.** Half the tokens are wasted per pass. Can one train on all positions at once with multiple masking patterns sharing a forward pass (an analogue of the multi-token prediction heads that help autoregressive models)? The attention computation is shared; only the readout differs. This looks like a free factor of two.
3. **Does AR initialisation cap the ceiling?** Dream-7B is competitive with its Qwen2.5-7B parent but trails it on several general benchmarks, while beating it decisively on planning. It is unclear whether adaptation transfers knowledge with a permanent tax, or whether enough continued training removes it. The experiment — matched-compute from-scratch versus adapted, at several scales — has not been published and would settle a question everyone building these models currently guesses at.
4. **Learned block structure.** Block diffusion fixes B by hand. A model that chooses its own block boundaries per input — parallelising where the content permits and serialising where it does not — is the natural generalisation, and the obvious vehicle for the "learned factorisation" direction raised in Chapter 1.
5. **Mixture-of-experts routed by noise level.** In a diffusion model the computation needed at $t \approx 1$ (generate structure from nothing) and at $t \approx 0$ (fix a few tokens) is qualitatively different. Routing experts by noise level, or using a smaller network at low noise, is an obvious efficiency play with no serious study behind it.
6. **Optimal ε and schedule endpoints.** The cutoffs at both ends of the time interval are chosen by folklore and silently change reported likelihoods. A principled treatment — analogous to the careful handling of the first and last steps in continuous diffusion likelihood evaluation — is missing, and it affects every perplexity number in Table 5.1.
7. **A tractable estimate of joint entropy.** The frontier method of Pynadath et al. (2026) uses *unigram* entropy as a proxy for the joint entropy in Equation (10.1), and they name this as a limitation. A better estimator — from a reference model's own conditional entropies, or a compression-based estimate — would make the frontier method substantially sharper and is a well-defined, self-contained project.

8. **Is frontier dominance ever actually observed?** The condition is sufficient for lower KL but it is a strong requirement, and it is unclear how often real model pairs are ordered by it rather than crossing. A survey across the published models would tell us whether the criterion is usable in practice or mostly vacuous.
9. **Metrics for conditional generation.** Nearly all of this literature is about *unconditional* text generation, which is not what anyone deploys. For infilling, constrained generation and instruction following — diffusion's actual strengths — there is no agreed methodology at all. This gap is arguably more consequential than the gen-PPL problem.
10. **An oracle benchmark suite.** Tang et al. (2026)'s design should be a standard, reusable benchmark: a family of exactly-solvable distributions (HMMs, Ising-like models, formal grammars) of varying correlation length, with exact posteriors, against which any sampler can be scored. Building and maintaining it would be a genuine service to the field and is not research-hard.
11. **Separating sampler quality from model quality, in general.** The oracle design works only where exact posteriors are available. A diagnostic that decomposes end-to-end error into model error and sampler error for a *real* model on real data does not exist. Approaches via self-consistency (does the model agree with itself across decoding orders?) look promising and would be widely used.
12. **Metrics that reward the capabilities we claim.** Diffusion's advertised advantages are infilling, revision, order-agnosticism and parallelism. None of the standard metrics measures any of them. A benchmark that does — scoring, say, the quality of a model's revision of its own flawed output — would both make the case for the approach and give it something to optimise.

CHAPTER 11

Diffusion Language Models

CHAPTER

Authors	Shen Nie et al. (LLaDA); Jiacheng Ye et al. (Dream); Jaeyeon Kim et al.; Marianne Arriola et al.
Year	2026
Published	NeurIPS 2025 / ICLR 2025 / preprints
Link	https://arxiv.org/abs/2502.09992
Tags	LLaDA, Dream, infilling, planning, reasoning, multimodal, dLLM

This chapter is about what diffusion language models are actually good for, based on the published evidence, and where the honest uncertainties are.

11.1. The state of play

Two open 7–8B models anchor the field. **LLaDA** (Nie et al., 2025b) was trained from scratch on 2.3T tokens with about 0.13M H800 GPU-hours, then supervised-fine-tuned on 4.5M pairs. Its authors report it competitive with LLaMA3-8B in in-context learning and on par with LLaMA3-8B Base across 15 zero/few-shot tasks — which is the striking part, since LLaMA3-8B saw roughly 6× more data. After SFT it is slightly behind LLaMA3-8B Instruct, which they attribute partly to not using RL alignment. They also report that LLaDA "effectively breaks the reversal curse", with consistent performance forwards and backwards, outperforming GPT-4o on a reversal poem-completion task. These are the authors' own results, from a single 8B run with no hyperparameter tuning.

Dream-7B (Ye et al., 2025) was initialised from Qwen2.5-7B and trained on 580B tokens, i.e. roughly a quarter of LLaDA's budget, and beats it across the board.

Table 11.1 Dream-7B against its autoregressive parent and against LLaDA-8B, as reported by Ye et al. (2025). Dream used 0.6T training tokens, LLaDA 2.3T.

Benchmark	Dream-7B	LLaDA-8B	Qwen2.5-7B
MMLU	69.5	65.9	**71.9**
GSM8K	77.2	70.9	**78.9**
HumanEval	**57.9**	32.9	—
Countdown	**16.0**	13.2	6.2
Sudoku	**81.0**	46.0	21.0
Trip planning	**17.8**	—	3.6

Table 11.1 is the single most informative table about diffusion language models, and it says two things clearly.

On general knowledge and reasoning benchmarks, diffusion is close but behind. MMLU 69.5 versus 71.9; GSM8K 77.2 versus 78.9. Close enough that the approach is viable, not close

enough to claim superiority. And Dream-7B's instruction-tuned variant generally trails Qwen2.5-7B's SFT+RL results, which is a reminder that the post-training stack for diffusion LMs is years less mature.

On planning and constraint satisfaction, diffusion wins decisively. Sudoku 81.0 versus 21.0 — a factor of four. Countdown 16.0 versus 6.2. Trip planning 17.8 versus 3.6. These are not marginal differences and they are exactly the tasks whose solutions are global assignments rather than left-to-right narratives, as predicted in Chapter 1.

Intuition: why planning tasks favour diffusion

In Sudoku, the constraint linking cell $(1,1)$ to cell $(9,9)$ is as strong as the one linking adjacent cells. An autoregressive model must commit to $(1,1)$ before it has any information about $(9,9)$, and cannot revise. A diffusion model commits in an order of its own choosing — in practice, the most-determined cells first — which is exactly how a human solves Sudoku, and (with remasking) can revise. Note how much of this advantage lives in the *sampler*: Kim et al. (2025) get from under 7% to around 90% on Sudoku by changing the decoding order of an already-trained model.

11.2. Capabilities that come for free

Infilling and completion. No task-specific training required: clamp the known tokens as unmasked and denoise the rest. Ye et al. (2025) report infilling guided by a required ending sentence working out of the box. For an autoregressive model this needs a fill-in-the-middle training objective.

Arbitrary-order generation. Decoding can follow left-to-right, partially random, or fully random order, all from the same model.

Controllable quality/speed trade-off. The number of denoising steps is an inference-time dial. Ye et al. (2025) report that on Countdown, 5–20 steps gave better speed *and* quality than Qwen2.5-7B. Note the caveat from Section 10.8: such comparisons are only meaningful with the sampler fully specified.

Mitigating the reversal curse. Nie et al. (2025a) identify this explicitly. A bidirectional model trained on arbitrary conditioning sets has no privileged direction, so "A is B" and "B is?" are the same kind of query.

11.3. What is genuinely hard

Latency, honestly accounted. A diffusion LM with N steps on L tokens does N full-sequence forward passes. An autoregressive model does L passes but each is $O(1)$ in new computation thanks to KV caching. So the comparison is $N \times \text{cost}(L)$ against $L \times \text{cost}(1)$, and diffusion wins only when $N \ll L$ *and* the per-pass cost is controlled. This is why caching (Chapter 8) and block diffusion (Chapter 10) matter so much: they are what make the arithmetic work. The commercial systems — Mercury, Gemini Diffusion, Seed Diffusion — report strong latency numbers, and their technical details are limited.

Variable length. Masked diffusion fixes L in advance. Practice pads to a maximum and relies on end-of-sequence tokens, which biases length and wastes compute. Block diffusion and semi-autoregressive generation are the workarounds.

The post-training gap. Instruction tuning, preference optimisation, tool use, long-context handling and inference-time reasoning are all mature for autoregressive models and immature here, largely because the intractable sequence log-probability complicates every RL method (Chapter 9).

Evaluation. See Section 10.8. A substantial fraction of the published comparisons in this area do not survive careful scrutiny.

11.4. Multimodal diffusion language models

A fast-growing area, because masked diffusion handles heterogeneous token streams naturally: an image as VQ codes and text as tokens are both just categorical sequences, and a single masked diffusion model can denoise both.

The main systems: LLaDA-V and LaViDa (vision–language diffusion models), Dimple, and MMaDA, which unifies understanding and generation and introduces UniGRPO for RL across modalities. FUDOKI and Muddit pursue unified generation on flow-matching foundations. D-DiT does dual diffusion for joint image generation and understanding.

Intuition: why multimodality is a natural fit

An autoregressive model must linearise everything into one sequence and pick an order, which for an image is arbitrary (raster scan?) and for interleaved text-and-image data is awkward. A masked diffusion model conditions on *whatever is unmasked*, so "describe this image" and "generate an image from this text" and "fill in the missing patch given the caption" are all the same operation with different masking patterns. One model, one objective, every task. This is the strongest conceptual argument for diffusion in the multimodal setting and it is why the area is moving quickly.

11.5. Code generation

A natural application, because code has strong non-local structure (a function signature constrains its body and its call sites) and because infilling is the dominant real-world use case. DiffuCoder and its coupled-GRPO training, Seed Diffusion (ByteDance, code-focused), and Mercury's coding models all target this. Dream-7B's HumanEval 57.9 against LLaDA's 32.9 shows how much room there still is.

11.6. Reasoning

The most contested area. The theoretical picture from Feng et al. (2025) is the place to start, and it is nuanced:

- For **token-level** error (measured via perplexity), masked diffusion models can reach near-optimal performance in a **constant** number of sampling steps, independent of sequence length. This is the positive result and it is strong.
- For **sequence-level** error — the probability of generating a sequence outside the target language's support — the required number of steps must, in the worst case, scale **linearly** with sequence length. Because sequence-level correctness requires the *entire* sequence to be error-free, the model can only safely commit a few tokens per step.

Their analysis is within a hidden Markov model framework with a bounded sequence length, and assumes a small learning error (the KL between the trained reverse model and the true reverse process is bounded, which the ELBO-optimal solution satisfies).

Intuition: the separation is about what counts as a failure

If you are judged on average per-token quality, parallel decoding is fine: a few wrong tokens cost you a little perplexity. If you are judged on whether the *whole* answer is right — a proof, a program, an arithmetic derivation — then every token must be correct, and committing many tokens per step from a factorised distribution is a compounding risk. That is exactly Remark 2.4 again, and it says that **reasoning tasks are structurally the worst case for parallel generation**, which sits uncomfortably beside the empirical finding that diffusion models are good at planning. The resolution is probably that planning tasks benefit from *order freedom* (a model advantage) while suffering from *parallelism* (a sampler cost), and the published wins come from models that take the order freedom and decode nearly serially.

The empirical picture: RL post-training helps substantially on reasoning, and most on planning (Table 9.1). TraDo/TraceRL, LLaDA 1.5 and d1 are the main entries. A reasonable summary is that diffusion LMs are not yet better reasoners than autoregressive models of the same size, but that the gap is closing and the *profile* of what they are good at differs.

11.7. Reading the recent literature critically

Three 2026 papers that should shape how you read everything else in this area:

- Franca and Tong (2026): generative perplexity is hackable by zero-parameter samplers.
- Pynadath et al. (2026): likelihood bounds do not compare across latent structures; use entropy–perplexity frontiers.
- Tang et al. (2026): few-step samplers are distributionally incorrect even with an exact oracle denoiser.

There are also broad surveys (Yu et al., 2026; Li et al., 2025; Yuan et al., 2026) and experimental analyses (Bertolani et al., 2026) worth consulting for coverage, with the usual caveat that survey tables inherit whatever methodological problems the underlying papers have.

Key takeaways

- LLaDA (8B, 2.3T tokens, from scratch) and Dream-7B (0.58T tokens, adapted from Qwen2.5-7B) are the reference open models. Adaptation is dramatically cheaper and currently better. LLaDA matches LLaMA3-8B Base on 15 zero/few-shot tasks despite $6\times$ less data, and reportedly breaks the reversal curse.
- On general benchmarks diffusion LMs are close to but behind autoregressive models of the same size. On planning and constraint satisfaction they win by large margins (Sudoku 81.0 vs 21.0).
- Infilling, arbitrary-order generation, a quality/speed dial and reversal-curse mitigation all come for free.
- The latency case requires $N \ll L$ and controlled per-pass cost, which is what caching and block diffusion deliver. Account for it honestly.

- Theory says diffusion can reach near-optimal *token*-level error in a constant number of steps, but worst-case *sequence*-level error needs steps linear in length (Feng et al., 2025). Reasoning tasks are the structurally hardest case for parallel decoding.
- Multimodality is the most natural fit: every task is one masking pattern of one model.
- Much published evaluation in this area is unsound. Read Section 10.8 first.

Open questions and research directions

1. **Will diffusion LMs match autoregressive ones at frontier scale?** Genuinely unknown. The largest public open models are 7–8B; the scaling exponents look similar with a worse constant (Nie et al., 2025a); and Sahoo et al. (2026) suggest the family choice is not settled. Nothing in the literature answers this, and it is the question the field exists to answer.
2. **Resolve the planning/parallelism tension.** Diffusion wins on planning tasks, and theory says parallel decoding is worst for exactly the tasks needing whole-sequence correctness. The hypothesis in the box above — that the wins come from order freedom, not parallelism, and that published planning results use nearly-serial decoding — is testable by reporting planning accuracy against NFE. If true, the marketing of "fast *and* better at reasoning" is claiming two things that trade off against each other, and that should be said out loud.
3. **Long context.** Every advantage of bidirectional attention is also a cost: attention is $O(L^2)$ with no causal sparsity to exploit and no KV cache across steps. How diffusion LMs behave at 100k+ context is barely studied and may be where the approach is weakest.
4. **Tool use and agentic behaviour.** These are inherently sequential and interactive: you cannot denoise a tool call whose result you have not seen. Block diffusion is the obvious substrate, but how a diffusion LM should interleave generation with external observations is unexplored and is a prerequisite for the approach to matter commercially.
5. **Does order freedom help reasoning in a way we can characterise?** The intuition that a model which can write a proof's conclusion before its lemmas might reason better is appealing and untested. One could measure, for a trained model, the order it *chooses* on reasoning problems and ask whether it correlates with correctness. Nobody appears to have looked, and it would be a nice empirical paper.
6. **Conditional-generation benchmarks.** Flagged in Section 10.8 and worth repeating because it is the most actionable item here: diffusion's real strengths (infilling, constrained generation, revision) have no agreed benchmark. Building one would let the field optimise for what it is actually good at rather than for unconditional perplexity.

CHAPTER 12

Molecular, Graph and Biological Design

CHAPTER

Authors	Clément Vignac et al.; Seul Lee et al.; Sarah Alamdari et al.; Andrew Campbell et al.; Hannes Stark et al.
Year	2025
Published	ICLR 2023 / ICML 2024–25 / NeurIPS 2023
Link	https://arxiv.org/abs/2209.14734
Tags	DiGress, graph diffusion, GenMol, fragments, proteins, DNA, co-design

Molecular generation is where discrete diffusion arguably has the clearest advantage over autoregressive modelling, for a reason specific to the domain: **a molecule has no canonical order**. The SMILES string of a molecule is the output of a depth-first traversal whose starting atom and branch order are conventions. Committing an autoregressive model to that order is committing it to an artefact. This chapter covers the two main representations — graphs and strings — and the methods for each.

12.1. Graphs: DiGress

Vignac et al. (2023a) established the template for discrete graph diffusion, and it remains the reference method.

12.1.1. The representation and the noise model

A graph on n nodes is a pair $(\mathbf{X}, \mathbf{E})$: node types $\mathbf{X} \in \{0,1\}^{n \times a}$ (atom identities) and edge types $\mathbf{E} \in \{0,1\}^{n \times n \times b}$ (bond orders, with "no bond" as one of the b types). The crucial representational decision is that **absence of an edge is a category**, not a missing value — so the state space is a fixed-size categorical array and the standard machinery applies.

Diffusion is applied *separately and independently to each node and each edge*, with transition matrices $\mathbf{Q}^t_{\mathbf{X}}$ and $\mathbf{Q}^t_{\mathbf{E}}$. For undirected graphs, noise is applied only to the upper triangle of the edge tensor, which is then symmetrised. As usual the cumulative products let one sample a noisy graph directly from a clean one.

12.1.2. Why marginal noise, not uniform

This is DiGress's most transferable idea. Real graphs are *sparse*: the "no bond" category dominates. Under uniform noise the limiting distribution is uniform over edge types, so a fully noised graph is nearly complete — and the reverse process must carve a sparse molecule out of a dense mess, having been trained on dense intermediate states that look nothing like real molecules.

The fix (Proposition 3.2) is to use a prior close to the data: a single node distribution $\boldsymbol{u}$ applied to all nodes and a single edge distribution $\boldsymbol{v}$ applied to all edges, both set to the empirical marginals. Exchangeability is preserved. Listing 3.1 quantifies the difference: for a realistic

edge-type marginal, a fully noised 20-node graph has 19 edges out of 190 candidate pairs under marginal noise, against 142 under uniform noise.

Intuition: the prior should look like the data

The general principle, stated once: **the limiting distribution of your forward process is the distribution your reverse process starts from, and you should make it as close to the data as you can afford**. Masking takes this to one extreme (the prior is a point mass, which carries no information but is trivially easy to start from). Uniform noise takes a different extreme (maximum entropy, maximally far from sparse data). Marginal matching is the sensible middle, and it costs nothing — you just estimate two histograms from your training set. This is one of the cheapest wins in the whole book, and it is routinely skipped by people porting text methods to graphs.

12.1.3. Structural features

Message-passing graph neural networks have well-known representational limits: they cannot count cycles or distinguish certain non-isomorphic graphs. DiGress augments the denoiser's input with auxiliary *structural and spectral* features computed from the noisy graph. This is affordable *because* the discrete noise model keeps intermediate graphs sparse — spectral features of a near-complete graph would be both expensive and uninformative. The sparsity and the features are therefore coupled design decisions.

12.1.4. The loss

Cross-entropy between predicted and true node types summed over nodes, plus a λ-weighted cross-entropy summed over all node *pairs* for edge types. Note that this reduces graph generation to per-node and per-edge classification — no graph matching is required, because the noisy graph supplies the correspondence.

12.1.5. Why equivariance of the denoiser is enough

A graph has no canonical node ordering, but the model operates on adjacency tensors, which do. Every graph-diffusion paper asserts that an equivariant denoiser makes this harmless. It is worth proving, because the proof says exactly which properties of the *noise model* are being used, and because the same argument fails for the forward process in a way Remark 12.3 makes precise.

Write a graph on n labelled nodes as $G = (\mathbf{X}, \mathbf{E})$ with $\mathbf{X} \in \Delta^{n \times a}$ node types and $\mathbf{E} \in \Delta^{n \times n \times b}$ edge types, and let a permutation $\sigma \in S_n$ act by $\sigma \cdot G := (\mathbf{X}_{\sigma^{-1}(\cdot)}, \mathbf{E}_{\sigma^{-1}(\cdot),\sigma^{-1}(\cdot)})$.

Theorem 12.1 (Equivariant denoiser ⇒ invariant model). Suppose

1. *(Exchangeable noise.)* The forward process applies the same kernel $\mathbf{Q}_t$ independently to every node and every node pair, so that $q(\sigma \cdot \boldsymbol{z}_t \mid \sigma \cdot G) = q(\boldsymbol{z}_t \mid G)$ for all σ, and the prior $p(\boldsymbol{z}_T)$ is a product of identical per-element distributions;
2. *(Equivariant denoiser.)* $\boldsymbol{x}_\theta(\sigma \cdot \boldsymbol{z}_t, t) = \sigma \cdot \boldsymbol{x}_\theta(\boldsymbol{z}_t, t)$ for all $\sigma \in S_n$, t;
3. *(Factorised reverse kernel.)* The reverse step factorises over nodes and pairs given $\boldsymbol{x}_\theta$, as in Equation (3.4).

Then the model distribution is permutation-invariant: $p_{\text{model}}(\sigma \cdot G) = p_{\text{model}}(G)$ for every G and every $\sigma \in S_n$.

Proof. Let $p_\theta(\boldsymbol{z}_s \mid \boldsymbol{z}_t)$ be the reverse kernel. By (iii) it is a product over elements e (nodes and pairs) of factors depending only on $[\boldsymbol{z}_t]_e$ and $[\boldsymbol{x}_\theta(\boldsymbol{z}_t, t)]_e$, with the same functional form for every node and the same form for every pair. Relabelling by σ permutes the elements and, by (ii), permutes the denoiser outputs the same way, so each factor is matched with its image and

$$p_\theta(\sigma \cdot \boldsymbol{z}_s \mid \sigma \cdot \boldsymbol{z}_t) = p_\theta(\boldsymbol{z}_s \mid \boldsymbol{z}_t). \tag{12.1}$$

By (i) the prior satisfies $p(\sigma \cdot \boldsymbol{z}_T) = p(\boldsymbol{z}_T)$. Now marginalise over the trajectory and substitute $\boldsymbol{z}_t \to \sigma^{-1} \cdot \boldsymbol{z}_t$, which is a bijection of the summation domain:

$$\begin{aligned} p_{\text{model}}(\sigma \cdot G) &= \sum_{\boldsymbol{z}_{T:1}} p(\boldsymbol{z}_T) \prod_t p_\theta(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t)\Big|_{\boldsymbol{z}_0 = \sigma \cdot G} \\ &= \sum_{\boldsymbol{z}_{T:1}} p(\sigma \cdot \boldsymbol{z}_T) \prod_t p_\theta(\sigma \cdot \boldsymbol{z}_{t-1} \mid \sigma \cdot \boldsymbol{z}_t)\Big|_{\boldsymbol{z}_0 = G} \;=\; p_{\text{model}}(G), \end{aligned} \tag{12.2}$$

using Equation (12.1) and prior invariance in the last step. ∎

Corollary 12.2 (Likelihoods per graph and per isomorphism class differ). If p_{model} is permutation-invariant, then the probability it assigns to the *isomorphism class* of G is

$$p_{\text{model}}\big([G]\big) = \frac{n!}{|\mathrm{Aut}(G)|}\, p_{\text{model}}(G), \tag{12.3}$$

since the orbit of G under S_n has size $n!/|\mathrm{Aut}(G)|$ by the orbit–stabiliser theorem. Consequently a reported negative log-likelihood per graph and one per isomorphism class differ by $\log n! - \log|\mathrm{Aut}(G)|$, a quantity that varies across the test set (it is $\log n!$ for an asymmetric graph and much smaller for a symmetric one). Verified by enumeration in Listing C.3.

Remark 12.3 (What the theorem does *not* give). Assumption (i) is a statement about *exchangeability* — the same kernel everywhere — and that is all Theorem 12.1 needs. It is strictly weaker than the property one might want, namely that the forward process respect $\mathrm{Aut}(G)$ *pathwise*: two automorphic nodes of the same molecule are noised *independently*, so a single trajectory breaks the symmetry even though the law over trajectories does not. The model distribution is invariant; individual training trajectories are not, and the denoiser must therefore spend capacity learning to be equivariant on noisy inputs whose symmetry has already been destroyed. Whether an automorphism-aware forward process would help is the open question raised in Chapter 1 and restated at the end of this chapter; Theorem 12.1 is precisely the reason the question is about *efficiency* rather than correctness.

12.2. Jointly 2D and 3D: MiDi

Many applications need coordinates, not just connectivity. Vignac et al. (2023b) (MiDi) generate the molecular graph *and* the 3D conformation jointly: discrete diffusion on atom and bond types, continuous (equivariant) diffusion on coordinates, in one model with a shared denoiser. This is the problem that Campbell et al. (2024) later addressed in the flow-matching framework (Chapter 7), and multimodal flows are now the more common formulation.

Related: MolDiff addresses the atom–bond inconsistency problem (generating 3D atom positions and bond types that disagree); MUDiff and JODO also do joint 2D/3D generation. Purely-3D methods (EDM, GeoLDM, GeoBFN) use continuous diffusion on coordinates and infer bonds afterwards, which is simpler but loses the chemistry.

Remark 12.4 (Equivariance). For 3D molecules the denoiser should be $E(3)$-equivariant: rotating and translating the input should rotate and translate the output. This is a constraint on the

architecture, and it interacts with the discrete part only through the shared network. A deeper question — whether the *forward process* should respect graph automorphisms, so that two labellings of the same molecule follow the same noising trajectory — is raised in Chapter 1 and remains open.

12.3. Strings and fragments: GenMol

A different tack. Rather than diffusing over the graph, diffuse over a *string* representation — but choose the representation so that the order genuinely does not matter.

Lee et al. (2025) use **SAFE** (Sequential Attachment-based Fragment Embedding), which represents a molecule as an *unordered set* of SMILES fragment blocks. Their argument is the one that opens this chapter — sequence representations usually assume an order from a heuristic traversal, which makes left-to-right autoregression a poor fit; masked discrete diffusion with a BERT-style bidirectional architecture does not depend on token order and decodes non-autoregressively.

The three components:

- **Masked discrete diffusion over SAFE tokens** with bidirectional attention.
- **Fragment remasking**: replace parts of an existing molecule with mask tokens and let diffusion regenerate new fragments in their place. The unit of exploration is a *fragment*, which matches how chemists actually optimise molecules.
- **Molecular context guidance (MCG)**: calibrate the model's predictions using information from a given molecular context.

They report significantly outperforming the GPT-based SAFE-GPT on de novo and fragment-constrained generation, and state-of-the-art results on goal-directed hit generation and lead optimisation.

> **Intuition: choose the units, then the model**
>
> GenMol's main lesson is representational, not algorithmic. Diffusing over *characters* of a SMILES string asks the model to learn chemical valence rules from scratch and produces invalid molecules when parallel updates conflict. Diffusing over *fragments* bakes the valence rules into the vocabulary: every token is a chemically sensible unit, and every rearrangement of tokens is closer to a real molecule. The same move is available in other domains — protein motifs, code AST nodes, musical phrases — and is underexploited. Lee et al. (2026) push further with fragment-level discrete flow matching.

12.4. Property optimisation

The actual objective in drug discovery is not "generate a plausible molecule" but "generate a molecule with these properties". Three mechanisms, in increasing order of strength and cost:

1. **Guidance** (Chapter 9). Exact for discrete states (Theorem 9.1). Nisonoff et al. (2025) condition SMILES generation on ring count and lipophilicity. Kerby and Moon (2024) explore training-free guidance specifically for molecular generation. Note that for molecular alphabets K is small enough that the *exact* guided rate is affordable — you do not need the Taylor approximation.
2. **Conditional training.** Graph DiT (Liu et al., 2024) handles multi-conditional molecular generation with a graph diffusion transformer, conditioning on several properties at once.

3. **Reinforcement learning.** Fine-tune against a reward (Zekri and Boullé, 2025); MolEditRL combines discrete diffusion with RL for structure-preserving editing. The key advantage over guidance is that the reward need not be differentiable — docking scores and synthesis-accessibility heuristics are not.

The reward-hacking problem. Standard in this domain and worth stating. A generator optimised against a property predictor will find the predictor's blind spots, producing molecules that score beautifully and are chemically absurd. The defences are to report distributional similarity to known chemistry alongside any property score, to use ensembles or uncertainty-aware predictors, and to validate a sample with a slower, more trustworthy method (docking, DFT, or a chemist).

12.5. Evaluation

Molecular metrics are better behaved than text metrics, but not unproblematic.

- **Validity** (parses, and satisfies valence rules), **uniqueness**, **novelty**. Necessary, easily saturated, and gameable: a model that generates methane repeatedly is 100% valid.
- **Distributional**: Fréchet ChemNet distance; distances between distributions of molecular descriptors. The analogue of MAUVE, and preferable for the same reasons.
- **Property-targeted**: for benchmark suites such as PMO (practical molecular optimisation), the standard is area under the top-k curve, which measures sample-efficiency rather than best-case performance — a good design that text benchmarks could learn from.
- **Graph-specific**: degree, clustering and orbit-count statistics compared against the training distribution; spectral statistics.

Caution. Benchmarks such as QM9 and ZINC are small and well-fitted, and reported differences between strong methods are often within noise of the dataset. Report multiple seeds, and prefer benchmarks that measure optimisation sample-efficiency over ones that measure unconditional distribution matching.

12.6. A representative map

12.7. Biological sequences: proteins, DNA and RNA

Biological sequence design is, in my view, the application where discrete diffusion is most clearly the right tool rather than merely a viable one. The reasons are domain-specific and worth stating up front:

- **The vocabularies are tiny.** 4 nucleotides, 20 amino acids. Everything that is expensive at $K = 50{,}000$ — the $L(K-1)$ concrete score, exact guidance, uniform-state diffusion — is cheap here.
- **The tasks are inherently conditional.** Scaffold a binding site; design a linker between two domains; generate a sequence with a target activity. These are infilling and guidance problems, which is what discrete diffusion does natively and autoregressive models do awkwardly.
- **There is no natural order.** An α-helix in the middle of a protein does not depend on the N-terminus in a way that makes left-to-right generation sensible.
- **The objective is a property, not a likelihood.** Nobody wants the most probable protein; they want one that binds, folds and is stable. So guidance and reward optimisation are the whole game, and discrete guidance is exact (Theorem 9.1).

Table 12.1 Discrete diffusion and flow methods for molecules and graphs.

Method	Representation	Contribution
DiGress (Vignac et al., 2023a)	2D graph	the template: per-node/per-edge categorical diffusion, marginal noise, structural features
MiDi (Vignac et al., 2023b)	2D + 3D	joint discrete graph and continuous coordinate diffusion
MolDiff	2D + 3D	fixes atom–bond inconsistency
Graph DiT (Liu et al., 2024)	2D graph	multi-conditional generation
GenMol (Lee et al., 2025)	SAFE fragments	fragment-level masked diffusion; fragment remasking; MCG
FragFM (Lee et al., 2026)	fragments	fragment-level discrete flow matching
Multiflow (Campbell et al., 2024)	discrete + continuous	the flow-matching framework for joint modalities
UDLM (Schiff et al., 2025)	SELFIES/graph	uniform-state diffusion; SOTA at small vocabularies
ReMDM (Wang et al., 2025b)	any	remasking improves the novelty/property Pareto frontier on QM9

12.7.1. Protein sequences: EvoDiff

Alamdari et al. (2023) brought discrete diffusion to protein sequence generation at scale. Two model families: an order-agnostic autoregressive diffusion over sequences (OADM, which by Theorem 6.1 is masked diffusion), and a variant operating on multiple sequence alignments, which lets the model condition on evolutionary context rather than a single sequence.

The claim in the title — "sequence is all you need" — is the substantive one: that generating in sequence space, rather than in structure space and then inverse-folding, is sufficient and has advantages. Specifically, sequence generation can produce intrinsically disordered regions, which structure-based methods cannot represent, and it supports scaffolding tasks by conditioning on arbitrary subsets of positions.

Intuition: why sequence space, and why masked diffusion

A structure-first pipeline (generate a backbone, then inverse-fold) presumes the protein has a single well-defined structure. A large fraction of the proteome does not — disordered regions are functional, not failures — and that whole class is unreachable by construction. Sequence-space generation has no such presumption.

Masked diffusion is then the natural generative model in sequence space, because every protein design task is literally a masking pattern: "keep these catalytic residues, design everything else" is a conditional sample with a specific set of positions unmasked. No task-specific head, no retraining.

Related: diffusion language models have been applied as general-purpose protein learners, and mask-prior-guided denoising has been used for inverse protein folding.

12.7.2. Joint sequence–structure co-design

The flagship application of Chapter 7's machinery. Campbell et al. (2024) generate protein sequence and structure jointly: a discrete flow over amino-acid identities coupled to a continuous

flow over backbone frames, in one model with a shared time variable. The motivation is that sequence and structure are not independent — a residue identity makes sense only relative to its structural environment — so generating them jointly beats generating one and then the other.

This framing has become standard for the problem, and the "multimodal flow" construction generalises well beyond proteins: anything that is a mix of categorical labels and continuous coordinates (molecules with conformers, materials with lattice parameters, scenes with object classes and poses) fits.

Earlier related work includes SE(3)-discrete diffusion for nucleic acid and protein complexes and various antibody-design methods.

12.7.3. DNA: regulatory sequence design

The most striking quantitative results in this chapter are here.

Dirichlet flow matching. Stark et al. (2024) apply the simplex construction of Section 7.6 to DNA. On distributional similarity for enhancer design (Fréchet biological distance, lower better):

Table 12.2 Enhancer design, FBD (lower is better), after Stark et al. (2024).

Dataset	Autoregressive LM	Dirichlet FM
Melanoma DNA	36.0	**1.9**
Fly Brain DNA	25.2	**1.0**

That is better than an order of magnitude, which is unusual and worth taking seriously. They also report that classifier-free guidance produces cell-type-specific enhancers improving on experimentally validated sequences from the literature, and that the deterministic sampling map distils to a single step with $O(L)$ speedups over autoregressive generation.

Guided design. Nisonoff et al. (2025) apply exact discrete guidance to cell-type-conditioned enhancer design. Zekri and Boullé (2025) fine-tune a masked diffusion model for enhancer activity with SEPO and lead on most metrics (99.5% ATAC-Acc, 7.55 median predicted activity), though not on 3-mer correlation — a reminder to look at more than one metric.

Earlier work. Dirichlet diffusion score models (Avdeyev et al., 2023) used Jacobi processes on the simplex for DNA design and remain a standard baseline.

Remark 12.5 (Why DNA favours the simplex). With $K = 4$ the simplex is three-dimensional. The pathology that kills linear simplex interpolation — Proposition 7.7's support collapse, which worsens as K grows — is mild at $K = 4$, and the Dirichlet construction removes it entirely. Meanwhile the benefits of a continuous relaxation (gradient guidance, a deterministic flow to distil) are fully available. DNA is close to the best possible case for a relaxation-based method, and the results reflect that. Do not expect these margins to transfer to protein ($K = 20$), let alone to text.

12.7.4. RNA and other targets

RNA structure prediction and design (RNADiffFold and similar), D-peptide design, and antibody generation have all received discrete-diffusion treatments. Materials generation — MatterGen and successors — is a neighbouring area with the same multimodal structure (discrete element identities plus continuous lattice and positions).

12.7.5. Guidance and reward optimisation here

This is where Chapter 9 earns its keep. The practical points:

1. **Use exact guidance, not the Taylor approximation.** At $K = 4$ or $K = 20$, evaluating the predictor at all $L(K-1)$ Hamming-1 neighbours is affordable, so Equation (9.1) can be used as written. Most of the literature imports TAG from the large-vocabulary setting out of habit. This is a free accuracy improvement and, as far as I can tell, nobody has quantified what it is worth.
2. **Rewards are not differentiable.** Predicted $\Delta\Delta G$, docking scores, ATAC-seq accessibility, wet-lab assays — none of these is a differentiable function you can backpropagate through. SEPO's gradient estimator, which explicitly does not require differentiability (Zekri and Boullé, 2025), is the right tool.
3. **Validate distributionally.** A sequence that maximises a predictor's output is usually an adversarial example for that predictor. Report distributional similarity to natural sequences (FBD, k-mer statistics) alongside any predicted-activity number. Table 12.2's FBD is exactly this kind of metric, which is part of why those results are credible.

12.7.6. What to use for what

Table 12.3 A rough guide for biological sequence tasks.

Task	Method	Notes
Protein sequence generation, scaffolding	masked diffusion (EvoDiff-style)	every task is a masking pattern; handles disordered regions
Sequence + structure co-design	multimodal flow (Campbell et al., 2024)	the standard formulation
DNA regulatory element design	Dirichlet FM (Stark et al., 2024)	$> 10\times$ better FBD than AR; distils to one step
Small-vocabulary likelihood modelling	uniform-state diffusion (Schiff et al., 2025)	SOTA at small K; can revise
Property-targeted design	exact discrete guidance, or SEPO	use exact guidance at small K; SEPO for non-differentiable rewards

Key takeaways

- Molecules have no canonical order, so an autoregressive model over SMILES is committed to an artefact. This is the clearest domain argument for discrete diffusion anywhere.
- DiGress: treat "no edge" as a category, diffuse each node and edge independently, use **marginal-matched** noise so that noised graphs stay sparse, and add structural/spectral features (which the sparsity makes affordable).
- Match the prior to the data. 19 edges versus 142 in a noised 20-node graph is the whole argument.
- For 3D, generate graph and coordinates jointly (MiDi), now usually formulated as a multimodal flow (Campbell et al., 2024).

- Choose your units before your model: fragment-level vocabularies (SAFE) bake in chemistry and give large gains (Lee et al., 2025).
- At molecular vocabulary sizes, *exact* discrete guidance is affordable — use it rather than the Taylor approximation.
- Guard against reward hacking: report distributional similarity alongside any property score.
- Biology is the best case for discrete diffusion: tiny vocabularies make the expensive machinery cheap, the tasks are natively conditional, there is no canonical order, and the objective is a property rather than a likelihood.
- EvoDiff put masked (order-agnostic) diffusion in protein sequence space, with the argument that sequence-space generation reaches disordered regions that structure-first pipelines cannot.
- Joint sequence–structure co-design via multimodal flows (Campbell et al., 2024) is now the standard formulation and generalises to any discrete+continuous object.
- Dirichlet flow matching on DNA gives order-of-magnitude better distributional similarity than autoregressive baselines (Table 12.2) and distils to one step — but $K = 4$ is the best possible case for a simplex relaxation, so do not extrapolate.
- At small K, use *exact* discrete guidance rather than the Taylor approximation. Use non-differentiable-reward RL (SEPO) when the objective is an assay or a docking score.
- Always report distributional similarity alongside predicted activity; optimising a predictor produces adversarial examples for it.

Open questions and research directions

1. **Symmetry in the forward process.** A molecular graph is an equivalence class under node permutation, but the forward process noises a *labelled* representative. Current practice handles this with an equivariant denoiser and exchangeable noise, which is sufficient for correctness but means the model spends capacity on labelling. A forward process defined on isomorphism classes — or an objective that quotients them out — is an open and genuinely interesting problem, and the same question arises for sets and permutations.
2. **Synthesisability as part of the generative process.** A generated molecule is useless if it cannot be made. Current practice bolts a synthesis-accessibility score on as a reward or filter. A forward process whose corruption steps are *retrosynthetic disconnections* — so that the reverse process is a synthesis route by construction — would make synthesisability structural rather than a penalty. I am not aware of this having been done, and it is the molecular analogue of the "structured multi-site jumps" direction from Chapter 4.
3. **Scale.** Graph diffusion is $O(n^2)$ in the number of nodes because every pair is a categorical variable. For proteins, materials and polymers this is prohibitive. Sparse formulations exist (Qin et al., 2023; Siraudin et al., 2024) but the field's standard benchmarks (QM9, ZINC) are small enough that the problem is easy to ignore — which is itself a reason it is under-addressed.
4. **Hierarchical / multi-resolution generation.** Chemists think in scaffolds and decorations; GenMol's fragments are one level of a hierarchy. Generating coarse-to-fine —

scaffold, then fragments, then atoms, then coordinates — maps onto a hierarchical forward process and would give the reverse process a curriculum. This connects to the hierarchical-mask direction raised in Chapter 5.

5. **Which representation actually wins, and why?** Graph, SMILES character, SELFIES token, SAFE fragment, 3D point cloud — all have papers claiming advantages, with no controlled comparison at matched compute and matched denoiser. This is an unglamorous but genuinely useful study that nobody has run.
6. **Guidance under parallel updates, for molecules.** Flagged generally in Chapter 9, but it bites hardest here: a property like LogP depends on the whole molecule, so guiding each position's marginal independently is exactly the wrong decomposition. Since molecular guidance is the main practical use of discrete guidance, this gap matters.
7. **Evolutionary structure in the forward process.** Protein sequences are related by evolution, and substitution matrices (BLOSUM, PAM) encode which amino-acid substitutions are likely. A forward process whose rate matrix *is* a substitution matrix would corrupt proteins along evolutionarily plausible paths rather than arbitrarily — the biological analogue of DiGress's marginal matching, and of D3PM's embedding-distance kernel. This seems obviously worth trying, has a ready-made $\mathbf{Q}$, and I cannot find it done properly.
8. **Does uniform-state diffusion beat masked diffusion for proteins?** Schiff et al. (2025) show uniform wins at small K on molecules and DNA. Proteins are at $K = 20$, in between, and most protein work uses masking out of convention. A controlled comparison is cheap and would settle a question that affects a lot of applied work.
9. **Length and insertion/deletion.** Fixed-length diffusion cannot natively represent indels, which are fundamental to biological sequence evolution and to design (you want to *insert* a loop, not just substitute residues). This is a sharper instance of the general variable-length problem from Chapter 5 and it is arguably more consequential here than anywhere else.
10. **Coupled multimodal dynamics.** In current multimodal flows, the discrete and continuous channels share a network but their dynamics are independent. Physically they should not be: the rate at which a residue identity changes ought to depend on its local structural environment. Writing down a correctly coupled process, and checking whether the marginalisation trick of Theorem 7.3 survives, is a well-posed theoretical problem with direct application.
11. **Guidance with experimental feedback.** The real design loop is iterative: generate, assay, update. That is Bayesian optimisation with a discrete diffusion prior, and the discrete setting's exact guidance makes posterior sampling unusually tractable. Active-learning formulations of discrete diffusion design are, as far as I can see, almost absent from the literature despite being what practitioners actually need.
12. **Does the DNA result transfer?** Table 12.2's margins are large enough to be suspicious of over-extrapolation. An honest follow-up would test Dirichlet FM at $K = 20$ (protein) and $K \approx 30$ (SELFIES) to find where Proposition 7.7's pathology begins to bite in practice. This would be a genuinely useful negative result if it is one.

CHAPTER 13

Theory: Convergence, Expressivity and Limits

CHAPTER

Authors	Yuchen Liang, Renxiang Huang, Lifeng Lai, Ness Shroff and Yingbin Liang; Guhao Feng et al.; Jaeyeon Kim et al.
Year	2025
Published	NeurIPS 2025 / ICML 2025
Link	https://arxiv.org/abs/2506.02318
Tags	convergence, sample complexity, hardness, schedule optimality, ELBO tightness

The theory of discrete diffusion is young but has produced several results that should change how you design experiments. This chapter collects them. The organising question is: *given a good enough score or denoiser, how many steps do you need, and what can go wrong that more steps cannot fix?*

13.1. Convergence: how many steps?

The sharpest results are for absorbing-state diffusion. Liang et al. (2025) analyse both τ-leaping and uniformization. Write d for the sequence length, $\varepsilon_{\text{score}}$ for the score estimation error, N for the number of steps, T for the terminal time and δ for an early-stopping cutoff. All bounds hide constants.

Theorem 13.1 (Absorbing-state convergence, after Liang et al., 2025). **1. Initialisation.** With the surrogate initialisation, $D_{\mathrm{KL}}(q_T \,\|\, p_{\text{init}}) \lesssim d\, e^{-T}$.

2. **τ-leaping with early stopping.**

$$D_{\mathrm{KL}}(q_\delta \,\|\, p_{T-\delta}) \lesssim d\, e^{-T} + \varepsilon_{\text{score}} + \frac{d\,(T + \log(M\delta^{-1}))(T + \log \delta^{-1})^2}{N}, \tag{13.1}$$

with $\mathrm{TV}(q_0, q_\delta) \lesssim d\delta$. Choosing $T = \log(d/\varepsilon)$ and $N = \tilde{O}(d/\varepsilon)$ gives $\mathrm{KL} \le \varepsilon$.

3. **Uniformization with early stopping.** $D_{\mathrm{KL}}(q_\delta \,\|\, p_{T-\delta}) \lesssim d\, e^{-T} + \varepsilon'_{\text{score}}$ — *no explicit discretisation term* — with expected step count $\mathbb{E}[N] = O\big(d(\log\log(d/\varepsilon) + \log \delta^{-1})\big)$.

4. **Without early stopping** the same guarantees hold for q_0 directly, but only under an additional assumption: a lower bound γ on the relative likelihood of `[MASK]` in the data. Then τ-leaping needs $N = \tilde{O}(d\gamma^{-2}/\varepsilon)$.

Three things to extract.

Absorbing beats uniform by a factor of d. Their Table 1 compares against the best prior uniform-state results with the same algorithm and state space: $\tilde{O}(d/\varepsilon)$ steps for absorbing against $\tilde{O}(d^2/\varepsilon)$ for uniform. That is a genuine theoretical advantage for masking, and it is the best formal justification currently available for the field's consolidation on it.

Early stopping is not optional. The score diverges as $t \to 0$ (we saw why in Section 4.7: near-deterministic data makes neighbouring states' probabilities wildly different), so the analysis holds on $[\delta, T]$ and you pay $\mathrm{TV} \lesssim d\delta$ for the truncation. Without early stopping you need the γ assumption, which Liang et al. argue is close to necessary.

The step count is $\Omega(d)$, always. Every bound in Theorem 13.1 is linear in d at best. Nothing in this theory licenses $N \ll L$. That is consistent with the remark after Equation (4.1): the exact reverse process makes one jump per token.

Other convergence results. There are discrete-time analyses of score-based discrete diffusion (Zhang et al., 2025b), convergence analyses via uniformization, and a growing set of results for discrete flow matching (Su et al., 2025). The picture is broadly consistent.

13.2. Expressivity: the token/sequence separation

The most consequential theoretical result for practice is Feng et al. (2025)'s separation. Their setting is a hidden Markov model with sequence length bounded by L, under an assumption that the trained reverse model is close in KL to the true reverse process (which the ELBO-optimal solution satisfies).

> **Theorem 13.2 (Token versus sequence error, after Feng et al., 2025).**
> 1. **Positive.** Masked diffusion models can achieve near-optimal token error rate (defined via perplexity) in a number of sampling steps that is **constant**, independent of sequence length.
> 2. **Negative.** To achieve low sequence error rate — the probability of generating a sequence outside the target language's support — the required number of sampling steps must, in the worst case, scale **linearly** with sequence length.

Intuition: the separation is about what counts as a failure

Suppose each parallel step introduces an independent per-token error with probability η. Token-level metrics average over positions, so the damage is $O(\eta)$ regardless of L: you can afford to commit many tokens at once. Sequence-level correctness requires *every* token to be right, so the failure probability is $1 - (1-\eta)^L \approx L\eta$, and to keep that bounded you need $\eta = O(1/L)$ — which means committing $O(1)$ tokens per step, hence $\Omega(L)$ steps.

The practical reading: **if your task is judged on whole-answer correctness — a proof, a program, a valid molecule — do not expect few-step parallel sampling to work**, and no improvement to the model will change this. If your task tolerates per-token imperfection, few-step sampling is fine. This single distinction explains most of the apparent contradictions in the empirical literature about diffusion sampling speed.

13.3. Hardness: masked diffusion trains on harder subproblems

Kim et al. (2025) supply the complementary negative result, and it is about *learning* rather than sampling. The finding, in their words, is that "MDMs indeed train on computationally intractable subproblems compared to their autoregressive counterparts". Specifically: for simple distributions with a natural left-to-right order, autoregressive models can provably sample efficiently, yet at certain noise levels a large fraction of the corresponding masked-diffusion subproblems is provably computationally intractable.

The constructive half is the striking part. The hard subproblems are only a fraction, and the model's own logits already contain enough information to *avoid* them by choosing a good

unmasking order. Hence the title: train for the worst (you have no choice — training samples masking patterns uniformly), plan for the best (at inference, pick the easy subproblems). The empirical payoff is Sudoku accuracy from under 7% to around 90%, and beating autoregressive models with $7\times$ the parameters.

Remark 13.3 (The two negative results point the same way). Theorem 13.2 says parallel *sampling* is limited by whole-sequence correctness. Kim et al. (2025) say uniform *training* spends capacity on intractable subproblems. Both say that the uniform measure over masking patterns — which is simply what independent masking gives you, not a choice anyone defended — is the wrong thing to train on and to sample from. That is a strong convergent signal, and it is why "reweight the measure over orderings" and "sampler-aware training" appear as open problems in Chapters 5, 6 and 8. If I had to bet on one direction in this book paying off, it would be that one.

13.4. Tightness of the bound

Theorem 6.1 established that the continuous-time masked diffusion NELBO equals the exact NLL when the denoiser is exact. So for masked models the variational gap is zero at the optimum and any observed gap is model error.

For other noise processes this does not hold, and the gap depends on the latent structure — which is exactly why Pynadath et al. (2026) argue that cross-family likelihood comparison is unsound (Section 10.8). Ivanov et al. (2026) propose an alternative, tangent-based upper bound on the evidence as a sharper diagnostic.

Shi and Titsias (2025) add a result that is both practically useful and mildly surprising: certain *reweighted* losses are *tighter* variational bounds than the canonical weighting. This matters because the folklore (four independent instances, collected in Remark 7.6) is that reweighting improves training at the cost of the bound — and it turns out that is not always a trade-off.

13.5. Schedule optimality

One clean result. Zhang and Syed (2025) show that the cosine schedule is **Fisher–Rao optimal** for masked discrete diffusion models. This is the only schedule-optimality theorem I am aware of in this literature, and it is satisfying for two reasons: it justifies the schedule practitioners already use, and the Fisher–Rao metric is the same geometry that appears in simplex-based flow matching (Section 7.6) — which suggests a deeper unifying story that has not been written.

13.6. Bounding the factorisation error

The most direct theoretical attack on Remark 2.4 is Lavenant and Zanella (2025), and it is the state of the art on the question that matters most for practice.

They split the sampling error in KL into a *learning* term and a *factorisation* term, the latter arising purely from unmasking several tokens per step as if they were conditionally independent. For random-order schedules their main bound is

$$E_{\text{fact}} \;\le\; \big(\mathbb{E}[s_{\max}] - 1\big)\, D(\pi) \;\le\; \big(\mathbb{E}[s_{\max}] - 1\big) \log |\mathcal{X}|, \tag{13.2}$$

where $s_{\max}$ is the largest number of tokens unmasked in a single step and $D(\pi)$ is an average measure of correlation among the coordinates. With near-constant schedules this gives $E_{\text{fact}} \lesssim \lceil (N-K)/K \rceil \log |\mathcal{X}|$ for K steps on N tokens.

Equation (13.2) is exactly the right *shape*: the error is the number of tokens committed per step, times how correlated the data is. If $D(\pi)$ were small you could parallelise freely.

Remark 13.4 (The exact identity behind the bound). The reason Equation (13.2) has that shape is that the quantity it bounds has a closed form. For a *fixed* block partition $S_1, \dots, S_N$ of the positions, and an oracle denoiser that gets every conditional marginal right, Theorem 8.2 gives

$$D_{\mathrm{KL}}(p \,\|\, q) \;=\; \sum_{k=1}^{N} \mathbb{E}_{\boldsymbol{x}^{S_{<k}} \sim p}\Big[\, \mathrm{TC}\left(\boldsymbol{x}^{S_k} \,\middle|\, \boldsymbol{x}^{S_{<k}}\right)\Big], \tag{13.3}$$

with no inequality anywhere. So the factorisation error *is* a sum of conditional total correlations, and $D(\pi)$ in Equation (13.2) is an order-averaged surrogate for it, chosen because it is estimable without knowing the schedule. Three things follow for how to read this literature:

- A lower bound on parallel-decoding error is not a modelling artefact to be engineered away. Equation (13.3) holds for the *best possible* factorised denoiser, so any positive parallel-sampling result must come from the data — correlation that decays with distance, or blocks chosen to cut along weak dependencies — and never from a better architecture.
- The two results are complementary rather than competing: Equation (13.3) is exact but schedule-specific and needs p; Equation (13.2) is a bound but uniform over random-order schedules and expressed in an estimable quantity.
- The conjecture in the open questions below is now precisely stated: it asks for a class of distributions on which the right-hand side of Equation (13.3) is $o(1)$ for partitions with $o(L)$ blocks.

They also derive the optimal schedule. For a fixed step count the best random-order schedule is deterministic; in the scaling limit $N, K \to \infty$ with $N/K \to \infty$ the factorisation error is asymptotically $\frac{1}{2} D(\pi)(N/K) \int_0^1 g(\alpha_t) \dot{\alpha}_t^2 \, \mathrm{d}t$ for a function g built from the information profile, the optimal schedule is $\alpha_t = G^{-1}(t\,G(1))$ with $G(y) = \int_0^y \sqrt{g}$, and its cost relative to the uniform schedule is $(\int_0^1 \sqrt{g})^2 / \int_0^1 g \le 1$ by Jensen. So **the more non-linear the information profile, the more a non-uniform schedule buys you** — a clean and actionable conclusion.

Remark 13.5 (What is still missing, precisely). Equation (13.2) is a bound in terms of a correlation measure, but it is *not* a correlation-decay theorem: the right-hand side is a constant multiple of $D(\pi)$ rather than a term that vanishes as the step count N grows, and the authors are explicit that stronger concentration results are future work. So the positive parallel-sampling result conjectured in the open questions below remains open — but the natural quantity to control, $D(\pi)$, is now identified, and the schedule side of the problem is solved. This is the most promising entry point into the field's central theoretical question.

13.7. Sampler correctness, formally

Tang et al. (2026)'s contribution is empirical but has a theoretical core: matching per-position marginals implies matching the joint only under independence assumptions that natural data violates, so a parallel sampler is provably targeting the wrong distribution, and the discrepancy shrinks only as the step count approaches the sequence length. This is Remark 2.4 plus Theorem 13.2, confirmed by an oracle experiment — and reproduced in miniature in Remark 7.5, where an exact enumerated posterior still puts 72% of mass off-support at two steps.

Table 13.1 What the theory says. "d" is the sequence length.

Question	Answer
Steps for ε-accurate absorbing diffusion?	$\tilde{O}(d/\varepsilon)$ with τ-leaping; $O(d \log\log(d/\varepsilon))$ expected with uniformization (Liang et al., 2025)
Absorbing versus uniform?	absorbing better by a factor of d in step count (Liang et al., 2025)
Can I use $N \ll L$ steps?	for token-level metrics yes, in $O(1)$ steps; for sequence-level correctness no, $\Omega(L)$ in the worst case (Feng et al., 2025)
Is the masked ELBO tight?	yes at the optimum (Theorem 6.1); not comparable across families (Pynadath et al., 2026)
Does reweighting the loss break the bound?	not always — some reweightings are tighter (Shi and Titsias, 2025)
Which schedule?	cosine is Fisher–Rao optimal for masked diffusion (Zhang and Syed, 2025)
Is my sampler correct?	not at low step counts, even with an exact denoiser (Tang et al., 2026)
Why is training hard?	a large fraction of masked subproblems is computationally intractable; inference can route around them (Kim et al., 2025)
How large is the parallel-update error?	at most (tokens per step -1) times a correlation measure $D(\pi)$, and the optimal schedule is known in the scaling limit (Lavenant and Zanella, 2025)

13.8. Summary table

Key takeaways

- Absorbing-state diffusion converges in $\tilde{O}(d/\varepsilon)$ τ-leaping steps, a factor of d better than uniform-state. Early stopping is necessary because the score diverges at $t \to 0$.
- Every convergence bound is $\Omega(d)$ in the step count. Theory does not support few-step sampling.
- Theorem 13.2 is the key practical result: token-level near-optimality in $O(1)$ steps; sequence-level correctness needs $\Omega(L)$ steps in the worst case. Know which regime your task is in before promising speed.
- Masked diffusion trains on a measure over masking patterns that includes computationally intractable subproblems, but inference can avoid them using the model's own confidence (Kim et al., 2025).
- The masked ELBO is tight at the optimum; cross-family likelihood comparison is unsound; and some reweighted losses are tighter bounds than the canonical one.
- The cosine schedule is Fisher–Rao optimal for masked diffusion — the only schedule-optimality result in the literature.

Open questions and research directions

1. **A positive parallel-sampling result.** We have a worst-case $\Omega(L)$ lower bound for sequence-level correctness (Feng et al., 2025) and, now, an upper bound on the factorisation error in terms of a correlation measure $D(\pi)$ (Lavenant and Zanella, 2025). What

is still missing (Remark 13.5) is the step that turns the latter into a *positive* statement: a characterisation of distribution classes for which $o(L)$ parallel steps suffice, presumably under a decay-of-correlation hypothesis that makes $D(\pi)$ small for the relevant subsets rather than on average. The natural statement is "if the conditional total correlation among any k positions given the rest is below ε, then L/k steps suffice". This remains the single most valuable open theoretical problem in the field — it would tell practitioners, from measurable properties of their data, how much parallelism they can afford — and Lavenant and Zanella (2025) have identified the quantity to control and solved the schedule half of it, so the entry point is now clear.

2. **Convergence for uniform-state and interpolating diffusion at modern tightness.** The absorbing analysis is now sharp; the uniform case lags by a factor of d, and it is unclear whether that is intrinsic or an artefact of the proof technique. Given Sahoo et al. (2026)'s empirical finding that uniform-state diffusion is competitive, resolving this matters.
3. **Sample complexity, not just convergence.** All the results above assume a score or denoiser with error $\varepsilon_{\text{score}}$ and bound the sampling error given it. Nobody has bounded how much *data* is needed to reach $\varepsilon_{\text{score}}$ for a natural function class — which is where the 2^L-conditionals capacity problem of Chapter 6 would show up. This is the honest statement of why masked diffusion needs more compute than autoregression, and it is unproved.
4. **Schedule optimality beyond masking.** Zhang and Syed (2025) settle the masked case. Uniform-state, interpolating and general CTMC schedules have no corresponding theory, and the Fisher–Rao framing looks like the right tool to extend.
5. **Formalise the capacity/order trade-off.** Theorem 6.1 says masked diffusion is AO-ARM. Kim et al. (2025) say some orderings are intractable. Block diffusion interpolates by fixing part of the order. A theory that, given a parameter budget and a distribution, says *how much* of the ordering you should fix would unify all three and would directly answer a design question practitioners currently guess at.
6. **Theory for remasking and correctors.** Theorem 8.7 guarantees marginal preservation and Proposition 4.3 guarantees the right fixed point, but neither gives a *rate*. A mixing-time analysis of the corrector generator $\mathbf{R}_t^c$ would convert corrector scheduling from a hyperparameter sweep into a calculation, and Table 8.2 suggests the stakes are large.
7. **Why does the unweighted loss train better?** Raised four times in this book from four different literatures. Shi and Titsias (2025) show the trade-off is not always real. A complete characterisation — which reweightings are bounds, and which minimise estimator variance — is a tractable and overdue piece of theory.

CHAPTER 14

A Research Playbook

CHAPTER

Authors	Synthesis; practical guidance
Year	2026
Published	How to start a project in this area
Tags	research practice, baselines, experiment design, project ideas

This chapter is advice rather than content. It is what I would tell someone starting a PhD project on discrete diffusion in late 2026.

14.1. Build the smallest honest thing first

Before touching a real dataset, build the toy pipeline of Chapter C: a 12-token toy grammar, a tiny transformer, the masked diffusion loss, and an ancestral sampler. It trains in minutes on a laptop. Then verify, in order:

1. **The forward process.** Check $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ against repeated single-step application (Listing 2.1). Check the stationary distribution.
2. **The loss is a bound.** Construct a distribution whose exact NLL you can compute and check that your estimated NELBO exceeds it, and that it *equals* it with an exact denoiser (Listing 6.1). This catches schedule and weighting errors that nothing else will.
3. **SUBS is implemented.** Assert that the model cannot emit `[MASK]` and that unmasked positions are carried over. Both are easy to get wrong silently and both invalidate your bound.
4. **The sampler, against an exact denoiser.** Replace your network with an enumerated exact posterior on a small instance and measure the sampler's error as a function of step count (Remark 7.5). This separates sampler error from model error permanently, and almost nobody does it.

Why this order

Every one of these four checks has caught a real bug in code I have seen (or written). They are cheap, they are decisive, and they are in increasing order of what they let you claim. In particular, step 4 gives you something most papers in this field cannot produce: a quantitative statement about which half of your system is responsible for your results.

14.2. Choosing a problem

Pick a domain where the advantages are real. From Chapters 1 and 12 and Section 12.7: small vocabularies, conditional tasks, no canonical order, property objectives rather than likelihood objectives. Biological sequence design and molecular generation satisfy all four. Text satisfies

none of them cleanly, which is why text results are closely fought and why a text project needs a much larger compute budget to say anything.

Pick a slot in the four-part framework. Forward process, learned object, objective, sampler (Chapter 1). Being explicit about which slot you are pushing on prevents the most common failure mode in this area, which is a paper that changes three things at once and cannot attribute its gains.

Prefer sampler and objective work if you are compute-poor. Inference-time contributions can be evaluated on *other people's checkpoints*. Kim et al. (2025) got one of the largest reported gains in the field with no retraining at all. Wang et al. (2025b) did likewise, and Table 8.2 reproduces the shape of their result on a laptop. By contrast, a pretraining contribution needs to be demonstrated at scale to be believed. And note Pynadath et al. (2026)'s finding that 50k pretraining steps give frontiers close to those at 1M steps — you can iterate on methods much more cheaply than you might think.

14.3. Baselines you must include

A list, because reviewers will ask and because the comparisons are informative.

1. **An autoregressive model at matched parameters and matched tokens.** Not a published number from a different setup. This is the comparison that matters and it is frequently fudged.
2. **MDLM or MD4** as the masked diffusion baseline (Sahoo et al., 2024; Shi et al., 2024).
3. **UDLM** if your vocabulary is small (Schiff et al., 2025) — it is genuinely state of the art there and is often omitted.
4. **Your own method with a *plain* sampler**, so the reader can separate model gains from sampler gains.
5. **Confidence-ordered decoding** on every baseline, not just yours. It is free and it changes rankings.

14.4. Experimental hygiene

The reporting checklist is in Section 10.8 and I will not repeat it. Three additional process points:

Fix the sampler before you compare models. Inference settings move results by more than most architectural choices (Table 8.2). A comparison with free sampler hyperparameters is not a comparison.

Report the ε cutoff and the evaluation discretisation. Both silently change your reported bound. Most papers do not state them, which is why cross-paper perplexity comparisons in this area should be treated with suspicion — including the ones reproduced in this book.

Report per-sample variance. Diffusion sampling is stochastic in several places at once, and dataset-level aggregates hide severe per-generation variability (Fang et al., 2026).

14.5. Fifteen project ideas

Collected from the open-questions boxes, sorted by how much compute they need. These are my judgements about tractability, not endorsements from the literature.

Small (a laptop or one GPU, weeks).

1. **An oracle benchmark suite for samplers.** A family of exactly-solvable distributions (HMMs, grammars, Ising-like models) with exact posteriors and tunable correlation length, against which any sampler can be scored. Not research-hard, immediately useful to everyone, and the obvious systematisation of Tang et al. (2026). (Section 10.8)
2. **Exact versus Taylor guidance at small K.** For DNA and proteins, Equation (9.1) is affordable exactly. Quantify what TAG is costing the biological-design literature. (Chapter 9 and Section 12.7)
3. **Substitution-matrix forward processes for proteins.** Use BLOSUM as $\mathbf{Q}$. The biological analogue of DiGress's marginal matching, with a ready-made rate matrix. (Section 12.7)
4. **Time warping for masked diffusion.** Port CDCD's adaptive t-distribution (Dieleman et al., 2022) to the masked objective, whose estimator variance is a known problem. (Section 7.6 and Chapter 10)
5. **Minimum-Hamming couplings for discrete flow matching.** The discrete analogue of minibatch optimal transport; cheap, and should shorten trajectories. (Chapter 7)
6. **Uniform versus masked for proteins.** A controlled comparison at $K = 20$, where the existing evidence points both ways. (Section 12.7)

Medium (a few GPUs, a term).

7. **Reweight the measure over orderings.** Equation (6.3) exposes the weight on each masked set. Train with a non-uniform measure — spans, or the patterns a confidence sampler visits — and test against Kim et al. (2025)'s hardness story. I think this is the highest-expected-value idea in the book. (Chapters 5, 6 and 13)
8. **Sampler-aware training.** Roll out the inference sampler during training so the model sees the masking distribution it will be evaluated on. Closes a real train/test gap nobody has closed. (Chapter 8)
9. **RL on the decoding policy.** Freeze the denoiser, train the unmasking policy by RL against the task reward. Small action space, clear reward, and a strong prior from Kim et al. (2025) that it will work. (Chapter 9)
10. **Total-correlation-budgeted parallel decoding.** Estimate the dependence among candidate positions and commit a set only if its estimated total correlation is below a tolerance. Turns a heuristic threshold into an error budget. (Chapter 8)
11. **Shared-forward-pass multi-pattern training.** Compute attention once and read out several masking patterns, recovering the half of your tokens that currently produce no gradient. (Chapter 10)
12. **An architecture for the actual task.** Asymmetric treatment of clean versus masked positions, rather than a decoder-only transformer with the causal mask removed. (Chapter 10)

Large (serious compute, or a strong theory background).

13. **A duality for absorbing-state diffusion**, or a proof that none exists. This unlocks consistency distillation for the entire LLaDA/Dream family and is on the critical path for the field's practical case (Remark 5.10). (Section 5.8)
14. **A positive parallel-sampling theorem.** Characterise the distribution classes for which $o(L)$ parallel steps suffice, presumably via decay of correlation. The matching upper bound to Feng et al. (2025)'s lower bound, and the most valuable open theoretical problem here. (Chapter 13)
15. **Structured multi-site forward processes.** A forward process with controlled multi-token jumps has an exact reverse that moves several tokens at once — the only principled escape

from the $\Theta(L)$ barrier. The ELBO machinery already supports it; nobody has built one at scale. (Chapter 4)

14.6. How to position a paper

Three framings that work in this area, and one that does not.

Works: "this is a sampler contribution and here is the oracle experiment". Because sampler claims are cheap to verify and currently under-scrutinised, a paper that isolates its sampler gain credibly stands out.

Works: "this domain has property X, which makes discrete diffusion the right tool". The molecular and biological papers that have landed hardest all do this — GenMol's fragment argument, EvoDiff's disordered-region argument, Dirichlet FM's small-K argument. A domain argument is more durable than a benchmark number.

Works: "the received wisdom is wrong, and here is the measurement". Franca and Tong (2026), Pynadath et al. (2026), Tang et al. (2026) and Ou et al. (2025) are all of this type and all highly cited relative to their age. The field has accumulated enough folklore to make this a productive genre.

Does not work: "our diffusion model beats autoregressive on perplexity". It probably does not at matched scale (Table 5.1), the metric does not transfer across families (Section 10.8), and a reviewer who knows the recent evaluation literature will say so.

Key takeaways

- Build the toy pipeline first and run the four verification steps: forward process, bound-is-a-bound, SUBS, and sampler-against-exact-denoiser.
- Pick a domain where the structural advantages are real — small vocabulary, conditional tasks, no canonical order, property objectives — rather than fighting for perplexity on text.
- Be explicit about which of the four slots (forward process, learned object, objective, sampler) you are changing, and change one at a time.
- If compute-constrained, do sampler or objective work: it evaluates on other people's checkpoints, and 50k pretraining steps suffice for method iteration (Pynadath et al., 2026).
- Fix the sampler before comparing models; report ε, the evaluation discretisation, and per-sample variance.
- The three paper framings that currently work: isolate a sampler gain, make a domain argument, or correct received wisdom with a measurement.

Open questions and research directions

1. **The field needs shared infrastructure.** There is no standard oracle benchmark, no agreed conditional-generation benchmark, no standard sampler-ablation protocol, and no common evaluation harness. Each of these is a service contribution that would be widely used and is not research-hard. Historically such contributions have had outsized influence in young subfields.

2. **Where should the next five years go?** My own reading, offered as a hypothesis rather than a conclusion: the modelling side is close to settled (masked diffusion with an AR initialisation is a good local optimum, and Theorem 6.1 tells us what it is), while the *sampling* side is wide open and is where the structural limitations bite (Theorem 13.2, Tang et al. (2026), Table 8.2). If that is right, the highest-value work is in samplers, distillation, and the training/inference-measure mismatch — not in new objectives.
3. **What would falsify the case for discrete diffusion?** Worth asking explicitly, and worth your taking a position on before you invest years. Two candidates: a demonstration that the step count required for sequence-level correctness is irreducibly $\Theta(L)$ in practice as well as in the worst case, which would eliminate the speed argument; or autoregressive models acquiring order-agnostic conditioning cheaply through training-objective changes (fill-in-the-middle taken seriously, permutation-augmented training), which would eliminate the capability argument. Neither has happened. Both are plausible, and a careful attempt at either would be a valuable paper even — especially — if it failed.

APPENDIX A

Notation

A.1. Spaces and variables

Symbol	Meaning
$\mathcal{V} = \{1, \dots, K\}$	vocabulary; K is the vocabulary size
L (or d)	sequence length. Theory papers usually write d
$\mathcal{X} = \mathcal{V}^L$	state space, $\lvert\mathcal{X}\rvert = K^L$
`[MASK]`, $\mathbf{m} = \mathbf{e}_{\texttt{[MASK]}}$	the mask symbol and its one-hot vector
$\mathbf{e}_x$	one-hot vector for token x
Δ^K	probability simplex in $\mathbb{R}^K$
$\boldsymbol{x}, \boldsymbol{x}_0$	clean data
$\boldsymbol{x}_t, \boldsymbol{z}_t$	noisy state at time t. $\boldsymbol{z}$ is MDLM's notation
$\boldsymbol{x}^\ell$	token at position ℓ
ℓ	position index, $1 \le \ell \le L$
$\mathrm{Ham}(\boldsymbol{x}, \boldsymbol{y})$	Hamming distance
$M \subseteq [L]$	set of masked positions

A.2. Processes

Symbol	Meaning
t	time. $t = 0$ is clean data, $t = 1$ (or T) is noise. **Flow matching reverses this**; Chapter 7 flags it
s	an earlier time, $s < t$
$\mathbf{Q}_t$	discrete-time transition matrix (**row**-stochastic, Chapter 3) or continuous-time rate matrix (**column** sums zero, Chapter 4 onwards)
$\overline{\mathbf{Q}}_t = \mathbf{Q}_1 \cdots \mathbf{Q}_t$	cumulative transition matrix
$\mathbf{R}_t$	rate matrix (Campbell et al.'s notation, row convention)
$\widehat{\mathbf{R}}_t$	reverse-time rate matrix
α_t	survival probability; $\alpha_0 \approx 1$, $\alpha_1 \approx 0$
$\alpha_{t\lvert s} = \alpha_t / \alpha_s$	conditional survival probability
β_t	per-step corruption probability, $\approx 1 - \alpha_{t\lvert t-1}$
$\overline{\sigma}(t)$	total noise; $\alpha_t = e^{-\overline{\sigma}(t)}$ for absorbing noise
$\boldsymbol{\pi}$	limiting / prior distribution of the forward process
$\boldsymbol{m}$	empirical marginal over token types (marginal matching)
κ_t	flow-matching scheduler ($\kappa_0 = 0$, $\kappa_1 = 1$)
$u_t(y, x)$	probability velocity, rate of flow $x \to y$
p_t	marginal distribution at time t
$p_{t\lvert 0}$, $p_{0\lvert t}$	forward kernel and denoising posterior

A.3. Learned objects and losses

Symbol	Meaning
$x_\theta(z_t, t)$	denoiser: predicted distribution over clean tokens, in Δ^K per position
$s_\theta(x, t)_y$	learned concrete score, $\approx p_t(y)/p_t(x)$
p_θ	the model's reverse kernel or implied data distribution
$\mathcal{L}$	negative ELBO, i.e. an upper bound on $-\log p_\theta$
$\mathcal{L}_{\mathrm{SE}}, \mathcal{L}_{\mathrm{ISE}}, \mathcal{L}_{\mathrm{DSE}}$	score entropy, implicit and denoising variants
$K(a) = a(\log a - 1)$	the Bregman generator of score entropy
w_{xy}	score-entropy weights
γ	guidance strength
σ_t	ReMDM remasking probability
ε	time cutoff; the integral runs on $[\varepsilon, 1]$
δ	early-stopping cutoff in convergence analyses
N	number of sampling steps (NFE, network function evaluations)

A.4. Operators

Symbol	Meaning
$\mathrm{Cat}(x; p)$	categorical distribution with probability vector p
$D_{\mathrm{KL}}(q \,\|\, p)$, $\mathrm{TV}(q, p)$	KL divergence, total variation distance
$\langle p, \mathbf{e}_x \rangle = p_x$	inner product picks a coordinate
$\odot$	elementwise (Hadamard) product
$\mathbf{1}$	all-ones vector
$\mathrm{div}_x(v)$	discrete divergence, $\sum_z [v(z, x) - v(x, z)]$
$\exp(\mathbf{A})$	matrix exponential
$\tilde{O}, \lesssim$	up to logarithmic factors / up to constants

A.5. Abbreviations

AO-ARM	any-order autoregressive model
AR	autoregressive
CTMC	continuous-time Markov chain
D3PM	discrete denoising diffusion probabilistic model
DFM	discrete flow matching
dLLM	diffusion large language model
DSE / ISE / DWDSE	denoising / implicit / diffusion-weighted denoising score entropy
ELBO / NELBO	evidence lower bound / its negative
MDM	masked diffusion model
NFE	number of function evaluations (network calls at sampling time)
SUBS	substitution parameterisation (zero mask probability + carry-over unmasking)
TAG	Taylor-approximated guidance

A.6. A reading aid: the same object in four notations

The following all denote the clean-data posterior that the network computes, and you will meet all four.

Notation	Source
$\tilde{p}_\theta(\tilde{\boldsymbol{x}}_0 \mid \boldsymbol{x}_t)$	D3PM (Austin et al., 2021)
$p^\theta_{0\mid t}(\boldsymbol{x}_0 \mid \boldsymbol{x})$	Campbell et al. (2022)
$\boldsymbol{x}_\theta(\boldsymbol{z}_t, t)$	MDLM (Sahoo et al., 2024); μ_θ in MD4
$p_{1\mid t}(\cdot \mid z)$	discrete flow matching (Gat et al., 2024) (note: their $t = 1$ is data)

And the concrete score appears as $\boldsymbol{s}_\theta(\boldsymbol{x}, t)_{\boldsymbol{y}}$ (SEDD), as $q_t(\tilde{x})/q_t(x)$ inside a rate (Campbell et al.), and — for absorbing noise — as $c(t)\, c_\theta(\boldsymbol{x}^{\mathrm{UM}})$ after the RADD factorisation (Ou et al., 2025).

APPENDIX B

Complete Derivations

The derivations deferred from the main text, in full. None requires anything beyond Bayes' rule, the chain rule and patience.

B.1. The diffusion ELBO telescopes

Claim. For a forward Markov chain $q(\boldsymbol{x}_{1:T} \mid \boldsymbol{x}_0) = \prod_{t=1}^{T} q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1})$ and a reverse model $p_\theta(\boldsymbol{x}_{0:T}) = p(\boldsymbol{x}_T) \prod_{t=1}^{T} p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$,

$$-\log p_\theta(\boldsymbol{x}_0) \;\le\; \mathbb{E}_q\Big[\, D_{\mathrm{KL}}(q(\boldsymbol{x}_T \mid \boldsymbol{x}_0) \,\|\, p(\boldsymbol{x}_T)) \;-\; \log p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_1) \\ + \sum_{t=2}^{T} D_{\mathrm{KL}}(q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)) \Big].$$

Proof. Start from Jensen:

$$-\log p_\theta(\boldsymbol{x}_0) = -\log \sum_{\boldsymbol{x}_{1:T}} q(\boldsymbol{x}_{1:T} \mid \boldsymbol{x}_0)\, \frac{p_\theta(\boldsymbol{x}_{0:T})}{q(\boldsymbol{x}_{1:T} \mid \boldsymbol{x}_0)} \;\le\; \mathbb{E}_q\left[-\log \frac{p_\theta(\boldsymbol{x}_{0:T})}{q(\boldsymbol{x}_{1:T} \mid \boldsymbol{x}_0)}\right].$$

Expand both products:

$$\mathbb{E}_q\left[-\log \frac{p_\theta(\boldsymbol{x}_{0:T})}{q(\boldsymbol{x}_{1:T} \mid \boldsymbol{x}_0)}\right] = \mathbb{E}_q\left[-\log p(\boldsymbol{x}_T) - \sum_{t=1}^{T} \log \frac{p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)}{q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1})}\right].$$

Now the key step. For $t \ge 2$, rewrite the forward kernel using Bayes' rule *conditioned on* $\boldsymbol{x}_0$:

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}, \boldsymbol{x}_0) = \frac{q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)\, q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)}{q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_0)}, \tag{B.1}$$

where the first equality is the Markov property. Substituting, the terms $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)/q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_0)$ telescope across $t = 2, \dots, T$, leaving $q(\boldsymbol{x}_T \mid \boldsymbol{x}_0)/q(\boldsymbol{x}_1 \mid \boldsymbol{x}_0)$. Collecting:

$$= \mathbb{E}_q\left[-\log \frac{p(\boldsymbol{x}_T)}{q(\boldsymbol{x}_T \mid \boldsymbol{x}_0)} - \sum_{t=2}^{T} \log \frac{p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)}{q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)} - \log p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_1)\right],$$

and each of the first two groups is an expectation of a log-ratio of two distributions over the same variable, i.e. a KL. This gives the claim. ∎

Remark B.1. Nothing here used the kernel's form. Equation (B.1) is the only non-obvious step, and it is why the *posterior conditioned on* $\boldsymbol{x}_0$ is the central object of every diffusion model: it is what the telescoping produces.

B.2. The D3PM posterior

Claim (Equation (3.2)). With row one-hot vectors and row-stochastic $\mathbf{Q}_t$,

$$q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) = \mathrm{Cat}\left(\boldsymbol{x}_{t-1};\; \frac{\boldsymbol{x}_t \mathbf{Q}_t^{\top} \odot \boldsymbol{x}_0 \overline{\mathbf{Q}}_{t-1}}{\boldsymbol{x}_0 \overline{\mathbf{Q}}_t \boldsymbol{x}_t^{\top}}\right).$$

Proof. By Bayes and the Markov property,

$$q(x_{t-1}=j \mid x_t=k, x_0=i)=\frac{q(x_t=k \mid x_{t-1}=j)\, q(x_{t-1}=j \mid x_0=i)}{q(x_t=k \mid x_0=i)}.$$

Now read off each factor from Equation (3.1): $q(x_t=k \mid x_{t-1}=j)=[Q_t]_{jk}=[Q_t^\top]_{kj}=[x_tQ_t^\top]_j$ when $x_t=e_k$; $q(x_{t-1}=j \mid x_0=i)=[\bar{Q}_{t-1}]_{ij}=[x_0\bar{Q}_{t-1}]_j$; and the denominator is $[\bar{Q}_t]_{ik}=x_0\bar{Q}_tx_t^\top$, a scalar. The numerator is a product of two vectors coordinatewise, hence the $\odot$. ∎

B.3. The masked posterior and the masked NELBO

Done in full in the main text: Proposition 5.1 and Theorem 5.3. The only step worth repeating is the KL computation, because it is where the "log of one coordinate" comes from.

Let $u=\frac{1-\alpha_s}{1-\alpha_t}$ be the shared mass on m and $v=\frac{\alpha_s-\alpha_t}{1-\alpha_t}=1-u$ the mass distributed over $\mathcal{V}$. The true posterior puts $v\,x_w$ on token w and the model puts $v\,[x_\theta]_w$. Then

$$D_{\mathrm{KL}}(q\,\|\,p_\theta)=u\log\frac{u}{u}+\sum_{w\in\mathcal{V}}v\,x_w\log\frac{v\,x_w}{v\,[x_\theta]_w}=v\sum_w x_w\log\frac{x_w}{[x_\theta]_w}.$$

Because x is one-hot, $x_w\log x_w=0$ for every w, and $\sum_w x_w\log[x_\theta]_w=\log\langle x_\theta,x\rangle$ by Equation (2.2). Hence $D_{\mathrm{KL}}(q\,\|\,p_\theta)=-v\log\langle x_\theta,x\rangle$, which is Equation (5.5).

B.4. Time reversal of a CTMC

Done in Theorem 2.12; the computation verifying $\widehat{R}_tp_t=-Q_tp_t$ is given there. The one additional point worth making explicit is the *schedule factor*, which is a common source of error. If $Q_t=\bar{\sigma}'(t)Q$ then the forward rate $x\to y$ is $\bar{\sigma}'(t)[Q]_{yx}$ and the reverse rate is

$$\left[\widehat{R}_t\right]_{yx}=\bar{\sigma}'(t)\,[Q]_{xy}\,\frac{p_t(y)}{p_t(x)}. \tag{B.2}$$

Omitting $\bar{\sigma}'(t)$ gives a reverse process that integrates to the wrong marginals by exactly the factor $\bar{\sigma}'$, which is easy to miss because the qualitative behaviour looks right. Listing 2.2 checks the identity with the factor included, at several times, to machine precision.

B.5. The continuous-time ELBO integrand

Claim. The θ-dependent part of Equation (4.4) has the form $b-a\log b$ with a the true reverse rate and b the model's.

Sketch. Take the discrete-time bound of Section B.1 with uniform steps of size h and expand each KL to first order in h. For a CTMC, the one-step kernel is $\delta_{yx}+h[R]_{yx}+O(h^2)$. For two such kernels with rates a_{yx} and b_{yx},

$$\begin{aligned}D_{\mathrm{KL}}(\delta+ha\,\|\,\delta+hb)&=\sum_y(\delta_{yx}+ha_{yx})\log\frac{\delta_{yx}+ha_{yx}}{\delta_{yx}+hb_{yx}}\\&=\underbrace{\left(1-h\textstyle\sum_{y\neq x}a_{yx}\right)\log\frac{1-h\sum_{y\neq x}a_{yx}}{1-h\sum_{y\neq x}b_{yx}}}_{\text{diagonal}}+\sum_{y\neq x}h\,a_{yx}\log\frac{a_{yx}}{b_{yx}}.\end{aligned}$$

Expanding $\log(1-\epsilon) = -\epsilon + O(\epsilon^2)$, the diagonal term is $h\sum_{y\neq x}(b_{yx} - a_{yx}) + O(h^2)$. Adding the off-diagonal sum and dividing by h gives the integrand

$$\sum_{y\neq x}\Big[b_{yx} - a_{yx} + a_{yx}\log\frac{a_{yx}}{b_{yx}}\Big] = \sum_{y\neq x}\Big[b_{yx} - a_{yx}\log b_{yx} + K(a_{yx})\Big], \tag{B.3}$$

with $K(a) = a(\log a - 1)$. Summing over steps and letting $h \to 0$ turns the sum into $\int_0^T \mathrm{d}t$, which is Equation (4.4). The constant $K(a_{yx})$ does not involve θ. ∎

This is the derivation promised in Chapter 4: score entropy's functional form is *forced* by the continuous-time likelihood bound, not chosen for convenience. Notice also that this is exactly a generalised KL (Bregman) divergence between the unnormalised rate vectors.

B.6. Denoising score entropy recovers the marginal score

Done in Theorem 4.9. Restating the structure, because it is the discrete analogue of the single most important lemma in continuous diffusion: for each fixed pair (x, y) the objective is, as a function of $b = s_\theta(x)_y$,

$$b - \bar{a}\log b + \text{const}, \qquad \bar{a} = \mathbb{E}_{x_0\sim p_{0|t}(\cdot|x)}\left[\frac{p_{t|0}(y \mid x_0)}{p_{t|0}(x \mid x_0)}\right]. \tag{B.4}$$

By Proposition 4.6 the minimiser is $b = \bar{a}$, and $\bar{a} = p_t(y)/p_t(x)$ by Equation (4.3).

B.7. Masked diffusion equals any-order autoregression

Done in full in Theorem 6.1. Two details worth isolating.

The Beta integral. With $\alpha_t = 1 - t$, each position is masked independently with probability t, and the ELBO weight is $1/t$. So the coefficient on a masked set M with $|M| = m$ is

$$\int_0^1 \frac{1}{t}\, t^m (1-t)^{L-m}\,\mathrm{d}t = \int_0^1 t^{m-1}(1-t)^{L-m}\,\mathrm{d}t = B(m, L-m+1) = \frac{(m-1)!\,(L-m)!}{L!}. \tag{B.5}$$

The permutation count. The number of permutations σ of $[L]$ for which the masked set at step i is exactly M (so $i = L - m + 1$) and $\sigma(i) = \ell$ for a given $\ell \in M$ is $(L-m)!\,(m-1)!$: the $L-m$ revealed positions may be ordered arbitrarily, $\sigma(i)$ is forced, and the remaining $m-1$ masked positions may be ordered arbitrarily. Dividing by $L!$ gives the same coefficient.

That the two coefficients agree is the whole content of the theorem. The numbers are checked in Listing 6.1, where all three quantities — the exact NLL, the AO-ARM objective and the masked NELBO — come out as 11.733854.

B.8. Closed-form kernels for uniform and absorbing noise

Proposition 2.9 is proved in Chapter 2; the mechanism in both cases is that the generator is $c(\mathbf{P} - \mathbf{I})$ for an idempotent $\mathbf{P}$, so

$$\exp\big(s\,c(\mathbf{P}-\mathbf{I})\big) = e^{-sc}\exp(sc\,\mathbf{P}) = e^{-sc}\sum_{n\geq 0}\frac{(sc)^n}{n!}\mathbf{P}^n = e^{-sc}\big(\mathbf{I} + (e^{sc}-1)\mathbf{P}\big), \tag{B.6}$$

using $\mathbf{P}^n = \mathbf{P}$ for $n \geq 1$. For uniform noise $\mathbf{P} = \frac{1}{K}\mathbf{1}\mathbf{1}^\top$ and $c = K$; for absorbing noise $\mathbf{P} = \mathbf{e}_{[\mathtt{MASK}]}\mathbf{1}^\top$ and $c = 1$.

B.9. ReMDM's validity bound

Claim (the constraint in Theorem 8.7). The remasking probability must satisfy $\sigma_t \leq \min\{1, (1-\alpha_s)/\alpha_t\}$.

Where the bound comes from. The requirement is that the modified backward kernel remain a valid probability distribution while preserving the forward marginals. Preserving the marginal of $\mathbf{m}$ at time s means the total mass that ends up masked must equal $1-\alpha_s$. That mass has two sources: states already masked at t (mass $1-\alpha_t$, of which a fraction is kept masked) and states decoded at t that we now remask (mass α_t, remasked with probability σ_t). For the second contribution alone not to exceed the budget we need $\sigma_t \alpha_t \leq 1-\alpha_s$, i.e. $\sigma_t \leq (1-\alpha_s)/\alpha_t$; and σ_t is a probability, so also $\sigma_t \leq 1$. Hence the stated bound. Wang et al. (2025b) give the full argument and verify that the resulting marginals match. ∎

Note the behaviour at the endpoints: at t near 1, $\alpha_t \approx 0$ and the bound is vacuous (remask freely — almost everything is masked anyway); at t near 0, $\alpha_t \approx 1$ and $1-\alpha_s \approx 0$, so $\sigma_t \approx 0$ (do not remask — you are nearly finished). This is exactly the annealing that Table 8.2 shows you want.

B.10. Exact guidance

Done in Theorem 9.1. The one-line version: the conditional marginals are $p_t(\boldsymbol{x} \mid c) \propto p_t(\boldsymbol{x}) p(c \mid \boldsymbol{x}, t)$, so the ratio in Equation (2.17) factorises into the unconditional ratio times the likelihood ratio, and the unknown normalising constant cancels because it appears in both numerator and denominator. The entire content of discrete classifier guidance is that a ratio of normalised quantities is a product of ratios.

APPENDIX C

A Working Masked Diffusion Model

This appendix contains a complete, trainable masked diffusion language model in about a hundred lines of PyTorch, plus the decoding experiments of Chapter 8. Everything here was executed; the printed numbers are the numbers the code produced.

C.1. The model

Listing C.1 `appC_masked_lm.py` — a complete masked diffusion model: noise schedule, denoiser with SUBS, the continuous-time NELBO, an ancestral sampler, and a toy task with a global constraint.

```
"""A complete, minimal masked diffusion language model (MDLM / MD4 style).

Trains on a toy grammar and samples from it.  Importable: `get_trained_model()`
trains once and caches the weights to disk, so the decoding experiments of
Chapter 12 can reuse it.

Run directly to train and evaluate:   python3 appC_masked_lm.py
"""
import os
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
V, L, MASK = 16, 12, 16          # vocab 0..15, plus the mask token with id 16
D, NHEAD, NLAYER = 128, 4, 4
CKPT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "mdlm_toy.pt")


# ---------------------------------------------------------------------------- #
# 1.  Noise schedule.  alpha_t = 1 - t  (linear), so the ELBO weight
#     -alpha'_t / (1 - alpha_t)  is exactly  1/t.
# ---------------------------------------------------------------------------- #
def alpha(t):
    return 1.0 - t                       # alpha_0 = 1, alpha_1 = 0


# ---------------------------------------------------------------------------- #
# 2.  Denoiser: a bidirectional transformer with time conditioning.
#     SUBS is enforced at the output: there is no mask logit at all (zero
#     masking probability) and unmasked positions are copied by the sampler
#     (carry-over unmasking).
# ---------------------------------------------------------------------------- #
class Denoiser(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok = nn.Embedding(V + 1, D)
        self.pos = nn.Embedding(L, D)
        self.time = nn.Sequential(nn.Linear(1, D), nn.SiLU(), nn.Linear(D, D))
```

```
        layer = nn.TransformerEncoderLayer(D, NHEAD, 4 * D, batch_first=True,
                                           norm_first=True, dropout=0.0)
        self.body = nn.TransformerEncoder(layer, NLAYER)
        self.head = nn.Linear(D, V)      # no mask logit  ->  SUBS condition 1

    def forward(self, z, t):
        h = self.tok(z) + self.pos(torch.arange(L, device=z.device))
        h = h + self.time(t[:, None])[:, None, :]
        return self.head(self.body(h))   # (B, L, V) logits over clean tokens


# ---------------------------------------------------------------------------- #
# 3.  Forward process + loss.  Each token is masked independently with
#     probability 1 - alpha_t, and
#       L = E_t E_z  (1/t) * sum_{masked l}  -log p_theta(x^l | z_t).
# ---------------------------------------------------------------------------- #
def nelbo(model, x, eps=1e-3):
    B = x.shape[0]
    t = torch.rand(B, device=x.device) * (1 - eps) + eps
    masked = torch.rand(B, L, device=x.device) < (1.0 - alpha(t))[:, None]
    z = torch.where(masked, torch.full_like(x, MASK), x)
    logits = model(z, t)
    ce = F.cross_entropy(logits.reshape(-1, V), x.reshape(-1),
                         reduction="none").reshape(B, L)
    return ((ce * masked).sum(-1) / t).mean()      # nats per sequence


# ---------------------------------------------------------------------------- #
# 4.  Ancestral sampler.  Going from t to s < t, a masked token is revealed
#     with probability (alpha_s - alpha_t) / (1 - alpha_t) and then drawn from
#     p_theta(. | z_t);  unmasked tokens are carried over unchanged.
# ---------------------------------------------------------------------------- #
@torch.no_grad()
def sample(model, n=8, steps=32, device="cpu"):
    z = torch.full((n, L), MASK, dtype=torch.long, device=device)
    ts = torch.linspace(1.0, 0.0, steps + 1, device=device)
    for i in range(steps):
        t, s = ts[i], ts[i + 1]
        p_unmask = ((alpha(s) - alpha(t)) / (1.0 - alpha(t))).clamp(0, 1)
        probs = model(z, t.expand(n)).softmax(-1)
        draw = torch.multinomial(probs.reshape(-1, V), 1).reshape(n, L)
        do = (torch.rand(n, L, device=device) < p_unmask) & (z == MASK)
        z = torch.where(do, draw, z)
    probs = model(z, torch.zeros(n, device=device)).softmax(-1)
    return torch.where(z == MASK, probs.argmax(-1), z)


# ---------------------------------------------------------------------------- #
# 5.  Toy data.  x_l = (start + l * step) mod 8, plus an 8-offset "flag" bit
#     that must be the same at every position -- a global constraint that a
#     factorised parallel update cannot respect.  Exactly 8 * 3 * 2 = 48
#     equally likely sequences, so the data entropy is log 48 = 3.87 nats.
# ---------------------------------------------------------------------------- #
def batch(n):
    start = torch.randint(0, 8, (n, 1))
    step = torch.randint(1, 4, (n, 1))
    seq = (start + step * torch.arange(L)) % 8
    flag = torch.randint(0, 2, (n, 1))
    return seq + 8 * flag

```

```

def valid(x):
    s, f = x % 8, x // 8
    ok_flag = (f == f[:, :1]).all(-1)
    d = (s[:, 1:] - s[:, :-1]) % 8
    ok_step = (d == d[:, :1]).all(-1) & (d[:, 0] >= 1) & (d[:, 0] <= 3)
    return ok_flag & ok_step


def get_trained_model(steps=3000, bs=256, lr=3e-4, verbose=True):
    model = Denoiser()
    if os.path.exists(CKPT):
        model.load_state_dict(torch.load(CKPT))
        return model
    opt = torch.optim.AdamW(model.parameters(), lr=lr)
    for step in range(steps):
        loss = nelbo(model, batch(bs))
        opt.zero_grad(); loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
        if verbose and step % 500 == 0:
            print(f"step {step:5d}   NELBO {loss.item():8.3f} nats/seq",
                  flush=True)
    torch.save(model.state_dict(), CKPT)
    return model


if __name__ == "__main__":
    model = get_trained_model()
    model.eval()
    print(f"\ndata entropy = log 48 = {math.log(48):.3f} nats/seq")
    for steps in (4, 12, 48):
        s = sample(model, n=512, steps=steps)
        print(f"{steps:3d} denoising steps -> "
              f"{100 * valid(s).float().mean():5.1f}% valid")
    print("ground-truth data validity:",
          f"{100 * valid(batch(512)).float().mean():.1f}%")
```

What to look at. Four lines carry the whole method.

- `self.head = nn.Linear(D, V)` — the output head has V logits, not $V + 1$. That is SUBS condition 1 (zero masking probability), enforced architecturally so it cannot be got wrong.
- `((ce * masked).sum(-1) / t).mean()` — the loss. Cross-entropy at masked positions only (which is both SUBS condition 2 and the Rao–Blackwellisation of Section 5.4), divided by t, which is $-\alpha'_t/(1-\alpha_t)$ for the linear schedule. This one line is Equation (5.8).
- `p_unmask = (alpha(s) - alpha(t)) / (1 - alpha(t))` — the sampler. This is Equation (5.3) verbatim.
- `z = torch.where(do, draw, z)` — carry-over unmasking: only currently-masked positions are ever written.

The task. A 12-token sequence encodes an arithmetic progression mod 8 (common difference in $\{1, 2, 3\}$) plus a flag bit that must be the *same at every position*. There are exactly $8 \times 3 \times 2 = 48$ valid sequences, so the data entropy is $\log 48 = 3.871$ nats per sequence. The flag is the interesting part: it is a global constraint that no local computation can satisfy, so a factorised parallel update will violate it.

Output.

```
step    0            NELBO  45.310 nats/seq
step  500            NELBO    3.774 nats/seq
step 1000            NELBO    3.690 nats/seq
step 1500            NELBO    4.546 nats/seq
step 2000            NELBO    3.069 nats/seq
step 2500            NELBO    3.236 nats/seq
data entropy = log 48 = 3.871 nats/seq
  4 denoising steps   →   6.2% valid
 12 denoising steps   →  44.1% valid
 48 denoising steps   →  82.0% valid
ground-truth data     → 100.0% valid
```

Remark C.1 (Reading the NELBO). The reported NELBO fluctuates around and sometimes below $\log 48 = 3.871$ (note the 4.546 at step 1500, after 3.690 at step 1000). That is not a violation of the bound: the printed value is a single-batch Monte Carlo estimate of an expectation, and the $1/t$ weight gives it high variance (the same phenomenon as the 11.53-versus-11.73 discrepancy in Listing 6.1). In expectation the bound holds. If you want a number you can quote, average over many batches and many t values, and state your ε cutoff.

C.2. Decoding experiments

Listing C.2 `ch12_decoding.py` — random versus confidence-ordered decoding, with optional ReMDM remasking, at matched NFE.

```python
"""Decoding strategies for a masked diffusion model, at matched NFE.

Imports the trained toy model of Appendix C and compares
  (a) random unmasking order            -- what the ELBO prescribes
  (b) confidence-ordered unmasking      -- top-probability first
  (c) confidence order + ReMDM remasking
on the number of samples that satisfy the data's global constraints.
"""
import torch

from appC_masked_lm import (Denoiser, L, V, MASK, alpha, batch, valid,
                            get_trained_model)

torch.manual_seed(0)
model = get_trained_model()           # trains once and caches to disk
model.eval()


@torch.no_grad()
def decode(model, n, steps, order="confidence", sigma=0.0):
    """Unmask ceil(L/steps) tokens per step.

    order  : "random" or "confidence"
    sigma  : ReMDM remasking probability applied to already-decoded tokens.
             Theorem 3.1 of Wang et al. (2025) requires
             0 <= sigma <= min{1, (1 - alpha_s) / alpha_t}.
    """
    z = torch.full((n, L), MASK, dtype=torch.long)
    per = max(1, (L + steps - 1) // steps)
    nsteps = (L + per - 1) // per
    for i in range(nsteps):
```

```
        t = max(1e-3, 1.0 - i / nsteps)
        s = max(0.0, 1.0 - (i + 1) / nsteps)
        probs = model(z, torch.full((n,), t)).softmax(-1)
        conf, _ = probs.max(-1)
        draw = torch.multinomial(probs.reshape(-1, V), 1).reshape(n, L)

        # --- optional ReMDM corrector step: remask some decoded tokens -----
        if sigma > 0.0 and i > 0:
            cap = min(1.0, (1.0 - alpha(s)) / max(alpha(t), 1e-6))
            sig = min(sigma, cap)
            remask = (torch.rand(n, L) < sig) & (z != MASK)
            z = torch.where(remask, torch.full_like(z, MASK), z)

        # --- choose which masked positions to commit -----------------------
        score = torch.rand_like(conf) if order == "random" else conf.clone()
        score[z != MASK] = -1.0
        k = min(per, int((z == MASK).sum(-1).max().item()))
        if k == 0:
            continue
        idx = score.topk(k, dim=-1).indices
        pick = torch.zeros_like(z, dtype=torch.bool).scatter_(1, idx, True)
        pick &= (z == MASK)
        z = torch.where(pick, draw, z)

    # fill anything still masked
    probs = model(z, torch.full((z.shape[0],), 1e-3)).softmax(-1)
    return torch.where(z == MASK, probs.argmax(-1), z)


print(f"{'NFE':>5} {'random':>10} {'confidence':>12} {'conf+ReMDM':>12}")
for steps in (2, 3, 4, 6, 12):
    r = valid(decode(model, 1024, steps, "random")).float().mean()
    c = valid(decode(model, 1024, steps, "confidence")).float().mean()
    m = valid(decode(model, 1024, steps, "confidence", sigma=0.3)).float().mean()
    print(f"{steps:5d} {100 * r:9.1f}% {100 * c:11.1f}% {100 * m:11.1f}%")
print(f"{'data':>5} {100 * valid(batch(1024)).float().mean():9.1f}%")
```

Output (reproduced as Table 8.2):

NFE	random	confidence	conf+ReMDM
2	0.0%	0.0%	0.0%
3	0.0%	0.1%	2.3%
4	1.5%	0.8%	23.6%
6	16.2%	17.7%	66.0%
12	97.8%	99.3%	95.9%
data	100.0%		

C.3. Checking the theory

Every theorem, proposition and corollary in this book that admits a finite check is checked numerically by the following script. It is deliberately written against the *statements* rather than against the proofs — brute-force enumeration where enumeration is possible, dense linear algebra otherwise — so that it can catch an error in a derivation rather than confirming one.

Listing C.3 `verify_theory.py` — a numerical check of each theoretical result in this book: the Cat mixture property (Proposition 2.1), closure of the idempotent noise families (Theorem 3.3), the generator spectra and exact contraction rates (Proposition 2.10), flux reversal and the corrected adjoint identity (Proposition 2.13), f-divergence contraction (Proposition 2.11), score entropy as a Bregman divergence (Proposition 4.7), the general interpolating posterior (Theorem 5.5), the parallel-decoding total-correlation identity (Theorem 8.2), the any-order Jensen gap (Proposition 6.2), the gauge freedom of generating velocities (Proposition 7.2), graph orbit sizes (Corollary 12.2), the γ-guidance bias (Proposition 9.2) and the generative-perplexity decomposition (Proposition 10.2).

```python
"""Numerical verification of the theoretical results added to the book.

Every proposition and theorem that admits a finite check is checked here.
Run:  python3 verify_theory.py
"""
import itertools
import math

import numpy as np
from scipy.linalg import expm

rng = np.random.default_rng(0)
ok = lambda name, cond: print(f"  [{'OK ' if cond else 'FAIL'}] {name}")


# ============================================================================ #
print("\nProposition: Cat is affine in its parameter (mixture property)")
# Cat(x; sum_i lam_i p_i) = sum_i lam_i Cat(x; p_i)
K = 6
ps = rng.dirichlet(np.ones(K), size=4)
lam = rng.dirichlet(np.ones(4))
mix_param = lam @ ps                        # mix the parameters
mix_dist = sum(l * p for l, p in zip(lam, ps))   # mix the distributions
ok("Cat(.; sum lam p) == sum lam Cat(.; p)", np.allclose(mix_param, mix_dist))


# ============================================================================ #
print("\nTheorem: idempotent-M families are closed under products")
# Q(beta) = (1-beta) I + beta M  with M row-stochastic and M^2 = M.
# Claim:  Q(b1) Q(b2) = Q(1 - (1-b1)(1-b2)),  hence Qbar_t = abar_t I + (1-abar_t) M
one = np.ones((K, 1))
m_marg = rng.dirichlet(np.ones(K))
Ms = {
    "uniform  M = 11^T/K": np.ones((K, K)) / K,
    "absorbing M = 1 e_m^T": one @ np.eye(K)[[K - 1]],
    "marginal  M = 1 m^T": one @ m_marg[None, :],
}
Q = lambda b, M: (1 - b) * np.eye(K) + b * M
for name, M in Ms.items():
    idem = np.allclose(M @ M, M)
    stoch = np.allclose(M.sum(1), 1)
    b1, b2 = 0.3, 0.45
    closed = np.allclose(Q(b1, M) @ Q(b2, M), Q(1 - (1 - b1) * (1 - b2), M))
    betas = rng.uniform(0.02, 0.3, 40)
    acc = np.eye(K)
    for b in betas:
        acc = acc @ Q(b, M)
    abar = np.prod(1 - betas)
    cum = np.allclose(acc, abar * np.eye(K) + (1 - abar) * M)
    ok(f"{name}: M^2=M {idem}, rows sum 1 {stoch}, closed {closed}, "
       f"Qbar = abar I + (1-abar) M {cum}", idem and stoch and closed and cum)
```

```

# ========================================================================== #
print("\nProposition: spectra of the uniform and absorbing generators")
Qu = np.ones((K, K)) - K * np.eye(K)              # columns sum to zero
em = np.zeros((K, 1)); em[K - 1] = 1
Qa = em @ np.ones((1, K)) - np.eye(K)
eu = np.sort(np.linalg.eigvals(Qu).real)
ea = np.sort(np.linalg.eigvals(Qa).real)
ok(f"uniform  spectrum = {{0, -K}} with -K of multiplicity K-1   {eu.round(6)}",
   np.allclose(np.sort(eu), np.sort([-K] * (K - 1) + [0.0])))
ok(f"absorbing spectrum = {{0, -1}} with -1 of multiplicity K-1  {ea.round(6)}",
   np.allclose(np.sort(ea), np.sort([-1.0] * (K - 1) + [0.0])))
# exact contraction rates
p0 = rng.dirichlet(np.ones(K) * 0.5)
for s in (0.2, 0.7, 2.0):
    pu = expm(s * Qu) @ p0
    pa = expm(s * Qa) @ p0
    dev_u = np.abs(pu - np.ones(K) / K).sum()
    dev_u0 = np.abs(p0 - np.ones(K) / K).sum()
    #  1 - p_s(m) = e^{-s} (1 - p_0(m)):  the unabsorbed mass decays by
    #  exactly the survival factor alpha_s, whatever p_0 is.
    dev_a = 1 - pa[K - 1]
    pred_a = math.exp(-s) * (1 - p0[K - 1])
    ok(f"s={s}: ||p_s - pi||_1 = e^{{-Ks}}||p_0 - pi||_1  "
       f"({dev_u:.6f} vs {math.exp(-K * s) * dev_u0:.6f})",
       abs(dev_u - math.exp(-K * s) * dev_u0) < 1e-9)
    ok(f"s={s}: unabsorbed mass = e^{{-s}}(1 - p_0(m))  "
       f"({dev_a:.6f} vs {pred_a:.6f})", abs(dev_a - pred_a) < 1e-9)


# ========================================================================== #
print("\nProposition: time reversal is an adjoint, up to d/dt log p_t")
# A dedicated stream, so that the draws below stay reproducible.
rng2 = np.random.default_rng(7)


def rand_generator(n, sym=False):
    """A valid rate matrix in the column convention (columns sum to zero)."""
    A = rng2.uniform(0.1, 1.0, (n, n))
    if sym:
        A = (A + A.T) / 2
    np.fill_diagonal(A, 0.0)
    np.fill_diagonal(A, -A.sum(0))
    return A


def reverse_rates(Qm, p):
    """Rhat[y,x] = Q[x,y] p(y)/p(x) off-diagonal; diagonal by zero column sum."""
    n = len(p)
    R = np.array([[Qm[xi, yi] * p[yi] / p[xi] if xi != yi else 0.0
                   for xi in range(n)] for yi in range(n)])
    np.fill_diagonal(R, -R.sum(0))
    return R


Qt = rand_generator(K)
pt = rng2.dirichlet(np.ones(K))
Rhat = reverse_rates(Qt, pt)
f, g = rng2.normal(size=K), rng2.normal(size=K)
```

```
# (i) flux reversal: Rhat[y,x] p(x) == Q[x,y] p(y) off the diagonal
flux = all(abs(Rhat[yi, xi] * pt[xi] - Qt[xi, yi] * pt[yi]) < 1e-12
           for xi in range(K) for yi in range(K) if xi != yi)
ok("flux reversal  Rhat[y,x] p(x) = Q[x,y] p(y)", flux)
# (ii) <L f, g>_p  ==  <f, (Lhat + dt log p) g>_p   with dt p = Q p
dtp = Qt @ pt
lhs = float(np.sum(pt * g * (Qt.T @ f)))
rhs = float(np.sum(pt * f * (Rhat.T @ g)) + np.sum(f * g * dtp))
ok(f"<Lf,g>_p = {lhs:.10f} == <f,(Lhat + dt log p)g>_p = {rhs:.10f}",
   abs(lhs - rhs) < 1e-10)
# naive claim without the correction term is FALSE -- check that it is
naive = float(np.sum(pt * f * (Rhat.T @ g)))
ok(f"...and the uncorrected adjoint identity genuinely fails "
   f"({naive:.6f} != {lhs:.6f})", abs(naive - lhs) > 1e-3)
# (iii) stationary p => exact adjoint; detailed balance => self-adjoint
w, V = np.linalg.eig(Qt)
pi_t = np.real(V[:, np.argmin(np.abs(w))]); pi_t = pi_t / pi_t.sum()
Rstat = reverse_rates(Qt, pi_t)
lhs_s = float(np.sum(pi_t * g * (Qt.T @ f)))
rhs_s = float(np.sum(pi_t * f * (Rstat.T @ g)))
ok(f"stationary p: exact adjoint ({lhs_s:.10f} == {rhs_s:.10f})",
   abs(lhs_s - rhs_s) < 1e-10)
Qsym = rand_generator(K, sym=True)          # symmetric Q is uniform-reversible
ok("detailed balance: Rhat == Q (the process is its own reversal)",
   np.allclose(reverse_rates(Qsym, np.ones(K) / K), Qsym))


# ============================================================================ #
print("\nProposition: the forward process contracts every f-divergence")
# KL(p_t || p'_t) is non-increasing under a shared forward kernel
pA, pB = rng2.dirichlet(np.ones(K) * 0.3), rng2.dirichlet(np.ones(K) * 0.3)
kls, tvs = [], []
for s in np.linspace(0.0, 2.0, 9):
    P = expm(s * (np.ones((K, K)) - K * np.eye(K)))
    a, b = P @ pA, P @ pB
    kls.append(float(np.sum(a * np.log(a / b))))
    tvs.append(float(0.5 * np.abs(a - b).sum()))
ok(f"KL non-increasing: {np.round(kls, 5).tolist()}",
   all(kls[i + 1] <= kls[i] + 1e-12 for i in range(len(kls) - 1)))
ok(f"TV non-increasing: {np.round(tvs, 5).tolist()}",
   all(tvs[i + 1] <= tvs[i] + 1e-12 for i in range(len(tvs) - 1)))


# ============================================================================ #
print("\nProposition: score entropy is the Bregman divergence of phi(b)=b log b - b")
# D_phi(a || b) = phi(a) - phi(b) - phi'(b)(a - b),  phi'(b) = log b
# claim: D_phi(a || b) == b - a log b + K(a),  K(a) = a(log a - 1)
for a, b in [(0.3, 1.7), (2.0, 0.4), (12.0, 9.0), (1e-3, 5.0)]:
    phi = lambda z: z * math.log(z) - z
    breg = phi(a) - phi(b) - math.log(b) * (a - b)
    se = b - a * math.log(b) + a * (math.log(a) - 1)
    ok(f"a={a}, b={b}: Bregman {breg:.10f} == score entropy {se:.10f}",
       abs(breg - se) < 1e-12)


# ============================================================================ #
print("\nTheorem: the general interpolating posterior")
# q(z_t | x) = Cat(alpha_t x + (1 - alpha_t) pi)
# claim:  q(z_s | z_t, x) propto [a_ts e_{z_t} + (1-a_ts) pi_{z_t} 1] * [a_s x + (1-a_s) pi]
def check_posterior(pi, a_s, a_t, x_idx, zt_idx):
```

```
    a_ts = a_t / a_s
    x = np.eye(K)[x_idx]
    # brute force: enumerate z_s
    num = np.zeros(K)
    for k in range(K):
        q_zs = a_s * x[k] + (1 - a_s) * pi[k]                 # q(z_s = k | x)
        q_zt = a_ts * (1.0 if k == zt_idx else 0.0) + (1 - a_ts) * pi[zt_idx]
        num[k] = q_zs * q_zt
    brute = num / num.sum()
    # closed form
    left = a_ts * np.eye(K)[zt_idx] + (1 - a_ts) * pi[zt_idx] * np.ones(K)
    right = a_s * x + (1 - a_s) * pi
    closed = left * right
    norm = a_t * x[zt_idx] + (1 - a_t) * pi[zt_idx]
    return np.allclose(brute, closed / norm), np.allclose(num.sum(), norm)

pi_mask = np.eye(K)[K - 1]
pi_unif = np.ones(K) / K
pi_rand = rng.dirichlet(np.ones(K))
for name, pi in [("mask", pi_mask), ("uniform", pi_unif), ("random", pi_rand)]:
    allok = True
    for x_idx in range(K):
        for zt_idx in range(K):
            if pi[zt_idx] == 0 and zt_idx != x_idx:
                continue                                  # zero-probability state
            a, b = check_posterior(pi, 0.7, 0.35, x_idx, zt_idx)
            allok &= a and b
    ok(f"pi = {name}: closed form matches brute-force Bayes, normaliser correct",
       allok)

# the masked specialisation quoted in the book
a_s, a_t = 0.7, 0.35
left = (a_t / a_s) * np.eye(K)[K - 1] + (1 - a_t / a_s) * 1.0 * np.ones(K)
right = a_s * np.eye(K)[2] + (1 - a_s) * pi_mask
post = left * right / (1 - a_t)
expected = ((a_s - a_t) * np.eye(K)[2] + (1 - a_s) * pi_mask) / (1 - a_t)
ok("masked case reduces to ((a_s-a_t)x + (1-a_s)m)/(1-a_t)",
   np.allclose(post, expected))


# ============================================================================ #
print("\nTheorem: parallel-decoding error = sum of conditional total correlations")
# KL(p || q) = sum_k E[ TC(X_{S_k} | X_{S_<k}) ]
L, V = 5, 3
joint = rng.dirichlet(np.ones(V ** L) * 0.4).reshape((V,) * L)
seqs = list(itertools.product(range(V), repeat=L))


def marg(idx_vals):
    """P(x_i = v for (i,v) in idx_vals)."""
    tot = 0.0
    for s in seqs:
        if all(s[i] == v for i, v in idx_vals.items()):
            tot += joint[s]
    return tot


def sampler_dist(partition):
    """Distribution produced by revealing each block from exact marginals."""
    q = {}
```

```
    for s in seqs:
        prob, observed = 1.0, {}
        for block in partition:
            denom = marg(observed) if observed else 1.0
            for i in block:                       # independent within a block
                prob *= marg({**observed, i: s[i]}) / denom
            observed.update({i: s[i] for i in block})
        q[s] = prob
    return q


def total_corr_sum(partition):
    """sum_k E[ TC(X_{S_k} | X_{S_<k}) ], expectation under the TRUE joint."""
    total = 0.0
    for k, block in enumerate(partition):
        prev = [i for b in partition[:k] for i in b]
        for s in seqs:
            if joint[s] == 0:
                continue
            observed = {i: s[i] for i in prev}
            denom = marg(observed) if observed else 1.0
            p_joint = marg({**observed, **{i: s[i] for i in block}}) / denom
            p_prod = np.prod([marg({**observed, i: s[i]}) / denom
                              for i in block])
            total += joint[s] * math.log(p_joint / p_prod)
    return total


for partition in [[[0], [1], [2], [3], [4]],            # fully serial
                  [[0, 1], [2, 3], [4]],                 # two at a time
                  [[0, 1, 2], [3, 4]],
                  [[0, 1, 2, 3, 4]]]:                    # one shot
    q = sampler_dist(partition)
    kl = sum(joint[s] * math.log(joint[s] / q[s]) for s in seqs if joint[s] > 0)
    tc = total_corr_sum(partition)
    ok(f"partition {partition}:  KL(p||q) = {kl:.10f}  ==  sum TC = {tc:.10f}",
       abs(kl - tc) < 1e-9)

# corollary: one-shot error is the total correlation of the whole sequence
H_joint = -sum(joint[s] * math.log(joint[s]) for s in seqs if joint[s] > 0)
H_marg = 0.0
for i in range(L):
    for v in range(V):
        pv = marg({i: v})
        if pv > 0:
            H_marg -= pv * math.log(pv)
q = sampler_dist([[0, 1, 2, 3, 4]])
kl = sum(joint[s] * math.log(joint[s] / q[s]) for s in seqs if joint[s] > 0)
ok(f"one-shot KL = sum_i H(X_i) - H(X) = {H_marg - H_joint:.10f}",
   abs(kl - (H_marg - H_joint)) < 1e-9)


# ============================================================================ #
print("\nProposition: the any-order ELBO gap is a Jensen gap")
# NELBO_AO(x) - (-log p_mix(x)) = log p_mix(x) - (1/L!) sum_sigma log p_sigma(x)
# with p_mix = (1/L!) sum_sigma p_sigma.  Build deliberately INCONSISTENT
# conditionals so the gap is strictly positive.
Ls, Vs = 4, 2
true = rng.dirichlet(np.ones(Vs ** Ls) * 0.5).reshape((Vs,) * Ls)
seqs_s = list(itertools.product(range(Vs), repeat=Ls))
```

```
# a "model": true conditionals perturbed independently per conditioning set
pert = {}


def model_cond(i, observed, v):
    key = (i, tuple(sorted(observed.items())))
    if key not in pert:
        pert[key] = rng.uniform(0.5, 1.5, Vs)
    num = np.array([sum(true[s] for s in seqs_s
                        if s[i] == w and all(s[j] == u for j, u in observed.items()))
                    for w in range(Vs)])
    num = num * pert[key]
    return num[v] / num.sum()


x = seqs_s[rng.integers(len(seqs_s))]
logps = []
for sigma in itertools.permutations(range(Ls)):
    acc, observed = 0.0, {}
    for i in sigma:
        acc += math.log(model_cond(i, observed, x[i]))
        observed[i] = x[i]
    logps.append(acc)
logps = np.array(logps)
nelbo_ao = -logps.mean()
log_pmix = math.log(np.exp(logps).mean())
gap = nelbo_ao + log_pmix
jensen = log_pmix - logps.mean()
ok(f"gap {gap:.10f} == Jensen gap {jensen:.10f} >= 0",
   abs(gap - jensen) < 1e-12 and gap >= -1e-12)
# and it vanishes exactly when the orderings agree
ok(f"gap > 0 for inconsistent conditionals (spread of log p_sigma = "
   f"{logps.std():.4f})", gap > 1e-6)


# ============================================================================ #
print("\nProposition: generating velocities are unique only for K <= 2")
# The continuity equation constrains only the DIVERGENCE of the net flux, so
# the solution set is an affine space whose direction is the cycle space of
# the state graph: dimension E - (K-1) = (K-1)(K-2)/2 on the complete graph.
for Kv in range(2, 8):
    edges = list(itertools.combinations(range(Kv), 2))
    Div = np.zeros((Kv, len(edges)))
    for e, (va, vb) in enumerate(edges):
        Div[va, e], Div[vb, e] = +1.0, -1.0      # net flow va -> vb
    rank = np.linalg.matrix_rank(Div)
    nullity = len(edges) - rank
    ok(f"K={Kv}: div has rank K-1={Kv-1} ({rank}), divergence-free net flows "
       f"have dim (K-1)(K-2)/2={(Kv-1)*(Kv-2)//2} ({nullity})",
       rank == Kv - 1 and nullity == (Kv - 1) * (Kv - 2) // 2)
# and an explicit pair of distinct velocities generating the same path
Kv = 3
pv = rng2.dirichlet(np.ones(Kv))
Uone = rand_generator(Kv)                        # some rate matrix
cycle = np.array([[0.0, -1.0, 1.0],              # the single 3-cycle, as rates
                  [1.0, 0.0, -1.0],
                  [-1.0, 1.0, 0.0]])
# add the cycle as a FLUX (divide by p to turn a flux into a rate), scaled so
# that every off-diagonal rate stays positive
scale = 0.5 * min(Uone[i, j] * pv[j]
```

```
                  for i in range(Kv) for j in range(Kv) if i != j)
Utwo = Uone + scale * cycle / pv[None, :]
np.fill_diagonal(Utwo, 0.0)
np.fill_diagonal(Utwo, -Utwo.sum(0))
ok("the two velocities genuinely differ",
   not np.allclose(Uone, Utwo)
   and all(Utwo[i, j] > 0 for i in range(Kv) for j in range(Kv) if i != j))
ok(f"...yet both give the same dp/dt  (max diff "
   f"{np.abs(Uone @ pv - Utwo @ pv).max():.2e})",
   np.allclose(Uone @ pv, Utwo @ pv))


# ============================================================================ #
print("\nCorollary: orbit size of a labelled graph is n!/|Aut(G)|")
# Checked by brute-force enumeration of S_n on small graphs.
def orbit_and_aut(n, edge_set):
    edges = frozenset(frozenset(e) for e in edge_set)
    orbit, aut = set(), 0
    for perm in itertools.permutations(range(n)):
        img = frozenset(frozenset((perm[u], perm[v])) for u, v in edge_set)
        orbit.add(img)
        if img == edges:
            aut += 1
    return len(orbit), aut


GRAPHS = {
    "triangle (n=3)":        (3, [(0, 1), (1, 2), (0, 2)]),
    "path  0-1-2 (n=3)":     (3, [(0, 1), (1, 2)]),
    "star  K_1,3 (n=4)":     (4, [(0, 1), (0, 2), (0, 3)]),
    "path  0-1-2-3 (n=4)":   (4, [(0, 1), (1, 2), (2, 3)]),
    "cycle C_4 (n=4)":       (4, [(0, 1), (1, 2), (2, 3), (3, 0)]),
    "asymmetric (n=5)":      (5, [(0, 1), (1, 2), (2, 3), (3, 4), (1, 3)]),
}
for gname, (n, es) in GRAPHS.items():
    orb, aut = orbit_and_aut(n, es)
    fact = math.factorial(n)
    ok(f"{gname}: |orbit|={orb} == n!/|Aut|={fact}/{aut}={fact // aut}",
       aut > 0 and orb * aut == fact)


# ============================================================================ #
print("\nProposition: gamma-guidance is exact only at gamma = 1")
# Reverse rates  R^g[y,x] = (p_t(y)/p_t(x)) (w_t(y)/w_t(x))^g Q[x,y]  have the
# form of a reversal of the tilted family  mu_t propto p_t w_t^g,  but mu_t
# solves the forward Kolmogorov equation only when g = 1.  So integrating the
# guided reverse process from mu_T should land on mu_0 only for g = 1.
Kg, Tend, EPS = 5, 1.0, 1e-3
Qg = np.ones((Kg, Kg)) - Kg * np.eye(Kg)            # uniform generator
p0g = np.random.default_rng(3).dirichlet(np.ones(Kg) * 0.4)
w0g = np.array([0.02, 1.0, 0.05, 0.6, 0.3])         # clean classifier p(c|x_0)
bias = {}
for Nsteps in (20_000, 80_000):
    tsg = np.linspace(Tend, EPS, Nsteps + 1)
    PTg = np.empty((Nsteps + 1, Kg))
    WTg = np.empty((Nsteps + 1, Kg))
    for i, t in enumerate(tsg):
        Pm = expm(t * Qg)
        ptg = Pm @ p0g
        PTg[i] = ptg
```

```
        #  p(c | x_t = y, t) = sum_{x0} p_{0|t}(x0 | y) p(c | x0)
        WTg[i] = [(Pm[y, :] * p0g / ptg[y]) @ w0g for y in range(Kg)]
    tilt = lambda i, g: (PTg[i] * WTg[i] ** g) / (PTg[i] * WTg[i] ** g).sum()
    for g in (1.0, 2.0, 4.0):
        nu = tilt(0, g).copy()
        for i in range(Nsteps):
            ratio = (np.outer(PTg[i], 1 / PTg[i])
                     * np.outer(WTg[i] ** g, 1 / WTg[i] ** g))
            Rg = ratio * Qg.T                       # Rg[y,x]: rate x -> y
            np.fill_diagonal(Rg, 0.0)
            np.fill_diagonal(Rg, -Rg.sum(0))
            nu = np.clip(nu + (tsg[i] - tsg[i + 1]) * (Rg @ nu), 1e-300, None)
            nu /= nu.sum()
        bias[(Nsteps, g)] = 0.5 * float(np.abs(nu - tilt(Nsteps, g)).sum())
ok(f"gamma=1 is exact: TV -> 0 as steps grow "
   f"({bias[(20_000, 1.0)]:.6f} -> {bias[(80_000, 1.0)]:.6f})",
   bias[(80_000, 1.0)] < bias[(20_000, 1.0)] / 2 and bias[(80_000, 1.0)] < 1e-4)
for g in (2.0, 4.0):
    coarse, fine = bias[(20_000, g)], bias[(80_000, g)]
    ok(f"gamma={g} bias is NOT discretisation: TV stays at "
       f"{coarse:.6f} -> {fine:.6f}",
       fine > 1e-3 and abs(fine - coarse) < 0.05 * fine)


# ============================================================================ #
print("\nProposition: generative-perplexity decomposition")
# KL(p || r) = CE(p, r) - H(p)
p = rng.dirichlet(np.ones(20))
r = rng.dirichlet(np.ones(20))
kl = sum(p * np.log(p / r))
ce = -sum(p * np.log(r))
H = -sum(p * np.log(p))
ok(f"KL {kl:.10f} == CE {ce:.10f} - H {H:.10f}", abs(kl - (ce - H)) < 1e-12)

print("\nall checks complete")
```

All 52 checks pass. Four of them are worth singling out, because they caught real errors in earlier drafts of this book rather than merely confirming algebra:

- The adjoint identity of Proposition 2.13 was first written without the $\partial_t \log p_t$ correction — the form that is correct for a *stationary* chain and appears in most textbook treatments. The script reports both the corrected identity and the failure of the uncorrected one, so the distinction cannot quietly go missing again.
- The absorbing contraction rate was initially stated as a decay of $\|p_s - \delta_{[\mathrm{MASK}]}\|_1$ with the wrong constant; the correct statement is Equation (2.16), about the *unabsorbed mass*, and the test is what forced it.
- The total-correlation identity (Theorem 8.2) is checked against brute-force enumeration over 3^5 sequences for four different block partitions, including the two degenerate ones, rather than against a re-derivation of the same formula.
- The γ-guidance bias (Proposition 9.2) is measured at two step counts rather than one, because a single number could not distinguish a genuine bias from Euler error — and distinguishing them is the entire content of the result.

C.4. Other listings in this book

All code is in the `code/` directory of this project and all of it runs with only `numpy`, `scipy` and `torch`.

File	What it demonstrates	Chapter
`ch02_d3pm_matrices.py`	transition matrices, marginals, exact posterior versus brute-force Bayes	2, 3
`ch02_ctmc.py`	the time-reversal identity to machine precision; reverse-ODE integration; the DSE minimiser equals the true ratio	2, 4.7
`ch04_marginal_Q.py`	marginal-matched noise keeps noised graphs sparse (19 versus 142 edges)	3, 12
`ch06_score_entropy.py`	training a concrete-score network with DSE; relative score error falls to $\approx 1\%$	4.7
`ch09_equivalence.py`	masked NELBO = any-order AR NLL = exact NLL, to nine decimals	6
`ch10_dfm.py`	discrete flow matching with an *exact* denoiser: 28.4% on-support at 2 steps, 99.5% at 128	7, 8
`ch12_decoding.py`	decoding orders and remasking at matched NFE	8
`ch13_guidance.py`	exact CTMC guidance versus the Taylor approximation	9
`verify_theory.py`	52 numerical checks of the book's theorems and propositions	all; Section C.3
`appC_masked_lm.py`	the complete trainable model	5, this appendix

C.5. Scaling this up

The listing above is deliberately minimal. To go from it to something research-grade:

1. **Replace the toy task** with a real tokenised corpus, or with SELFIES/SAFE strings for molecules, or with amino-acid sequences.
2. **Replace the schedule.** Use a cosine schedule — which is Fisher–Rao optimal for masked diffusion (Zhang and Syed, 2025) — and state your ε cutoff.
3. **Drop time conditioning** if you are using absorbing noise (Ou et al., 2025), and cache across non-unmasking steps.
4. **Reduce the gradient variance.** Low-discrepancy t sampling, antithetic masking, and consider dropping the $1/t$ weight entirely — worth about 12% of FLOPs (Sahoo et al., 2026).
5. **Initialise from an autoregressive checkpoint** and keep the AR shift (Ye et al., 2025).
6. **Use a real sampler**: confidence ordering, adaptive parallelism, and an annealed remasking schedule (Kim et al., 2025; Wang et al., 2025b).
7. **Evaluate properly.** Section 10.8's checklist, and build the oracle instance for your sampler.

Or start from a maintained repository — Chapter D lists them. There is no reason to write a training loop from scratch for real work; the point of the listing above is that you should be able to, and should have read one that you can hold in your head.

APPENDIX D

Resources and a Reading Order

D.1. A reading order

The irreducible core (five papers). If you read nothing else:

1. Austin et al. (2021) — D3PM. The foundation.
2. Campbell et al. (2022) — the continuous-time CTMC framework.
3. Lou et al. (2024) — score entropy. ICML 2024 best paper.
4. Sahoo et al. (2024) or Shi et al. (2024) — the masked diffusion simplification. Read one; they agree.
5. Ou et al. (2025) — the any-order autoregressive equivalence, which reframes all of the above.

Then, depending on your direction.

- *Language models:* Nie et al. (2025b), Ye et al. (2025), Nie et al. (2025a), Arriola et al. (2025).
- *Molecules:* Vignac et al. (2023a), Vignac et al. (2023b), Lee et al. (2025).
- *Biological sequences:* Alamdari et al. (2023), Campbell et al. (2024), Stark et al. (2024).
- *Flows:* Gat et al. (2024), Campbell et al. (2024).
- *Sampling:* Kim et al. (2025), Wang et al. (2025b), Zheng et al. (2025), Liu et al. (2025).
- *Theory:* Feng et al. (2025), Liang et al. (2025), Kim et al. (2025).
- *Guidance and RL:* Nisonoff et al. (2025), Schiff et al. (2025), Zekri and Boullé (2025), Zhao et al. (2025), Zhang et al. (2025a).

Read before you write anything. The 2025–26 critical literature: Zheng et al. (2025) (the float32 bug and time-agnosticism), Franca and Tong (2026) (gen-PPL is hackable), Pynadath et al. (2026) (entropy frontiers), Tang et al. (2026) (samplers are incorrect even with oracle denoisers), Sahoo et al. (2026) (perplexity does not transfer across families).

Surveys, for coverage. Yu et al. (2026), Li et al. (2025), Yuan et al. (2026), Bertolani et al. (2026). Use these to find papers, not to learn the methods — and remember that a survey table inherits every methodological problem of the papers it tabulates.

D.2. Codebases

The repositories below were the reference implementations at the time of writing. Repository locations change; search for the paper title if a link has moved.

Project	Repository
MDLM	`https://github.com/kuleshov-group/mdlm`
SEDD	`https://github.com/louaaron/Score-Entropy-Discrete-Diffusion`

Other code to look for by name: `google-deepmind/md4` (MD4); the D3PM reference code in `google-research`; `cvignac/DiGress` and `cvignac/MiDi` for graphs and molecules; `microsoft/evodiff` for proteins; `HannesStark/dirichlet-flow-matching` for DNA; `facebookresearch` for discrete flow

matching; the Kuleshov group's repositories for UDLM, Duo, BD3-LM and ReMDM, which share a codebase with MDLM and are the easiest family to work in; `ML-GSAI/LLaDA` and the HKU NLP group's Dream release for the large open models; and the `dllm` toolkit for inference. The curated lists (`Awesome-DLMs`, `awesome-diffusion-llm`, `awesome-molecular-diffusion-models`, `DLLM-Survey`) are the fastest way to find current code.

Which codebase to start in

For research on *methods*, the Kuleshov group's MDLM codebase and its descendants (UDLM, Duo, BD3-LM, ReMDM) are the most productive starting point: they share infrastructure, so a sampler or objective change can be evaluated against five baselines without reimplementation, and the checkpoints are public. For work on *large* models, LLaDA and Dream are the open checkpoints; note that inference-time contributions (Chapter 8) can be evaluated on these without any training at all, which is the cheapest route to a credible result.

D.3. Datasets and benchmarks

Domain	Standard choices
Text (likelihood)	OpenWebText (preferred), LM1B (legacy; sentence-shuffled, so it cannot assess beyond-sentence quality), text8, WikiText
Text (downstream)	GSM8K, MATH500, MMLU, HumanEval; Countdown and Sudoku for planning
Molecules	QM9, ZINC250k, MOSES, GuacaMol; PMO for optimisation sample-efficiency
Graphs	QM9, ZINC, plus synthetic (planar, SBM, tree) sets for structural statistics
Proteins	UniRef, and the EvoDiff evaluation suite
DNA	enhancer datasets (melanoma, fly brain), promoter activity datasets
Images (pixel-level)	CIFAR-10, ImageNet 64×64 (MD4 reports 2.75 and 3.40 bits/dim)

A note on text benchmarks: Pynadath et al. (2026) recommend OpenWebText with GPT-2 Large as the reference scorer, comparison within the empirical unigram-entropy interquartile range $[5.37, 5.55]$, and — usefully for small budgets — note that 50k pretraining steps gave frontiers close to 1M steps for MDLM, Duo and CANDI.

D.4. A compute-budget guide

Rough orders of magnitude, to help you scope a project honestly.

Budget	What is feasible
A laptop	everything in Chapter C; oracle sampler studies; all of the verification in Chapter 14
1 GPU, days	molecules and DNA at full scale (small K, short sequences); sampler research on public checkpoints; guidance experiments
4–8 GPUs, weeks	GPT-2-small-scale text pretraining (the scale at which MDLM, SEDD, Duo and UDLM were compared); method iteration at 50k steps
32+ GPUs, months	1B-parameter scaling studies
Industrial	7–8B from scratch (LLaDA used 2.3T tokens); adaptation from an AR checkpoint is far cheaper (Dream used 580B)

D.5. Where to follow the field

New work appears on arXiv under cs.LG and cs.CL faster than any survey can track, and the curated GitHub lists above are updated continuously. The venues that carry most of this work are

ICML, NeurIPS and ICLR; for the biological applications also look at the ML-for-structural-biology workshops and at the journals in that area. For molecular work, note that a significant fraction appears first at domain venues rather than ML ones.

A closing caution

This appendix, and the book, describe a field as it stood in October 2026. Several of the central claims here were established in the preceding twelve months and some of them reversed claims that had been standard for two years. Expect more of that. The parts of this book most likely to survive are the derivations in Chapter B, which are mathematics, and the methodological advice in Section 10.8 and Chapter 14, which is about how to avoid fooling yourself. The parts most likely to date are the tables of benchmark numbers — treat them as calibration, not as facts.

Bibliography

Sarah Alamdari, Nitya Thakkar, Rianne van den Berg, Alex X. Lu, Nicolo Fusi, Ava P. Amini, and Kevin K. Yang. Protein generation with evolutionary diffusion: sequence is all you need. *bioRxiv*, 2023. doi:10.1101/2023.09.11.556673.

Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

Marianne Arriola, Aaron Kerem Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations (ICLR)*, 2025. Oral. arXiv:2503.09573.

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2107.03006.

Pavel Avdeyev, Chenlai Shi, Yuxuan Tan, Kseniia Dudnyk, and Jian Zhou. Dirichlet diffusion score model for biological sequence generation. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2305.10699.

Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. The reversal curse: LLMs trained on "a is b" fail to learn "b is a". In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.12288.

Thomas Bertolani, Davide Bucciarelli, Leonardo Zini, Marcella Cornia, and Lorenzo Baraldi. Diffusion language models: An experimental analysis. *arXiv preprint arXiv:2606.19475*, 2026.

Umberto Borso, Davide Paglieri, Jude Wells, and Tim Rocktäschel. Preference-based alignment of discrete diffusion models. *arXiv preprint arXiv:2503.08295*, 2025. v1 was titled *D3PO*.

Andrew Campbell, Joe Benton, Valentin De Bortoli, Tom Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14987.

Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.04997.

Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. Analog bits: Generating discrete data using diffusion models with self-conditioning. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2208.04202.

Sander Dieleman, Laurent Sartran, Arman Roshannai, Nikolay Savinov, Yaroslav Ganin, Pierre H. Richemond, Arnaud Doucet, Robin Strudel, Chris Dyer, Conor Durkan, Curtis Hawthorne, Rémi Leblond, Will Grathwohl, and Jonas Adler. Continuous diffusion for categorical data. *arXiv preprint arXiv:2211.15089*, 2022.

Zhengyu Fang, Zhimeng Jiang, Huiyuan Chen, Xiaoge Zhang, Tianyi Li, Kaiyu Tang, Xiao Li, and Jing Li. Dataset-level metrics attenuate non-determinism: A fine-grained non-determinism evaluation in diffusion language models. *arXiv preprint arXiv:2604.13413*, 2026.

Guhao Feng, Yihan Geng, Jian Guan, Wei Wu, Liwei Wang, and Di He. Theoretical benefit and limitation of diffusion language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.09622.

Antonio Franca and Alexander Tong. Hacking generative perplexity: Why unconditional text evaluation needs distributional metrics. *ICML 2026 Workshop on Structured Probabilistic Inference and Generative Modeling (SPIGM)*, 2026. arXiv:2606.08417.

Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.15595.

Daniel T. Gillespie. Approximate accelerated stochastic simulation of chemically reacting systems. *The Journal of Chemical Physics*, 115(4):1716–1733, 2001.

Ishaan Gulrajani and Tatsunori B. Hashimoto. Likelihood-based diffusion language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.18619.

Jiaqi Han, Austin Wang, Minkai Xu, Wenda Chu, Meihua Dang, Haotian Ye, Huayu Chen, Yisong Yue, and Stefano Ermon. Discrete diffusion trajectory alignment via stepwise decomposition. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.04832.

Satoshi Hayakawa, Yuhta Takida, Masaaki Imaizumi, Hiromi Wakaki, and Yuki Mitsufuji. Distillation of discrete diffusion through dimensional correlations. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2410.08709.

Michael Hersche, Samuel Moor-Smith, Thomas Hofmann, and Abbas Rahimi. Soft-masked diffusion language models. *arXiv preprint arXiv:2510.17206*, 2025.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Chunsan Hong, Sanghyun Lee, and Jong Chul Ye. Unifying masked diffusion models with various generation orders and beyond. *arXiv preprint arXiv:2602.02112*, 2026. ICML 2026.

Emiel Hoogeboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. Argmax flows and multinomial diffusion: Learning categorical distributions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2102.05379.

Emiel Hoogeboom, Alexey A. Gritsenko, Jasmijn Bastings, Ben Poole, Rianne van den Berg, and Tim Salimans. Autoregressive diffusion models. In *International Conference on Learning Representations (ICLR)*, 2022.

Arseny Ivanov, Sergei Kholkin, Vladislav Gromadskii, Grigoriy Ksenofontov, Ivan Oseledets, and Alexander Korotin. TUBE: Tangent upper bound on evidence for discrete diffusion language models. *arXiv preprint arXiv:2605.24292*, 2026. ICML 2026.

Mengni Jia, Mengyu Zhou, Yihao Liu, Xiaoxi Jiang, and Guanjun Jiang. Bringing stability to diffusion: Decomposing and reducing variance of training masked diffusion models. *arXiv preprint arXiv:2511.18159*, 2025. ICLR 2026.

Thomas J. Kerby and Kevin R. Moon. Training-free guidance for discrete diffusion models for molecular generation. *arXiv preprint arXiv:2409.07359*, 2024.

Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.06768.

Amandeep Kumar, Nithin Gopalakrishnan Nair, and Vishal M. Patel. Scale-wise VAR is secretly discrete diffusion. *arXiv preprint arXiv:2509.22636*, 2025.

Hugo Lavenant and Giacomo Zanella. Error bounds and optimal schedules for masked diffusions with factorized approximations. *arXiv preprint arXiv:2510.25544*, 2025.

Joongwon Lee, Seonghwan Kim, Seokhyun Moon, Hyunwoo Kim, and Woo Youn Kim. FragFM: Hierarchical framework for efficient molecule generation via fragment-level discrete flow matching. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2502.15805.

Seul Lee, Karsten Kreis, Srimukh Prasad Veccham, Meng Liu, Danny Reidenbach, Yuxing Peng, Saee Paliwal, Weili Nie, and Arash Vahdat. GenMol: A drug discovery generalist with discrete diffusion. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2501.06158.

Tianyi Li, Mingda Chen, Bowei Guo, and Zhiqiang Shen. A survey on diffusion language models. *arXiv preprint arXiv:2508.10875*, 2025.

Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. Diffusion-LM improves controllable text generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14217.

Yuchen Liang, Renxiang Huang, Lifeng Lai, Ness Shroff, and Yingbin Liang. Absorb and converge: Provable convergence guarantee for absorbing discrete diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.02318.

Anji Liu, Oliver Broadrick, Mathias Niepert, and Guy Van den Broeck. Discrete copula diffusion. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.01949.

Gang Liu, Jiaxin Xu, Tengfei Luo, and Meng Jiang. Graph diffusion transformers for multi-conditional molecular generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2401.13858.

Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. Best paper award. arXiv:2310.16834.

Chenlin Meng, Kristy Choi, Jiaming Song, and Stefano Ermon. Concrete score matching: Generalized score matching for discrete data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2211.00802.

Kou Misaki and Takuya Akiba. UnMaskFork: Test-time scaling for masked diffusion via deterministic action branching. *arXiv preprint arXiv:2602.04344*, 2026.

Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

Shen Nie, Fengqi Zhu, Chao Du, Tianyu Pang, Qian Liu, Guangtao Zeng, Min Lin, and Chongxuan Li. Scaling up masked diffusion models on text. In *International Conference on Learning Representations (ICLR)*, 2025a. arXiv:2410.18514.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b. arXiv:2502.09992.

Hunter Nisonoff, Junhao Xiong, Stephan Allenspach, and Jennifer Listgarten. Unlocking guidance for discrete state-space diffusion and flow models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.01572.

Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.03736.

Yong-Hyun Park, Chieh-Hsin Lai, Satoshi Hayakawa, Yuhta Takida, and Yuki Mitsufuji. Jump your steps: Optimizing sampling schedule of discrete diffusion models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.07761.

Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. CANDI: Hybrid discrete-continuous diffusion models. *arXiv preprint arXiv:2510.22510*, 2025. ICML 2026.

Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. Generative frontiers: Why evaluation matters for diffusion language models. *arXiv preprint arXiv:2604.02718*, 2026.

Yiming Qin, Clément Vignac, and Pascal Frossard. Sparse training of discrete diffusion models for graph generation. *arXiv preprint arXiv:2311.02142*, 2023.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.18290.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.07524.

Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, Guanghan Wang, Justin Chiu, and Volodymyr Kuleshov. The diffusion duality. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2506.10892.

Subham Sekhar Sahoo, Jean-Marie Lemercier, Zhihan Yang, Justin Deschenaux, Jingyu Liu, John Thickstun, and Ante Jukic. Scaling beyond masked diffusion language models. *arXiv preprint arXiv:2602.15014*, 2026. ICML 2026.

Yair Schiff, Subham Sekhar Sahoo, Hao Phung, Guanghan Wang, Sam Boshar, Hugo Dalla-Torre, Bernardo P. de Almeida, Alexander Rush, Thomas Pierrot, and Volodymyr Kuleshov. Simple guidance mechanisms for discrete diffusion models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.10193.

Hyunjin Seo, Taewon Kim, Sihyun Yu, and SungSoo Ahn. Learning flexible forward trajectories for masked molecular diffusion. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.16790.

Jiaxin Shi and Michalis K. Titsias. Demystifying diffusion objectives: Reweighted losses are better variational bounds. *arXiv preprint arXiv:2511.19664*, 2025.

Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.04329.

Antoine Siraudin, Fragkiskos D. Malliaros, and Christopher Morris. Cometh: A continuous-time discrete-state graph diffusion model. *arXiv preprint arXiv:2406.06449*, 2024.

Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

Hannes Stark, Bowen Jing, Chenyu Wang, Gabriele Corso, Bonnie Berger, Regina Barzilay, and Tommi Jaakkola. Dirichlet flow matching with applications to DNA sequence design. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.05841.

Maojiang Su, Mingcheng Lu, Jerry Yao-Chieh Hu, Shang Wu, Zhao Song, Alex Reneau, and Han Liu. A theoretical analysis of discrete flow matching generative models. *arXiv preprint arXiv:2509.22623*, 2025.

Luhan Tang, Longxuan Yu, Shaorong Zhang, and Greg Ver Steeg. Is your diffusion sampler actually correct? a sampler-centric evaluation of discrete diffusion language models. *arXiv preprint arXiv:2602.19619*, 2026.

Benigno Uria, Iain Murray, and Hugo Larochelle. A deep and tractable density estimator. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014.

Clément Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. DiGress: Discrete denoising diffusion for graph generation. In *International Conference on Learning Representations (ICLR)*, 2023a. arXiv:2209.14734.

Clément Vignac, Nagham Osman, Laura Toni, and Pascal Frossard. MiDi: Mixed graph and 3d denoising diffusion for molecule generation. In *European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML-PKDD)*, 2023b. arXiv:2302.09048.

Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

Chenyu Wang, Masatoshi Uehara, Yichun He, Amy Wang, Tommaso Biancalani, Avantika Lal, Tommi Jaakkola, Sergey Levine, Hanchen Wang, and Aviv Regev. Fine-tuning discrete diffusion models via reward optimization with applications to DNA and protein design. In *International Conference on Learning Representations (ICLR)*, 2025a. arXiv:2410.13643.

Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b. arXiv:2503.00307.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

Runpeng Yu, Qi Li, and Xinchao Wang. Discrete diffusion in large language and multimodal models: A survey. *Transactions on Machine Learning Research (TMLR)*, 2026. arXiv:2506.13759.

Ye Yuan, Weien Li, Rui Song, Zeyu Li, Haochen Liu, Xiangyu Kong, Zixuan Dong, Linfeng Du, Zipeng Sun, Weixu Zhang, Jiaxin Huang, Changjiang Han, Yonghan Yang, Zichen Zhao, Xiuyuan Hu, Haolun Wu, Yankai Chen, Fengran Mo, Jikun Kang, Bowei He, Dawn Song, Philip S. Yu, and Xue Liu. Discrete diffusion models: A unified framework from tokenization to generation. *arXiv preprint arXiv:2607.13431*, 2026.

Oussama Zekri and Nicolas Boullé. Fine-tuning discrete diffusion models with policy gradient methods. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.01384.

Leo Zhang and Saifuddin Syed. The cosine schedule is fisher-rao-optimal for masked discrete diffusion models. *arXiv preprint arXiv:2508.04884*, 2025.

Ruixiang Zhang, Shuangfei Zhai, Yizhe Zhang, James Thornton, Zijing Ou, Joshua Susskind, and Navdeep Jaitly. Target concrete score matching: A holistic framework for discrete diffusion. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025a. arXiv:2504.16431.

Zikun Zhang, Quanquan Gu, and Zixiang Chen. Convergence of score-based discrete diffusion models: A discrete-time analysis. In *International Conference on Learning Representations (ICLR)*, 2025b. arXiv:2410.02321.

Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2504.12216.

Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Qinsheng Zhang, and Jun Zhu. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.02908.

Cai Zhou, Chenyu Wang, Dinghuai Zhang, Shangyuan Tong, Yifei Wang, Stephen Bates, and Tommi Jaakkola. Next semantic scale prediction via hierarchical diffusion language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2510.08632.

Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. LLaDA 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.